CFGS Desarrollo de Aplicaciones Multiplataforma

Acceso a Datos

con Python

Módulo profesional 0486 · 126 horas

Segundo curso

Apuntes del módulo — versión 0.6.2 (borrador)
Fecha: 21/10/2026
Curso 2026–2027

Basado en el currículo del módulo para la Comunidad de Castilla y León.

Contenido generado con ayuda de inteligencia artificial: puede contener errores.

Presentación

Aviso sobre el contenido de estos apuntes. Los contenidos de este documento (explicaciones, ejemplos de código, ejercicios y proyectos) se han **generado con ayuda de herramientas de inteligencia artificial** y **pueden contener errores**, imprecisiones u omisiones. Los ejemplos de código pueden comportarse de forma distinta según la versión de Python, de las bibliotecas o de los gestores de bases de datos que utilices.
Úsalos como material de apoyo, contrasta la información con la documentación oficial de cada herramienta (enlazada a lo largo del texto y recogida en el Anexo F) y, si encuentras un error, comunícaselo al profesor para corregirlo en las siguientes versiones.

Estos apuntes acompañan las clases del módulo *Acceso a Datos* del segundo curso del ciclo de grado superior de Desarrollo de Aplicaciones Multiplataforma. El objetivo del módulo es que aprendas a construir programas que **guardan, recuperan y transforman información** en los distintos lugares donde puede vivir: ficheros, bases de datos relacionales, herramientas de mapeo objeto-relacional, bases de datos de objetos y bases de datos documentales.

A lo largo del curso trabajaremos con **Python**. Es un lenguaje con una sintaxis muy legible, una biblioteca estándar enorme (que ya trae módulos para ficheros, CSV, JSON, XML o SQLite) y conectores para prácticamente cualquier gestor de bases de datos. Como no todo el mundo lo ha usado antes, el Tema 0 repasa los conceptos que necesitarás durante el curso.

Cómo están organizados los apuntes

Cada tema sigue la misma estructura:

- Un recuadro inicial con lo que aprenderás en el tema.
- Explicaciones teóricas acompañadas de **ejemplos de código completos** que puedes copiar y ejecutar.
- Recuadros de **Nota** con aclaraciones y de **Cuidado** con errores habituales.
- **Ejercicios** cortos para practicar cada apartado, con una indicación de dificultad (★ a ★★★).
- Uno o varios **Proyectos** finales, más largos, para trabajar de forma autónoma.

Relación entre temas y resultados de aprendizaje

Tema	Contenido	RA
0	Repaso de Python	—
1	Manejo de ficheros	RA1
2	Manejo de conectores	RA2
3	Herramientas de mapeo objeto-relacional (ORM)	RA3
4	Bases de datos objeto-relacionales	RA4
5	Bases de datos documentales: MongoDB	RA5
6	Programación de componentes de acceso a datos	RA6
B	Anexo: procesamiento avanzado de XML (DOM, SAX, XPath, XSLT)	RA1
C	Anexo: bases de datos orientadas a objetos (ODMG, OQL, ZODB)	RA4
D	Anexo: bases de datos XML nativas (BaseX, XQuery)	RA5
E	Anexo: pruebas y documentación del código (`pytest`, *docstrings*)	Todos

Los anexos B, C y D recogen contenidos que el currículo incluye pero que hoy tienen poca presencia en el mundo profesional; se pueden trabajar como ampliación o según decida el profesorado. El RA6 es transversal: los componentes del Tema 6 se construyen sobre lo aprendido en los temas 1 a 5, por lo que el proyecto final del curso consiste en empaquetar como componentes reutilizables el trabajo de los temas anteriores.

Convenciones

- El código Python aparece así:

```python
def saludar(nombre: str) -> str:
    return f"Hola, {nombre}"   # comentario
```

- Las órdenes que se escriben en la terminal aparecen sobre fondo oscuro:

```
$ python -m venv .venv
$ pip install sqlalchemy
```

- Los nombres de bibliotecas y módulos aparecen en este color y enlazan con su documentación oficial. La bibliografía completa está en el Anexo F.
- Todos los ejemplos están pensados para **Python 3.12 o superior**.

Software necesario

El Apéndice A explica cómo preparar el entorno de trabajo: Python, un editor (Visual Studio Code o PyCharm), entornos virtuales y Docker para levantar los servidores de bases de datos (PostgreSQL, MySQL y MongoDB, y opcionalmente BaseX para el Anexo D) sin instalarlos en el equipo.

Índice

Presentación **i**

0 Repaso de Python **1**

0.1 Primeros pasos . . . 1
0.2 Tipos de datos básicos . . . 2
0.3 Colecciones . . . 4
0.4 Control de flujo . . . 5
0.5 Comprensiones . . . 11
0.6 Funciones . . . 13
0.7 Anotaciones de tipo . . . 15
0.8 Decoradores . . . 17
0.9 Módulos y paquetes . . . 18
0.10 Excepciones . . . 19
0.11 Gestores de contexto: la sentencia `with` . . . 20
0.12 Programación orientada a objetos . . . 21
0.13 Iteradores y generadores . . . 28
0.14 Enumeraciones . . . 28
0.15 Fechas y horas . . . 28
0.16 Argumentos de línea de comandos . . . 32
0.17 Ejercicios . . . 36

1 Manejo de ficheros **39**

1.1 Gestión de ficheros y directorios . . . 39
1.2 Formas de acceso a un fichero . . . 41
1.3 Flujos . . . 42
1.4 Operaciones básicas con ficheros de texto . . . 42
1.5 Ficheros binarios . . . 43
1.6 Ficheros CSV . . . 46
1.7 Ficheros JSON . . . 47
1.8 Ficheros XML . . . 57
1.9 Conversión entre formatos . . . 63
1.10 Extracción de datos de páginas web (*web scraping*) . . . 66
1.11 Excepciones en el manejo de ficheros . . . 68
1.12 Pruebas y documentación . . . 69
1.13 Ejercicios . . . 70

2 Manejo de conectores **74**
2.1 El desfase objeto-relacional . . . 74
2.2 Protocolos de acceso a bases de datos y conectores . . . 75
2.3 Establecimiento de conexiones . . . 76
2.4 Ejecución de sentencias de descripción de datos (DDL) . . . 81
2.5 Ejecución de sentencias de modificación de datos (DML) . . . 81
2.6 Ejecución de consultas . . . 83
2.7 Ejecución de procedimientos almacenados . . . 85
2.8 Gestión de transacciones . . . 86
2.9 Excepciones de la DB-API . . . 88
2.10 Ejercicios . . . 89

3 Herramientas de mapeo objeto-relacional (ORM) **91**
3.1 Concepto de mapeo objeto-relacional . . . 91
3.2 Características de las herramientas ORM . . . 92
3.3 Instalación de SQLAlchemy . . . 92
3.4 Configuración de la herramienta . . . 93
3.5 Clases persistentes y estructura del mapeo . . . 94
3.6 Mapeo de relaciones y colecciones . . . 95
3.7 Mapeo de la herencia . . . 98
3.8 Sesiones y estados de un objeto . . . 99
3.9 Carga, almacenamiento y modificación de objetos . . . 100
3.10 Consultas . . . 101
3.11 Gestión de transacciones . . . 102
3.12 Organización de una aplicación con ORM . . . 103
3.13 Ejercicios . . . 104

4 Bases de datos objeto-relacionales **106**
4.1 Bases de datos objeto-relacionales . . . 106
4.2 Gestión de objetos con SQL:1999 . . . 108
4.3 Acceso a las funciones del gestor desde Python . . . 111
4.4 Pruebas y documentación . . . 116
4.5 Ejercicios . . . 117

5 Bases de datos documentales: MongoDB **119**
5.1 Bases de datos NoSQL . . . 119
5.2 El modelo de datos de MongoDB . . . 121
5.3 Instalación y herramientas . . . 123
5.4 El lenguaje de consultas de MongoDB . . . 123
5.5 Modelado de datos . . . 127
5.6 Índices . . . 128

5.7 MongoDB desde Python: `pymongo` . . . 129
5.8 Ejercicios . . . 135

6 Programación de componentes de acceso a datos 138
6.1 Concepto de componente . . . 138
6.2 Programación orientada a componentes . . . 139
6.3 Propiedades y atributos . . . 140
6.4 Eventos y asociación de acciones a eventos . . . 142
6.5 Propiedades compartidas y restringidas . . . 143
6.6 Introspección y reflexión . . . 145
6.7 Editores de propiedades . . . 147
6.8 Persistencia del componente . . . 147
6.9 Componentes de acceso a datos . . . 148
6.10 Herramientas para el desarrollo de componentes no visuales . . . 151
6.11 Empaquetado de componentes . . . 152
6.12 Ejercicios . . . 153

A Preparación del entorno de trabajo 156
A.1 Python y el editor . . . 156
A.2 Servidores de bases de datos con Docker . . . 156

B Procesamiento avanzado de XML 159
B.1 Los modelos DOM y SAX en la práctica . . . 159
B.2 XPath completo con `lxml` . . . 160
B.3 Vinculación de datos ampliada . . . 161
B.4 Transformaciones XSLT . . . 162
B.5 Ejercicios . . . 162

C Bases de datos orientadas a objetos 164
C.1 El modelo orientado a objetos . . . 164
C.2 Tipos de datos: básicos y estructurados . . . 165
C.3 El lenguaje de consultas OQL . . . 166
C.4 Una base de datos de objetos en Python: ZODB . . . 167
C.5 Ejercicios . . . 171

D Bases de datos XML nativas 172
D.1 Bases de datos XML nativas . . . 172
D.2 Instalación y configuración de BaseX . . . 173
D.3 Establecimiento y cierre de conexiones . . . 174
D.4 Colecciones y documentos . . . 175
D.5 Lenguajes de consulta: XPath y XQuery . . . 176
D.6 Realización de consultas desde Python . . . 178

D.7 Indexación . . . 179
D.8 Identificadores únicos . . . 179
D.9 Gestión de transacciones . . . 180
D.10 Tratamiento de excepciones . . . 180
D.11 Ejercicios . . . 181

E Pruebas y documentación del código **182**
E.1 Por qué escribir pruebas automáticas . . . 182
E.2 De `assert` a las pruebas . . . 182
E.3 Un ejemplo completo . . . 182
E.4 Herramientas de `pytest` . . . 184
E.5 Documentar el código . . . 187
E.6 Guía de estilo . . . 189
E.7 Ejercicios . . . 190

F Bibliografía y recursos **191**
F.1 Documentación general . . . 191
F.2 Tema 1: Manejo de ficheros . . . 191
F.3 Tema 2: Manejo de conectores . . . 192
F.4 Tema 3: Herramientas ORM . . . 192
F.5 Tema 4: Bases de datos objeto-relacionales . . . 192
F.6 Tema 5: Bases de datos documentales (MongoDB) . . . 193
F.7 Tema 6: Componentes de acceso a datos . . . 193
F.8 Anexos B, C y D . . . 193
F.9 Anexo E: pruebas y documentación . . . 193
F.10 Especificaciones y estándares . . . 194
F.11 Libros . . . 194
F.12 Datos abiertos y sitios de práctica . . . 194

TEMA 0

Repaso de Python

En este tema aprenderás a…

- Ejecutar programas Python y gestionar dependencias con entornos virtuales y `pip`.
- Manejar los tipos de datos básicos y las colecciones (listas, tuplas, diccionarios y conjuntos).
- Usar las estructuras de control (`if`, `for`, `while`, `match`) y las comprensiones.
- Escribir funciones, anotar sus tipos y comprobarlos con herramientas de análisis estático.
- Organizar el código en módulos y paquetes.
- Tratar errores mediante excepciones, comprobar suposiciones con `assert` y usar gestores de contexto (`with`).
- Programar con clases, distinguiendo atributos de clase y de instancia y aplicando los convenios de visibilidad, herencia y propiedades.
- Elegir entre clases normales, *dataclasses* y modelos de Pydantic para representar datos.
- Entender iteradores, generadores y decoradores, que aparecerán en el resto de temas.
- Trabajar con fechas, horas y zonas horarias, y escribir programas de línea de comandos con `argparse`.

Este tema no pretende ser un curso completo de Python, sino un repaso de las piezas que usaremos constantemente en el módulo. Si ya programas en Java o C#, verás que los conceptos son los mismos; lo que cambia es la sintaxis y algunas costumbres del lenguaje.

0.1 Primeros pasos

0.1.1 El intérprete y los scripts

Python es un lenguaje **interpretado** y de **tipado dinámico**: no hay fase de compilación explícita y las variables no se declaran con un tipo. Hay dos formas de ejecutar código:

- En el **intérprete interactivo** (REPL), escribiendo `python` en la terminal. Es muy útil para probar cosas rápidamente.
- Guardando el código en un fichero `.py` y ejecutándolo con `python fichero.py`.

```
$ python
>>> 2 + 3
5
>>> nombre = "Ada"
```

```
>>> f"Hola, {nombre}"
'Hola, Ada'
>>> exit()
```

En Python **la indentación forma parte de la sintaxis**: los bloques no se delimitan con llaves sino con sangría (por convenio, cuatro espacios), y la línea que abre un bloque termina en dos puntos.

```
edad = 19
if edad >= 18:
    print("Mayor de edad")
else:
    print("Menor de edad")
```

0.1.2 Entornos virtuales y paquetes

Durante el curso instalaremos bibliotecas de terceros (`lxml`, `psycopg`, `sqlalchemy`, `pymongo`...). Para que cada proyecto tenga sus propias dependencias y no interfieran entre sí, se usa un **entorno virtual**: una carpeta con una copia aislada del intérprete y sus paquetes.

```
$ python -m venv .venv               # crea el entorno en la carpeta .venv
$ source .venv/bin/activate          # Linux / macOS
> .venv\Scripts\activate             # Windows
(.venv) $ pip install lxml           # instala un paquete
(.venv) $ pip freeze > requirements.txt   # guarda las dependencias
(.venv) $ pip install -r requirements.txt # las reinstala en otro equipo
```

> **Nota.** Existen herramientas más modernas que combinan entornos virtuales y gestión de dependencias, como `uv` o `poetry`. En el Tema 6 veremos cómo describir un proyecto con `pyproject.toml`. Mientras tanto, `venv` + `pip` es suficiente.

0.2 Tipos de datos básicos

Tipo	Ejemplo	Comentario
`int`	`42`, `-7`, `1_000_000`	Enteros de precisión ilimitada.
`float`	`3.14`, `1e-3`	Coma flotante de doble precisión.
`bool`	`True`, `False`	Subtipo de `int`.
`str`	`"hola"`, `'hola'`	Cadenas Unicode inmutables.
`bytes`	`b"\x00\xff"`	Secuencias inmutables de bytes (Tema 1).
`NoneType`	`None`	Ausencia de valor (equivale a `null`).

```
x = 10            # int
x = "diez"        # ahora es str: el tipo lo tiene el valor, no la variable
print(type(x))    # <class 'str'>
print(7 / 2)      # 3.5  (división real)
print(7 // 2)     # 3    (división entera)
print(7 % 2)      # 1    (resto)
print(2 ** 10)    # 1024 (potencia)
```

0.2.1 Cadenas de caracteres

Las cadenas son inmutables y admiten índices (empezando en 0), índices negativos (contando desde el final) y *slicing* `[inicio:fin:paso]`.

```
s = "Acceso a Datos"
print(s[0], s[-1])        # A s
print(s[0:6])             # Acceso
print(s.upper())          # ACCESO A DATOS
print(s.split(" "))       # ['Acceso', 'a', 'Datos']
print("-".join(["a", "b", "c"]))   # a-b-c
print("  texto  ".strip())         # 'texto'
print(s.replace("Datos", "Ficheros"))
print("Datos" in s)       # True
```

Las **f-strings** son la forma recomendada de construir cadenas con valores:

```
nombre, nota = "Lucía", 8.456
print(f"{nombre} ha sacado un {nota:.2f}")   # Lucía ha sacado un 8.46
print(f"{nombre:>10}|")                       # alineado a la derecha en 10 posiciones
print(f"{nota=}")                             # nota=8.456 (útil para depurar)
```

Resumen de modificadores de formato

Dentro de las llaves de una f-string puede ir cualquier expresión de Python (`f"{x * 2}"`, `f"{nombre.upper()}"`), seguida opcionalmente de una **conversión** (tras `!`) y de una **especificación de formato** (tras `:`):

```
{expresión!conversión:formato}

formato = [[relleno]alineación][signo][#][0][ancho][agrupación][.precisión][tipo]
```

Todas las partes son opcionales, pero si aparecen deben ir en ese orden. Los resultados de la tabla se han obtenido con `nombre = "Lucía"`, `nota = 8.456`, `x = 42`, `n = 1234567.891`, `r = 0.256` y `hoy = date(2026, 10, 19)`; el símbolo ␣ representa un espacio.

Modificador	Ejemplo	Resultado
Precisión y tipo		
Decimales: `.N` con el tipo `f`	`f"{nota:.2f}"`	`8.46`
Notación científica: `e`	`f"{n:.3e}"`	`1.235e+06`
Porcentaje (multiplica por 100): `%`	`f"{r:.1%}"`	`25.6%`
Entero: `d`	`f"{x:d}"`	`42`
Binario, octal, hexadecimal: `b`, `o`, `x`, `X`	`f"{x:b} {x:x}"`	`101010 2a`
Con prefijo de base: `#`	`f"{x:#x}"`	`0x2a`
Ancho, alineación y relleno		
Ancho mínimo (los números se alinean a la derecha)	`f"{x:5d}|"`	`␣␣␣42|`
Relleno con ceros: `0` delante del ancho	`f"{x:05d}"`	`00042`
Alinear a la izquierda: `<`	`f"{nombre:<10}|"`	`Lucía␣␣␣␣␣|`
Alinear a la derecha: `>`	`f"{nombre:>10}|"`	`␣␣␣␣␣Lucía|`
Centrar: `^`	`f"{nombre:^10}|"`	`␣␣Lucía␣␣␣|`
Carácter de relleno (antes de la alineación)	`f"{nombre:*^11}"`	`***Lucía***`
Ancho tomado de una variable	`f"{nota:{ancho}.2f}"`	`␣␣␣␣8.46`
Signo y separador de miles		
Mostrar siempre el signo: `+`	`f"{x:+d}"`	`+42`
Separador de miles: `,` o `_`	`f"{n:,.2f}"`	`1,234,567.89`
Otros		
Fechas: códigos de `strftime`	`f"{hoy:%d/%m/%Y}"`	`19/10/2026`
Depuración: `=` muestra la expresión y su valor	`f"{nota = :.1f}"`	`nota = 8.5`
Llaves literales: se duplican	`f"{{x}} vale {x}"`	`{x} vale 42`

Conversiones: `!r` y `!s`. Antes de aplicar el formato, la f-string convierte el valor en texto. Por defecto usa `str()`, la representación pensada para el usuario, pero con `!r` se usa `repr()`, la pensada para el programador (son los métodos `__str__` y `__repr__` que veremos en el apartado de clases). La diferencia se aprecia bien con una fecha, y `!r` es especialmente útil en mensajes de error, porque las comillas dejan ver espacios sobrantes o si el valor es una cadena:

```
hoy = date(2026, 10, 19)
print(f"{hoy}")        # 2026-10-19                  (str, por defecto)
print(f"{hoy!r}")      # datetime.date(2026, 10, 19)  (repr)

nombre = "Ana "
print(f"No existe el alumno {nombre!r}")   # No existe el alumno 'Ana '
```

El especificador `=` de depuración usa `repr()` automáticamente (`f"{nombre=}"` muestra `nombre='Ana '`). Existe también `!a`, que usa `ascii()`, pero apenas tiene uso práctico.

> **Nota.** El separador de miles y el punto decimal siguen siempre el formato inglés (`1,234,567.89`). Para obtener el formato español (`1.234.567,89`) se puede usar el módulo `locale` o, de forma sencilla, intercambiar los caracteres: `f"{n:,.2f}".replace(",", "X").replace(".", ",").replace("X", ".")`.

0.3 Colecciones

0.3.1 Listas

Secuencias **mutables** y ordenadas. Pueden contener elementos de cualquier tipo.

```
modulos = ["Acceso a Datos", "PSP", "DI"]
modulos.append("PMDM")          # añade al final
modulos.insert(0, "SGE")        # inserta en una posición
modulos.remove("DI")            # elimina por valor
ultimo = modulos.pop()          # extrae el último
print(len(modulos), modulos)
print(sorted(modulos))          # nueva lista ordenada
modulos.sort(key=len)           # ordena en el sitio por longitud
```

0.3.2 Tuplas

Secuencias **inmutables**. Se usan para agrupar valores relacionados, por ejemplo una fila devuelta por una consulta SQL.

```
fila = (1, "Ana", 7.5)
ident, nombre, nota = fila      # desempaquetado
print(nombre)
```

0.3.3 Diccionarios

Colecciones de pares *clave: valor*. Son, con diferencia, la estructura más utilizada en Python y la que encaja de forma natural con JSON y con los documentos de MongoDB.

```
alumno = {"nombre": "Ana", "edad": 20, "modulos": ["AD", "PSP"]}
print(alumno["nombre"])
print(alumno.get("email", "sin email"))   # valor por defecto si no existe
alumno["email"] = "ana@ejemplo.es"         # añadir o modificar
del alumno["edad"]
for clave, valor in alumno.items():
    print(clave, "->", valor)
```

0.3.4 Conjuntos

Colecciones sin orden y **sin elementos repetidos**.

```
a = {"python", "java", "sql"}
b = {"sql", "xml"}
print(a | b, a & b, a - b)     # unión, intersección, diferencia
```

0.4 Control de flujo

0.4.1 Condicionales: `if`, `elif` y `else`

La sentencia `if` ejecuta un bloque sólo si se cumple una condición. Se pueden encadenar tantas alternativas `elif` (*else if*) como se quiera y, opcionalmente, un `else` final para el resto de casos.

Python evalúa las condiciones en orden y ejecuta **sólo el primer bloque** cuya condición sea verdadera.

```
nota = 7
if nota >= 9:
    calificacion = "Sobresaliente"
elif nota >= 7:
    calificacion = "Notable"          # se ejecuta este y no se mira nada más
elif nota >= 5:
    calificacion = "Aprobado"
else:
    calificacion = "Suspenso"
```

Algunos detalles útiles de las condiciones en Python:

- **Operadores lógicos**: se escriben con palabras: `and`, `or` y `not`.
- **Comparaciones encadenadas**: `0 <= nota <= 10` equivale a `0 <= nota and nota <= 10`.
- **Pertenencia**: `in` y `not in` comprueban si un elemento está en una cadena, lista, diccionario (sus claves) o conjunto.
- **Comparación con `None`**: se usa `is None` / `is not None`, no `== None`.
- **Valores "verdaderos" y "falsos"**: cualquier valor puede usarse como condición. Se consideran falsos `False`, `None`, el cero (`0`, `0.0`) y las colecciones vacías (`""`, `[]`, `{}`, `set()`); todo lo demás es verdadero. Así, `if lista:` significa "si la lista no está vacía".

```
modulos = ["AD", "PSP"]
if "AD" in modulos and "DI" not in modulos:
    print("Matriculado en AD pero no en DI")

email = None
if email is None:
    print("Sin email")

if not modulos:                       # lista vacía
    print("No hay módulos")
```

Para elegir entre dos valores en una sola línea existe la **expresión condicional** (el equivalente al operador ternario `? :` de Java):

```
resultado = "Aprobado" if nota >= 5 else "Suspenso"
```

0.4.2 El bucle `for`

En Python, `for` no cuenta de un número a otro como en Java o C, sino que **recorre los elementos de un iterable**: una lista, una cadena, un diccionario, un fichero, el resultado de una consulta… En cada vuelta, la variable del bucle toma el valor del siguiente elemento.

```
for modulo in ["AD", "PSP", "DI"]:
    print(modulo)

for letra in "Hola":                  # una cadena es una secuencia de caracteres
    print(letra)
```

```python
notas = {"Ana": 8, "Luis": 4}
for nombre in notas:                  # un diccionario se recorre por sus claves
    print(nombre)
for nombre, nota in notas.items():    # claves y valores a la vez
    print(nombre, nota)
```

`range`: secuencias de números

Cuando sí se necesita repetir algo un número de veces o recorrer posiciones, se usa `range`, que genera números enteros **sin incluir el final**:

Llamada	Números que genera
`range(5)`	`0, 1, 2, 3, 4`
`range(1, 6)`	`1, 2, 3, 4, 5`
`range(0, 10, 2)`	`0, 2, 4, 6, 8`
`range(10, 0, -3)`	`10, 7, 4, 1`

`range` no crea una lista en memoria: va generando los números a medida que el bucle los pide, por lo que `range(1_000_000)` no ocupa más que `range(5)`. Si se necesita la lista, se convierte con `list(range(5))`.

`enumerate`: elemento y posición

Si además del elemento se necesita su posición, no hay que recurrir a `range(len(lista))`: `enumerate` devuelve en cada vuelta una tupla `(posición, elemento)`, que se desempaqueta en dos variables. El parámetro `start` indica desde qué número contar.

```python
modulos = ["AD", "PSP", "DI"]
for i, modulo in enumerate(modulos, start=1):
    print(f"{i}. {modulo}")           # 1. AD   2. PSP   3. DI

# Equivalente, pero menos claro y más propenso a errores:
for i in range(len(modulos)):
    print(f"{i + 1}. {modulos[i]}")
```

`zip`: recorrer varias secuencias a la vez

`zip` combina varias secuencias elemento a elemento: en cada vuelta devuelve una tupla con el primer elemento de cada una, luego el segundo, etc. Se detiene al acabarse la secuencia más corta; con `strict=True` lanza un `ValueError` si no tienen la misma longitud, lo que evita perder datos sin darse cuenta.

```python
nombres = ["Ana", "Luis", "Eva"]
notas = [8, 4, 6]
for nombre, nota in zip(nombres, notas):
    print(nombre, nota)               # Ana 8 / Luis 4 / Eva 6

print(dict(zip(nombres, notas)))      # {'Ana': 8, 'Luis': 4, 'Eva': 6}
list(zip(nombres, [8, 4], strict=True))   # ValueError: zip() argument 2 is shorter...
```

`break`, `continue` y `else`

- `break` sale del bucle inmediatamente.
- `continue` salta a la siguiente vuelta sin ejecutar el resto del bloque.
- Un bucle puede llevar un bloque `else`, que se ejecuta cuando el bucle **termina normalmente**, es decir, cuando se recorren todos los elementos **sin salir por un `break`**.

El `else` es útil en las búsquedas: permite saber si el bucle encontró lo que buscaba sin necesidad de una variable auxiliar.

```
alumnos = ["Ana", "Luis", "Eva"]
buscado = "Marta"

for alumno in alumnos:
    if alumno == buscado:
        print("Encontrado")
        break                          # se sale del bucle: NO se ejecuta el else
else:
    print(f"{buscado} no está en la lista")   # sólo si no hubo break

for n in range(10):
    if n % 2 == 0:
        continue                       # se salta los pares
    print(n)                           # 1 3 5 7 9
```

> **Nota.** El nombre `else` resulta poco intuitivo en un bucle. Puedes leerlo como *"si no se salió con break"*.

0.4.3 El bucle `while`

`while` repite un bloque **mientras se cumpla una condición**, que se comprueba antes de cada vuelta. Se usa cuando no se sabe de antemano cuántas vueltas habrá: leer datos hasta que se acaben, reintentar una conexión, esperar una respuesta válida del usuario... Hay que asegurarse de que algo dentro del bucle acabe haciendo falsa la condición; si no, el bucle no termina nunca.

```
intentos = 0
while intentos < 3:
    intentos += 1
    print("Intento", intentos)
```

Admite también `break`, `continue` y `else` (que se ejecuta cuando la condición deja de cumplirse sin haber salido por `break`). Un patrón muy habitual es el bucle "infinito" con `while True` del que se sale con `break` cuando se cumple cierta condición:

```
while True:
    texto = input("Edad: ")
    if texto.isdigit() and 0 < int(texto) < 120:
        edad = int(texto)
        break                          # dato válido: salimos del bucle
    print("Edad no válida, inténtalo de nuevo")
```

0.4.4 `match`: coincidencia de patrones

La sentencia `match` (Python 3.10) compara un valor con una serie de **patrones** y ejecuta el bloque del primero que coincida. En su forma más sencilla sustituye a una cadena de `if/elif` que compara con valores concretos, como el `switch` de Java:

```
opcion = input("Opción: ")
match opcion:
    case "1":
        print("Alta")
    case "2" | "3":                    # varios valores con |
        print("Modificación o baja")
    case "q" | "Q":
        print("Salir")
    case _:                            # comodín: cualquier otro valor
        print("Opción no válida")
```

A diferencia del `switch` de otros lenguajes, no hace falta `break`: sólo se ejecuta un caso. Pero `match` es mucho más potente que un `switch`, porque los patrones pueden describir la **estructura** del valor y **capturar** partes de él en variables:

```
def procesar(orden: list[str]) -> str:
    match orden:
        case ["listar"]:
            return "Listado completo"
        case ["buscar", texto]:                # lista de 2 elementos: captura el 2.º
            return f"Buscando {texto!r}"
        case ["borrar", *ids] if ids:          # 1 o más elementos tras "borrar" (
    ↪ guarda)
            return f"Borrando {len(ids)} registros"
        case _:
            return "Orden no reconocida"

print(procesar("buscar Ana".split()))          # Buscando 'Ana'
print(procesar("borrar 3 7 9".split()))        # Borrando 3 registros

def describir(evento: dict) -> str:
    match evento:
        case {"tipo": "alta", "usuario": nombre}:    # diccionario con esas claves
            return f"Nuevo usuario: {nombre}"
        case {"tipo": "error", "codigo": int(c)} if c >= 500:
            return f"Error del servidor ({c})"
        case _:
            return "Evento desconocido"

print(describir({"tipo": "error", "codigo": 503}))  # Error del servidor (503)
```

Patrón	Coincide con
`"1"`, `42`, `None`	Ese valor concreto (literal).
`"a" | "b"`	Cualquiera de las alternativas.
`nombre`	Cualquier valor, que se guarda en la variable `nombre`.
`_`	Cualquier valor, sin guardarlo (comodín).
`[a, b]`, `[a, *resto]`	Una secuencia con esa forma; `*resto` recoge los elementos sobrantes.
`{"clave": valor}`	Un diccionario que tenga (al menos) esas claves.
`int(x)`, `Punto(x=0)`	Un objeto de ese tipo (con esos atributos).
`patrón if condición`	El patrón, si además se cumple la condición (*guarda*).

Este tipo de patrones resulta muy cómodo para procesar documentos JSON (Tema 1) o los mensajes de una API.

0.4.5 El operador morsa (:=)

El operador `:=`, llamado **operador morsa** (*walrus operator*) por su parecido con los ojos y colmillos de una morsa, se introdujo en Python 3.8. Es una **expresión de asignación**: asigna un valor a una variable y, a la vez, **devuelve** ese valor, de modo que se puede usar dentro de otra expresión (una condición, un `while`, una comprensión...). El `=` normal es una *sentencia*: asigna, pero no devuelve nada y no puede ir dentro de una condición.

Su uso más habitual es en los bucles que leen datos hasta que se acaban. Compara las dos versiones:

```
# Sin operador morsa: hay que repetir la llamada a input()
nombres = []
nombre = input("Nombre (vacío para terminar): ")
while nombre != "":
    nombres.append(nombre)
    nombre = input("Nombre (vacío para terminar): ")

# Con operador morsa: se lee y se comprueba en la misma línea
nombres = []
while (nombre := input("Nombre (vacío para terminar): ")) != "":
    nombres.append(nombre)
```

Es exactamente el patrón que usaremos en el Tema 1 para leer un fichero por bloques: `read()` devuelve un bloque vacío (que se evalúa como falso) al llegar al final del fichero.

```
with open("foto.jpg", "rb") as f:
    while bloque := f.read(64 * 1024):     # se detiene cuando bloque == b""
        procesar(bloque)
```

También evita llamar dos veces a una función cuyo resultado se comprueba y luego se usa (las comprensiones del segundo ejemplo se explican en el apartado siguiente):

```
agenda = {"Ana": "600111222", "Luis": "611222333"}
if (telefono := agenda.get("Ana")) is not None:
    print("Llamando al", telefono)

# En una comprensión: calcular el precio con IVA una sola vez por elemento
```

```
precios = [10.0, 25.5, 8.0]
caros = [con_iva for p in precios if (con_iva := round(p * 1.21, 2)) > 15]
print(caros)           # [30.86]
```

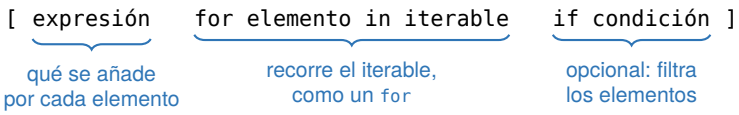
Cuidado.

- **Paréntesis**: `:=` tiene muy poca prioridad, así que casi siempre hay que rodear la asignación con paréntesis. `if n := len(nombres) > 1:` asigna a `n` el resultado de la comparación (`True`), no la longitud; lo correcto es `if (n := len(nombres)) > 1:`.
- No se puede usar como sentencia suelta: `x := 5` en una línea da `SyntaxError`. Para asignar sin más, se usa `=`.
- Úsalo sólo cuando haga el código más claro (bucles de lectura, evitar repetir una llamada). Encadenar varias asignaciones dentro de una misma expresión lo hace difícil de leer.

0.5 Comprensiones

Una **comprensión** construye una lista, un diccionario o un conjunto a partir de otro iterable en una sola expresión. Es una forma compacta, y muy habitual en Python, de escribir un bucle `for` cuyo único propósito es llenar una colección.

0.5.1 Sintaxis



Una comprensión equivale exactamente a este bucle:

```
# Comprensión
cuadrados_pares = [n * n for n in range(10) if n % 2 == 0]

# Bucle equivalente
cuadrados_pares = []
for n in range(10):              # for elemento in iterable
    if n % 2 == 0:               # if condición
        cuadrados_pares.append(n * n)    # expresión
print(cuadrados_pares)           # [0, 4, 16, 36, 64]
```

Se lee casi como una frase: "*el cuadrado de n, para cada n de 0 a 9, si n es par*".

0.5.2 Tipos de comprensiones

Según los delimitadores, se obtiene una colección u otra:

Tipo	Sintaxis	Resultado
Lista	`[expr for x in it]`	`list` (con orden y repetidos)
Conjunto	`{expr for x in it}`	`set` (sin repetidos)
Diccionario	`{clave: valor for x in it}`	`dict`
Generador	`(expr for x in it)`	Un generador: produce los valores de uno en uno, sin crear la colección en memoria (ver Iteradores y generadores).

```
palabras = ["Python", "java", "SQL", "python", "XML"]

minusculas = [p.lower() for p in palabras]
# ['python', 'java', 'sql', 'python', 'xml']

distintas = {p.lower() for p in palabras}
# {'java', 'python', 'sql', 'xml'}  (sin repetidos; el orden puede variar)

longitudes = {p: len(p) for p in palabras}
# {'Python': 6, 'java': 4, 'SQL': 3, 'python': 6, 'XML': 3}

notas = {"Ana": 8, "Luis": 4, "Eva": 6}
aprobados = {nombre: nota for nombre, nota in notas.items() if nota >= 5}
# {'Ana': 8, 'Eva': 6}

total = sum(len(p) for p in palabras)    # generador: no crea ninguna lista
# 22
```

0.5.3 Condiciones: filtrar o transformar

Hay que distinguir dos usos del `if`, que se escriben en lugares distintos:

- **Al final**, `if condición` **filtra**: los elementos que no la cumplen no aparecen en el resultado.
- **En la expresión**, `valor1 if condición else valor2` es una expresión condicional que **transforma**: todos los elementos aparecen, pero con un valor u otro. En este caso el `else` es obligatorio.

```
notas = [8, 4, 6, 3]
[n for n in notas if n >= 5]                          # [8, 6]  -> filtra
["Apto" if n >= 5 else "No apto" for n in notas]      # ['Apto', 'No apto', 'Apto', 'No
    ↪ apto']
```

0.5.4 Varios `for`

Una comprensión puede tener varios `for`, que se leen **en el mismo orden que si fueran bucles anidados**:

```
grupos = [["Ana", "Luis"], ["Eva"], ["Marta", "Iker"]]
alumnos = [alumno for grupo in grupos for alumno in grupo]
# ['Ana', 'Luis', 'Eva', 'Marta', 'Iker']
```

```python
# Equivale a:
alumnos = []
for grupo in grupos:
    for alumno in grupo:
        alumnos.append(alumno)

pares = [(x, y) for x in "AB" for y in range(2)]
# [('A', 0), ('A', 1), ('B', 0), ('B', 1)]
```

Cuándo no usarlas. Las comprensiones son ideales para transformaciones y filtros sencillos. Si la expresión se vuelve larga, hay más de dos `for` o hace falta más de una condición compleja, un bucle normal es más legible. Tampoco deben usarse por sus efectos secundarios (por ejemplo, `[print(x) for x in lista]`): si no se necesita la colección resultante, se usa un `for`.

0.6 Funciones

Una función se define con `def`, seguida del nombre, los parámetros entre paréntesis y dos puntos. Con `return` devuelve un valor; si no hay `return`, devuelve `None`. La cadena entre triples comillas al principio es la documentación de la función (*docstring*), que muestran los editores y `help(media)`.

```python
def media(valores: list[float], decimales: int = 2) -> float:
    """Devuelve la media aritmética redondeada."""
    if not valores:
        raise ValueError("La lista está vacía")
    return round(sum(valores) / len(valores), decimales)

print(media([7, 8, 9]))
print(media([7, 8, 9], decimales=0))   # argumento por nombre
```

Las anotaciones `list[float]` y `-> float` indican los tipos de los parámetros y del resultado; las veremos con detalle en el apartado siguiente.

0.6.1 Argumentos posicionales, por nombre y valores por defecto

Al llamar a una función, los argumentos se pueden pasar de dos formas, que se pueden combinar (primero los posicionales):

- **Posicionales**: se asignan a los parámetros por orden. `media([7, 8], 1)`.
- **Por nombre** (*keyword arguments*): se indica el nombre del parámetro, y el orden da igual. `media(decimales=1, valores=[7, 8])`. Hacen las llamadas más legibles, sobre todo con valores booleanos o numéricos.

Un parámetro con **valor por defecto** (`decimales: int = 2`) es opcional en la llamada. Los parámetros con valor por defecto deben ir después de los que no lo tienen.

Cuidado. Nunca uses una lista o un diccionario como valor por defecto (`def f(datos=[])`): el valor por defecto se crea una sola vez y lo comparten todas las llamadas.

> Usa `None` y crea la colección dentro: `if datos is None: datos = []`. Volveremos sobre este problema en el apartado de *dataclasses*.

Una función puede devolver varios valores separándolos por comas; en realidad devuelve una tupla, que se puede desempaquetar:

```
def minimo_maximo(valores: list[float]) -> tuple[float, float]:
    return min(valores), max(valores)

menor, mayor = minimo_maximo([7, 3, 9])     # menor = 3, mayor = 9
```

0.6.2 Número variable de argumentos: `*args` y `**kwargs`

A veces una función debe aceptar **cualquier número de argumentos**, como hace `print`, que admite uno, dos o veinte. Para ello existen dos parámetros especiales:

- `*args`: recoge en una **tupla** todos los argumentos **posicionales** que sobran.
- `**kwargs`: recoge en un **diccionario** todos los argumentos **por nombre** que no corresponden a ningún parámetro (las claves son los nombres y los valores, los valores).

Lo que importa son los asteriscos; los nombres `args` y `kwargs` (*keyword arguments*) son sólo un convenio y se pueden cambiar por otros más descriptivos:

```
def sumar(*numeros: float) -> float:
    print(numeros)                    # una tupla
    return sum(numeros)

sumar()                               # ()          -> 0
sumar(4)                              # (4,)        -> 4
sumar(1, 2, 3)                        # (1, 2, 3)   -> 6

def crear_alumno(nombre: str, **datos) -> dict:
    print(datos)                      # un diccionario
    return {"nombre": nombre, **datos}

crear_alumno("Ana", edad=20, ciclo="DAM")
# {'edad': 20, 'ciclo': 'DAM'}  -> {'nombre': 'Ana', 'edad': 20, 'ciclo': 'DAM'}
```

Se pueden combinar con parámetros normales. El orden en la definición es siempre: parámetros normales, `*args`, parámetros que sólo pueden pasarse por nombre y `**kwargs`:

```
def registrar(mensaje: str, *etiquetas: str, nivel: str = "INFO", **datos) -> None:
    print(f"[{nivel}] {mensaje} {etiquetas} {datos}")

registrar("Alta", "alumno", "2DAM", id=3, nombre="Ana")
# [INFO] Alta ('alumno', '2DAM') {'id': 3, 'nombre': 'Ana'}
registrar("Error al guardar", nivel="ERROR")
# [ERROR] Error al guardar () {}
```

Como `nivel` va después de `*etiquetas`, sólo se puede indicar por nombre: cualquier argumento posicional extra irá a `etiquetas`. Para obligar a pasar ciertos parámetros por nombre sin admitir argumentos variables, se usa un `*` solo: `def conectar(host, *,`

`puerto=5432, usuario="dam")` obliga a escribir `conectar("localhost", puerto=5433)`.

Desempaquetar al llamar

Los asteriscos también se usan en la **llamada**, con el efecto contrario: `*` reparte los elementos de una secuencia como argumentos posicionales, y `**` reparte un diccionario como argumentos por nombre. Es muy habitual para pasar a una función datos que ya tenemos en una lista o en un diccionario (por ejemplo, una fila leída de un CSV o de una base de datos):

```
def presentar(nombre: str, edad: int, ciclo: str) -> str:
    return f"{nombre}, {edad} años, {ciclo}"

datos = ["Ana", 20, "DAM"]
print(presentar(*datos))              # presentar("Ana", 20, "DAM")

fila = {"nombre": "Luis", "edad": 22, "ciclo": "DAW"}
print(presentar(**fila))              # presentar(nombre="Luis", edad=22, ciclo="DAW")
```

El uso más frecuente de `*args, **kwargs` juntos es en funciones que **reenvían** sus argumentos a otra sin necesidad de conocerlos, como los decoradores que veremos un poco más adelante.

0.6.3 Funciones como objetos y funciones lambda

Las funciones son objetos: se pueden guardar en variables y pasar como argumento a otras funciones. Las **funciones lambda** son funciones anónimas de una sola expresión, útiles para pasar una función corta como argumento:

```
alumnos = [("Ana", 8), ("Luis", 4), ("Eva", 6)]
alumnos.sort(key=lambda a: a[1], reverse=True)   # ordenar por nota
# equivale a definir: def nota(a): return a[1]   y usar key=nota
```

0.7 Anotaciones de tipo

Python es un lenguaje de **tipado dinámico**: el tipo lo tienen los valores, no las variables, y el intérprete no comprueba los tipos antes de ejecutar. Aun así, desde Python 3.5 se pueden **anotar** los tipos esperados de variables, parámetros y valores devueltos. En este curso las usaremos siempre.

0.7.1 Sintaxis

```
edad: int = 20                        # variable
nombres: list[str] = []               # variable de tipo colección

def media(valores: list[float], decimales: int = 2) -> float:
    #      parámetro: tipo            valor por defecto  -> tipo devuelto
    return round(sum(valores) / len(valores), decimales)

def saludar(nombre: str) -> None:     # None: no devuelve nada
    print(f"Hola, {nombre}")
```

0.7.2 Tipos más habituales

Anotación	Significado
`int`, `float`, `str`, `bool`, `bytes`	Tipos básicos.
`list[str]`, `set[int]`	Lista de cadenas, conjunto de enteros.
`dict[str, float]`	Diccionario con claves `str` y valores `float`.
`tuple[int, str]`	Tupla de exactamente dos elementos: un entero y una cadena.
`tuple[int, ...]`	Tupla de enteros de cualquier longitud.
`int | None`	Un entero o `None` (antes se escribía `Optional[int]`).
`int | str`	Un entero o una cadena (antes, `Union[int, str]`).
`Alumno`, `"Alumno"`	Un objeto de una clase. Entre comillas si la clase todavía no está definida (por ejemplo, dentro de la propia clase).
`Any`	Cualquier tipo: desactiva la comprobación (`from typing import Any`).
`Callable[[int, int], int]`	Una función que recibe dos enteros y devuelve un entero (`from collections.abc import Callable`).
`Iterator[str]`	Un iterador o generador que produce cadenas (`collections.abc`).
`Literal["r", "w"]`	Sólo esos valores concretos (`from typing import Literal`).

Se pueden definir **alias** para tipos complejos y escribir funciones **genéricas**, que funcionan con cualquier tipo manteniendo la relación entre parámetros y resultado (sintaxis de Python 3.12):

```
type Notas = dict[str, list[float]]     # alias: nombre del alumno -> sus notas

def mejores(notas: Notas, minimo: float) -> list[str]:
    return [n for n, lista in notas.items() if min(lista) >= minimo]

def primero[T](elementos: list[T]) -> T:   # T: "el tipo que sea"
    return elementos[0]

x = primero([3, 4])          # el editor sabe que x es int
s = primero(["a", "b"])      # y que s es str
```

0.7.3 ¿Para qué sirven si no se comprueban?

Las anotaciones **no cambian la ejecución**: si se pasa un tipo distinto del anotado, Python no protesta (el error, si lo hay, aparecerá más tarde, cuando se use el valor). Su utilidad está en otra parte:

- **Documentación**: la firma de la función dice qué recibe y qué devuelve.
- **Ayuda del editor**: autocompletado y avisos mientras se escribe.
- **Comprobación estática**: herramientas como `mypy` (o la integrada en el editor) analizan el código *sin ejecutarlo* y detectan errores de tipos.
- **Bibliotecas que las leen en tiempo de ejecución**: las *dataclasses* generan el constructor a partir de ellas, Pydantic las usa para validar y convertir datos, y SQLAlchemy para

definir las columnas de las tablas (Tema 3). Se obtienen con `typing.get_type_hints()`, como veremos en el Tema 6.

```python
# tipos.py
def media(valores: list[float], decimales: int = 2) -> float:
    return round(sum(valores) / len(valores), decimales)

def buscar(nombre: str, agenda: dict[str, str]) -> str | None:
    return agenda.get(nombre)

nota = media([7, 8, 9], decimales="2")              # error de tipo
telefono = buscar("Ana", {"Ana": "600111222"})
print(telefono.upper())                              # ?`y si es None?
```

```
$ pip install mypy
$ mypy tipos.py
tipos.py:7: error: Argument "decimales" to "media" has incompatible type "str"; expected
    ↪  "int"
tipos.py:9: error: Item "None" of "str | None" has no attribute "upper"
Found 2 errors in 1 file (checked 1 source file)
```

`mypy` ha encontrado los dos errores sin ejecutar el programa. El segundo es especialmente valioso: obliga a tratar el caso en que `buscar` no encuentra nada, un error muy frecuente al trabajar con datos.

0.8 Decoradores

Un **decorador** es una función que recibe otra función y devuelve una versión modificada de ella, que hace algo más (antes, después o en lugar de la original) sin tocar su código. Se aplica escribiendo `@nombre` justo encima de la definición. Aparecerán enseguida en estos apuntes: `@contextmanager` en los gestores de contexto y `@property`, `@classmethod` y `@dataclass` en la programación orientada a objetos. Escribir uno sencillo ayuda a entender cómo funcionan:

```python
def anunciar(funcion):
    def envoltorio(*args, **kwargs):
        print(f"Empieza {funcion.__name__}")
        resultado = funcion(*args, **kwargs)      # se llama a la función original
        print(f"Termina {funcion.__name__}")
        return resultado
    return envoltorio

@anunciar
def sumar(a: int, b: int) -> int:
    return a + b

print(sumar(2, 3))
# Empieza sumar
# Termina sumar
# 5
```

Paso a paso:

1. `anunciar` recibe una función (`funcion`) y define dentro otra, `envoltorio`, que la "envuelve": muestra un mensaje, llama a la original con los mismos argumentos (`*args`,

`**kwargs`, apartado de funciones), muestra otro mensaje y devuelve el resultado.

2. `anunciar` devuelve `envoltorio`, **sin llamarla**.
3. Escribir `@anunciar` encima de `def sumar` es sólo una forma abreviada de escribir, después de definirla:

```
sumar = anunciar(sumar)     # "sumar" pasa a ser el envoltorio
```

4. Por eso, al llamar a `sumar(2, 3)`, en realidad se ejecuta `envoltorio(2, 3)`, que a su vez llama a la `sumar` original.

Los decoradores se usan para añadir a muchas funciones el mismo comportamiento: registrar llamadas, medir tiempos, comprobar permisos, reintentar operaciones que fallan... Por ejemplo, para medir cuánto tarda una consulta:

```
import time

def cronometrar(funcion):
    def envoltorio(*args, **kwargs):
        inicio = time.perf_counter()
        resultado = funcion(*args, **kwargs)
        print(f"{funcion.__name__} tardó {time.perf_counter() - inicio:.3f} s")
        return resultado
    return envoltorio

@cronometrar
def contar(n: int) -> int:
    return sum(range(n))

contar(1_000_000)          # contar tardó 0.022 s
```

> **Nota.** Como la función decorada es en realidad el envoltorio, pierde su nombre y su documentación: `sumar.__name__` vale `"envoltorio"`. Para conservarlos, en el código profesional se añade `@functools.wraps(funcion)` justo encima de `def envoltorio`. No cambia nada más del funcionamiento.

0.9 Módulos y paquetes

Cada fichero `.py` es un **módulo**. Una carpeta con módulos es un **paquete**. Para usar el código de otro módulo se importa:

```
import json                         # módulo completo
from pathlib import Path            # un nombre concreto
from datetime import date as Fecha  # con alias
```

Un módulo puede ejecutarse como programa o importarse desde otro. Para que cierto código se ejecute sólo en el primer caso se usa este patrón, que veremos en todos los ejemplos:

```
def main() -> None:
    print("Programa principal")

if __name__ == "__main__":
    main()
```

Una estructura típica de proyecto del curso será:

```
proyecto/
├── .venv/
├── requirements.txt
├── datos/
│   └── alumnos.csv
└── gestion/            <- paquete
    ├── __init__.py
    ├── modelo.py
    ├── repositorio.py
    └── main.py
```

Y se ejecuta desde la carpeta `proyecto` con `python -m gestion.main`.

0.10 Excepciones

Cuando algo va mal (un fichero no existe, una conexión falla, un dato no tiene el formato esperado) Python **lanza una excepción**. Si nadie la captura, el programa termina mostrando la traza del error. Este mecanismo es central en el módulo: el criterio *"se han previsto y gestionado las excepciones"* aparece en varios resultados de aprendizaje.

```python
def leer_entero(texto: str) -> int | None:
    try:
        valor = int(texto)
    except ValueError as e:
        print(f"'{texto}' no es un número: {e}")
        return None
    else:
        # se ejecuta sólo si no hubo excepción
        return valor
    finally:
        # se ejecuta SIEMPRE, haya excepción o no
        print("Conversión terminada")
```

Algunas excepciones habituales:

`ValueError`	Valor con tipo correcto pero contenido inválido (`int("abc")`).
`TypeError`	Operación con un tipo inadecuado (`"a" + 1`).
`KeyError` / `IndexError`	Clave de diccionario o índice de lista inexistente.
`FileNotFoundError`	El fichero no existe (subclase de `OSError`).
`PermissionError`	Sin permisos sobre un fichero (subclase de `OSError`).
`sqlite3.Error`	Base de la jerarquía de errores de SQLite (Tema 2).

Podemos lanzar excepciones con `raise` y definir las nuestras heredando de `Exception`:

```python
class AlumnoNoEncontrado(Exception):
    """El alumno solicitado no existe en el almacén de datos."""

def buscar(alumnos: dict[int, str], ident: int) -> str:
    if ident not in alumnos:
```

```
        raise AlumnoNoEncontrado(f"No existe el alumno {ident}")
    return alumnos[ident]
```

Cuidado. Evita el `except:` vacío o `except Exception:` sin más, porque ocultan errores de programación. Captura sólo las excepciones que sabes tratar y deja que el resto se propague.

0.10.1 La sentencia `assert`

`assert` comprueba que una condición se cumple y, si no es así, lanza un `AssertionError`. Se puede añadir un mensaje tras una coma:

```
def media(valores: list[float]) -> float:
    resultado = sum(valores) / len(valores)
    assert min(valores) <= resultado <= max(valores), "media fuera de rango"
    return resultado

assert media([4, 6]) == 5
assert media([4, 6]) == 6, "la media debería ser 6"
# AssertionError: la media debería ser 6
```

Su utilidad es expresar **suposiciones del programador**: condiciones que, si el programa es correcto, siempre deben cumplirse. Si alguna falla, el error está en el código, y `assert` lo detecta en el punto exacto donde aparece en lugar de dejar que se propague y provoque un fallo difícil de rastrear más adelante. Por eso es la herramienta básica para escribir **pruebas**: cada prueba ejecuta un fragmento de código y comprueba con `assert` que el resultado es el esperado. El Anexo E explica cómo organizar y ejecutar pruebas con `pytest`.

Cuidado. `assert` **no sirve para validar datos** que vienen de fuera del programa (lo que escribe el usuario, el contenido de un fichero, la respuesta de un servidor). Si se ejecuta Python con la opción de optimización (`python -O programa.py`), todas las sentencias `assert` se eliminan y la comprobación desaparece. Para validar datos externos se usa `if` y se lanza la excepción adecuada:

```
if edad < 0:
    raise ValueError("La edad no puede ser negativa")    # correcto
assert edad >= 0, "La edad no puede ser negativa"       # incorrecto para datos
    ↪ externos
```

0.11 Gestores de contexto: la sentencia `with`

Muchos recursos (ficheros, conexiones a bases de datos, transacciones) deben **liberarse siempre**, incluso si se produce un error. La sentencia `with` lo garantiza:

```
with open("datos.txt", "w", encoding="utf-8") as f:
    f.write("Hola\n")
# aquí el fichero ya está cerrado, aunque write() hubiera fallado
```

Un **gestor de contexto** es cualquier objeto que tenga los métodos `__enter__` y `__exit__`, y

por tanto puede usarse en una sentencia `with`. Al entrar en el bloque, `with` llama a `__enter__`, cuyo resultado se asigna a la variable que sigue a `as`. Al salir, llama siempre a `__exit__`, tanto si el bloque termina bien como si lanza una excepción; ahí es donde se liberan los recursos.

También se pueden crear gestores de contexto de forma sencilla con el decorador `contextlib.contextmanager`:

```
from contextlib import contextmanager
import time

@contextmanager
def cronometro(etiqueta: str):
    inicio = time.perf_counter()
    try:
        yield                       # aquí se ejecuta el bloque del with
    finally:
        print(f"{etiqueta}: {time.perf_counter() - inicio:.3f} s")

with cronometro("Carga de datos"):
    total = sum(range(10_000_000))
```

0.12 Programación orientada a objetos

0.12.1 Clases y objetos

```
class Alumno:
    centro = "IES Ejemplo"            # atributo de clase (compartido)

    def __init__(self, nombre: str, edad: int) -> None:
        self.nombre = nombre          # atributos de instancia
        self.edad = edad
        self.notas: list[float] = []

    def anadir_nota(self, nota: float) -> None:
        self.notas.append(nota)

    def media(self) -> float:
        return sum(self.notas) / len(self.notas) if self.notas else 0.0

    def __str__(self) -> str:         # representación para el usuario
        return f"{self.nombre} ({self.edad} años)"

    def __repr__(self) -> str:        # representación para el programador
        return f"Alumno(nombre={self.nombre!r}, edad={self.edad})"

a = Alumno("Ana", 20)
a.anadir_nota(8)
print(a, a.media())
```

- `__init__` es el inicializador (equivalente al constructor).
- `self` es la referencia al propio objeto y se declara explícitamente como primer parámetro.
- No existen `private` ni `protected`: la visibilidad se indica con guiones bajos en el nombre (lo vemos más abajo).

- Los métodos con doble guion bajo (`__str__`, `__eq__`, `__len__`...) se llaman **métodos especiales** y permiten que nuestros objetos se integren con el lenguaje.

0.12.2 Atributos de clase y atributos de instancia

- Un **atributo de instancia** pertenece a cada objeto. Se crea asignándolo a través de `self` (normalmente en `__init__`) y cada objeto tiene su propio valor.
- Un **atributo de clase** se define en el cuerpo de la clase, fuera de los métodos. Pertenece a la clase y lo **comparten todos sus objetos** (equivale a un atributo `static` de Java).

```
class Alumno:
    centro = "IES Ejemplo"          # atributo de clase
    total = 0                       # contador compartido

    def __init__(self, nombre: str) -> None:
        self.nombre = nombre        # atributo de instancia
        Alumno.total += 1           # se modifica a través de la clase

a, b = Alumno("Ana"), Alumno("Luis")
print(a.nombre, b.nombre)           # Ana Luis    -> uno por objeto
print(a.centro, b.centro)           # IES Ejemplo -> compartido
print(Alumno.total)                 # 2

Alumno.centro = "IES Nuevo"         # cambia para todos los objetos
a.centro = "Otro"                   # !`crea un atributo de INSTANCIA en a!
print(a.centro, b.centro)           # Otro IES Nuevo
```

Cuidado.

- Al leer `objeto.atributo`, Python busca primero en el objeto y, si no lo encuentra, en la clase. Pero al *asignar* `objeto.atributo = valor` siempre se crea o modifica un atributo de instancia. Para cambiar un atributo de clase, asígnalo a través de la clase (`Alumno.total += 1`).
- No uses listas o diccionarios como atributos de clase para guardar datos de cada objeto: al ser compartidos, lo que añada un objeto lo verán todos. Créalos en `__init__`.

Los métodos también pueden ser de clase: `@classmethod` recibe la clase (`cls`) en lugar del objeto y se usa a menudo como constructor alternativo (`Libro.desde_dict(d)`, Tema 1); `@staticmethod` no recibe ni la clase ni el objeto y es una función normal agrupada dentro de la clase.

0.12.3 Visibilidad: el guion bajo

Python no tiene modificadores de acceso como `private`, `protected` o `public`. La visibilidad de atributos y métodos se indica con **convenciones en el nombre**:

Nombre	Equivale a	Significado
`nombre`	público	Forma parte de la interfaz de la clase; se puede usar desde fuera.
`_nombre`	protegido / interno	Convenio: "es un detalle interno, no lo uses desde fuera". Python no lo impide, pero los editores avisan y `from modulo import *` no importa los nombres que empiezan por _.
`__nombre`	privado	Python **renombra** el atributo a `_Clase__nombre` (*name mangling*). Así se evita que una subclase lo sobrescriba por accidente; no es una protección real.
`__nombre__`	—	Métodos especiales del lenguaje (`__init__`, `__str__`...). No hay que inventar nombres nuevos con este formato.

```python
class Cuenta:
    def __init__(self, titular: str, saldo: float, pin: str) -> None:
        self.titular = titular        # público
        self._saldo = saldo           # interno: úsese a través de métodos
        self.__pin = pin              # "privado": se guarda como _Cuenta__pin

    def comprobar_pin(self, pin: str) -> bool:     # método público
        return self.__pin == pin                   # dentro de la clase, normal

    def _registrar(self, texto: str) -> None:      # método interno
        print(f"[{self.titular}] {texto}")

c = Cuenta("Ana", 100.0, "1234")
print(c.titular)          # Ana
print(c._saldo)           # 100.0 (funciona, pero no se debe hacer)
print(c.__pin)            # AttributeError: 'Cuenta' object has no attribute '__pin'
print(c._Cuenta__pin)     # 1234: el nombre real; la "privacidad" es un convenio
```

Nota. La filosofía de Python se resume en la frase "*we are all consenting adults here*": el lenguaje confía en que el programador respete los convenios. Para controlar de verdad cómo se lee o modifica un atributo se usan las **propiedades** del apartado siguiente.

0.12.4 Herencia

```python
class Persona:
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre

    def presentarse(self) -> str:
        return f"Soy {self.nombre}"

class Profesor(Persona):
    def __init__(self, nombre: str, departamento: str) -> None:
        super().__init__(nombre)
        self.departamento = departamento

    def presentarse(self) -> str:      # sobrescritura
```

```
        return f"{super().presentarse()} y trabajo en {self.departamento}"
```

0.12.5 Propiedades

En lugar de escribir *getters* y *setters* como en Java, Python permite acceder a los atributos directamente y, si más adelante hace falta validar, convertirlos en **propiedades** sin cambiar el código que los usa. Las propiedades serán muy importantes en el Tema 6.

```
class Producto:
    def __init__(self, nombre: str, precio: float) -> None:
        self.nombre = nombre
        self.precio = precio           # pasa por el setter

    @property
    def precio(self) -> float:
        return self._precio

    @precio.setter
    def precio(self, valor: float) -> None:
        if valor < 0:
            raise ValueError("El precio no puede ser negativo")
        self._precio = valor

p = Producto("Teclado", 25.0)
p.precio = 30            # llama al setter
p.precio = -1            # ValueError
```

Fíjate en que en el código aparecen dos nombres distintos, y es intencionado:

- `precio` es la **propiedad**: la interfaz pública de la clase. Toda lectura (`p.precio`) pasa por el *getter* y toda asignación (`p.precio = 30`) pasa por el *setter*.
- `_precio` es el **atributo interno** donde se guarda realmente el valor. Lleva guion bajo porque es un detalle de implementación (ver el apartado de visibilidad).

Por eso la línea `self.precio = precio` del constructor **no crea un atributo** `precio`: como `precio` es una propiedad definida en la clase, esa asignación llama al *setter*, que valida el valor y lo guarda en `self._precio`. La ventaja es que la validación se aplica también al crear el objeto:

```
p = Producto("Teclado", 25.0)
print(vars(p))           # {'nombre': 'Teclado', '_precio': 25.0}
Producto("Ratón", -5)         # ValueError: El precio no puede ser negativo
```

> **Cuidado.** Dentro del *getter* y del *setter* hay que usar el atributo interno (`self._precio`), nunca la propiedad. Si en el *setter* se escribiera `self.precio = valor`, esa asignación volvería a llamar al *setter*, que se llamaría a sí mismo una y otra vez hasta provocar un `RecursionError`.

0.12.6 Dataclasses

Para clases cuyo propósito principal es **guardar datos** (lo que en Java serían los POJO o los *records*), el decorador `@dataclass` genera automáticamente `__init__`, `__repr__` y `__eq__`. Las

usaremos constantemente para representar los registros que leemos de ficheros y bases de datos.

```
from dataclasses import dataclass, field, asdict
from datetime import date

@dataclass
class Libro:
    isbn: str
    titulo: str
    autor: str
    anio: int
    disponible: bool = True
    etiquetas: list[str] = field(default_factory=list)

l = Libro("978-84-00000-00-0", "El Quijote", "Cervantes", 1605)
print(l)              # Libro(isbn='978-...', titulo='El Quijote', ...)
print(asdict(l))      # convierte a diccionario (útil para JSON)

@dataclass(frozen=True)     # inmutable
class Punto:
    x: float
    y: float
```

La función `field`

En el ejemplo, el valor por defecto del campo `etiquetas` no es una lista vacía, sino una llamada a `field(default_factory=list)`. La función `field` del módulo `dataclasses` permite configurar un campo concreto cuando no basta con escribir su tipo y un valor por defecto.

Su uso más habitual es dar como valor por defecto una **colección vacía**. En Python, los valores por defecto se crean **una sola vez**, al definir la clase o la función, no cada vez que se usan. Si ese valor es una lista, todos los objetos comparten la misma lista, y lo que añade uno lo ven todos. Con una clase normal es un error clásico y silencioso:

```
class LibroNormal:
    def __init__(self, titulo: str, etiquetas: list[str] = []):   # !`una sola lista!
        self.titulo = titulo
        self.etiquetas = etiquetas

x, y = LibroNormal("Nada"), LibroNormal("La colmena")
x.etiquetas.append("novela")
print(y.etiquetas)        # ['novela']  <- y también la tiene
```

Las *dataclasses* impiden este error: si se escribe `etiquetas: list[str] = []`, la clase ni siquiera se crea y se produce `ValueError: mutable default <class 'list'> for field etiquetas is not allowed: use default_factory`. La solución es `default_factory`, que recibe una **función sin argumentos** y la llama **cada vez que se crea un objeto**:

```
a, b = Libro("1", "Nada", "Laforet", 1945), Libro("2", "La colmena", "Cela", 1951)
a.etiquetas.append("novela")
print(b.etiquetas)                  # []: cada libro tiene su propia lista
print(a.etiquetas is b.etiquetas)   # False
```

Para diccionarios y conjuntos se hace lo mismo, pasando `dict` o `set` como `default_factory`.

Con valores inmutables (números, cadenas, `None`, tuplas) no hace falta: `disponible: bool = True` es correcto.

`field` admite otras opciones útiles:

`field(default_factory=f)`	El valor por defecto se obtiene llamando a `f()` para cada objeto.
`field(repr=False)`	El campo no aparece al imprimir el objeto (útil para contraseñas o datos internos).
`field(compare=False)`	El campo no se tiene en cuenta al comparar objetos con ==.
`field(init=False)`	El campo no es un parámetro del constructor (se calcula después, por ejemplo en `__post_init__`).

0.12.7 Clases de datos con validación: Pydantic

Las *dataclasses* de la biblioteca estándar **no comprueban** que los valores coincidan con las anotaciones: `Libro("978-...", "Nada", "Laforet", "1945")` crea un libro cuyo año es la cadena `"1945"`. Cuando los datos llegan de fuera del programa (un fichero, un formulario, una API, una base de datos documental), conviene validarlos. Para eso se usa la biblioteca Pydantic, que ofrece dos opciones con la misma sintaxis basada en anotaciones:

```
$ pip install pydantic
```

Dataclasses de Pydantic. Se usan exactamente igual que las de la biblioteca estándar, cambiando sólo el `import`, pero validan y convierten los datos al crear el objeto:

```
from pydantic import ValidationError
from pydantic.dataclasses import dataclass    # en lugar de "from dataclasses"

@dataclass
class Libro:
    titulo: str
    anio: int

print(Libro("Nada", "1945"))      # Libro(titulo='Nada', anio=1945): convierte "1945"
try:
    Libro("Nada", "mil")
except ValidationError as e:
    print(e.errors()[0]["msg"])   # Input should be a valid integer...
```

`BaseModel`. Es la forma principal de usar Pydantic. Además de validar, permite añadir restricciones a cada campo y convertir los objetos desde y hacia diccionarios y JSON directamente. Lo usaremos en el Tema 1 para leer y escribir ficheros JSON.

```
from datetime import date
from pydantic import BaseModel, Field, ValidationError

class Libro(BaseModel):
    titulo: str = Field(min_length=1)
    anio: int = Field(ge=1450)                # restricción: >= 1450
    publicado: date | None = None
```

```python
l = Libro(titulo="Nada", anio="1945", publicado="1945-05-01")   # convierte tipos
print(l.publicado.year)                        # 1945: es un objeto date
print(l.model_dump())                          # -> dict
print(l.model_dump_json())                     # -> '{"titulo":"Nada","anio":1945,...}'
l2 = Libro.model_validate_json('{"titulo": "La colmena", "anio": 1951}')

try:
    Libro(titulo="", anio=1000)
except ValidationError as e:
    print(len(e.errors()), "errores")          # 2 errores: título vacío y año
```

Nota. Por defecto, Pydantic valida al *crear* el objeto, pero no cuando después se asigna un atributo (`l.anio = "abc"` no da error). Si se necesita, se activa con `model_config = ConfigDict(validate_assignment=True)`.

0.12.8 Comparación de las clases para almacenar datos

	Clase normal	**`@dataclass`**	**`@dataclass` de Pydantic**	**`BaseModel`**
Origen	Lenguaje	Biblioteca estándar	Pydantic	Pydantic
`__init__`, `__repr__`, `__eq__`	Se escriben a mano	Generados	Generados	Generados
Comprueba los tipos al crear	No (salvo que se programe)	No	Sí	Sí
Convierte tipos (`"1945"` → `1945`)	No	No	Sí	Sí
Restricciones por campo (mínimo, longitud, patrón)	Con propiedades	No	Sí (`Field`)	Sí (`Field`)
Conversión a `dict` / JSON	A mano	`asdict()` / a mano	Con `TypeAdapter`	`model_dump()` `model_dump_json()`
Lectura desde JSON	A mano	A mano	Con `TypeAdapter`	`model_validate_json()`
Generación de JSON Schema	No	No	Sí	`model_json_schema()`
Inmutabilidad	A mano	`frozen=True`	`frozen=True`	`frozen=True` en la configuración
Coste	Mínimo	Mínimo	Validar tiene un coste	Validar tiene un coste
Cuándo usarla	Objetos con comportamiento y estado propio (un servicio, un repositorio)	Datos internos del programa, ya fiables	Igual que `@dataclass`, pero con datos de origen externo	Datos que entran o salen del programa: ficheros, API, configuración

Existen otras opciones que verás en el curso: `typing.NamedTuple` (una tupla inmutable con nombres de campo, que usaremos en el Tema 4) y, cuando no se necesita una clase, un simple

`dict`, que es como llegan los datos de JSON y MongoDB.

0.13 Iteradores y generadores

Un objeto es **iterable** si se puede recorrer con `for`. Los ficheros, los cursores de base de datos y los resultados de muchas consultas son iterables: los datos se van obteniendo *a medida que se piden*, sin cargarlo todo en memoria.

Una función que contiene `yield` es un **generador**: cada vez que se le pide un valor, se ejecuta hasta el siguiente `yield` y se detiene.

```
def lineas_no_vacias(ruta: str):
    with open(ruta, encoding="utf-8") as f:
        for linea in f:
            linea = linea.strip()
            if linea:
                yield linea

for l in lineas_no_vacias("datos.txt"):
    print(l)
```

Las **expresiones generadoras** son como las comprensiones de listas pero con paréntesis, y no construyen la lista en memoria:

```
total = sum(len(l) for l in lineas_no_vacias("datos.txt"))
```

0.14 Enumeraciones

Una **enumeración** define un conjunto cerrado de valores con nombre. Es más segura que usar cadenas o números sueltos, porque un error de escritura se detecta en lugar de pasar inadvertido.

```
from enum import Enum

class Estado(Enum):
    PENDIENTE = "P"
    ENVIADO = "E"
    ENTREGADO = "D"

e = Estado("E")          # Estado.ENVIADO (a partir del valor guardado)
print(e.name, e.value)   # ENVIADO E
```

0.15 Fechas y horas

Prácticamente todas las aplicaciones de acceso a datos manejan fechas: de nacimiento, de alta, de un pedido, de la última modificación de un fichero… Y casi todos los formatos y gestores que veremos las representan de forma distinta. Merece la pena conocer bien el módulo `datetime` de la biblioteca estándar.

0.15.1 Los tipos de `datetime`

Tipo	Representa	Ejemplo
`date`	Una fecha (año, mes, día).	`2026-10-19`
`time`	Una hora del día (hora, minuto, segundo, microsegundo).	`09:30:00`
`datetime`	Una fecha con su hora: un instante concreto.	`2026-10-19 09:30:00`
`timedelta`	Una **duración**: la diferencia entre dos fechas u horas.	`67 days, 0:00:00`

Todos ellos son **inmutables**: las operaciones no modifican el objeto, sino que devuelven uno nuevo.

0.15.2 Crear fechas y acceder a sus partes

```
from datetime import date, time, datetime, timedelta

hoy = date.today()                        # fecha actual
ahora = datetime.now()                    # fecha y hora actuales

d = date(2026, 10, 19)                    # año, mes, día
t = time(9, 30)                           # hora, minuto (segundos = 0)
dt = datetime(2026, 10, 19, 9, 30, 15)    # año, mes, día, hora, minuto, segundo
print(datetime.combine(d, t))             # 2026-10-19 09:30:00

print(d.year, d.month, d.day)             # 2026 10 19
print(d.weekday())                        # 0 -> lunes (0 = lunes ... 6 = domingo)
print(d.isoweekday())                     # 1 -> lunes (1 = lunes ... 7 = domingo)
print(dt.date(), dt.time())               # 2026-10-19 09:30:15
print(dt.hour, dt.minute, dt.second)      # 9 30 15

print(dt.replace(day=1))                  # 2026-10-01 09:30:15 (nuevo objeto)
print(dt.replace(hour=0, minute=0, second=0))   # 2026-10-19 00:00:00
```

Crear una fecha imposible lanza `ValueError` (por ejemplo, `date(2026, 2, 31)`), lo que resulta muy útil para validar datos.

0.15.3 Operaciones: duraciones y comparaciones

Sumar o restar un `timedelta` a una fecha da otra fecha; restar dos fechas da un `timedelta`:

```
d = date(2026, 10, 19)
print(d + timedelta(days=30))             # 2026-11-18
print(d - timedelta(weeks=2))             # 2026-10-05

hasta_navidad = date(2026, 12, 25) - d
print(hasta_navidad)                      # 67 days, 0:00:00
print(hasta_navidad.days)                 # 67

jornada = datetime(2026, 10, 19, 14, 0) - datetime(2026, 10, 19, 9, 30)
print(jornada)                            # 4:30:00
print(jornada.total_seconds() / 3600)     # 4.5 (horas)
```

```
print(date(2026, 1, 1) < d)              # True: las fechas se comparan
print(max(date(2025, 5, 1), d))          # 2026-10-19
```

Cuidado con los meses y los años. `timedelta` admite días, semanas, horas, minutos y segundos, pero **no meses ni años**, porque no duran siempre lo mismo. Tampoco conviene calcular la edad dividiendo los días entre 365, porque los años bisiestos acumulan error justo alrededor del cumpleaños:

```
nacimiento, hoy = date(2008, 10, 20), date(2026, 10, 19)
print((hoy - nacimiento).days // 365)    # 18  !`incorrecto: cumple 18 mañana!

def edad(nacimiento: date, hoy: date) -> int:
    ya_cumplio = (hoy.month, hoy.day) >= (nacimiento.month, nacimiento.day)
    return hoy.year - nacimiento.year - (0 if ya_cumplio else 1)

print(edad(nacimiento, hoy))             # 17  correcto
```

0.15.4 De fecha a texto y de texto a fecha

Para mostrar una fecha con un formato concreto se usa el método `strftime` (*string format time*); para interpretar un texto como fecha, `datetime.strptime` (*string parse time*). Ambos usan los mismos códigos de formato:

Código	Significado	Ejemplo
`%d` / `%m` / `%Y`	Día / mes (2 dígitos) / año (4 dígitos)	`19, 10, 2026`
`%y`	Año con 2 dígitos	`26`
`%H` / `%M` / `%S`	Hora (00–23) / minutos / segundos	`09, 30, 15`
`%f`	Microsegundos	`000000`
`%A` / `%a`	Nombre del día / abreviado	`Monday, Mon`
`%B` / `%b`	Nombre del mes / abreviado	`October, Oct`
`%j`	Día del año (001–366)	`292`
`%z` / `%Z`	Desfase horario / nombre de la zona	`+0200, CEST`

```
dt = datetime(2026, 10, 19, 9, 30, 15)
print(dt.strftime("%d/%m/%Y %H:%M"))             # 19/10/2026 09:30
print(f"{dt:%d/%m/%Y}")                          # 19/10/2026 (lo mismo en una f-string)

leida = datetime.strptime("19/10/2026 09:30", "%d/%m/%Y %H:%M")
print(leida)                                     # 2026-10-19 09:30:00

try:
    datetime.strptime("31/02/2026", "%d/%m/%Y")
except ValueError as e:
    print("Fecha no válida:", e)                 # day is out of range for month
```

Nota. Los nombres de días y meses (`%A`, `%B`) salen en inglés salvo que se cambie la configuración regional con el módulo `locale`, cuyo resultado depende del sistema

operativo. Para programas que deben funcionar igual en cualquier equipo, es más fiable usar listas propias de nombres en español.

0.15.5 ISO 8601: el formato para guardar e intercambiar fechas

El estándar **ISO 8601** escribe las fechas de mayor a menor: `2026-10-19` y `2026-10-19T09:30:15`. Es el formato que debes usar siempre que guardes o intercambies fechas como texto (ficheros CSV, JSON, XML, APIs), porque no es ambiguo (¿`05/04` es 5 de abril o 4 de mayo?), lo entienden todos los lenguajes y gestores y, además, al ordenar las cadenas alfabéticamente quedan en orden cronológico.

```
print(d.isoformat())                              # 2026-10-19
print(dt.isoformat())                             # 2026-10-19T09:30:15
print(date.fromisoformat("2026-10-19"))           # objeto date
print(datetime.fromisoformat("2026-10-19T09:30:00"))
```

0.15.6 Zonas horarias

Un `datetime` puede ser de dos tipos:

- **Ingenuo** (*naive*): no sabe a qué zona horaria pertenece. `datetime.now()` y `datetime(2026, 10, 19, 9, 30)` son ingenuos. Las 09:30, pero ¿de dónde?
- **Consciente** (*aware*): incluye su zona horaria en el atributo `tzinfo`, y por tanto representa un instante exacto.

Mientras un programa sólo trabaje en una zona, las fechas ingenuas son suficientes. Pero en cuanto los datos se comparten entre equipos, servidores o países (una base de datos en la nube, una API), hay que usar fechas conscientes. Las zonas horarias se obtienen con el módulo `zoneinfo`, que además aplica automáticamente los cambios de horario de verano e invierno:

```
from datetime import datetime, timezone
from zoneinfo import ZoneInfo

madrid = ZoneInfo("Europe/Madrid")
reunion = datetime(2026, 10, 19, 9, 30, tzinfo=madrid)
print(reunion.isoformat())             # 2026-10-19T09:30:00+02:00 (horario de verano)
print(datetime(2026, 12, 19, 9, 30, tzinfo=madrid).isoformat())
                                       # 2026-12-19T09:30:00+01:00 (horario de invierno)

print(reunion.astimezone(timezone.utc))                    # 2026-10-19 07:30:00+00:00
print(reunion.astimezone(ZoneInfo("America/Mexico_City")))   # 2026-10-19 01:30:00-06:00

ahora_utc = datetime.now(timezone.utc)   # instante actual en UTC (consciente)
```

Cuidado.

- No se pueden comparar ni restar fechas ingenuas y conscientes: `reunion < datetime(2026, 10, 19, 9, 30)` lanza `TypeError: can't compare offset-naive and offset-aware datetimes`.
- La práctica recomendada es **guardar siempre en UTC** y convertir a la hora local

sólo al mostrar los datos al usuario.

- En Windows, `zoneinfo` puede necesitar el paquete `tzdata` (`pip install tzdata`).

0.15.7 Fechas en los formatos y gestores del curso

Dónde	Cómo se guardan las fechas
CSV, JSON, XML (Tema 1)	No tienen tipo fecha: se guardan como **texto en ISO 8601** y se convierten con `isoformat()` / `fromisoformat()`.
SQLite (Tema 2)	Tampoco tiene tipo fecha: se guardan como texto ISO 8601 (`TEXT`).
PostgreSQL (Tema 2)	Tipos `DATE`, `TIME`, `TIMESTAMP` y `TIMESTAMPTZ` (con zona horaria). psycopg los convierte automáticamente en `date` y `datetime`.
SQLAlchemy (Tema 3)	Basta anotar el atributo como `Mapped[date]` o `Mapped[datetime]`.
MongoDB (Tema 5)	Tipo `Date` de BSON: siempre un instante en UTC con precisión de milisegundos; no existe un tipo sólo fecha.

0.16 Argumentos de línea de comandos

Muchos de los programas del curso serán **utilidades de terminal**: un conversor de formatos, un importador de datos, un programa de copias de seguridad... Lo habitual es que reciban sus datos como **argumentos** al ejecutarlos:

```
$ python convertir.py alumnos.csv -f json -o alumnos.json --sobrescribir
```

0.16.1 `sys.argv`

Python guarda los argumentos en la lista `sys.argv`: el primer elemento es el nombre del programa y el resto, las palabras escritas detrás, siempre como cadenas.

```
import sys
print(sys.argv)
# ['convertir.py', 'alumnos.csv', '-f', 'json', '-o', 'alumnos.json', '--sobrescribir']
```

Procesar esta lista a mano es sencillo para un argumento, pero en cuanto hay opciones, valores por defecto, argumentos que pueden aparecer en cualquier orden o errores de escritura, se vuelve largo y propenso a fallos. Para eso existe el módulo `argparse` de la biblioteca estándar: describimos qué argumentos acepta el programa y él se encarga de leerlos, convertirlos, validarlos, mostrar errores claros y generar la ayuda.

0.16.2 Un ejemplo completo

```
# convertir.py
import argparse
from pathlib import Path
```

```python
def crear_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="convertir",
        description="Convierte ficheros de datos entre CSV, JSON y XML.")
    parser.add_argument("entrada", type=Path, help="fichero de entrada")
    parser.add_argument("-o", "--salida", type=Path, metavar="FICHERO",
                        help="fichero de salida (por defecto, el mismo nombre con otra
    ↪ extensión)")
    parser.add_argument("-f", "--formato", choices=["csv", "json", "xml"], required=True,
    ↪
                        help="formato de salida")
    parser.add_argument("-d", "--delimitador", default=";",
                        help="separador de campos para CSV (por defecto: %(default)s)")
    parser.add_argument("--sobrescribir", action="store_true",
                        help="sobrescribe el fichero de salida si ya existe")
    parser.add_argument("-v", "--verbose", action="count", default=0,
                        help="muestra más información (-v, -vv)")
    return parser

def main(argv: list[str] | None = None) -> int:
    parser = crear_parser()
    args = parser.parse_args(argv)           # None -> usa sys.argv
    if not args.entrada.exists():
        parser.error(f"no existe el fichero {args.entrada}")
    salida = args.salida or args.entrada.with_suffix("." + args.formato)
    if salida.exists() and not args.sobrescribir:
        parser.error(f"{salida} ya existe (usa --sobrescribir)")
    if args.verbose >= 1:
        print(f"Convirtiendo {args.entrada} -> {salida}")
    # ... aquí iría la conversión (Tema 1) ...
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

`parse_args()` devuelve un objeto con un atributo por argumento (`args.entrada`, `args.formato`...), ya convertido al tipo indicado. Los guiones del nombre largo se convierten en guiones bajos (`--orden-por` → `args.orden_por`).

0.16.3 Ayuda y errores automáticos

Sin escribir ni una línea más, el programa responde a `-h` o `--help`:

```
$ python convertir.py -h
usage: convertir [-h] [-o FICHERO] -f {csv,json,xml} [-d DELIMITADOR]
                 [--sobrescribir] [-v]
                 entrada

Convierte ficheros de datos entre CSV, JSON y XML.

positional arguments:
  entrada               fichero de entrada

options:
  -h, --help            show this help message and exit
  -o, --salida FICHERO  fichero de salida (por defecto, el mismo nombre con
```

```
                        otra extensión)
  -f, --formato {csv,json,xml}
                        formato de salida
  -d, --delimitador DELIMITADOR
                        separador de campos para CSV (por defecto: ;)
  --sobrescribir        sobrescribe el fichero de salida si ya existe
  -v, --verbose         muestra más información (-v, -vv)
```

Y si el usuario se equivoca, muestra el uso correcto y un mensaje de error, y termina con el **código de salida 2** (el convenio para "uso incorrecto"):

```
$ python convertir.py alumnos.csv
convertir: error: the following arguments are required: -f/--formato
$ python convertir.py alumnos.csv -f yaml
convertir: error: argument -f/--formato: invalid choice: 'yaml' (choose from 'csv', '
    ↪ json', 'xml')
$ python convertir.py noexiste.csv -f json
convertir: error: no existe el fichero noexiste.csv
```

El último error lo genera nuestro propio código con `parser.error()`, que produce el mismo formato y el mismo código de salida. Los textos fijos (*usage*, *options*…) aparecen en inglés, y su formato exacto puede variar ligeramente entre versiones de Python.

0.16.4 Tipos de argumentos y opciones de `add_argument`

- **Posicionales** (`"entrada"`): obligatorios, se identifican por su posición.
- **Opcionales** (`"-o", "--salida"`): empiezan por guiones, pueden ir en cualquier orden y tener forma corta y larga. Son opcionales salvo que se indique `required=True`.

Parámetro	Para qué sirve
`help`	Texto que aparece en la ayuda. `%(default)s` se sustituye por el valor por defecto.
`type`	Función que convierte el texto: `int`, `float`, `Path`… Si falla, se muestra un error.
`default`	Valor si el argumento no aparece (si no se indica, `None`).
`required`	Hace obligatorio un argumento opcional.
`choices`	Lista de valores permitidos.
`metavar`	Nombre del valor en la ayuda (`FICHERO` en lugar de `SALIDA`).
`dest`	Nombre del atributo en `args`, si no se quiere el deducido.
`action="store_true"`	Interruptor sin valor: `True` si aparece, `False` si no.
`action="count"`	Cuenta las repeticiones: `-v` = 1, `-vv` = 2.
`action="append"`	Opción repetible: acumula los valores en una lista.
`nargs`	Número de valores: `"?"` (0 o 1), `"*"` (0 o más), `"+"` (1 o más) o un número.

```
parser = argparse.ArgumentParser(prog="unir")
parser.add_argument("ficheros", nargs="+", help="uno o más ficheros")
parser.add_argument("-e", "--excluir", action="append", default=[],
                    help="columna a excluir (se puede repetir)")
```

```
grupo = parser.add_mutually_exclusive_group()     # opciones incompatibles entre sí
grupo.add_argument("--json", action="store_true")
grupo.add_argument("--csv", action="store_true")

args = parser.parse_args(["a.csv", "b.csv", "-e", "id", "-e", "dni", "--json"])
print(args)   # Namespace(ficheros=['a.csv', 'b.csv'], excluir=['id', 'dni'], json=True,
    ↪ csv=False)
# unir a.csv --json --csv  ->  error: argument --csv: not allowed with argument --json
```

0.16.5 Subcomandos

Cuando un programa hace varias cosas distintas, lo más claro es organizarlo en **subcomandos**, como `git add` y `git commit` o `pip install` y `pip list`. Cada subcomando tiene sus propios argumentos:

```
# agenda.py
import argparse

def crear_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(prog="agenda", description="Gestión de contactos")
    parser.add_argument("--fichero", default="agenda.json", help="fichero de datos")
    sub = parser.add_subparsers(dest="orden", required=True, metavar="ORDEN")

    alta = sub.add_parser("alta", help="añade un contacto")
    alta.add_argument("nombre")
    alta.add_argument("telefono")
    alta.add_argument("--email")

    listar = sub.add_parser("listar", help="muestra los contactos")
    listar.add_argument("--orden-por", choices=["nombre", "telefono"], default="nombre")

    borrar = sub.add_parser("borrar", help="elimina un contacto")
    borrar.add_argument("nombre")
    return parser

def main(argv: list[str] | None = None) -> int:
    args = crear_parser().parse_args(argv)
    match args.orden:
        case "alta":
            print(f"Alta de {args.nombre} ({args.telefono}) en {args.fichero}")
        case "listar":
            print(f"Listado ordenado por {args.orden_por}")
        case "borrar":
            print(f"Baja de {args.nombre}")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

```
$ python agenda.py alta Ana 600111222 --email ana@ejemplo.es
Alta de Ana (600111222) en agenda.json
$ python agenda.py --fichero otra.json borrar Ana
Baja de Ana
$ python agenda.py alta -h
```

```
usage: agenda alta [-h] [--email EMAIL] nombre telefono
...
```

0.16.6 Estructura recomendada y pruebas

En los ejemplos anteriores, el programa está organizado en una función `main(argv=None)` que devuelve el código de salida:

- Al ejecutarlo desde la terminal, `argv` vale `None` y `parse_args` lee `sys.argv`.
- En una prueba, se le pasa la lista de argumentos directamente, sin necesidad de lanzar un proceso.
- `raise SystemExit(main())` hace que el programa termine con el código que devuelve `main`: 0 si todo ha ido bien y otro valor si ha habido un error, lo que permite usarlo en *scripts* y tareas programadas.

```python
# test_convertir.py
import pytest
from convertir import main

def test_formato_no_valido(capsys):
    with pytest.raises(SystemExit) as e:      # argparse termina el programa
        main(["datos.csv", "-f", "yaml"])
    assert e.value.code == 2
    assert "invalid choice" in capsys.readouterr().err

def test_conversion(tmp_path, capsys):
    entrada = tmp_path / "datos.csv"
    entrada.write_text("a;b\n1;2\n", encoding="utf-8")
    assert main([str(entrada), "-f", "json", "-v"]) == 0
    assert "datos.json" in capsys.readouterr().out
```

0.17 Ejercicios

Ejercicio 0.1 — Calificaciones

Dificultad: ★

Escribe un programa que pida por teclado notas numéricas hasta que el usuario introduzca una línea vacía. Si una entrada no es un número o no está entre 0 y 10, muestra un aviso y sigue pidiendo. Al terminar, muestra cuántas notas se han introducido, la media, la máxima, la mínima y cuántas hay de cada calificación (Suspenso, Aprobado, Notable, Sobresaliente) usando un diccionario.

Ejercicio 0.2 — Comprensiones

Dificultad: ★

Dada la lista `palabras = ["python", "Java", "sql", "XML", "json", "Kotlin"]`, obtén usando comprensiones:

a) Una lista con las palabras en minúsculas.

b) Un diccionario *palabra* → *longitud*.

c) Un conjunto con las letras iniciales (en mayúsculas).

d) La lista de palabras con más de cuatro letras, ordenada alfabéticamente sin distinguir mayúsculas.

Ejercicio 0.3 — Funciones y excepciones

Dificultad: ★★

Implementa una función `validar_dni(dni: str) -> str` que reciba un DNI (8 dígitos y una letra), compruebe que el formato es correcto y que la letra coincide con la calculada (`"TRWAGMYFPDXBNJZSQVHLCKE"[numero % 23]`). Si es válido, lo devuelve normalizado en mayúsculas; si no, lanza una excepción propia `DNIInvalido` con un mensaje que explique el problema. Escribe un pequeño programa que pruebe la función con varios DNI correctos e incorrectos.

Ejercicio 0.4 — Clases

Dificultad: ★★

Crea una clase `CuentaBancaria` con titular, IBAN y saldo. El saldo debe ser una propiedad de sólo lectura; las operaciones `ingresar` y `retirar` lanzarán `ValueError` si la cantidad no es positiva o si no hay saldo suficiente. Añade un historial de movimientos (fecha, tipo, cantidad) e implementa `__str__` y `__len__` (número de movimientos). Crea después una subclase `CuentaAhorro` con un tipo de interés y un método `aplicar_intereses()`.

Ejercicio 0.5 — Dataclasses y ordenación

Dificultad: ★★

Define una *dataclass* `Pelicula` (título, director, año, duración en minutos, géneros). Crea una lista con al menos ocho películas y escribe funciones que:

a) Devuelvan las películas de un género dado, ordenadas por año.

b) Calculen la duración media por director (diccionario).

c) Devuelvan, mediante un generador, las películas que duran más de un valor dado.

Ejercicio 0.6 — Fechas

Dificultad: ★★

Escribe un módulo `fechas_util.py` con estas funciones y pruébalas con `pytest`:

a) `edad(nacimiento, hoy=None) -> int`, correcta también el día anterior y el día del cumpleaños, y para nacidos un 29 de febrero.

b) `leer_fecha(texto) -> date`, que acepte los formatos `19/10/2026`, `19-10-2026` y `2026-10-19` y lance `ValueError` con un mensaje claro si el texto no es una fecha válida.

c) `dias_laborables(inicio, fin) -> int`, que cuente los días de lunes a viernes entre dos fechas (ambas incluidas).

d) `a_utc(texto, zona="Europe/Madrid") -> datetime`, que reciba una fecha y hora local como `"25/03/2026 10:00"` y devuelva el instante equivalente en UTC. Comprueba el resultado antes y después del cambio de horario de marzo.

Ejercicio 0.7 — Utilidad de línea de comandos

Dificultad: ★★

Usando la clase `Pelicula` del ejercicio anterior, escribe un programa `peliculas.py` con `argparse` y tres subcomandos: `listar` (con opciones `--genero`, `--desde AÑO` y `--orden` con los valores `titulo`, `anio` o `duracion`), `buscar TEXTO` (busca en el título sin distinguir mayúsculas) y `estadisticas` (número de películas y duración media por director). Añade una opción global `-v` que se pueda repetir para mostrar más detalle. Organízalo con una función `main(argv=None)` y escribe al menos tres pruebas con `pytest`, una de ellas para un argumento no válido.

Ejercicio 0.8 — Gestor de contexto y decorador

Dificultad: ★★★

a) Escribe un decorador `@reintentar(veces=3)` que vuelva a ejecutar la función decorada si lanza una excepción, hasta el número de veces indicado, y relance la última excepción si todos los intentos fallan. (Pista: un decorador con parámetros es una función que devuelve un decorador.)

b) Escribe un gestor de contexto `directorio_temporal()` que cree una carpeta temporal, la devuelva para usarla dentro del `with` y la borre al salir. Puedes usar `tempfile.mkdtemp` y `shutil.rmtree`.

Proyecto: Agenda de contactos en memoria

Desarrolla una aplicación de consola para gestionar una agenda de contactos. En este tema los datos sólo se guardan en memoria; en el Tema 1 la ampliaremos para que persistan en ficheros.

Requisitos:

1. Un paquete `agenda` con, al menos, los módulos `modelo.py`, `servicio.py` y `main.py`.
2. En `modelo.py`, una *dataclass* `Contacto` (id, nombre, apellidos, teléfono, email, fecha de nacimiento opcional, etiquetas).
3. En `servicio.py`, una clase `Agenda` con métodos para añadir, buscar por id, buscar por texto (nombre o apellidos, sin distinguir mayúsculas), modificar, eliminar y listar ordenando por cualquier campo.
4. Excepciones propias: `ContactoNoEncontrado` y `ContactoDuplicado` (mismo email).
5. Validación del teléfono (9 dígitos) y del email (debe contener una sola `@` y un punto después) mediante propiedades o en `__post_init__`.
6. En `main.py`, un menú con `match` que use la clase `Agenda` y nunca termine con una traza de error: todos los errores previstos se muestran al usuario con un mensaje claro.
7. Una opción que muestre los próximos cumpleaños de los siguientes 30 días.

Entrega: el código del paquete y un fichero `README.md` con instrucciones para ejecutarlo.

TEMA 1

Manejo de ficheros

En este tema aprenderás a…

- Gestionar ficheros y directorios: crear, borrar, copiar, mover, recorrer y consultar sus propiedades.
- Distinguir entre acceso secuencial y acceso aleatorio, y entre flujos de bytes y de caracteres.
- Leer y escribir ficheros de texto, binarios, de registros de longitud fija, CSV y, sobre todo, JSON.
- Serializar objetos en JSON, validar documentos con JSON Schema y vincularlos a clases.
- Leer, modificar y validar documentos XML con ElementTree, XPath básico y XSD, y conocer los modelos DOM y SAX.
- Convertir datos entre formatos y extraer datos de páginas web (*web scraping*).
- Prever y tratar las excepciones propias del trabajo con ficheros, y probar el código con `pytest`.

Resultado de aprendizaje asociado: **RA1**.

1.1 Gestión de ficheros y directorios

Un **fichero** es un conjunto de bytes almacenado en un dispositivo y al que el sistema operativo asigna un nombre. Un **directorio** (o carpeta) es un fichero especial que contiene referencias a otros ficheros y directorios, formando un árbol.

En Python hay tres módulos de la biblioteca estándar para trabajar con el sistema de ficheros:

`pathlib`	Representa rutas como objetos (`Path`). Es la forma **moderna y recomendada**.
`shutil`	Operaciones de alto nivel: copiar, mover y borrar árboles de directorios, comprimir.
`os`, `os.path`	Interfaz clásica, basada en cadenas. La encontrarás en mucho código existente.

1.1.1 Rutas con `pathlib`

```
from pathlib import Path

base = Path.cwd()                       # directorio de trabajo actual
home = Path.home()                      # carpeta personal del usuario
ruta = base / "datos" / "alumnos.csv"   # el operador / une rutas
```

```
print(ruta)                 # /home/ana/proyecto/datos/alumnos.csv
print(ruta.name)            # alumnos.csv
print(ruta.stem)            # alumnos
print(ruta.suffix)          # .csv
print(ruta.parent)          # /home/ana/proyecto/datos
print(ruta.parts)           # ('/', 'home', 'ana', ...)
print(ruta.is_absolute())   # True
print(ruta.with_suffix(".json"))   # .../alumnos.json
print(Path("datos/../datos/a.txt").resolve())  # ruta absoluta normalizada
```

Rutas multiplataforma. Windows separa las carpetas con \ y Linux/macOS con /. Si construyes las rutas con `Path` y el operador /, tu programa funcionará en todos los sistemas sin cambios. Nunca concatenes rutas con +.

1.1.2 Consultar información

```
from pathlib import Path
from datetime import datetime

p = Path("datos/alumnos.csv")
if p.exists():
    print("?`Fichero?", p.is_file(), "?`Directorio?", p.is_dir())
    info = p.stat()
    print("Tamaño:", info.st_size, "bytes")
    print("Modificado:", datetime.fromtimestamp(info.st_mtime))
```

1.1.3 Crear, renombrar y borrar

```
from pathlib import Path

carpeta = Path("salida/informes/2026")
carpeta.mkdir(parents=True, exist_ok=True)   # crea toda la ruta si no existe

f = carpeta / "notas.txt"
f.touch()                                    # crea un fichero vacío
f.write_text("Primera línea\n", encoding="utf-8")
nuevo = f.rename(carpeta / "notas_1T.txt")   # renombrar o mover
nuevo.unlink(missing_ok=True)                # borrar un fichero
carpeta.rmdir()                              # borrar directorio (debe estar vacío)
```

1.1.4 Copiar, mover y borrar árboles: `shutil`

```
import shutil
from pathlib import Path

shutil.copy("datos/alumnos.csv", "copia/")          # copia el contenido
shutil.copy2("datos/alumnos.csv", "copia/a.csv")    # copia también fechas y permisos
shutil.copytree("datos", "copia_datos", dirs_exist_ok=True)  # copia un árbol
shutil.move("copia/a.csv", "historico/a.csv")       # mueve (o renombra)
shutil.rmtree("copia_datos")                        # !`borra el árbol completo!
```

```
shutil.make_archive("backup", "zip", "datos")       # crea backup.zip

total, usado, libre = shutil.disk_usage("/")
print(f"Libre: {libre / 2**30:.1f} GiB")
```

Cuidado. `shutil.rmtree` borra sin pasar por la papelera y sin pedir confirmación. Comprueba siempre la ruta antes de usarlo.

1.1.5 Recorrer directorios

```
from pathlib import Path

carpeta = Path("proyecto")
for elemento in carpeta.iterdir():              # sólo el primer nivel
    tipo = "DIR " if elemento.is_dir() else "FILE"
    print(tipo, elemento.name)

for py in carpeta.glob("*.py"):                  # patrón en el primer nivel
    print(py)

for py in carpeta.rglob("*.py"):                 # patrón recursivo
    print(py, py.stat().st_size)

# Tamaño total de una carpeta
total = sum(f.stat().st_size for f in carpeta.rglob("*") if f.is_file())
```

1.2 Formas de acceso a un fichero

Según cómo se llegue a los datos, distinguimos dos formas de acceso:

Acceso secuencial. Los datos se leen o escriben en orden, desde el principio hasta el final. Para llegar al registro 1000 hay que pasar por los 999 anteriores.

Acceso aleatorio (o directo). Se puede ir directamente a cualquier posición del fichero indicando su desplazamiento en bytes (`seek`). Requiere saber dónde está cada dato, lo que normalmente implica **registros de longitud fija** o un índice.

	Secuencial	**Aleatorio**
Ventajas	Sencillo; aprovecha todo el espacio; ideal para procesar todos los datos; registros de longitud variable.	Lectura y modificación de un registro concreto muy rápida, sin reescribir el fichero.
Inconvenientes	Buscar o modificar un registro concreto es lento; modificar suele implicar reescribir el fichero.	Registros de tamaño fijo (se desperdicia espacio); más complejo de programar; no apto para texto libre.
Aplicaciones	Logs, CSV, JSON, XML, exportaciones, copias de seguridad, ficheros de configuración.	Ficheros de índices, bases de datos, cachés, formatos binarios (imágenes, vídeo).

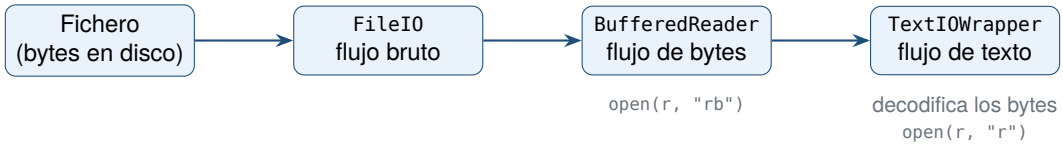
1.3 Flujos

Un **flujo** (*stream*) es una abstracción que representa una secuencia de datos que fluye desde un origen (fichero, teclado, conexión de red, memoria) hacia el programa, o desde el programa hacia un destino. Permite tratar todos esos orígenes y destinos de la misma forma.

Python distingue dos tipos de flujos:

Flujos de bytes (binarios). Trabajan con objetos `bytes`. No interpretan el contenido. Se usan para imágenes, ficheros comprimidos, ficheros de registros, etc. Se abren con modo `"b"`.

Flujos de caracteres (texto). Trabajan con objetos `str`. Internamente leen bytes y los **decodifican** usando una codificación de caracteres (UTF-8, Latin-1...). También traducen los saltos de línea de cada sistema operativo a `\n`.



Las clases que intervienen están en el módulo `io`, aunque casi nunca se instancian directamente: la función `open()` construye la cadena adecuada según el modo. El módulo `io` también ofrece flujos en memoria (`io.StringIO` e `io.BytesIO`), que se comportan como ficheros pero no tocan el disco; son muy útiles en pruebas.

1.3.1 La función `open` y los modos de apertura

Modo	Significado
`"r"`	Lectura (por defecto). Error si el fichero no existe.
`"w"`	Escritura. Crea el fichero o **lo vacía** si ya existe.
`"a"`	Añadir al final. Crea el fichero si no existe.
`"x"`	Creación exclusiva. Error si el fichero ya existe.
`"+"`	Añadido a otro modo, permite leer y escribir (`"r+"`, `"w+"`).
`"b"`	Modo binario (`"rb"`, `"wb"`, `"r+b"`...).
`"t"`	Modo texto (por defecto).

Indica siempre la codificación. Si abres un fichero de texto sin `encoding`, Python usa la codificación por defecto del sistema, que en Windows puede no ser UTF-8. El mismo programa podría leer bien las tildes en Linux y mal en Windows. Escribe siempre `encoding="utf-8"`.

1.4 Operaciones básicas con ficheros de texto

1.4.1 Lectura

```
from pathlib import Path
```

```python
# 1) Todo el contenido de golpe (ficheros pequeños)
texto = Path("poema.txt").read_text(encoding="utf-8")

# 2) Línea a línea (ficheros de cualquier tamaño: no se carga todo en memoria)
with open("poema.txt", encoding="utf-8") as f:
    for num, linea in enumerate(f, start=1):
        print(f"{num:3}: {linea.rstrip()}")

# 3) Otros métodos del objeto fichero
with open("poema.txt", encoding="utf-8") as f:
    primera = f.readline()       # una línea (incluye '\n')
    cinco = f.read(5)            # 5 caracteres
    resto = f.readlines()        # lista con las líneas restantes
```

1.4.2 Escritura

```python
lineas = ["Ana;8\n", "Luis;5\n"]
with open("notas.txt", "w", encoding="utf-8") as f:
    f.write("nombre;nota\n")
    f.writelines(lineas)
    print("Eva;9", file=f)       # print también puede escribir en un fichero

with open("notas.txt", "a", encoding="utf-8") as f:   # añadir al final
    f.write("Marta;7\n")
```

1.4.3 Modificar un fichero de texto

Un fichero de texto no permite insertar o borrar en medio de su contenido: la técnica habitual es **leer, transformar y reescribir**. Para no perder datos si el programa falla a mitad, se escribe en un fichero temporal y después se sustituye el original:

```python
import os
from pathlib import Path

def reemplazar_en_fichero(ruta: Path, viejo: str, nuevo: str) -> int:
    """Sustituye un texto en todo el fichero. Devuelve cuántas líneas cambiaron."""
    temporal = ruta.with_suffix(ruta.suffix + ".tmp")
    cambios = 0
    with open(ruta, encoding="utf-8") as origen, \
         open(temporal, "w", encoding="utf-8") as destino:
        for linea in origen:
            if viejo in linea:
                cambios += 1
                linea = linea.replace(viejo, nuevo)
            destino.write(linea)
    os.replace(temporal, ruta)     # sustitución atómica
    return cambios
```

1.5 Ficheros binarios

1.5.1 Lectura y escritura de bytes

```
# Copiar un fichero binario por bloques
TAM_BLOQUE = 64 * 1024
with open("foto.jpg", "rb") as origen, open("copia.jpg", "wb") as destino:
    while bloque := origen.read(TAM_BLOQUE):    # operador morsa (Tema 0)
        destino.write(bloque)

# Comprobar la "firma" de un fichero PNG
with open("imagen.png", "rb") as f:
    cabecera = f.read(8)
print(cabecera == b"\x89PNG\r\n\x1a\n")
```

1.5.2 Registros de longitud fija y acceso aleatorio con `struct`

Para guardar datos estructurados en binario y acceder directamente al registro n, todos los registros deben ocupar lo mismo. El módulo `struct` convierte valores de Python en bytes con un formato fijo:

Código	Tipo	Bytes
`i` / `I`	entero con / sin signo	4
`q`	entero largo	8
`d`	real de doble precisión	8
`?`	booleano	1
`30s`	cadena de 30 bytes	30
`<`	(prefijo) orden *little-endian*, sin relleno	—

El siguiente ejemplo implementa un pequeño fichero de productos con acceso directo por posición:

```
import struct
from dataclasses import dataclass
from pathlib import Path

FORMATO = "<i30sdi?"           # id, nombre, precio, stock, activo
TAM = struct.calcsize(FORMATO)  # 4 + 30 + 8 + 4 + 1 = 47 bytes

@dataclass
class Producto:
    id: int
    nombre: str
    precio: float
    stock: int
    activo: bool = True

    def a_bytes(self) -> bytes:
        nombre = self.nombre.encode("utf-8")[:30]   # se recorta/rellena a 30
        return struct.pack(FORMATO, self.id, nombre, self.precio,
                           self.stock, self.activo)

    @classmethod
    def desde_bytes(cls, datos: bytes) -> "Producto":
```

```
        id_, nombre, precio, stock, activo = struct.unpack(FORMATO, datos)
        nombre = nombre.rstrip(b"\x00").decode("utf-8", errors="ignore")
        return cls(id_, nombre, precio, stock, activo)


class FicheroProductos:
    def __init__(self, ruta: str | Path) -> None:
        self.ruta = Path(ruta)
        self.ruta.touch(exist_ok=True)

    def __len__(self) -> int:
        return self.ruta.stat().st_size // TAM

    def anadir(self, p: Producto) -> int:
        with open(self.ruta, "ab") as f:
            f.write(p.a_bytes())
        return len(self) - 1                    # posición del registro

    def leer(self, pos: int) -> Producto:
        if not 0 <= pos < len(self):
            raise IndexError(f"No existe el registro {pos}")
        with open(self.ruta, "rb") as f:
            f.seek(pos * TAM)                   # salto directo
            return Producto.desde_bytes(f.read(TAM))

    def escribir(self, pos: int, p: Producto) -> None:
        if not 0 <= pos < len(self):
            raise IndexError(f"No existe el registro {pos}")
        with open(self.ruta, "r+b") as f:       # lectura + escritura
            f.seek(pos * TAM)
            f.write(p.a_bytes())

    def borrar(self, pos: int) -> None:
        """Borrado lógico: se marca como inactivo."""
        p = self.leer(pos)
        p.activo = False
        self.escribir(pos, p)

    def __iter__(self):
        with open(self.ruta, "rb") as f:
            while datos := f.read(TAM):
                p = Producto.desde_bytes(datos)
                if p.activo:
                    yield p


if __name__ == "__main__":
    fp = FicheroProductos("productos.dat")
    fp.anadir(Producto(1, "Teclado", 24.90, 10))
    fp.anadir(Producto(2, "Ratón", 12.50, 25))
    p = fp.leer(1)
    p.stock -= 1
    fp.escribir(1, p)
    for prod in fp:
        print(prod)
```

Borrado lógico y compactación. En los ficheros de acceso aleatorio no se suele eliminar físicamente un registro (habría que desplazar todos los siguientes). Se marca como borrado y, de vez en cuando, se *compacta* el fichero copiando sólo los registros activos a uno nuevo.

1.5.3 Serialización de objetos con `pickle`

Serializar es convertir un objeto en memoria en una secuencia de bytes para guardarlo o enviarlo; **deserializar** es el proceso inverso. El módulo `pickle` serializa casi cualquier objeto de Python (equivale a la serialización de Java con `ObjectOutputStream`).

```
import pickle

agenda = {"Ana": ["600111222"], "Luis": ["611222333", "922000000"]}
with open("agenda.pkl", "wb") as f:
    pickle.dump(agenda, f)

with open("agenda.pkl", "rb") as f:
    recuperada = pickle.load(f)
```

Cuidado. `pickle` **sólo debe usarse con datos de confianza**: al deserializar se puede ejecutar código arbitrario. Además, el formato sólo lo entiende Python. Para intercambiar datos con otros programas usa formatos de texto estándar como JSON, CSV o XML.

1.6 Ficheros CSV

Un fichero CSV (*comma-separated values*) almacena una tabla en texto: una fila por línea y los campos separados por un delimitador (coma, punto y coma, tabulador...). Aunque parece sencillo, hay detalles (campos con el delimitador dentro, comillas, saltos de línea) que hacen que **no debamos procesarlo con `split`**, sino con el módulo `csv`.

```
import csv
from pathlib import Path

alumnos = [
    {"nombre": "Ana", "apellidos": "García, López", "nota": 8.5},
    {"nombre": "Luis", "apellidos": "Pérez", "nota": 4.25},
]

# Escritura a partir de diccionarios
with open("alumnos.csv", "w", newline="", encoding="utf-8") as f:
    escritor = csv.DictWriter(f, fieldnames=["nombre", "apellidos", "nota"],
                              delimiter=";")
    escritor.writeheader()
    escritor.writerows(alumnos)

# Lectura como diccionarios (la primera fila es la cabecera)
with open("alumnos.csv", newline="", encoding="utf-8") as f:
    for fila in csv.DictReader(f, delimiter=";"):
        print(fila["nombre"], float(fila["nota"]))   # ¡todo se lee como str!

# Lectura como listas
```

```python
with open("alumnos.csv", newline="", encoding="utf-8") as f:
    lector = csv.reader(f, delimiter=";")
    cabecera = next(lector)
    filas = list(lector)
```

> **Nota.** Se abre con `newline=""` porque el módulo `csv` gestiona los saltos de línea por su cuenta. Los ficheros CSV generados por Excel en español suelen usar `;` como separador y, a veces, la codificación `"utf-8-sig"` (UTF-8 con BOM) o `"cp1252"`.

1.7 Ficheros JSON

JSON (*JavaScript Object Notation*) es hoy el formato de intercambio de datos más utilizado: lo usan prácticamente todos los servicios web (API REST), muchos ficheros de configuración, las bases de datos documentales como MongoDB (Tema 5) e incluso los gestores relacionales (el tipo `JSONB` de PostgreSQL, Tema 4). Por eso será el **formato principal** de este tema.

1.7.1 Sintaxis

Un documento JSON es texto UTF-8 formado por **objetos** (pares *clave: valor* entre llaves), **arrays** (listas de valores entre corchetes) y valores simples:

```json
{
  "nombre": "Biblioteca del IES",
  "abierta": true,
  "libros": [
    {
      "isbn": "978-84-376-0494-7",
      "titulo": "Cien años de soledad",
      "autor": {"nombre": "Gabriel García Márquez", "pais": "Colombia"},
      "anio": 1967,
      "precio": 19.95,
      "generos": ["novela", "realismo mágico"],
      "prestado": false
    },
    {
      "isbn": "978-84-206-3311-6",
      "titulo": "La colmena",
      "autor": {"nombre": "Camilo José Cela", "pais": "España"},
      "anio": 1951,
      "precio": 12.50,
      "generos": ["novela"],
      "prestado": true
    }
  ]
}
```

Reglas que conviene recordar: las claves son **siempre cadenas** entre comillas dobles; no se admiten comillas simples, comentarios ni comas al final de una lista; no existen tipos para fechas, decimales exactos o binarios (hay que representarlos como cadenas o números).

La correspondencia con los tipos de Python es directa:

JSON	Python (al leer)	Python (al escribir)
objeto `{}`	`dict`	`dict`
array `[]`	`list`	`list`, `tuple`
cadena	`str`	`str`
número entero / real	`int` / `float`	`int`, `float`, `Enum` de enteros
`true` / `false`	`True` / `False`	`True` / `False`
`null`	`None`	`None`

1.7.2 Lectura y escritura con el módulo `json`

`json.load(f)`	Lee un documento JSON de un fichero abierto y devuelve el objeto Python.
`json.dump(obj, f)`	Escribe un objeto Python como JSON en un fichero abierto.
`json.loads(texto)`	Igual que `load`, pero a partir de una cadena.
`json.dumps(obj)`	Igual que `dump`, pero devuelve una cadena.

```python
import json
from pathlib import Path

ruta = Path("biblioteca.json")

with open(ruta, encoding="utf-8") as f:
    biblioteca = json.load(f)              # dict

for libro in biblioteca["libros"]:
    print(f"{libro['titulo']} ({libro['anio']}) - {libro['autor']['nombre']}")

baratos = [l for l in biblioteca["libros"] if l["precio"] < 15]
biblioteca["libros"].append({"isbn": "978-84-233-0000-1", "titulo": "Nada",
                             "autor": {"nombre": "Carmen Laforet", "pais": "España"},
                             "anio": 1945, "precio": 9.90, "generos": ["novela"],
                             "prestado": False})

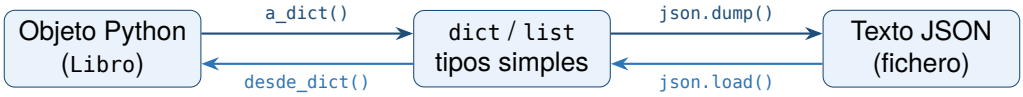
with open(ruta, "w", encoding="utf-8") as f:
    json.dump(biblioteca, f,
              indent=2,              # sangría: legible por personas
              ensure_ascii=False,    # escribe "ñ" en vez de "ñ"
              sort_keys=False)

texto = json.dumps({"ok": True, "total": 3})   # '{"ok": true, "total": 3}'
datos = json.loads(texto)
```

Lo que se pierde por el camino. Al guardar y volver a leer: las tuplas vuelven como listas; las claves numéricas de un diccionario vuelven como cadenas (`{1: "a"}` → `{"1": "a"}`); los `float` pueden perder precisión decimal (usa cadenas o `decimal.Decimal` para importes); y las fechas, conjuntos u objetos propios provocan `TypeError` si no se convierten antes.

1.7.3 Serialización de objetos

Serializar un objeto es convertirlo en una representación que se puede guardar o transmitir (aquí, texto JSON); **deserializar** es reconstruir el objeto a partir de esa representación. Como JSON sólo conoce diccionarios, listas y valores simples, nuestros objetos deben pasar por un diccionario intermedio.



Patrón `a_dict` / `desde_dict`

La forma más clara y controlable es que cada clase sepa convertirse en diccionario y reconstruirse a partir de uno. Así decidimos exactamente cómo se representa cada tipo especial (fechas, decimales, enumerados, objetos anidados):

```
import json
from dataclasses import dataclass, field
from datetime import date
from decimal import Decimal
from enum import Enum

class Estado(Enum):
    DISPONIBLE = "disponible"
    PRESTADO = "prestado"
    BAJA = "baja"

@dataclass
class Autor:
    nombre: str
    pais: str | None = None

    def a_dict(self) -> dict:
        return {"nombre": self.nombre, "pais": self.pais}

    @classmethod
    def desde_dict(cls, d: dict) -> "Autor":
        return cls(d["nombre"], d.get("pais"))

@dataclass
class Libro:
    isbn: str
    titulo: str
    autor: Autor                              # objeto anidado
    precio: Decimal                           # importe exacto
    alta: date
    estado: Estado = Estado.DISPONIBLE
    generos: list[str] = field(default_factory=list)

    def a_dict(self) -> dict:
        return {
            "isbn": self.isbn,
            "titulo": self.titulo,
            "autor": self.autor.a_dict(),
            "precio": str(self.precio),       # "19.95": sin perder precisión
```

```
            "alta": self.alta.isoformat(),    # "2026-10-14"
            "estado": self.estado.value,
            "generos": self.generos,
        }

    @classmethod
    def desde_dict(cls, d: dict) -> "Libro":
        return cls(
            isbn=d["isbn"],
            titulo=d["titulo"],
            autor=Autor.desde_dict(d["autor"]),
            precio=Decimal(d["precio"]),
            alta=date.fromisoformat(d["alta"]),
            estado=Estado(d.get("estado", "disponible")),
            generos=d.get("generos", []),
        )


def guardar_libros(libros: list[Libro], ruta: str) -> None:
    with open(ruta, "w", encoding="utf-8") as f:
        json.dump([l.a_dict() for l in libros], f, indent=2, ensure_ascii=False)

def cargar_libros(ruta: str) -> list[Libro]:
    with open(ruta, encoding="utf-8") as f:
        return [Libro.desde_dict(d) for d in json.load(f)]


libros = [Libro("978-84-376-0494-7", "Cien años de soledad",
                Autor("Gabriel García Márquez", "Colombia"),
                Decimal("19.95"), date(2026, 9, 15), generos=["novela"])]
guardar_libros(libros, "libros.json")
assert cargar_libros("libros.json") == libros     # ida y vuelta sin pérdidas
```

Codificadores y decodificadores genéricos

Si sólo hay que convertir unos pocos tipos, se puede indicar a `json` cómo hacerlo de forma general:

- Al **escribir**, el parámetro `default` recibe una función que se llama con cada objeto que `json` no sabe serializar.
- Al **leer**, el parámetro `object_hook` recibe una función que se llama con cada objeto JSON (ya convertido en `dict`) y puede devolver otra cosa.

```
import json
from datetime import date, datetime
from decimal import Decimal
from enum import Enum

def codificar(obj):
    if isinstance(obj, (date, datetime)):
        return obj.isoformat()
    if isinstance(obj, Decimal):
        return str(obj)
    if isinstance(obj, Enum):
        return obj.value
```

```
    if isinstance(obj, set):
        return sorted(obj)
    raise TypeError(f"Tipo no serializable: {type(obj).__name__}")

def decodificar(d: dict) -> dict:
    """Convierte en date los campos cuyo nombre empieza por 'fecha'."""
    for clave, valor in d.items():
        if clave.startswith("fecha") and isinstance(valor, str):
            d[clave] = date.fromisoformat(valor)
    return d

texto = json.dumps({"fecha_alta": date.today(), "importe": Decimal("10.50"),
                    "etiquetas": {"b", "a"}}, default=codificar)
datos = json.loads(texto, object_hook=decodificar)
print(type(datos["fecha_alta"]))          # <class 'datetime.date'>
```

> **`pickle` frente a JSON.** `pickle` serializa casi cualquier objeto sin esfuerzo, pero su formato sólo lo entiende Python y no es seguro con datos externos. JSON exige decidir cómo se representa cada tipo, pero el resultado es legible, estándar y lo puede leer cualquier lenguaje. Para intercambiar o almacenar datos, usa JSON.

1.7.4 JSON Lines: ficheros JSON grandes

Un documento JSON se lee entero en memoria con `json.load`, y para añadir un registro hay que reescribirlo completo. Para ficheros de registros muy grandes o que crecen continuamente (registros de actividad, exportaciones, datos recogidos por un programa de *scraping*) se usa **JSON Lines** (`.jsonl`): **un objeto JSON por línea**. Combina las ventajas de JSON con el acceso secuencial de un fichero de texto.

```
{"isbn": "978-84-376-0494-7", "titulo": "Cien años de soledad", "anio": 1967}
{"isbn": "978-84-206-3311-6", "titulo": "La colmena", "anio": 1951}
```

```
import json
from collections.abc import Iterator

def anadir_jsonl(ruta: str, registro: dict) -> None:
    with open(ruta, "a", encoding="utf-8") as f:          # sólo se añade al final
        f.write(json.dumps(registro, ensure_ascii=False) + "\n")

def leer_jsonl(ruta: str) -> Iterator[dict]:
    with open(ruta, encoding="utf-8") as f:
        for num, linea in enumerate(f, start=1):
            if linea.strip():
                try:
                    yield json.loads(linea)
                except json.JSONDecodeError as e:
                    print(f"Línea {num} ignorada: {e.msg}")

anadir_jsonl("libros.jsonl", {"isbn": "978-84-233-0000-1", "titulo": "Nada"})
antiguos = [l for l in leer_jsonl("libros.jsonl") if l.get("anio", 0) < 1960]
```

1.7.5 Validación con JSON Schema

JSON no impone ninguna estructura: un programa puede recibir un documento sintácticamente correcto pero al que le falta un campo, con un texto donde se esperaba un número o con valores fuera de rango. **JSON Schema** (`https://json-schema.org`) es un estándar para describir, *también en JSON*, la estructura que debe tener un documento: es el equivalente a XSD para XML. Empezamos con un ejemplo sencillo, el esquema de un libro, y después veremos la sintaxis con detalle.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Libro",
  "type": "object",
  "properties": {
    "isbn":    {"type": "string", "pattern": "^97[89]-\\d{1,5}-\\d{1,7}-\\d{1,7}-\\d$"},
    "titulo":  {"type": "string", "minLength": 1},
    "autor": {
      "type": "object",
      "properties": {"nombre": {"type": "string"}, "pais": {"type": "string"}},
      "required": ["nombre"]
    },
    "anio":    {"type": "integer", "minimum": 1450, "maximum": 2100},
    "precio":  {"type": "number", "exclusiveMinimum": 0},
    "generos": {"type": "array", "items": {"type": "string"}, "uniqueItems": true},
    "prestado": {"type": "boolean"},
    "estado":  {"enum": ["disponible", "prestado", "baja"]},
    "email_contacto": {"type": "string", "format": "email"}
  },
  "required": ["isbn", "titulo", "autor", "anio"],
  "additionalProperties": false
}
```

Sintaxis de un esquema

Un esquema es un objeto JSON cuyas claves son **palabras clave** (*keywords*) del estándar. Cada palabra clave impone una condición al documento que se valida (la *instancia*); el documento es válido si cumple **todas** las condiciones. Un esquema vacío (`{}`) admite cualquier documento.

La referencia completa está en la web oficial del estándar, `https://json-schema.org`, que incluye una guía muy recomendable, *Understanding JSON Schema* (`https://json-schema.org/understanding-json-schema`), con la descripción de cada palabra clave y ejemplos. Usaremos la versión 2020-12 del estándar.

Cabecera y anotaciones. Estas palabras clave no validan nada; describen el esquema y sirven de documentación:

`$schema`	Versión del estándar con la que está escrito el esquema.
`$id`	Identificador (URI) del esquema, para poder referenciarlo desde otros.
`title`, `description`	Título y descripción, del esquema o de cada propiedad.
`default`, `examples`	Valor por defecto y ejemplos. ¡Ojo!: `default` **no** rellena los valores que faltan; sólo los documenta.
`$comment`	Comentario para quien mantiene el esquema (JSON no admite comentarios).

Tipos. `type` indica el tipo de dato: `string`, `number`, `integer` (número sin decimales), `boolean`, `object`, `array` o `null`. Puede ser una lista para admitir varios: `"type": ["string", "null"]` permite una cadena o `null`. El resto de palabras clave sólo se aplican si el valor es del tipo al que se refieren: `minLength` no dice nada de un número.

Cadenas.

`minLength`, `maxLength`	Longitud mínima y máxima.
`pattern`	Expresión regular que debe cumplir (no está anclada: para exigir que toda la cadena la cumpla, se usan `^` y `$`).
`format`	Formato semántico: `date` (`2026-10-19`), `date-time`, `time`, `email`, `uri`, `uuid`, `ipv4`...

Números.

`minimum`, `maximum`	Valor mínimo y máximo (incluidos).
`exclusiveMinimum`, `exclusiveMaximum`	Límites sin incluir: `"exclusiveMinimum": 0` exige un valor mayor que 0.
`multipleOf`	El valor debe ser múltiplo del indicado (`"multipleOf": 5`). Con decimales no es fiable: por cómo se representan los números reales, `19.99` **no** se considera múltiplo de `0.01`. Para importes, valida los céntimos como enteros o usa cadenas.

Objetos.

`properties`	Esquema que debe cumplir cada propiedad, si aparece.
`required`	Lista de propiedades obligatorias (por defecto, ninguna lo es).
`additionalProperties`	Esquema para las propiedades no declaradas en `properties`; `false` las prohíbe.
`patternProperties`	Esquema para las propiedades cuyo nombre cumple una expresión regular.
`propertyNames`	Condiciones sobre los nombres de las propiedades.
`minProperties`, `maxProperties`	Número mínimo y máximo de propiedades.
`dependentRequired`	Si aparece una propiedad, otras pasan a ser obligatorias.

Arrays.

`items`	Esquema que deben cumplir todos los elementos.
`prefixItems`	Esquema de cada posición, para arrays que funcionan como tuplas (`[latitud, longitud]`).
`minItems`, `maxItems`	Número mínimo y máximo de elementos.
`uniqueItems`	Si es `true`, no puede haber elementos repetidos.
`contains`	Al menos un elemento debe cumplir el esquema indicado.

Valores concretos. `enum` da la lista de valores permitidos (`"enum": ["disponible", "prestado", "baja"]`) y `const`, el único valor permitido.

Combinación de esquemas. Permiten construir condiciones complejas a partir de varios esquemas:

`allOf`	Debe cumplir **todos** los esquemas de la lista (equivale a un "y").
`anyOf`	Debe cumplir **al menos uno** ("o").
`oneOf`	Debe cumplir **exactamente uno**.
`not`	**No** debe cumplir el esquema.
`if`, `then`, `else`	Si cumple el esquema de `if`, debe cumplir el de `then`; si no, el de `else`.

Reutilización: `$defs` y `$ref`. Las partes que se repiten se definen una vez en `$defs` y se referencian con `$ref`. `"$ref": "#/$defs/direccion"` significa "el esquema llamado `direccion` dentro de `$defs` de este mismo documento". Una referencia también puede apuntar a otro fichero o a una URL.

Un ejemplo completo. El siguiente esquema describe un pedido y reúne la mayoría de los elementos anteriores:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://ejemplo.es/esquemas/pedido.json",
  "title": "Pedido",
  "description": "Pedido de una tienda en línea",
  "type": "object",
  "properties": {
    "id":      {"type": "integer", "minimum": 1},
    "fecha":   {"type": "string", "format": "date"},
    "estado":  {"enum": ["pendiente", "enviado", "entregado"], "default": "pendiente"},
    "cliente": {"$ref": "#/$defs/cliente"},
    "lineas": {
      "type": "array",
      "items": {"$ref": "#/$defs/linea"},
      "minItems": 1
    },
    "envio":   {"enum": ["tienda", "domicilio"]},
    "direccion": {"$ref": "#/$defs/direccion"},
    "observaciones": {"type": ["string", "null"], "maxLength": 500}
  },
```

```
  "required": ["id", "fecha", "cliente", "lineas", "envio"],
  "additionalProperties": false,
  "if":   {"properties": {"envio": {"const": "domicilio"}}},
  "then": {"required": ["direccion"]},
  "$defs": {
    "cliente": {
      "type": "object",
      "properties": {
        "nombre": {"type": "string", "minLength": 1},
        "email":  {"type": "string", "format": "email"}
      },
      "required": ["nombre", "email"]
    },
    "linea": {
      "type": "object",
      "properties": {
        "producto": {"type": "string"},
        "cantidad": {"type": "integer", "minimum": 1},
        "precio":   {"type": "number", "exclusiveMinimum": 0}
      },
      "required": ["producto", "cantidad", "precio"]
    },
    "direccion": {
      "type": "object",
      "properties": {
        "calle":  {"type": "string"},
        "cp":     {"type": "string", "pattern": "^[0-9]{5}$"},
        "ciudad": {"type": "string"}
      },
      "required": ["calle", "cp", "ciudad"]
    }
  }
}
```

Fíjate en el par `if/then`: la dirección sólo es obligatoria cuando el envío es a domicilio. Y en que, por defecto, `format` es sólo una anotación: para que `"format": "email"` o `"format": "date"` se comprueben de verdad, hay que pedirlo expresamente al validador, con el parámetro `format_checker`, como se ve en el apartado siguiente.

Validar documentos desde Python

En Python se valida con la biblioteca `jsonschema`:

```
$ pip install jsonschema
```

```
import json
from jsonschema import Draft202012Validator, FormatChecker, ValidationError, validate

with open("libro.schema.json", encoding="utf-8") as f:
    esquema = json.load(f)

libro = {"isbn": "978-84-206-3311-6", "titulo": "La colmena",
         "autor": {"nombre": "Camilo José Cela"}, "anio": 1951}

# 1) Forma rápida: lanza ValidationError con el PRIMER error encontrado
try:
```

```
    validate(instance=libro, schema=esquema)
except ValidationError as e:
    print("Libro no válido:", e.message)

# 2) Informe con TODOS los errores y dónde están
validador = Draft202012Validator(esquema, format_checker=FormatChecker())
erroneo = {"isbn": "123", "titulo": "", "autor": {}, "anio": "1951", "color": "rojo"}
for error in sorted(validador.iter_errors(erroneo), key=lambda e: e.path):
    ruta = "/".join(str(p) for p in error.path) or "(raíz)"
    print(f"{ruta}: {error.message}")
```

```
(raíz): Additional properties are not allowed ('color' was unexpected)
anio: '1951' is not of type 'integer'
autor: 'nombre' is a required property
isbn: '123' does not match '^97[89]-\\d{1,5}-\\d{1,7}-\\d{1,7}-\\d$'
titulo: '' should be non-empty
```

Con el esquema del pedido ocurre lo mismo; observa que, al activar `format_checker`, también se comprueban la fecha y el email, y que el error de la dirección lo produce la condición `if/then`:

```
with open("pedido.schema.json", encoding="utf-8") as f:
    esquema_pedido = json.load(f)

pedido = {"id": 0, "fecha": "19/10/2026", "envio": "domicilio", "lineas": [],
          "cliente": {"nombre": "Ana", "email": "ana"}, "color": "rojo"}
validador = Draft202012Validator(esquema_pedido, format_checker=FormatChecker())
for error in sorted(validador.iter_errors(pedido), key=lambda e: list(e.path)):
    print("/".join(map(str, error.path)) or "(raíz)", ":", error.message)
```

```
(raíz) : Additional properties are not allowed ('color' was unexpected)
(raíz) : 'direccion' is a required property
cliente/email : 'ana' is not a 'email'
fecha : '19/10/2026' is not a 'date'
id : 0 is less than the minimum of 1
lineas : [] should be non-empty
```

> **Nota.** Valida siempre los datos que vienen de fuera del programa (ficheros que edita una persona, respuestas de servicios web, datos importados) **antes** de convertirlos en objetos. Es mucho más fácil dar un mensaje de error claro en ese momento que depurar un `KeyError` más adelante.

1.7.6 Vinculación de JSON a objetos con Pydantic

En el patrón `a_dict/desde_dict` la conversión y la validación las programamos nosotros. La **vinculación de datos** (*data binding*) automatiza ambas cosas: a partir de la definición de una clase con anotaciones de tipo, una biblioteca convierte el documento en objetos (*unmarshalling*) y los objetos en documento (*marshalling*), validando por el camino. En Python, la biblioteca de referencia para JSON es **Pydantic**.

```
$ pip install pydantic
```

```python
from datetime import date
from decimal import Decimal
from enum import Enum
from pydantic import BaseModel, Field, ValidationError

class Estado(str, Enum):
    DISPONIBLE = "disponible"
    PRESTADO = "prestado"

class Autor(BaseModel):
    nombre: str
    pais: str | None = None

class Libro(BaseModel):
    isbn: str = Field(pattern=r"^97[89]-")
    titulo: str = Field(min_length=1)
    autor: Autor
    anio: int = Field(ge=1450, le=2100)
    precio: Decimal = Field(gt=0, decimal_places=2)
    alta: date
    estado: Estado = Estado.DISPONIBLE
    generos: list[str] = []

class Catalogo(BaseModel):
    libros: list[Libro]

# JSON -> objetos (convierte los tipos: "2026-10-14" -> date, "12.50" -> Decimal)
with open("catalogo.json", encoding="utf-8") as f:
    try:
        catalogo = Catalogo.model_validate_json(f.read())
    except ValidationError as e:
        for err in e.errors():
            print(".".join(map(str, err["loc"])), "->", err["msg"])
        raise SystemExit(1)

print(catalogo.libros[0].alta.year, catalogo.libros[0].autor.nombre)

# objetos -> JSON
catalogo.libros[0].estado = Estado.PRESTADO
with open("catalogo.json", "w", encoding="utf-8") as f:
    f.write(catalogo.model_dump_json(indent=2))

# Pydantic también genera el JSON Schema de la clase
import json
print(json.dumps(Libro.model_json_schema(), indent=2, ensure_ascii=False))
```

Nota. Pydantic es la base de *frameworks* muy utilizados como FastAPI, y lo volverás a encontrar en otros módulos del ciclo. La biblioteca estándar no incluye nada equivalente.

1.8 Ficheros XML

XML (*eXtensible Markup Language*) fue durante muchos años el formato de intercambio dominante y sigue muy presente en ficheros de configuración (Maven, Android), documentos

ofimáticos (`.docx` y `.odt` son XML comprimido), facturación electrónica, servicios SOAP y administraciones públicas. En este apartado vemos lo imprescindible para leer, crear, modificar y validar documentos XML. El Anexo B amplía con ejemplos de código de DOM y SAX, XPath completo, transformaciones XSLT y más herramientas de vinculación y conversión.

1.8.1 Repaso de XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<biblioteca nombre="Biblioteca del IES">
  <libro isbn="978-84-376-0494-7" prestado="no">
    <titulo>Cien años de soledad</titulo>
    <autor>Gabriel García Márquez</autor>
    <anio>1967</anio>
    <precio moneda="EUR">19.95</precio>
  </libro>
  <libro isbn="978-84-206-3311-6" prestado="si">
    <titulo>La colmena</titulo>
    <autor>Camilo José Cela</autor>
    <anio>1951</anio>
    <precio moneda="EUR">12.50</precio>
  </libro>
</biblioteca>
```

- La información se organiza en **elementos** anidados, que pueden tener **atributos** y **texto**. El documento forma un **árbol** con un único elemento raíz.
- Un documento es **bien formado** si cumple las reglas sintácticas (un único elemento raíz, etiquetas cerradas y bien anidadas, atributos entrecomillados…).
- Es **válido** si además cumple un esquema (DTD o XSD) que define qué elementos y atributos se permiten.

Comparado con JSON, XML es más verboso pero admite atributos, espacios de nombres, contenido mixto (texto con etiquetas dentro, como en HTML) y tiene herramientas estándar de validación (XSD) y transformación (XSLT) muy maduras.

1.8.2 Modelos de procesamiento: DOM y SAX

Un **analizador sintáctico** (*parser*) lee el texto XML, comprueba que es correcto y ofrece su contenido al programa. Hay dos modelos clásicos:

	DOM (*Document Object Model*)	**SAX** (*Simple API for XML*)
Funcionamiento	Carga el documento completo en memoria como un **árbol de nodos**.	Recorre el documento de forma secuencial y **lanza eventos** (inicio de elemento, texto, fin de elemento) que el programa atiende.
Memoria	Proporcional al tamaño del documento.	Muy baja y constante.
Navegación	Libre: hijos, padres, hermanos, en cualquier orden.	Sólo hacia delante; el programa guarda el estado que necesite.
Modificación	Sí: se modifica el árbol y se vuelve a guardar.	No: sólo lectura.
Uso típico	Documentos pequeños o medianos que se consultan varias veces o se modifican.	Documentos muy grandes de los que se extraen unos pocos datos.

DOM es un estándar del W3C con la misma API en todos los lenguajes (métodos como `createElement` o `appendChild`); es el mismo modelo que usa JavaScript para manipular páginas web. SAX es el modelo de los analizadores dirigidos por eventos. Python incluye ambos (`xml.dom.minidom` y `xml.sax`); en el Anexo B tienes un ejemplo completo de cada uno.

En la práctica, en Python se usa casi siempre **ElementTree**, una API de tipo árbol (como DOM) pero mucho más sencilla, que además permite un procesamiento incremental parecido a SAX (`iterparse`) para ficheros grandes.

1.8.3 Lectura y modificación con ElementTree

ElementTree representa el documento como un árbol de objetos `Element`. Cada elemento tiene un nombre (`tag`), un diccionario de atributos (`attrib`), un texto (`text`) y se comporta como una lista de sus hijos.

```
import xml.etree.ElementTree as ET

arbol = ET.parse("biblioteca.xml")
raiz = arbol.getroot()
print(raiz.tag, raiz.get("nombre"))          # biblioteca Biblioteca del IES

for libro in raiz.findall("libro"):          # hijos <libro> de la raíz
    titulo = libro.findtext("titulo")        # texto del primer hijo <titulo>
    anio = int(libro.findtext("anio"))       # ¡`el texto siempre es str!
    precio = float(libro.find("precio").text)
    print(f"{libro.get('isbn')}: {titulo} ({anio}) {precio:.2f} €")
```

XPath básico

Los métodos `find`, `findall`, `findtext` e `iterfind` aceptan un subconjunto de **XPath**, el lenguaje estándar para seleccionar nodos de un documento XML:

Expresión	Selecciona
`libro`	Los hijos `<libro>` del elemento actual.
`libro/titulo`	Los `<titulo>` hijos de los `<libro>` hijos.
`.//titulo`	Todos los `<titulo>` a cualquier profundidad.
`*`	Todos los elementos hijos.
`..`	El elemento padre.
`libro[@isbn]`	Los `<libro>` que tienen el atributo `isbn`.
`libro[@prestado='si']`	Los `<libro>` con ese valor de atributo.
`libro[anio='1951']`	Los `<libro>` con un hijo `<anio>` de ese texto.
`libro[1]`, `libro[last()]`	El primero (las posiciones empiezan en 1) y el último.

```
prestados = raiz.findall("./libro[@prestado='si']")
titulos = [t.text for t in raiz.iterfind(".//titulo")]
primero = raiz.find("libro[1]/titulo").text
# ElementTree no evalúa comparaciones numéricas: se filtra en Python
antiguos = [l for l in raiz.findall("libro") if int(l.findtext("anio")) < 1960]
```

Modificar y crear documentos

```
# Modificar: subir un 10 % los precios
for precio in raiz.iter("precio"):
    precio.text = f"{float(precio.text) * 1.10:.2f}"
raiz.find("libro[@isbn='978-84-206-3311-6']").set("prestado", "no")

# Añadir un elemento con subelementos
libro = ET.SubElement(raiz, "libro", isbn="978-84-233-0000-1", prestado="no")
ET.SubElement(libro, "titulo").text = "Nada"
ET.SubElement(libro, "autor").text = "Carmen Laforet"
ET.SubElement(libro, "anio").text = "1945"

# Eliminar los libros anteriores a 1950
for libro in raiz.findall("libro"):
    if int(libro.findtext("anio")) < 1950:
        raiz.remove(libro)

ET.indent(arbol, space="  ")              # sangría legible
arbol.write("biblioteca_mod.xml", encoding="utf-8", xml_declaration=True)

# Crear un documento desde cero a partir de datos
alumnos = [{"dni": "11111111H", "nombre": "Ana"}, {"dni": "22222222J", "nombre": "Luis"
    ↪ }]
raiz = ET.Element("alumnos")
for a in alumnos:
    ET.SubElement(raiz, "alumno", dni=a["dni"]).text = a["nombre"]
nuevo = ET.ElementTree(raiz)
ET.indent(nuevo)
nuevo.write("alumnos.xml", encoding="utf-8", xml_declaration=True)
```

1.8.4 Validación con XSD

Un **XML Schema** (XSD) describe la estructura que debe tener un documento: qué elementos y atributos puede contener, en qué orden, cuántas veces y de qué tipo. ElementTree no valida; para ello se usa la biblioteca **lxml**, que implementa la misma API que ElementTree y añade validación, XPath completo y XSLT.

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- biblioteca.xsd -->
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="biblioteca">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="libro" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="titulo" type="xs:string"/>
              <xs:element name="autor"  type="xs:string"/>
              <xs:element name="anio"   type="xs:gYear"/>
              <xs:element name="precio" minOccurs="0">
                <xs:complexType>
                  <xs:simpleContent>
                    <xs:extension base="xs:decimal">
                      <xs:attribute name="moneda" type="xs:string" default="EUR"/>
                    </xs:extension>
                  </xs:simpleContent>
                </xs:complexType>
              </xs:element>
            </xs:sequence>
            <xs:attribute name="isbn" type="xs:string" use="required"/>
            <xs:attribute name="prestado" default="no">
              <xs:simpleType>
                <xs:restriction base="xs:string">
                  <xs:enumeration value="si"/>
                  <xs:enumeration value="no"/>
                </xs:restriction>
              </xs:simpleType>
            </xs:attribute>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
      <xs:attribute name="nombre" type="xs:string"/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

```
$ pip install lxml
```

```
from lxml import etree

esquema = etree.XMLSchema(etree.parse("biblioteca.xsd"))

try:
    documento = etree.parse("biblioteca.xml")       # comprueba que esté bien formado
except etree.XMLSyntaxError as e:
    raise SystemExit(f"XML mal formado: {e}")
```

```python
if esquema.validate(documento):
    print("Documento válido")
else:
    for error in esquema.error_log:
        print(f"Línea {error.line}: {error.message}")
```

1.8.5 Vinculación de XML a objetos

Hasta ahora hemos trabajado con lxml y ElementTree, que son **analizadores**: convierten el documento en un **árbol genérico de elementos**. Ese árbol sirve para recorrer el documento, hacer consultas XPath o validarlo con un XSD, pero no sabe nada de nuestro dominio: un libro es un `Element` más, todos sus valores son cadenas y, si queremos objetos `Libro` con un `anio` entero, tenemos que escribir la conversión a mano, igual que hicimos con `a_dict`/`desde_dict` en JSON.

La **vinculación** (*data binding*) automatiza ese paso: a partir de unas clases (o de un XSD) convierte el XML en objetos del dominio con sus tipos y, al revés, los objetos en XML. Es lo mismo que hicimos con Pydantic para JSON (en Java, la herramienta clásica es JAXB). En Python, una de las bibliotecas más completas es **xsdata**: se describen las clases como *dataclasses* con metadatos que indican qué campo es un atributo y cuál un elemento, y la biblioteca se encarga del resto con `XmlParser` (XML → objetos) y `XmlSerializer` (objetos → XML). Incluso puede generar las clases automáticamente a partir de un XSD (`xsdata generate biblioteca.xsd`).

```
$ pip install xsdata
```

```python
from dataclasses import dataclass, field
from pathlib import Path
from xsdata.formats.dataclass.parsers import XmlParser
from xsdata.formats.dataclass.serializers import XmlSerializer
from xsdata.formats.dataclass.serializers.config import SerializerConfig

@dataclass
class Libro:
    class Meta:
        name = "libro"
    isbn: str = field(metadata={"type": "Attribute"})
    titulo: str = field(metadata={"type": "Element"})
    autor: str = field(metadata={"type": "Element"})
    anio: int = field(metadata={"type": "Element"})

@dataclass
class Biblioteca:
    class Meta:
        name = "biblioteca"
    nombre: str = field(default="", metadata={"type": "Attribute"})
    libros: list[Libro] = field(default_factory=list,
                                metadata={"type": "Element", "name": "libro"})

# XML -> objetos (anio ya es int)
bib = XmlParser().from_path(Path("biblioteca_simple.xml"), Biblioteca)
print(bib.libros[0].titulo, bib.libros[0].anio + 1)

# objetos -> XML
```

```
bib.libros.append(Libro("978-84-233-0000-1", "Nada", "Carmen Laforet", 1945))
xml = XmlSerializer(config=SerializerConfig(indent="  ")).render(bib)
Path("biblioteca_nueva.xml").write_text(xml, encoding="utf-8")
```

Una alternativa ligera: `lxml.objectify`. El propio lxml incluye el módulo `lxml.objectify`, que ofrece una vinculación *dinámica*: no hay que declarar ninguna clase, y los elementos hijos se recorren como si fueran atributos de Python (`raiz.libro.titulo`). Además, deduce el tipo de los valores simples a partir de su texto:

```
from lxml import objectify

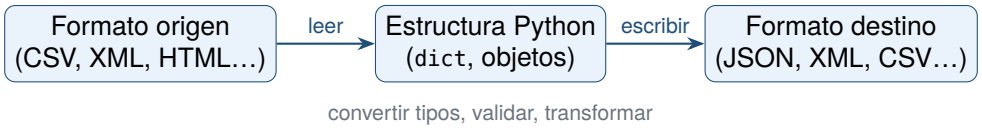
raiz = objectify.parse("biblioteca.xml").getroot()
print(raiz.get("nombre"))                  # los atributos XML, con get()
for libro in raiz.libro:                   # recorre todos los <libro>
    print(libro.get("isbn"), libro.titulo, libro.anio + 1)   # anio se trata como int
print(raiz.libro[1].autor)                 # el segundo <libro>
raiz.libro[1].titulo = "La colmena (ed. 2024)"   # modificar un valor
raiz.libro.editorial                       # AttributeError: no such child: editorial
```

Es muy cómodo para leer o retocar un documento con pocas líneas, pero tiene limitaciones frente a xsdata:

	`lxml.objectify`	**xsdata**
Clases del dominio	No: objetos genéricos creados al vuelo	Sí: *dataclasses* declaradas o generadas desde el XSD
Tipos	Deducidos del texto de cada valor (un ISBN numérico se convertiría en `int`)	Los declarados en la clase o en el XSD
Errores	Al acceder a un hijo que no existe (en ejecución)	Al analizar el documento; además, el editor y `mypy` conocen los campos
Uso típico	Scripts rápidos y documentos sencillos	Aplicaciones con un formato XML estable

1.9 Conversión entre formatos

Una tarea muy habitual en el acceso a datos es **convertir** información de un formato a otro: importar un CSV exportado de una hoja de cálculo, publicar como JSON datos que están en XML, generar un informe HTML... La técnica general es siempre la misma:



Leer con la herramienta adecuada al formato de origen, obtener una estructura de Python (lista de diccionarios u objetos), **convertir los tipos** (en CSV y XML todo es texto) y validar, y escribir con la herramienta del formato de destino.

1.9.1 CSV ↔ JSON

```python
import csv, json

def convertir_valor(texto: str):
    """Intenta interpretar un texto de CSV como número o booleano."""
    if texto == "":
        return None
    if texto.lower() in ("true", "false", "si", "no"):
        return texto.lower() in ("true", "si")
    for tipo in (int, float):
        try:
            return tipo(texto.replace(",", ".") if tipo is float else texto)
        except ValueError:
            pass
    return texto

def csv_a_json(origen: str, destino: str, delimitador: str = ";") -> int:
    with open(origen, newline="", encoding="utf-8") as f:
        filas = [{k: convertir_valor(v) for k, v in fila.items()}
                 for fila in csv.DictReader(f, delimiter=delimitador)]
    with open(destino, "w", encoding="utf-8") as f:
        json.dump(filas, f, indent=2, ensure_ascii=False)
    return len(filas)

def json_a_csv(origen: str, destino: str, delimitador: str = ";") -> int:
    with open(origen, encoding="utf-8") as f:
        datos = json.load(f)
    if not isinstance(datos, list) or not all(isinstance(d, dict) for d in datos):
        raise ValueError("El JSON debe ser una lista de objetos")
    columnas = list(dict.fromkeys(k for d in datos for k in d))   # unión ordenada
    with open(destino, "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=columnas, delimiter=delimitador)
        w.writeheader()
        w.writerows(datos)
    return len(datos)
```

Estructuras anidadas. CSV es plano: una tabla. Si los objetos JSON contienen otros objetos o listas (`"autor": {"nombre": …}`), hay que **aplanarlos** antes (por ejemplo, en columnas `autor.nombre`, `autor.pais`) o guardar las listas como texto separado por un carácter.

1.9.2 XML ↔ JSON

```python
import json
import xml.etree.ElementTree as ET

# XML -> JSON: decidimos explícitamente la estructura y los tipos
raiz = ET.parse("biblioteca.xml").getroot()
libros = [
    {
        "isbn": l.get("isbn"),
        "titulo": l.findtext("titulo"),
        "autor": l.findtext("autor"),
```

```
        "anio": int(l.findtext("anio")),
        "precio": float(l.findtext("precio", default="0")),
        "prestado": l.get("prestado") == "si",
    }
    for l in raiz.findall("libro")
]
with open("biblioteca.json", "w", encoding="utf-8") as f:
    json.dump({"nombre": raiz.get("nombre"), "libros": libros}, f,
              indent=2, ensure_ascii=False)

# JSON -> XML
with open("biblioteca.json", encoding="utf-8") as f:
    datos = json.load(f)
raiz = ET.Element("biblioteca", nombre=datos["nombre"])
for l in datos["libros"]:
    e = ET.SubElement(raiz, "libro", isbn=l["isbn"],
                      prestado="si" if l["prestado"] else "no")
    for campo in ("titulo", "autor", "anio"):
        ET.SubElement(e, campo).text = str(l[campo])      # ¡XML sólo tiene texto!
    ET.SubElement(e, "precio", moneda="EUR").text = f"{l['precio']:.2f}"
arbol = ET.ElementTree(raiz)
ET.indent(arbol)
arbol.write("biblioteca_desde_json.xml", encoding="utf-8", xml_declaration=True)
```

1.9.3 Otros formatos

- **TOML**: formato de configuración (`pyproject.toml`). Python lo lee con `tomllib` (biblioteca estándar); para escribir hace falta `tomli-w`.
- **YAML**: muy usado en configuración (Docker Compose, GitHub Actions, Kubernetes). Se lee y escribe con `pyyaml` (`yaml.safe_load`, `yaml.safe_dump`).
- **HTML**: para generar informes, lo más sencillo es construir el texto a partir de los datos (con *f-strings* o un motor de plantillas como Jinja2) escapando los valores con `html.escape`.
- **Hojas de cálculo**: `openpyxl` lee y escribe `.xlsx`; `pandas` convierte tablas entre CSV, JSON, XML, Excel y SQL con una sola llamada (`read_csv`, `to_json`...).

```
import html, json

with open("biblioteca.json", encoding="utf-8") as f:
    datos = json.load(f)

filas = "\n".join(
    f"<tr><td>{html.escape(l['titulo'])}</td><td>{html.escape(l['autor'])}</td>"
    f"<td>{l['anio']}</td></tr>"
    for l in sorted(datos["libros"], key=lambda l: l["anio"]))
pagina = f"""<!DOCTYPE html>
<html lang="es"><head><meta charset="utf-8"><title>{html.escape(datos['nombre'])}</title
    ↪ ></head>
<body><h1>{html.escape(datos['nombre'])}</h1>
<table><tr><th>Título</th><th>Autor</th><th>Año</th></tr>
{filas}
</table></body></html>"""
with open("biblioteca.html", "w", encoding="utf-8") as f:
```

```
    f.write(pagina)
```

1.10 Extracción de datos de páginas web (*web scraping*)

Mucha información útil sólo está publicada como páginas web, pensadas para personas y no para programas. El **web scraping** consiste en descargar esas páginas y extraer de su HTML los datos que nos interesan para guardarlos en un formato estructurado (JSON, CSV, una base de datos). Es, en el fondo, otro caso de *conversión entre formatos*: de HTML a datos.

Un documento HTML es, como XML, un **árbol de elementos** (el DOM que manipula JavaScript en el navegador), pero los navegadores toleran HTML mal formado, por lo que no se debe procesar con un parser XML. Usaremos dos bibliotecas:

- **requests**: descarga páginas por HTTP.
- **Beautiful Soup** (`bs4`): analiza el HTML, aunque esté mal formado, y permite buscar elementos por etiqueta, atributos o selectores CSS.

```
$ pip install requests beautifulsoup4
```

Scraping responsable.

- Antes de nada, comprueba si el sitio ofrece una **API** o **datos abiertos** descargables: siempre es preferible.
- Lee las **condiciones de uso** del sitio y respeta su fichero `robots.txt`, que indica qué rutas no deben recorrer los programas.
- No sobrecargues el servidor: haz pausas entre peticiones (`time.sleep`), no descargues lo mismo dos veces (guarda una copia local) e identifícate con una cabecera `User-Agent`.
- Ten en cuenta la normativa de protección de datos (RGPD) si la página contiene datos personales, y la propiedad intelectual de los contenidos.

Para practicar usaremos `https://books.toscrape.com` y `https://quotes.toscrape.com`, sitios creados precisamente para aprender scraping.

1.10.1 Descargar una página

```python
import requests

CABECERAS = {"User-Agent": "ScraperDAM/1.0 (practicas de Acceso a Datos)"}

def descargar(url: str) -> str:
    respuesta = requests.get(url, headers=CABECERAS, timeout=10)
    respuesta.raise_for_status()                # excepción si es 404, 500...
    respuesta.encoding = respuesta.apparent_encoding or "utf-8"
    return respuesta.text

html = descargar("https://books.toscrape.com/")
```

1.10.2 Analizar el HTML con Beautiful Soup

Para saber qué buscar, abre la página en el navegador y usa las herramientas de desarrollo (*Inspeccionar elemento*). En la portada de *books.toscrape.com*, cada libro tiene este aspecto:

```
<article class="product_pod">
  <h3><a href="catalogue/a-light-in-the-attic_1000/index.html"
         title="A Light in the Attic">A Light in the ...</a></h3>
  <div class="product_price">
    <p class="price_color">£51.77</p>
    <p class="instock availability">In stock</p>
  </div>
  <p class="star-rating Three"></p>
</article>
```

`BeautifulSoup(html, "html.parser")`	Analiza el documento y devuelve el árbol.
`sopa.find("a")`	Primer elemento con esa etiqueta (o `None`).
`sopa.find_all("p", class_="price_color")`	Lista de elementos que cumplen las condiciones.
`sopa.select("article.product_pod h3 a")`	Elementos que cumplen un **selector CSS**.
`sopa.select_one(selector)`	Primer elemento que cumple el selector.
`elem.get_text(strip=True)`	Texto del elemento (y de sus descendientes).
`elem["href"]`, `elem.get("title")`	Valor de un atributo.
`elem.parent`, `elem.find_next_sibling()`	Navegación por el árbol.

```python
from bs4 import BeautifulSoup
from urllib.parse import urljoin

URL_BASE = "https://books.toscrape.com/"
ESTRELLAS = {"One": 1, "Two": 2, "Three": 3, "Four": 4, "Five": 5}

def extraer_libros(html: str, url_pagina: str) -> list[dict]:
    sopa = BeautifulSoup(html, "html.parser")
    libros = []
    for articulo in sopa.select("article.product_pod"):
        enlace = articulo.select_one("h3 a")
        precio = articulo.select_one("p.price_color").get_text(strip=True)
        clases = articulo.select_one("p.star-rating")["class"]   # lista de clases
        libros.append({
            "titulo": enlace["title"],
            "url": urljoin(url_pagina, enlace["href"]),         # URL absoluta
            "precio": float(precio.lstrip("£Â")),
            "valoracion": next((ESTRELLAS[c] for c in clases if c in ESTRELLAS), None),
            "disponible": "In stock" in articulo.select_one(".availability").get_text(),
        })
    return libros

def siguiente_pagina(html: str, url_pagina: str) -> str | None:
    enlace = BeautifulSoup(html, "html.parser").select_one("li.next a")
    return urljoin(url_pagina, enlace["href"]) if enlace else None
```

1.10.3 Recorrer varias páginas y guardar los datos

```
import json, time
import requests

def recorrer_catalogo(url: str, max_paginas: int = 5) -> list[dict]:
    resultado: list[dict] = []
    paginas = 0
    while url and paginas < max_paginas:
        try:
            html = descargar(url)
        except requests.RequestException as e:
            print(f"No se pudo descargar {url}: {e}")
            break
        resultado.extend(extraer_libros(html, url))
        url = siguiente_pagina(html, url)
        paginas += 1
        time.sleep(1)                        # !`no saturar el servidor!
    return resultado

libros = recorrer_catalogo(URL_BASE)
with open("libros_scraping.json", "w", encoding="utf-8") as f:
    json.dump(libros, f, indent=2, ensure_ascii=False)
print(f"{len(libros)} libros guardados")
```

1.10.4 Respetar `robots.txt`

```
from urllib.robotparser import RobotFileParser

robots = RobotFileParser("https://books.toscrape.com/robots.txt")
robots.read()
if not robots.can_fetch(CABECERAS["User-Agent"], url):
    print("robots.txt no permite descargar", url)
```

Páginas dinámicas. Beautiful Soup analiza el HTML que devuelve el servidor. Muchas webs actuales construyen su contenido en el navegador con JavaScript, y ese contenido no aparece en el HTML descargado. En esos casos, mira en las herramientas de desarrollo (pestaña *Red*) si la página obtiene los datos de una API en JSON (lo más sencillo es usarla directamente) o recurre a un navegador automatizado como **Playwright** o Selenium.

1.11 Excepciones en el manejo de ficheros

Casi cualquier operación con ficheros puede fallar por motivos ajenos al programa: el fichero no existe, no hay permisos, el disco está lleno, el contenido está corrupto... Un programa robusto debe **prever** estos casos y **tratarlos**.

Excepción	Cuándo se produce
`OSError`	Base de todos los errores del sistema de ficheros.
`FileNotFoundError`	El fichero o directorio no existe.
`FileExistsError`	Ya existe (modo `"x"`, `mkdir` sin `exist_ok`).
`PermissionError`	Sin permisos de lectura/escritura.
`IsADirectoryError`	Se intenta abrir un directorio como fichero.
`UnicodeDecodeError`	El contenido no corresponde a la codificación indicada.
`json.JSONDecodeError`	JSON mal formado (subclase de `ValueError`).
`csv.Error`	CSV mal formado.
`jsonschema.ValidationError`	El documento JSON no cumple su esquema.
`pydantic.ValidationError`	Los datos no se pueden convertir al modelo Pydantic.
`xml.etree.ElementTree.ParseError`	XML mal formado (con ElementTree).
`lxml.etree.XMLSyntaxError`	XML mal formado (con lxml).
`struct.error`	Datos binarios con tamaño o formato incorrecto.
`requests.RequestException`	Error de red al descargar una página: sin conexión, tiempo agotado (`Timeout`), respuesta 404 o 500 tras `raise_for_status()` (`HTTPError`).

```python
import json
from pathlib import Path

def cargar_configuracion(ruta: Path) -> dict:
    try:
        with open(ruta, encoding="utf-8") as f:
            return json.load(f)
    except FileNotFoundError:
        print(f"No existe {ruta}; se usará la configuración por defecto")
        return {}
    except PermissionError:
        raise SystemExit(f"Sin permiso para leer {ruta}")
    except json.JSONDecodeError as e:
        raise SystemExit(f"{ruta} no es JSON válido (línea {e.lineno}): {e.msg}")
```

> **¿Comprobar antes o capturar después?** Podríamos comprobar con `ruta.exists()` antes de abrir, pero entre la comprobación y la apertura otro proceso podría borrar el fichero. En Python se prefiere el estilo **EAFP** (*easier to ask forgiveness than permission*): intentar la operación y capturar la excepción.

1.12 Pruebas y documentación

El RA1 exige probar y documentar las aplicaciones. Aquí vemos un resumen aplicado a ficheros; el Anexo E lo explica con detalle (cómo se escriben y ejecutan las pruebas, pruebas parametrizadas, *fixtures*, cobertura y cómo documentar un *script*). Usaremos dos herramientas:

- **Docstrings**: cadenas de documentación al principio de módulos, clases y funciones. Las muestran los editores y `help()`, y herramientas como `pdoc` generan documentación

HTML a partir de ellas.

- **pytest**: el marco de pruebas más usado en Python. Las pruebas son funciones cuyo nombre empieza por `test_` y usan `assert`. La *fixture* `tmp_path` proporciona un directorio temporal limpio para cada prueba, ideal para probar código que trabaja con ficheros.

```python
# test_productos.py
import pytest
from productos import FicheroProductos, Producto

def test_anadir_y_leer(tmp_path):
    fp = FicheroProductos(tmp_path / "p.dat")
    pos = fp.anadir(Producto(1, "Teclado", 24.9, 10))
    assert pos == 0
    assert fp.leer(0).nombre == "Teclado"

def test_leer_posicion_inexistente(tmp_path):
    fp = FicheroProductos(tmp_path / "p.dat")
    with pytest.raises(IndexError):
        fp.leer(5)

def test_borrado_logico(tmp_path):
    fp = FicheroProductos(tmp_path / "p.dat")
    fp.anadir(Producto(1, "A", 1.0, 1))
    fp.anadir(Producto(2, "B", 2.0, 2))
    fp.borrar(0)
    assert [p.id for p in fp] == [2]
```

```
$ pip install pytest
$ pytest -v
```

1.13 Ejercicios

Ejercicio 1.1 — Explorador de directorios

Dificultad: ★

Escribe un programa que reciba una ruta por línea de comandos (`argparse`) y muestre su contenido en forma de árbol, con sangría según la profundidad, indicando el tamaño de cada fichero en un formato legible (B, KB, MB). Al final debe mostrar el número total de ficheros, de directorios y el tamaño total. Si la ruta no existe o no es un directorio, muestra un mensaje de error y termina con código 1. Añade una opción `--json` que, en lugar de mostrar el árbol, lo guarde como un documento JSON anidado.

Ejercicio 1.2 — Organizador de descargas

Dificultad: ★★

Programa una utilidad que ordene los ficheros de una carpeta moviéndolos a subcarpetas según su extensión (`Imagenes`, `Documentos`, `Audio`, `Comprimidos`, `Otros`). Las correspondencias extensión–carpeta se leen de un fichero `reglas.json` que se valida con un JSON Schema antes de usarlo. Si en el destino ya existe un fichero con el mismo nombre, añade un sufijo numérico. Incluye una opción `--simular` que muestre lo que haría sin

mover nada, y registra cada movimiento en `organizador.jsonl` (JSON Lines) con fecha, origen y destino.

Ejercicio 1.3 — Estadísticas de un fichero de texto

Dificultad: ★

Lee un fichero de texto (por ejemplo, un libro descargado del Proyecto Gutenberg) y genera un fichero `estadisticas.json` con: número de líneas, palabras y caracteres; las 20 palabras más frecuentes (ignorando mayúsculas y signos de puntuación) y la longitud media de palabra. Prueba a leerlo con una codificación incorrecta y trata la excepción.

Ejercicio 1.4 — Fichero de acceso aleatorio

Dificultad: ★★

Partiendo del ejemplo `FicheroProductos`, crea un fichero de acceso aleatorio de **empleados** (id, DNI, nombre, departamento, salario) en el que la posición del registro sea `id - 1`. Implementa: alta (si el id ya existe y está activo, error), consulta por id, modificación del salario, baja lógica, listado de activos y una operación `compactar()` que elimine físicamente los registros borrados. Añade la exportación de los empleados activos a JSON. Escribe pruebas con `pytest` para cada operación.

Ejercicio 1.5 — Serialización de objetos anidados

Dificultad: ★★

Modela una tienda con *dataclasses*: `Cliente`, `Pedido` (fecha y hora, estado como `Enum`, cliente, líneas) y `LineaPedido` (producto, cantidad, precio unitario como `Decimal`). Implementa `a_dict`/`desde_dict` en cada clase y funciones para guardar y cargar una lista de pedidos en JSON. Escribe una prueba que compruebe que, tras guardar y volver a cargar, los objetos son iguales (`==`), y otra que demuestre qué ocurriría con los importes si se guardaran como `float`.

Ejercicio 1.6 — Validación con JSON Schema

Dificultad: ★★

Escribe el JSON Schema de los pedidos del ejercicio anterior (usa `$defs` para el cliente y las líneas, `enum` para el estado, `format: date-time` para la fecha y `minItems` para que un pedido tenga al menos una línea). Crea un programa `validar.py esquema.json datos.json` que muestre todos los errores con la ruta del dato erróneo y termine con código 0 o 1 según el resultado. Prepara un fichero de datos con al menos cinco errores distintos.

Ejercicio 1.7 — Conversor de formatos

Dificultad: ★★

Desarrolla un conversor de línea de comandos: `python conversor.py entrada salida`, que convierta entre CSV, JSON, JSON Lines y XML deduciendo el formato de las extensiones. En la conversión desde CSV o XML intenta convertir automáticamente los valores numéricos y booleanos; en la conversión a CSV aplana los objetos anidados (`autor.nombre`). Permite indicar el delimitador y la codificación. Gestiona: fichero inexistente, extensión no soportada, JSON que no es una lista de objetos y filas CSV con distinto número de campos.

Ejercicio 1.8 — XML con ElementTree y XSD

Dificultad: ★★

A partir de `biblioteca.xml`: amplía el esquema XSD para que el ISBN cumpla un patrón (`xs:pattern`) y el año esté entre 1450 y 2100; valida varios documentos mostrando los errores con su número de línea; y, con ElementTree y XPath básico, obtén los títulos de los libros prestados, el autor del libro más caro y el número de libros por autor. Finalmente, genera `biblioteca.json` con todos los libros, con los tipos de datos correctos.

Ejercicio 1.9 — Primeros pasos con Beautiful Soup

Dificultad: ★★

Extrae de `https://quotes.toscrape.com` todas las citas (texto, autor y etiquetas) recorriendo todas las páginas, con una pausa de un segundo entre peticiones. Guárdalas en `citas.jsonl` y genera después `autores.json` con el número de citas de cada autor y sus etiquetas más frecuentes. Comprueba antes de empezar qué indica el `robots.txt` del sitio.

Proyecto: Agenda persistente multiformato

Amplía la agenda del proyecto del Tema 0 para que los contactos se guarden en ficheros.
Requisitos:

1. Define una clase abstracta `AlmacenContactos` (módulo `abc`) con los métodos `cargar() -> list[Contacto]` y `guardar(contactos)`.
2. Implementa cuatro almacenes: `AlmacenJSON` (el formato principal), `AlmacenCSV`, `AlmacenXML` (con ElementTree) y `AlmacenBinario` (registros de longitud fija con `struct`).
3. El almacén JSON valida el fichero con un **JSON Schema** al cargarlo e informa de los contactos erróneos sin detener la aplicación. Opcionalmente, modela `Contacto` con Pydantic.
4. La clase `Agenda` recibe el almacén en su constructor (no sabe qué formato se usa). El formato se elige en un fichero `config.json` o con un argumento de línea de comandos.
5. Opción de **exportar** la agenda a otro formato y de **importar** contactos desde un CSV, informando de las filas que no se pudieron importar y por qué.
6. Exportación de un listado en HTML.
7. Copias de seguridad: cada vez que se guarda, la versión anterior se copia a `backup/` con la fecha y hora en el nombre, conservando sólo las 5 más recientes.
8. Todas las excepciones de E/S se tratan; el programa nunca termina con una traza.
9. Pruebas con `pytest` (usando `tmp_path`) que comprueben que cada almacén guarda y recupera exactamente los mismos contactos.

Entrega: código, pruebas, JSON Schema de la agenda y `README.md` con instrucciones y una explicación razonada de las ventajas e inconvenientes de cada formato.

Proyecto: Catálogo de libros mediante web scraping

Construye una herramienta de línea de comandos que obtenga, mediante *scraping* con **requests** y **Beautiful Soup**, el catálogo completo de `https://books.toscrape.com` (unos 1000 libros en 50 categorías) y lo convierta en un conjunto de datos estructurado.

Requisitos:

1. **Recorrido**: obtener la lista de categorías desde la barra lateral y, para cada una, recorrer todas sus páginas (enlace *next*). Para cada libro, entrar en su página de detalle y extraer: título, categoría, precio, valoración (1–5), disponibilidad y número de unidades en stock, UPC, descripción y URL de la imagen.
2. **Modelo**: *dataclass* o modelo Pydantic `Libro`, con los tipos correctos (el precio como `Decimal`, el stock como entero…).
3. **Scraping responsable**: comprobar `robots.txt`, cabecera `User-Agent` propia, pausa configurable entre peticiones, tiempo máximo de espera y reintentos limitados ante errores de red.
4. **Caché en disco**: cada página descargada se guarda en una carpeta `cache/` (con un nombre de fichero derivado de la URL); si ya existe, se lee del disco en lugar de descargarla otra vez. Una opción `--sin-cache` fuerza la descarga.
5. **Reanudación**: los libros se van añadiendo a `libros.jsonl` a medida que se obtienen, y el progreso (categorías y páginas completadas) se guarda en `estado.json`, de modo que si el programa se interrumpe, al volver a ejecutarlo continúa donde lo dejó.
6. **Validación**: al terminar, todos los registros se validan con un JSON Schema; los que no lo cumplan se escriben en `errores.jsonl` con el motivo.
7. **Exportación**: generar `catalogo.json` (agrupado por categoría), `catalogo.csv` y `catalogo.xml`.
8. **Informe**: `informe.html` con el número de libros y el precio medio por categoría, los 10 libros mejor valorados más baratos y los libros sin stock.
9. **Registro**: un fichero de log con las URLs descargadas, los errores y el tiempo total.
10. **Pruebas**: guarda dos o tres páginas HTML de ejemplo en `tests/datos/` y prueba con `pytest` las funciones de extracción **sin acceder a Internet**.

Ampliación: añade una opción `--comparar` que, a partir de dos ejecuciones en fechas distintas, genere un JSON con los libros nuevos, los eliminados y los que han cambiado de precio o de stock.

TEMA 2

Manejo de conectores

En este tema aprenderás a…

- Comprender el desfase objeto-relacional y cómo afecta al diseño de las aplicaciones.
- Conocer los protocolos y conectores de acceso a bases de datos, en especial la especificación DB-API de Python.
- Conectar con un gestor embebido (SQLite) y con uno cliente-servidor (PostgreSQL).
- Ejecutar sentencias de definición y modificación de datos y consultas de forma segura, con parámetros.
- Recoger los resultados de una consulta en objetos del programa.
- Ejecutar funciones y procedimientos almacenados y gestionar transacciones.

Resultado de aprendizaje asociado: **RA2**.

2.1 El desfase objeto-relacional

Nuestros programas están escritos con un lenguaje **orientado a objetos**, pero los datos se guardan casi siempre en bases de datos **relacionales**, que organizan la información en tablas. Ambos modelos representan la realidad de forma distinta, y a esa diferencia se la llama **desfase objeto-relacional** (*object-relational impedance mismatch*).

Aspecto	Modelo de objetos	Modelo relacional
Estructura	Clases con atributos y métodos.	Tablas con columnas; sin comportamiento.
Tipos	Tipos ricos: listas, diccionarios, objetos anidados, enumerados.	Tipos atómicos (primera forma normal): no hay listas dentro de una celda.
Identidad	Cada objeto tiene identidad propia (`id(obj)`, `is`).	Un registro se identifica por el valor de su clave primaria.
Relaciones	Referencias directas entre objetos, en uno o ambos sentidos.	Claves ajenas; las relaciones N:M necesitan una tabla intermedia.
Navegación	Se recorre el grafo: `pedido.cliente.direccion`.	Se combinan tablas con `JOIN`.
Herencia	Nativa.	No existe; hay que simularla.
Encapsulación	Atributos internos y métodos.	Todos los datos son visibles.

Por ejemplo, un objeto `Pedido` con una lista de `Linea` y una referencia a su `Cliente` se convierte en tres tablas relacionadas por claves ajenas. Para guardar el pedido hay que

"desmontar" el objeto en filas, y para recuperarlo hay que ejecutar consultas y "volver a montarlo".

```python
@dataclass
class Cliente:
    id: int
    nombre: str

@dataclass
class Linea:
    producto: str
    cantidad: int
    precio: float

@dataclass
class Pedido:
    id: int
    cliente: Cliente              # referencia a otro objeto
    lineas: list[Linea]           # colección
```

```sql
CREATE TABLE cliente (id INTEGER PRIMARY KEY, nombre TEXT NOT NULL);
CREATE TABLE pedido  (id INTEGER PRIMARY KEY,
                      cliente_id INTEGER NOT NULL REFERENCES cliente(id));
CREATE TABLE linea   (pedido_id INTEGER REFERENCES pedido(id),
                      num INTEGER, producto TEXT, cantidad INTEGER, precio REAL,
                      PRIMARY KEY (pedido_id, num));
```

Este tema trata de cómo hacer esa traducción **a mano** usando conectores y SQL. En el Tema 3 veremos cómo las herramientas ORM la automatizan, y en el Tema 4, cómo las bases de datos objeto-relacionales y orientadas a objetos intentan reducir el desfase desde el lado de la base de datos.

2.2 Protocolos de acceso a bases de datos y conectores

Cada gestor de bases de datos (SGBD) tiene su propio protocolo de red y su propia forma de comunicarse con los clientes. Un **conector** (o *driver*) es una biblioteca que implementa ese protocolo y ofrece al programador una API para conectarse, enviar sentencias SQL y recibir los resultados.

Para no tener que aprender una API distinta para cada SGBD, existen **especificaciones estándar** que los conectores implementan:

ODBC (*Open Database Connectivity*). Estándar de Microsoft, independiente del lenguaje, escrito en C. Muy extendido en Windows. En Python se usa con `pyodbc`.

JDBC (*Java Database Connectivity*). El estándar de Java.

ADO.NET. El estándar de la plataforma .NET.

DB-API 2.0 (PEP 249). La especificación de Python. Define los objetos `Connection` y `Cursor`, sus métodos (`execute`, `fetchone`, `commit`…) y una jerarquía común de excepciones. Todos los conectores de Python la siguen, por lo que el código cambia muy poco de un SGBD a otro.

SGBD	Conector (paquete que se instala con `pip`)	Marcadores
SQLite	`sqlite3` (incluido en Python)	`?` y `:nombre`
PostgreSQL	`psycopg` versión 3 (`psycopg[binary]`)	`%s` y `%(nombre)s`
MySQL / MariaDB	`mysql.connector` (`mysql-connector-python`)	`%s` y `%(nombre)s`
Oracle	`oracledb`	`:1` y `:nombre`
SQL Server	`pyodbc`	`?`

2.2.1 Ventajas e inconvenientes de usar conectores

+ Control total sobre el SQL que se ejecuta: se pueden optimizar las consultas y aprovechar características propias del SGBD.
+ Rendimiento máximo y poca "magia": lo que se escribe es lo que se ejecuta.
+ Pocas dependencias y curva de aprendizaje baja si se conoce SQL.
– Hay que escribir mucho código repetitivo para convertir filas en objetos y viceversa (el desfase objeto-relacional se resuelve a mano).
– El SQL queda mezclado con el código y, si usa características propias de un SGBD, cuesta cambiar de gestor.
– Los errores en el SQL sólo se detectan al ejecutar.

2.2.2 Gestores embebidos y gestores cliente-servidor

Embebidos. El motor de base de datos es una biblioteca que se ejecuta **dentro del propio proceso** de la aplicación, y la base de datos es un fichero local. No hay servidor, ni usuarios, ni instalación. Ejemplos: **SQLite**, H2, DuckDB. Son ideales para aplicaciones de escritorio y móviles (Android e iOS incluyen SQLite), pruebas y prototipos.

Cliente-servidor (independientes). El SGBD es un proceso independiente, normalmente en otra máquina, al que se conectan muchos clientes por red. Gestionan usuarios, permisos y concurrencia elevada. Ejemplos: **PostgreSQL**, MySQL/MariaDB, Oracle, SQL Server.

En este tema usaremos los dos: **SQLite** (no requiere instalar nada) y **PostgreSQL** (en un contenedor Docker; consulta el Apéndice A).

2.3 Establecimiento de conexiones

2.3.1 SQLite

```python
import sqlite3

conexion = sqlite3.connect("instituto.db")   # crea el fichero si no existe
# conexion = sqlite3.connect(":memory:")     # base de datos en memoria
print(sqlite3.sqlite_version)
conexion.execute("PRAGMA foreign_keys = ON") # SQLite no las comprueba por defecto
conexion.close()
```

2.3.2 PostgreSQL

```
import psycopg

conexion = psycopg.connect(
    host="localhost", port=5432,
    dbname="instituto", user="dam", password="dam",
)
print(conexion.info.server_version)
conexion.close()
```

> **No escribas las contraseñas en el código.** Los datos de conexión deben leerse de la configuración (variables de entorno, un fichero `.env` o un fichero de configuración que no se sube al repositorio).

```
import os
import psycopg

def conectar() -> psycopg.Connection:
    return psycopg.connect(
        host=os.getenv("DB_HOST", "localhost"),
        port=int(os.getenv("DB_PORT", "5432")),
        dbname=os.getenv("DB_NAME", "instituto"),
        user=os.getenv("DB_USER", "dam"),
        password=os.environ["DB_PASSWORD"],     # obligatoria
    )
```

2.3.3 MySQL con su conector oficial

MySQL (y su derivado MariaDB, compatible con él) es uno de los gestores cliente-servidor más utilizados, sobre todo en aplicaciones web. Oracle, su propietaria, publica un conector oficial para Python que sigue la DB-API: `mysql-connector-python`. Para levantar un servidor MySQL de pruebas, consulta el Apéndice A.

```
$ pip install mysql-connector-python
```

```
import mysql.connector
from mysql.connector import errorcode

config = {
    "host": "localhost",
    "port": 3306,                  # puerto por defecto de MySQL
    "user": "dam",
    "password": "dam",
    "database": "instituto",
}

try:
    conexion = mysql.connector.connect(**config)
except mysql.connector.Error as e:
    if e.errno == errorcode.ER_ACCESS_DENIED_ERROR:
        raise SystemExit("Usuario o contraseña incorrectos")
    raise SystemExit(f"No se pudo conectar: {e}")
```

```
print("Conectado a MySQL", conexion.server_info)

with conexion:                                  # cierra la conexión al salir
    with conexion.cursor() as cur:
        cur.execute("""CREATE TABLE IF NOT EXISTS curso (
                           id INT AUTO_INCREMENT PRIMARY KEY,
                           nombre VARCHAR(30) NOT NULL UNIQUE)""")
        cur.execute("INSERT INTO curso (nombre) VALUES (%s)", ("2º DAM",))
        print("id generado:", cur.lastrowid)
    conexion.commit()                           # sin commit, el INSERT se pierde

    with conexion.cursor(dictionary=True) as cur:   # filas como diccionarios
        cur.execute("SELECT id, nombre FROM curso WHERE nombre LIKE %s", ("%DAM",))
        for fila in cur:
            print(fila["id"], fila["nombre"])
```

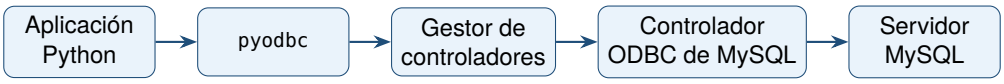
Lo que conviene saber de este conector:

- El marcador de parámetros es `%s` (también `%(nombre)s` con un diccionario), como en psycopg.
- La conexión **no** está en modo *autocommit*: hay que llamar a `commit()` para confirmar los cambios. Con `autocommit=True` en `connect()` se confirma cada sentencia.
- `cursor(dictionary=True)` devuelve cada fila como un diccionario, en lugar de una tupla.
- `with conexion:` **cierra** la conexión al salir, pero no hace `commit`.
- Todos los errores derivan de `mysql.connector.Error`; el atributo `errno` contiene el código de error de MySQL, y el módulo `errorcode` define constantes para cada uno (`ER_ACCESS_DENIED_ERROR`, `ER_DUP_ENTRY`, `ER_NO_SUCH_TABLE`…).
- Para los procedimientos almacenados ofrece `cursor.callproc("nombre", argumentos)`.

Nota. Además del conector oficial existen otros conectores para MySQL que siguen la DB-API, como `PyMySQL` (escrito sólo en Python) o `mysqlclient`. El conector oficial de MariaDB se llama `mariadb`. Todos se usan de forma muy parecida.

2.3.4 Conexión mediante ODBC

ODBC (*Open Database Connectivity*) es un estándar para acceder a bases de datos independientemente del gestor. Su idea es separar la aplicación del gestor concreto mediante dos piezas que se instalan en el sistema operativo:



- El **gestor de controladores** (*driver manager*) forma parte del sistema: en Windows viene incluido ("Orígenes de datos ODBC" en el Panel de control) y en Linux y macOS se usa **unixODBC**.
- Cada gestor de bases de datos proporciona su **controlador ODBC** (*driver*). Para MySQL es **MySQL Connector/ODBC**, que se descarga de la web de MySQL; para MariaDB, **MariaDB Connector/ODBC**.

- Desde Python se accede con `pyodbc`, que sigue la DB-API.

```
$ pip install pyodbc
```

Lo primero es comprobar qué controladores hay instalados y cómo se llaman exactamente, porque ese nombre es el que se usa en la conexión (incluye la versión instalada del controlador: por ejemplo, `MySQL ODBC 26.7 Unicode Driver`):

```
import pyodbc
print(pyodbc.drivers())     # ['MySQL ODBC 26.7 Unicode Driver', 'SQL Server', ...]
```

La conexión se describe con una **cadena de conexión** formada por pares `CLAVE=valor` separados por punto y coma. Hay dos formas:

Indicando el controlador y los datos de conexión (*DSN-less*):

```
import pyodbc

DRIVER = "MySQL ODBC 26.7 Unicode Driver"     # el nombre que muestra pyodbc.drivers()
cadena = (
    f"DRIVER={{{DRIVER}}};"                   # {{ }} -> llaves literales en la f-string
    "SERVER=localhost;PORT=3306;DATABASE=instituto;"
    "UID=dam;PWD=dam;CHARSET=utf8mb4"
)

try:
    conexion = pyodbc.connect(cadena)
except pyodbc.Error as e:
    raise SystemExit(f"No se pudo conectar: {e}")

print(conexion.getinfo(pyodbc.SQL_DBMS_NAME), conexion.getinfo(pyodbc.SQL_DBMS_VER))

with conexion:                                # hace commit al salir (rollback si hay
    ↪ error)
    cur = conexion.cursor()
    cur.execute("INSERT INTO curso (nombre) VALUES (?)", "1º DAM")
    cur.execute("SELECT id, nombre FROM curso ORDER BY id")
    for fila in cur.fetchall():
        print(fila.id, fila.nombre)           # acceso por nombre de columna
conexion.close()                              # el with NO cierra la conexión
```

Mediante un DSN. Un **DSN** (*Data Source Name*) es un origen de datos con nombre configurado en el sistema que guarda el controlador, el servidor, el puerto y la base de datos. En Windows se crea con la herramienta "Orígenes de datos ODBC"; en Linux, en el fichero `/etc/odbc.ini`:

```
[instituto]
Description = Base de datos del instituto (MySQL)
Driver      = MySQL ODBC 26.7 Unicode Driver
Server      = localhost
Port        = 3306
Database    = instituto
```

El programa sólo necesita conocer el nombre del DSN y las credenciales; si la base de datos cambia de servidor, basta con modificar el DSN, sin tocar el código:

```
conexion = pyodbc.connect("DSN=instituto;UID=dam;PWD=dam")
print(pyodbc.dataSources())                # {'instituto': 'MySQL ODBC 26.7 Unicode
    ↪ Driver'}
```

Diferencias con los conectores nativos que conviene recordar:

- El marcador de parámetros es siempre `?`, sea cual sea el gestor.
- Las filas (`pyodbc.Row`) se comportan como tuplas, pero también permiten acceder a las columnas por nombre (`fila.nombre`).
- `with conexion:` confirma la transacción al salir, pero no cierra la conexión.
- Los errores derivan de `pyodbc.Error` y su mensaje incluye el código de estado ODBC (*SQLSTATE*).

¿Conector nativo u ODBC? Con ODBC, el mismo programa puede trabajar con MySQL, SQL Server, Oracle o PostgreSQL cambiando sólo la cadena de conexión o el DSN, y es la única opción para algunos sistemas (como Microsoft Access o ciertas bases de datos corporativas). A cambio, añade una capa más, exige instalar y configurar el controlador en cada equipo y aprovecha peor las características propias de cada gestor. Por eso, en Python se suele preferir el conector nativo y reservar ODBC para cuando no existe otro o se necesita trabajar con varios gestores de forma uniforme.

2.3.5 Cerrar las conexiones

Una conexión consume recursos tanto en el cliente como en el servidor (que admite un número limitado de conexiones simultáneas). Debe cerrarse cuando ya no se necesita. Lo más seguro es usar `with`:

```
# psycopg: al salir del with se hace commit (o rollback si hubo error)
# y se CIERRA la conexión
with psycopg.connect(conninfo) as conn:
    ...

# sqlite3: al salir del with se hace commit/rollback pero NO se cierra
from contextlib import closing
with closing(sqlite3.connect("instituto.db")) as conn:
    with conn:            # transacción
        ...
```

Pool de conexiones. Abrir una conexión con un servidor es una operación costosa. Las aplicaciones con muchos usuarios (por ejemplo, servidores web) mantienen un **pool**: un conjunto de conexiones abiertas que se reutilizan. Para psycopg existe el paquete `psycopg-pool`.

2.4 Ejecución de sentencias de descripción de datos (DDL)

Las sentencias DDL (`CREATE`, `ALTER`, `DROP`) definen la estructura de la base de datos. Se ejecutan como cualquier otra sentencia, a través de un **cursor**, que es el objeto que envía las sentencias y recibe los resultados.

```
import sqlite3

ESQUEMA = """
CREATE TABLE IF NOT EXISTS curso (
    id      INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre  TEXT NOT NULL UNIQUE
);
CREATE TABLE IF NOT EXISTS alumno (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    dni        TEXT NOT NULL UNIQUE,
    nombre     TEXT NOT NULL,
    apellidos  TEXT NOT NULL,
    nacimiento TEXT,
    curso_id   INTEGER REFERENCES curso(id) ON DELETE SET NULL
);
"""

with sqlite3.connect("instituto.db") as conn:
    conn.executescript(ESQUEMA)      # varias sentencias separadas por ';'
```

En PostgreSQL los tipos y la sintaxis cambian ligeramente:

```
CREATE TABLE IF NOT EXISTS alumno (
    id         SERIAL PRIMARY KEY,           -- o INTEGER GENERATED ALWAYS AS IDENTITY
    dni        VARCHAR(9) NOT NULL UNIQUE,
    nombre     VARCHAR(50) NOT NULL,
    apellidos  VARCHAR(100) NOT NULL,
    nacimiento DATE,
    curso_id   INTEGER REFERENCES curso(id) ON DELETE SET NULL
);
```

```
with psycopg.connect(conninfo) as conn:
    conn.execute(open("esquema_pg.sql", encoding="utf-8").read())
```

2.5 Ejecución de sentencias de modificación de datos (DML)

2.5.1 Sentencias parametrizadas

Inyección SQL. **Nunca** construyas una sentencia SQL concatenando o formateando valores introducidos por el usuario:

```
dni = input("DNI: ")
cur.execute(f"DELETE FROM alumno WHERE dni = '{dni}'")   # !`!`MAL!!
```

Si el usuario escribe `' OR '1'='1`, la sentencia resultante borra **todos** los alumnos. Este ataque se llama **inyección SQL** y sigue estando entre las vulnerabilidades más

explotadas.

La forma correcta es usar **marcadores de parámetro** y pasar los valores aparte. El conector los envía al servidor de forma segura y además se encarga de convertir los tipos (fechas, `None` → `NULL`, etc.).

```
# SQLite: marcador ?
cur.execute("INSERT INTO alumno (dni, nombre, apellidos) VALUES (?, ?, ?)",
            ("12345678Z", "Ana", "García"))
# SQLite: marcadores con nombre
cur.execute("UPDATE alumno SET curso_id = :curso WHERE dni = :dni",
            {"curso": 2, "dni": "12345678Z"})

# PostgreSQL / MySQL: marcador %s (¡aunque el valor no sea una cadena!)
cur.execute("INSERT INTO alumno (dni, nombre, apellidos, nacimiento) "
            "VALUES (%s, %s, %s, %s)",
            ("12345678Z", "Ana", "García", date(2005, 3, 14)))
```

Nota. Los parámetros siempre se pasan como una **secuencia** (tupla o lista) o un diccionario. Para un único parámetro hay que escribir la tupla con coma: `(dni,)`. Los marcadores sólo sirven para *valores*, no para nombres de tablas o columnas. Si necesitas componer identificadores dinámicamente en PostgreSQL, usa `psycopg.sql.Identifier`.

2.5.2 Inserciones, modificaciones y borrados

```
import sqlite3

with sqlite3.connect("instituto.db") as conn:
    cur = conn.cursor()

    cur.execute("INSERT INTO curso (nombre) VALUES (?)", ("2º DAM",))
    curso_id = cur.lastrowid                # id generado automáticamente

    alumnos = [
        ("11111111H", "Ana", "García", "2005-03-14", curso_id),
        ("22222222J", "Luis", "Pérez", "2004-11-02", curso_id),
    ]
    cur.executemany(
        "INSERT INTO alumno (dni, nombre, apellidos, nacimiento, curso_id) "
        "VALUES (?, ?, ?, ?, ?)", alumnos)
    print(cur.rowcount, "alumnos insertados")

    cur.execute("UPDATE alumno SET apellidos = ? WHERE dni = ?",
                ("García López", "11111111H"))
    if cur.rowcount == 0:
        print("No existe ese alumno")

    cur.execute("DELETE FROM alumno WHERE dni = ?", ("22222222J",))
    cur.close()
# al salir del with: commit
```

En PostgreSQL (y en SQLite desde la versión 3.35) se puede usar `RETURNING` para obtener los valores generados:

```
cur.execute("INSERT INTO curso (nombre) VALUES (%s) RETURNING id", ("1º DAM",))
nuevo_id = cur.fetchone()[0]
```

2.6 Ejecución de consultas

2.6.1 Recuperar filas

Tras ejecutar un `SELECT`, el cursor permite obtener las filas resultantes:

`fetchone()`	Devuelve la siguiente fila (una tupla) o `None` si no quedan.
`fetchmany(n)`	Devuelve una lista con hasta `n` filas.
`fetchall()`	Devuelve una lista con todas las filas restantes.
`for fila in cur`	Itera sobre las filas una a una (recomendado para muchas filas).
`cur.description`	Metadatos de las columnas del resultado (nombre, tipo...).

```
with closing(sqlite3.connect("instituto.db")) as conn:
    cur = conn.execute(
        "SELECT a.dni, a.nombre, a.apellidos, c.nombre "
        "FROM alumno a LEFT JOIN curso c ON a.curso_id = c.id "
        "WHERE a.apellidos LIKE ? ORDER BY a.apellidos", ("G%",))

    print([col[0] for col in cur.description])   # nombres de columnas
    for dni, nombre, apellidos, curso in cur:
        print(f"{dni} {apellidos}, {nombre} ({curso or 'sin curso'})")

    total = conn.execute("SELECT COUNT(*) FROM alumno").fetchone()[0]
```

2.6.2 Objetos para almacenar el resultado de las consultas

Trabajar con tuplas y acceder a las columnas por posición es frágil. Es mejor que cada fila se convierta en un objeto del dominio. Hay varias opciones:

Filas con acceso por nombre. En SQLite se configura `row_factory`:

```
conn.row_factory = sqlite3.Row
fila = conn.execute("SELECT * FROM alumno WHERE id = ?", (1,)).fetchone()
print(fila["nombre"], fila["apellidos"], dict(fila))
```

Fábricas de filas en psycopg. psycopg incluye varias en `psycopg.rows`; `class_row` construye directamente objetos de una clase:

```
from dataclasses import dataclass
from datetime import date
from psycopg.rows import dict_row, class_row

@dataclass
class Alumno:
```

```
    id: int
    dni: str
    nombre: str
    apellidos: str
    nacimiento: date | None
    curso_id: int | None

with psycopg.connect(conninfo) as conn:
    with conn.cursor(row_factory=class_row(Alumno)) as cur:
        cur.execute("SELECT * FROM alumno ORDER BY apellidos")
        alumnos: list[Alumno] = cur.fetchall()

    with conn.cursor(row_factory=dict_row) as cur:
        cur.execute("SELECT nombre, apellidos FROM alumno")
        for d in cur:
            print(d["nombre"])
```

2.6.3 El patrón DAO

Para no repartir SQL por toda la aplicación, se agrupa el acceso a cada entidad en una clase **DAO** (*Data Access Object*) o **repositorio**. El resto del programa sólo trabaja con objetos y no sabe qué SGBD hay detrás.

```
import sqlite3
from dataclasses import dataclass

@dataclass
class Alumno:
    dni: str
    nombre: str
    apellidos: str
    nacimiento: str | None = None
    curso_id: int | None = None
    id: int | None = None


class AlumnoDAO:
    def __init__(self, conn: sqlite3.Connection) -> None:
        self.conn = conn

    @staticmethod
    def _a_objeto(fila: sqlite3.Row) -> Alumno:
        return Alumno(**dict(fila))

    def insertar(self, a: Alumno) -> Alumno:
        cur = self.conn.execute(
            "INSERT INTO alumno (dni, nombre, apellidos, nacimiento, curso_id) "
            "VALUES (?, ?, ?, ?, ?)",
            (a.dni, a.nombre, a.apellidos, a.nacimiento, a.curso_id))
        a.id = cur.lastrowid
        return a

    def buscar_por_id(self, ident: int) -> Alumno | None:
        fila = self.conn.execute("SELECT * FROM alumno WHERE id = ?",
                                 (ident,)).fetchone()
        return self._a_objeto(fila) if fila else None
```

```
    def listar(self) -> list[Alumno]:
        cur = self.conn.execute("SELECT * FROM alumno ORDER BY apellidos, nombre")
        return [self._a_objeto(f) for f in cur]

    def actualizar(self, a: Alumno) -> bool:
        cur = self.conn.execute(
            "UPDATE alumno SET dni=?, nombre=?, apellidos=?, nacimiento=?, "
            "curso_id=? WHERE id=?",
            (a.dni, a.nombre, a.apellidos, a.nacimiento, a.curso_id, a.id))
        return cur.rowcount == 1

    def borrar(self, ident: int) -> bool:
        return self.conn.execute("DELETE FROM alumno WHERE id = ?",
                                 (ident,)).rowcount == 1


if __name__ == "__main__":
    conn = sqlite3.connect("instituto.db")
    conn.row_factory = sqlite3.Row
    dao = AlumnoDAO(conn)
    with conn:                                    # una transacción
        nuevo = dao.insertar(Alumno("33333333P", "Eva", "Martín"))
    print(dao.buscar_por_id(nuevo.id))
    conn.close()
```

2.7 Ejecución de procedimientos almacenados

Un **procedimiento almacenado** es un programa escrito en un lenguaje procedimental del SGBD (PL/pgSQL en PostgreSQL, PL/SQL en Oracle, SQL/PSM en MySQL) que se guarda y ejecuta **dentro del servidor**. Ventajas: reducen el tráfico de red, centralizan reglas de negocio y permiten conceder permisos de ejecución sin dar acceso directo a las tablas. Inconvenientes: atan la aplicación a un SGBD concreto y son más difíciles de versionar y probar.

> **Nota.** SQLite no tiene procedimientos almacenados: al ser embebido, la lógica se escribe directamente en la aplicación. Para este apartado usaremos PostgreSQL.

PostgreSQL distingue entre **funciones** (devuelven un valor o un conjunto de filas y se usan dentro de un `SELECT`) y **procedimientos** (se invocan con `CALL` y pueden gestionar transacciones).

```
-- Función que devuelve un valor
CREATE OR REPLACE FUNCTION num_alumnos(p_curso INTEGER)
RETURNS INTEGER LANGUAGE sql AS $$
    SELECT COUNT(*) FROM alumno WHERE curso_id = p_curso;
$$;

-- Función que devuelve filas
CREATE OR REPLACE FUNCTION alumnos_de(p_curso TEXT)
RETURNS TABLE (dni VARCHAR, nombre VARCHAR, apellidos VARCHAR)
LANGUAGE sql AS $$
    SELECT a.dni, a.nombre, a.apellidos
    FROM alumno a JOIN curso c ON a.curso_id = c.id
    WHERE c.nombre = p_curso;
```

```
$$;

-- Procedimiento con parámetro de salida
CREATE OR REPLACE PROCEDURE matricular(p_dni VARCHAR, p_curso TEXT,
                                       INOUT p_resultado TEXT DEFAULT NULL)
LANGUAGE plpgsql AS $$
DECLARE v_curso INTEGER;
BEGIN
    SELECT id INTO v_curso FROM curso WHERE nombre = p_curso;
    IF v_curso IS NULL THEN
        RAISE EXCEPTION 'No existe el curso %', p_curso;
    END IF;
    UPDATE alumno SET curso_id = v_curso WHERE dni = p_dni;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'No existe el alumno %', p_dni;
    END IF;
    p_resultado := 'Alumno ' || p_dni || ' matriculado en ' || p_curso;
END;
$$;
```

Desde Python se invocan con sentencias normales:

```
import psycopg

with psycopg.connect(conninfo) as conn:
    n = conn.execute("SELECT num_alumnos(%s)", (1,)).fetchone()[0]

    for dni, nombre, apellidos in conn.execute(
            "SELECT * FROM alumnos_de(%s)", ("2º DAM",)):
        print(dni, nombre, apellidos)

    try:
        fila = conn.execute("CALL matricular(%s, %s, NULL)",
                            ("11111111H", "2º DAM")).fetchone()
        print(fila[0])        # valor del parámetro INOUT
    except psycopg.errors.RaiseException as e:
        print("Error del procedimiento:", e.diag.message_primary)
```

Nota. En MySQL, el conector `mysql-connector-python` ofrece además el método `cursor.callproc("nombre", args)` que define la DB-API para llamar a procedimientos.

2.8 Gestión de transacciones

Una **transacción** es un conjunto de operaciones que se ejecutan como una unidad: o se realizan todas, o no se realiza ninguna. El ejemplo clásico es una transferencia bancaria: restar de una cuenta y sumar en otra no puede quedarse a medias. Las transacciones garantizan las propiedades **ACID**:

Atomicidad. Todo o nada.

Consistencia. La base de datos pasa de un estado válido a otro estado válido (se respetan las restricciones).

Aislamiento. Las transacciones concurrentes no ven los cambios a medio hacer de las demás

(según el nivel de aislamiento).

Durabilidad. Una vez confirmada, la transacción persiste aunque falle el sistema.

2.8.1 Transacciones con la DB-API

Según la DB-API, una conexión **no está en modo autocommit**: la primera sentencia inicia implícitamente una transacción, que dura hasta que se llama a `commit()` (confirmar) o `rollback()` (deshacer).

```
def transferir(conn, origen: str, destino: str, cantidad: float) -> None:
    cur = conn.cursor()
    try:
        cur.execute("UPDATE cuenta SET saldo = saldo - ? WHERE iban = ?",
                    (cantidad, origen))
        cur.execute("SELECT saldo FROM cuenta WHERE iban = ?", (origen,))
        if cur.fetchone()[0] < 0:
            raise ValueError("Saldo insuficiente")
        cur.execute("UPDATE cuenta SET saldo = saldo + ? WHERE iban = ?",
                    (cantidad, destino))
        if cur.rowcount != 1:
            raise ValueError("La cuenta destino no existe")
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        cur.close()
```

El mismo comportamiento se consigue de forma más compacta con `with conn:` en `sqlite3`, que hace `commit` si el bloque termina bien y `rollback` si se lanza una excepción.

> **El modo de transacciones de `sqlite3`.** Por compatibilidad histórica, el módulo `sqlite3` no inicia transacciones antes de las sentencias DDL y tiene un comportamiento algo peculiar. Desde Python 3.12 se recomienda abrir la conexión con `sqlite3.connect("bd.db", autocommit=False)`, que sigue exactamente la DB-API. Con `autocommit=True` cada sentencia se confirma inmediatamente.

2.8.2 Transacciones en psycopg

psycopg ofrece el bloque `conn.transaction()`, que además permite anidar transacciones mediante *savepoints*: si falla el bloque interior, sólo se deshace esa parte.

```
with psycopg.connect(conninfo) as conn:
    with conn.transaction():
        conn.execute("INSERT INTO curso (nombre) VALUES (%s)", ("1º ASIR",))
        for dni, nombre, apellidos in nuevos_alumnos:
            try:
                with conn.transaction():          # SAVEPOINT
                    conn.execute(
                        "INSERT INTO alumno (dni, nombre, apellidos) "
                        "VALUES (%s, %s, %s)", (dni, nombre, apellidos))
            except psycopg.errors.UniqueViolation:
                print(f"DNI duplicado: {dni}; se omite")
    # aquí se confirma la transacción exterior
```

2.8.3 Niveles de aislamiento

`READ UNCOMMITTED`	Puede leer datos no confirmados por otras transacciones (lecturas sucias).
`READ COMMITTED`	Sólo lee datos confirmados. Es el nivel por defecto en PostgreSQL.
`REPEATABLE READ`	Si se relee una fila, se obtiene el mismo valor durante toda la transacción.
`SERIALIZABLE`	Equivale a ejecutar las transacciones una detrás de otra. Máxima seguridad, menor concurrencia.

```
from psycopg import IsolationLevel
conn.isolation_level = IsolationLevel.SERIALIZABLE
```

2.9 Excepciones de la DB-API

Todos los conectores definen la misma jerarquía de excepciones, accesible como atributos del módulo (`sqlite3.IntegrityError`, `psycopg.IntegrityError`...):

```
Exception
 └── Error
      ├── InterfaceError        error del propio conector
      └── DatabaseError         error de la base de datos
           ├── DataError        valor fuera de rango, tipo incorrecto...
           ├── OperationalError conexión perdida, BD no encontrada, bloqueo...
           ├── IntegrityError   clave duplicada, clave ajena, NOT NULL...
           ├── InternalError
           ├── ProgrammingError tabla inexistente, error de sintaxis SQL...
           └── NotSupportedError
```

```
try:
    with conn:
        dao.insertar(alumno)
except sqlite3.IntegrityError as e:
    print("No se pudo dar de alta: ya existe un alumno con ese DNI")
except sqlite3.OperationalError as e:
    print("Problema con la base de datos:", e)
```

psycopg define además subclases específicas para cada código de error de PostgreSQL en `psycopg.errors` (`UniqueViolation`, `ForeignKeyViolation`, `UndefinedTable`...).

2.10 Ejercicios

Ejercicio 2.1 — Primeros pasos con SQLite

Dificultad: ★

Crea una base de datos SQLite `biblioteca.db` con las tablas `autor`, `libro` (con clave ajena a autor) y `socio`. Escribe un programa que cree las tablas si no existen, inserte datos de ejemplo con `executemany` y muestre: todos los libros con el nombre de su autor; el número de libros por autor; los autores sin libros. Muestra los nombres de las columnas usando `cursor.description`.

Ejercicio 2.2 — Inyección SQL

Dificultad: ★

Escribe una función de *login* vulnerable que compruebe usuario y contraseña construyendo el SQL con una f-string, y demuestra que se puede entrar sin conocer la contraseña. Corrígela usando parámetros y almacenando las contraseñas con un *hash* (`hashlib.scrypt` o `hashlib.pbkdf2_hmac` con sal). Explica en un comentario por qué la versión corregida es segura.

Ejercicio 2.3 — De CSV a base de datos

Dificultad: ★★

Descarga un conjunto de datos abiertos en CSV (por ejemplo, del portal de datos abiertos de la Junta de Castilla y León) e impórtalo en SQLite: el programa debe crear la tabla deduciendo los nombres de columna de la cabecera, insertar los datos en una única transacción con `executemany` e informar de cuántas filas se han insertado y cuánto ha tardado. Compara el tiempo con el de hacer un `commit` por fila.

Ejercicio 2.4 — Mismo DAO, dos gestores

Dificultad: ★★

Implementa un DAO para la tabla `libro` del ejercicio 2.1 con una versión para SQLite y otra para PostgreSQL que compartan la misma interfaz (clase abstracta). El programa principal debe funcionar con cualquiera de las dos cambiando sólo una línea de configuración. Anota qué diferencias has tenido que resolver entre ambos gestores.

Ejercicio 2.5 — Procedimientos almacenados

Dificultad: ★★

En PostgreSQL, crea para la base de datos de la biblioteca: una función `libros_disponibles(autor TEXT)` que devuelva una tabla; y un procedimiento `prestar(isbn, id_socio)` que compruebe que el libro está disponible y que el socio no tiene más de 3 préstamos activos, registre el préstamo y lance una excepción con un mensaje claro en otro caso. Llámalos desde Python y trata las excepciones mostrando el mensaje del servidor.

Ejercicio 2.6 — Transacciones

Dificultad: ★★★

Crea una tabla `cuenta(iban, titular, saldo)` con la restricción `CHECK (saldo >= 0)` e

implementa `transferir(origen, destino, cantidad)`. Demuestra con dos programas ejecutándose a la vez (o dos hilos) qué ocurre si dos transferencias concurrentes leen el saldo antes de modificarlo. Soluciónalo usando `SELECT ... FOR UPDATE` o el nivel de aislamiento `SERIALIZABLE` con reintento, y explica la diferencia entre ambas soluciones.

Proyecto: Gestión de un videoclub (o plataforma de alquiler)

Desarrolla una aplicación de consola para gestionar el alquiler de películas.
Modelo de datos: `cliente`, `pelicula`, `copia` (cada película tiene varias copias físicas o licencias), `alquiler` (copia, cliente, fecha de alquiler, fecha prevista de devolución, fecha real de devolución, importe).
Requisitos:

1. Script SQL de creación con todas las restricciones (claves primarias, ajenas, `NOT NULL`, `UNIQUE`, `CHECK`).
2. Capa de acceso a datos con un DAO por entidad y *dataclasses* para el modelo. Los DAO nunca hacen `commit`: las transacciones se gestionan en una capa de servicio.
3. Operaciones: CRUD de clientes y películas; alquilar (comprueba que haya copia disponible y que el cliente no tenga alquileres vencidos); devolver (calcula el recargo por retraso); listados de películas disponibles, alquileres vencidos y ranking de películas más alquiladas.
4. Las operaciones de alquiler y devolución son transaccionales.
5. La aplicación funciona con SQLite **y** con PostgreSQL. En PostgreSQL, la devolución se implementa como procedimiento almacenado.
6. Configuración de la conexión en un fichero `.env` o `config.toml` (`tomllib`) que no se incluye en el repositorio.
7. Exportación del historial de alquileres de un cliente a CSV y JSON (reutilizando lo aprendido en el Tema 1).
8. Pruebas con `pytest` sobre una base de datos SQLite en memoria.

TEMA 3

Herramientas de mapeo objeto-relacional (ORM)

En este tema aprenderás a…

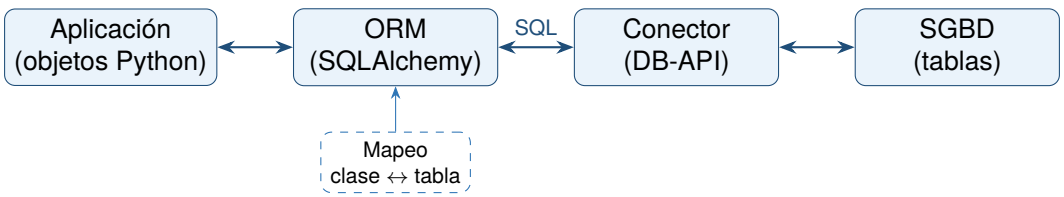
- Explicar qué es un ORM, qué problema resuelve y cuáles son las herramientas más utilizadas.
- Instalar y configurar SQLAlchemy y conectarlo con SQLite y PostgreSQL.
- Definir el mapeo de clases a tablas, incluyendo colecciones, relaciones y herencia.
- Comprender las sesiones y los estados de un objeto persistente.
- Cargar, guardar, modificar y borrar objetos; consultar con SQL y con el lenguaje propio del ORM.
- Gestionar transacciones con el ORM.

Resultado de aprendizaje asociado: **RA3**.

3.1 Concepto de mapeo objeto-relacional

En el tema anterior resolvimos el desfase objeto-relacional a mano: cada DAO contenía el SQL necesario y el código para convertir filas en objetos. Un **ORM** (*Object-Relational Mapping*) es una herramienta que automatiza esa traducción. A partir de una descripción de **cómo se corresponden las clases con las tablas** (el *mapeo*), el ORM:

- Genera las sentencias SQL para insertar, modificar, borrar y recuperar objetos.
- Convierte los tipos de datos en ambos sentidos.
- Traduce las relaciones entre objetos (referencias y colecciones) en claves ajenas y `JOIN`.
- Lleva la cuenta de qué objetos se han modificado para guardar sólo los cambios.
- Permite escribir consultas en términos de clases y atributos en lugar de tablas y columnas.



3.1.1 Ventajas e inconvenientes

+ Mucho menos código repetitivo: desaparecen los DAO escritos a mano.
+ Se programa con objetos; el modelo de dominio es el centro de la aplicación.

+ Independencia del SGBD: el ORM genera el SQL de cada dialecto.
+ Funcionalidades "gratis": caché de objetos, carga perezosa, control de cambios, protección frente a inyección SQL.
– Curva de aprendizaje: hay que entender cómo y cuándo el ORM ejecuta SQL.
– Riesgo de consultas ineficientes si no se controla (el famoso problema **N+1**).
– Sobrecarga de rendimiento en operaciones masivas.
– Algunas consultas complejas o características específicas del SGBD siguen requiriendo SQL.

3.2 Características de las herramientas ORM

Los ORM se suelen clasificar según el patrón que implementan:

Active Record. Cada objeto sabe guardarse a sí mismo (`alumno.save()`). Sencillo, pero mezcla el modelo con la persistencia. Ejemplos: Django ORM, Peewee, Ruby on Rails.

Data Mapper. Los objetos del modelo no saben nada de la base de datos; un componente aparte (la **sesión**) se encarga de guardarlos. Más flexible. Ejemplos: SQLAlchemy, Hibernate/JPA, Doctrine.

Lenguaje	ORM	Comentario
Python	**SQLAlchemy**	El ORM de referencia en Python; Data Mapper. Lo usaremos en el curso.
Python	Django ORM	Integrado en el framework web Django; Active Record.
Python	SQLModel, Peewee, Pony, Tortoise	Alternativas más ligeras o asíncronas. SQLModel está construido sobre SQLAlchemy.
Java	Hibernate / JPA	Estándar *de facto* en Java. Lenguaje de consultas HQL/JPQL.
C#	Entity Framework Core	ORM oficial de .NET. Consultas con LINQ.
PHP	Doctrine, Eloquent	Symfony y Laravel, respectivamente.
JS/TS	Prisma, TypeORM, Sequelize	Ecosistema Node.js.

3.3 Instalación de SQLAlchemy

```
$ pip install sqlalchemy               # el ORM
$ pip install "psycopg[binary]"        # el conector de PostgreSQL (si se usa)
$ python -c "import sqlalchemy; print(sqlalchemy.__version__)"
```

SQLAlchemy no sustituye al conector: lo usa por debajo. Para SQLite no hace falta instalar nada más.

> **Nota.** Todos los ejemplos usan la API de **SQLAlchemy 2.x**. En Internet encontrarás mucho código con el estilo antiguo (1.x: `session.query(...)`, `Column(...)` sin `Mapped`); sigue funcionando, pero no lo mezcles con el nuevo.

3.4 Configuración de la herramienta

3.4.1 El motor (*engine*)

El objeto `Engine` representa la base de datos: sabe qué dialecto de SQL usar, qué conector y cómo obtener conexiones (gestiona un *pool*). Se crea una única vez en la aplicación a partir de una **URL de conexión**:

```
dialecto+conector://usuario:contraseña@servidor:puerto/basededatos
```

```
from sqlalchemy import create_engine

engine = create_engine("sqlite:///instituto.db", echo=True)
engine = create_engine("sqlite:///:memory:")
engine = create_engine("postgresql+psycopg://dam:dam@localhost:5432/instituto")
engine = create_engine("mysql+mysqlconnector://dam:dam@localhost/instituto")
```

Propiedades configurables más habituales de `create_engine`:

`echo=True`	Muestra por consola todo el SQL generado. **Muy recomendable mientras se aprende.**
`pool_size`, `max_overflow`	Tamaño del pool de conexiones y conexiones extra permitidas.
`pool_pre_ping=True`	Comprueba que la conexión sigue viva antes de usarla.
`pool_recycle`	Segundos tras los que una conexión se renueva.
`isolation_level`	Nivel de aislamiento por defecto de las transacciones.
`connect_args`	Diccionario de argumentos que se pasan al conector.

3.4.2 Fichero de configuración

Igual que con los conectores, los datos de conexión deben estar fuera del código. Una opción sencilla es un fichero TOML, que Python lee con `tomllib`:

```
# config.toml
[database]
url = "postgresql+psycopg://dam:dam@localhost:5432/instituto"
echo = false
pool_size = 5
```

```
# basedatos.py
import tomllib
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

with open("config.toml", "rb") as f:
    cfg = tomllib.load(f)["database"]
```

```
engine = create_engine(cfg["url"], echo=cfg.get("echo", False),
                       pool_size=cfg.get("pool_size", 5))
Session = sessionmaker(bind=engine)     # fábrica de sesiones
```

3.5 Clases persistentes y estructura del mapeo

Una **clase persistente** es una clase cuyos objetos se pueden guardar en la base de datos. El **mapeo** indica a qué tabla corresponde, qué atributo corresponde a cada columna, cuál es la clave primaria y cómo se relaciona con otras clases.

Históricamente, herramientas como Hibernate describían el mapeo en **ficheros XML** separados (`Alumno.hbm.xml`); hoy lo habitual es describirlo en el propio código, con anotaciones (Java) o, en Python, con **anotaciones de tipo**. SQLAlchemy admite ambos estilos.

3.5.1 Mapeo declarativo

```python
# modelo.py
from datetime import date
from sqlalchemy import String, ForeignKey, CheckConstraint
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    """Clase base de todas las clases persistentes."""

class Alumno(Base):
    __tablename__ = "alumno"
    __table_args__ = (CheckConstraint("length(dni) = 9"),)

    id: Mapped[int] = mapped_column(primary_key=True)
    dni: Mapped[str] = mapped_column(String(9), unique=True)
    nombre: Mapped[str] = mapped_column(String(50))
    apellidos: Mapped[str] = mapped_column(String(100))
    nacimiento: Mapped[date | None]                 # nullable por ser Optional
    email: Mapped[str | None] = mapped_column(String(120), index=True)
    activo: Mapped[bool] = mapped_column(default=True)

    def __repr__(self) -> str:
        return f"Alumno(id={self.id}, dni={self.dni!r}, nombre={self.nombre!r})"
```

Elementos del mapeo:

`__tablename__`	Nombre de la tabla.
`__table_args__`	Restricciones e índices a nivel de tabla.
`Mapped[tipo]`	Declara un atributo persistente. El tipo Python determina el tipo SQL (`int` → `INTEGER`, `str` → `VARCHAR`, `date` → `DATE`...). `tipo | None` significa que admite `NULL`.
`mapped_column(...)`	Opciones de la columna: `primary_key`, `unique`, `index`, `nullable`, `default`, `server_default`, `name` (si la columna se llama distinto), tipo SQL explícito (`String(50)`, `Numeric(10, 2)`...).
`ForeignKey("tabla.col")`	Clave ajena.
`relationship()`	Relación con otra clase (siguiente apartado).

Una vez definidas las clases, el ORM puede crear las tablas:

```
from basedatos import engine
from modelo import Base

Base.metadata.create_all(engine)    # CREATE TABLE ... para las que no existan
# Base.metadata.drop_all(engine)    # DROP TABLE de todas
```

3.5.2 Mapeo imperativo

Si se prefiere mantener las clases del dominio totalmente independientes del ORM (como ocurría con los ficheros de mapeo XML), se describe la tabla por separado y se asocia a la clase:

```
from dataclasses import dataclass
from sqlalchemy import Table, Column, Integer, String, MetaData
from sqlalchemy.orm import registry

@dataclass
class Curso:                        # clase "limpia", no sabe nada del ORM
    nombre: str
    id: int | None = None

metadata = MetaData()
tabla_curso = Table("curso", metadata,
    Column("id", Integer, primary_key=True),
    Column("nombre", String(30), nullable=False, unique=True))

registry().map_imperatively(Curso, tabla_curso)   # el "fichero de mapeo"
```

Migraciones. `create_all` sólo crea tablas que no existen: si cambias una clase ya mapeada, no modifica la tabla. En proyectos reales se usa **Alembic**, la herramienta de migraciones de SQLAlchemy, que genera los `ALTER TABLE` necesarios y mantiene un historial de versiones del esquema.

3.6 Mapeo de relaciones y colecciones

3.6.1 Uno a muchos / muchos a uno

Un curso tiene muchos alumnos; un alumno pertenece a un curso. En la base de datos, la clave ajena está en `alumno`. En los objetos, el curso tiene una **colección** de alumnos y cada alumno una **referencia** a su curso:

```python
from sqlalchemy import ForeignKey, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

class Curso(Base):
    __tablename__ = "curso"
    id: Mapped[int] = mapped_column(primary_key=True)
    nombre: Mapped[str] = mapped_column(String(30), unique=True)

    alumnos: Mapped[list["Alumno"]] = relationship(
        back_populates="curso",
        cascade="all, delete-orphan",       # ver nota
        order_by="Alumno.apellidos")

class Alumno(Base):
    __tablename__ = "alumno"
    id: Mapped[int] = mapped_column(primary_key=True)
    nombre: Mapped[str] = mapped_column(String(50))
    apellidos: Mapped[str] = mapped_column(String(100))
    curso_id: Mapped[int | None] = mapped_column(ForeignKey("curso.id"))

    curso: Mapped["Curso | None"] = relationship(back_populates="alumnos")
```

```python
dam = Curso(nombre="2º DAM")
dam.alumnos.append(Alumno(nombre="Ana", apellidos="García"))
dam.alumnos.append(Alumno(nombre="Luis", apellidos="Pérez"))
session.add(dam)          # los alumnos se guardan en cascada
session.commit()
print(dam.alumnos[0].curso.nombre)    # navegación en ambos sentidos
```

> **Cascadas.** `cascade` indica qué operaciones sobre el objeto padre se propagan a los hijos. `"all, delete-orphan"` hace que al guardar o borrar un curso se guarden o borren sus alumnos, y que un alumno que se quite de la colección se borre. Úsalo sólo cuando los hijos no tengan sentido sin el padre (por ejemplo, las líneas de un pedido).

3.6.2 Uno a uno

Se mapea como uno a muchos pero indicando que el lado de la colección es un único objeto:

```python
class Expediente(Base):
    __tablename__ = "expediente"
    id: Mapped[int] = mapped_column(primary_key=True)
    alumno_id: Mapped[int] = mapped_column(ForeignKey("alumno.id"), unique=True)
    observaciones: Mapped[str | None]
    alumno: Mapped["Alumno"] = relationship(back_populates="expediente")

# En Alumno:
#   expediente: Mapped["Expediente | None"] = relationship(back_populates="alumno")
```

3.6.3 Muchos a muchos

Un alumno se matricula en muchos módulos y un módulo tiene muchos alumnos. Hace falta una tabla intermedia:

```
from sqlalchemy import Table, Column, ForeignKey

matricula = Table(
    "matricula", Base.metadata,
    Column("alumno_id", ForeignKey("alumno.id"), primary_key=True),
    Column("modulo_id", ForeignKey("modulo.id"), primary_key=True),
)

class Modulo(Base):
    __tablename__ = "modulo"
    id: Mapped[int] = mapped_column(primary_key=True)
    nombre: Mapped[str]
    horas: Mapped[int]
    alumnos: Mapped[list["Alumno"]] = relationship(
        secondary=matricula, back_populates="modulos")

# En Alumno:
#   modulos: Mapped[list["Modulo"]] = relationship(
#       secondary=matricula, back_populates="alumnos")
```

Si la relación tiene atributos propios (por ejemplo, la **nota** de cada alumno en cada módulo), la tabla intermedia se convierte en una clase, llamada **objeto de asociación**:

```
class Matricula(Base):
    __tablename__ = "matricula"
    alumno_id: Mapped[int] = mapped_column(ForeignKey("alumno.id"), primary_key=True)
    modulo_id: Mapped[int] = mapped_column(ForeignKey("modulo.id"), primary_key=True)
    nota: Mapped[float | None]
    alumno: Mapped["Alumno"] = relationship(back_populates="matriculas")
    modulo: Mapped["Modulo"] = relationship(back_populates="matriculas")
```

3.6.4 Colecciones de valores

Las colecciones no tienen por qué ser listas. Con `collection_class` se pueden mapear como conjuntos o como diccionarios indexados por un atributo:

```
from sqlalchemy.orm import attribute_keyed_dict

class Curso(Base):
    ...
    # diccionario dni -> Alumno, en lugar de una lista
    alumnos_por_dni: Mapped[dict[str, "Alumno"]] = relationship(
        collection_class=attribute_keyed_dict("dni"), viewonly=True)

curso.alumnos_por_dni["11111111H"].nombre
```

3.7 Mapeo de la herencia

Las bases de datos relacionales no tienen herencia, así que el ORM la simula con una de tres estrategias:

Estrategia	En la base de datos	Ventajas / inconvenientes
Tabla única (*single table*)	Una tabla para toda la jerarquía con una columna **discriminadora**; las columnas de las subclases admiten `NULL`.	Consultas rápidas, sin `JOIN`. Muchas columnas nulas; no se pueden poner `NOT NULL` en campos de subclases.
Tablas unidas (*joined*)	Una tabla por clase; la de cada subclase tiene sólo sus columnas y una clave ajena a la del padre.	Modelo normalizado. Requiere `JOIN` en cada consulta.
Tabla por clase concreta (*concrete*)	Una tabla completa por cada subclase, con todas las columnas.	Sin `JOIN` para consultar una subclase; consultas polimórficas costosas (`UNION`).

```python
# Tablas unidas
class Persona(Base):
    __tablename__ = "persona"
    id: Mapped[int] = mapped_column(primary_key=True)
    nombre: Mapped[str]
    tipo: Mapped[str]                       # discriminador
    __mapper_args__ = {"polymorphic_on": "tipo",
                       "polymorphic_identity": "persona"}

class Profesor(Persona):
    __tablename__ = "profesor"
    id: Mapped[int] = mapped_column(ForeignKey("persona.id"), primary_key=True)
    departamento: Mapped[str]
    __mapper_args__ = {"polymorphic_identity": "profesor"}

class Estudiante(Persona):
    __tablename__ = "estudiante"
    id: Mapped[int] = mapped_column(ForeignKey("persona.id"), primary_key=True)
    ciclo: Mapped[str]
    __mapper_args__ = {"polymorphic_identity": "estudiante"}

# Consulta polimórfica: devuelve objetos Profesor y Estudiante
for p in session.scalars(select(Persona)):
    print(type(p).__name__, p.nombre)
```

Para usar **tabla única**, basta con no definir `__tablename__` ni `id` en las subclases (sus columnas deben ser opcionales).

3.8 Sesiones y estados de un objeto

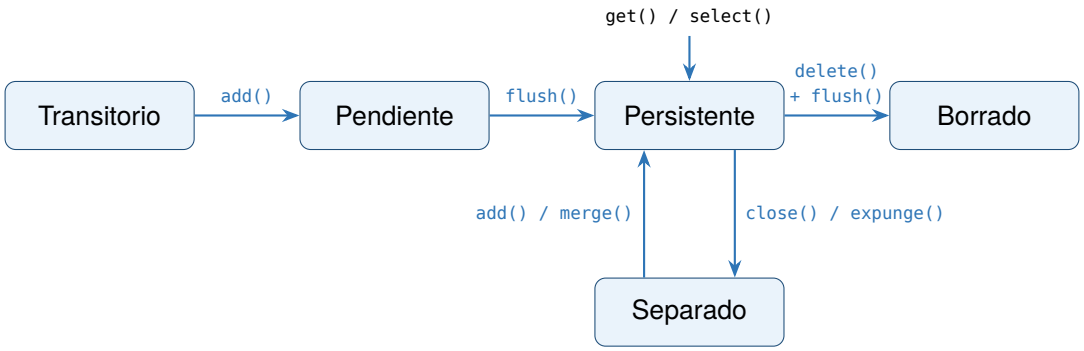
La **sesión** (`Session`) es el objeto central del ORM. Representa una conversación con la base de datos y actúa como:

- **Unidad de trabajo** (*unit of work*): acumula los objetos nuevos, modificados y borrados, y al hacer `flush` o `commit` genera todas las sentencias SQL necesarias en el orden correcto.
- **Mapa de identidad** (*identity map*): dentro de una sesión, a cada fila le corresponde un único objeto. Si se consulta dos veces el alumno con id 1, se obtiene *el mismo* objeto.
- Gestor de la **transacción** en curso.

```python
from sqlalchemy.orm import Session

with Session(engine) as session:       # se cierra al salir
    ...
    session.commit()
```

3.8.1 Estados de un objeto



Transitorio (*transient*). Objeto recién creado con `Alumno(...)`: no está en la sesión ni en la base de datos.

Pendiente (*pending*). Se ha añadido con `session.add()`, pero aún no se ha enviado el `INSERT`.

Persistente (*persistent*). Está asociado a la sesión y tiene su fila en la base de datos. Los cambios en sus atributos se detectan automáticamente.

Borrado (*deleted*). Se ha marcado con `session.delete()` y se ha enviado el `DELETE`; desaparece al confirmar.

Separado (*detached*). Tiene fila en la base de datos, pero ya no está asociado a ninguna sesión (se cerró). Sus cambios no se guardan salvo que se vuelva a asociar con `merge()`.

```python
from sqlalchemy import inspect

a = Alumno(nombre="Eva", apellidos="Martín")
print(inspect(a).transient)       # True
with Session(engine) as session:
    session.add(a)
    print(inspect(a).pending)     # True
    session.flush()               # envía el INSERT (aún sin commit)
    print(inspect(a).persistent, a.id)   # True y ya tiene id
```

```
    session.commit()
print(inspect(a).detached)        # True
```

3.9 Carga, almacenamiento y modificación de objetos

```
from sqlalchemy.orm import Session

with Session(engine) as session:
    # --- Almacenar ---
    eva = Alumno(nombre="Eva", apellidos="Martín")
    session.add(eva)
    session.add_all([Alumno(nombre="Iker", apellidos="Sanz"),
                     Alumno(nombre="Noa", apellidos="Vega")])
    session.commit()

    # --- Cargar por clave primaria ---
    a = session.get(Alumno, 1)            # None si no existe
    if a is None:
        print("No existe")

    # --- Modificar: basta con cambiar el atributo ---
    a.apellidos = "Martín Ruiz"           # la sesión detecta el cambio
    session.commit()                      # UPDATE alumno SET apellidos=...

    # --- Borrar ---
    session.delete(a)
    session.commit()

    # --- Recargar desde la base de datos ---
    session.refresh(eva)
```

3.9.1 Carga perezosa y carga ansiosa

Por defecto, las relaciones se cargan de forma **perezosa** (*lazy*): la colección `curso.alumnos` no se consulta hasta que se accede a ella. Es cómodo, pero puede provocar el **problema N+1**:

```
cursos = session.scalars(select(Curso)).all()     # 1 consulta
for c in cursos:
    print(c.nombre, len(c.alumnos))                # +1 consulta POR CADA curso
```

Si sabemos que vamos a necesitar la relación, se pide una carga **ansiosa** (*eager*):

```
from sqlalchemy.orm import selectinload, joinedload

cursos = session.scalars(
    select(Curso).options(selectinload(Curso.alumnos))).all()   # 2 consultas
alumnos = session.scalars(
    select(Alumno).options(joinedload(Alumno.curso))).all()      # 1 consulta con JOIN
```

Cuidado. Si se accede a una relación perezosa de un objeto **separado** (después de cerrar la sesión), se produce `DetachedInstanceError`. Carga lo que necesites antes de cerrar la sesión.

3.10 Consultas

3.10.1 El lenguaje de consultas propio: `select()`

Cada ORM tiene su propio lenguaje de consultas orientado a objetos: HQL/JPQL en Hibernate, LINQ en Entity Framework. En SQLAlchemy las consultas se construyen con **expresiones Python** sobre las clases y sus atributos, que el ORM traduce al dialecto SQL del gestor.

```python
from sqlalchemy import select, func, or_, desc

# Todos los alumnos ordenados
alumnos = session.scalars(select(Alumno).order_by(Alumno.apellidos)).all()

# Filtros
stmt = (select(Alumno)
        .where(Alumno.apellidos.like("G%"))
        .where(or_(Alumno.curso_id == 1, Alumno.curso_id.is_(None))))
for a in session.scalars(stmt):
    print(a)

# Un único resultado
ana = session.scalars(select(Alumno).where(Alumno.dni == "11111111H")).one_or_none()

# JOIN siguiendo una relación
stmt = select(Alumno).join(Alumno.curso).where(Curso.nombre == "2º DAM")

# Seleccionar columnas concretas -> filas (tuplas con nombre)
for nombre, curso in session.execute(
        select(Alumno.nombre, Curso.nombre).join(Alumno.curso)):
    print(nombre, curso)

# Agregados y agrupación
stmt = (select(Curso.nombre, func.count(Alumno.id).label("total"))
        .join(Curso.alumnos)
        .group_by(Curso.nombre)
        .having(func.count(Alumno.id) > 10)
        .order_by(desc("total")))
for fila in session.execute(stmt):
    print(fila.nombre, fila.total)

# Paginación
pagina = session.scalars(select(Alumno).order_by(Alumno.id)
                         .limit(20).offset(40)).all()
```

Nota. `session.scalars(stmt)` devuelve directamente los objetos (la primera columna de cada fila); `session.execute(stmt)` devuelve filas. Métodos de resultado útiles: `.all()`, `.first()`, `.one()` (error si no hay exactamente uno), `.one_or_none()`.

3.10.2 Modificaciones masivas

```
from sqlalchemy import update, delete

session.execute(update(Alumno).where(Alumno.curso_id == 1)
                .values(activo=False))
session.execute(delete(Alumno).where(Alumno.activo.is_(False)))
session.commit()
```

3.10.3 Consultas SQL

Cuando hace falta, se puede ejecutar SQL directamente con `text()`, siempre con parámetros con nombre:

```
from sqlalchemy import text

filas = session.execute(
    text("SELECT nombre, apellidos FROM alumno WHERE curso_id = :c"),
    {"c": 1}).all()

# Y convertir el resultado en objetos mapeados
stmt = select(Alumno).from_statement(
    text("SELECT * FROM alumno WHERE apellidos ILIKE :patron"))
alumnos = session.scalars(stmt, {"patron": "%garcía%"}).all()

# Llamar a una función almacenada de PostgreSQL
n = session.scalar(select(func.num_alumnos(1)))
```

3.11 Gestión de transacciones

La sesión inicia una transacción automáticamente con la primera operación y la mantiene hasta `commit()` o `rollback()`. El patrón recomendado es delimitar la transacción con un bloque:

```
from sqlalchemy.orm import sessionmaker

Session = sessionmaker(bind=engine)

# commit al terminar el bloque, rollback si hay excepción, y cierre
with Session.begin() as session:
    origen = session.get(Cuenta, "ES01")
    destino = session.get(Cuenta, "ES02")
    if origen.saldo < 100:
        raise ValueError("Saldo insuficiente")   # -> rollback
    origen.saldo -= 100
    destino.saldo += 100
```

Equivale a:

```
session = Session()
try:
    ...
    session.commit()
except Exception:
```

```
    session.rollback()
    raise
finally:
    session.close()
```

3.11.1 Transacciones anidadas

```
from sqlalchemy.exc import IntegrityError

with Session.begin() as session:
    for datos in alumnos_a_importar:
        try:
            with session.begin_nested():      # SAVEPOINT
                session.add(Alumno(**datos))
        except IntegrityError:
            print("Duplicado:", datos["dni"])
```

3.11.2 Excepciones

Las excepciones del conector se envuelven en las de `sqlalchemy.exc`, con los mismos nombres de la DB-API (`IntegrityError`, `OperationalError`...). Además, el ORM tiene las suyas, como `NoResultFound` y `MultipleResultsFound` (al usar `.one()`) o `DetachedInstanceError`.

3.12 Organización de una aplicación con ORM

Una estructura habitual separa:

```
instituto/
├── config.toml
├── instituto/
│   ├── __init__.py
│   ├── basedatos.py      engine y sessionmaker
│   ├── modelo.py         clases mapeadas
│   ├── repositorios.py   consultas (opcional, patrón repositorio)
│   ├── servicios.py      lógica de negocio y transacciones
│   └── main.py           interfaz (consola, GUI, API web...)
└── tests/
    └── test_servicios.py
```

```
# tests/conftest.py — base de datos nueva en memoria para cada prueba
import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import Session
from instituto.modelo import Base

@pytest.fixture
def session():
    engine = create_engine("sqlite:///:memory:")
    Base.metadata.create_all(engine)
    with Session(engine) as s:
```

```
        yield s
```

3.13 Ejercicios

Ejercicio 3.1 — Primer mapeo

Dificultad: ★

Mapea con SQLAlchemy una clase `Libro` (isbn como clave primaria, título, autor, año, precio con `Numeric(8,2)`, disponible). Con `echo=True`, crea las tablas en SQLite, inserta cinco libros, modifica el precio de uno, borra otro y realiza las consultas: libros de un autor; libros anteriores a un año; precio medio. Observa el SQL generado en cada operación y anota qué sentencias aparecen y cuándo.

Ejercicio 3.2 — Estados de un objeto

Dificultad: ★

Escribe un programa que muestre, usando `inspect()`, el estado de un objeto en cada momento de su ciclo de vida: creación, `add`, `flush`, `commit`, cierre de sesión, modificación estando separado, `merge` en una sesión nueva, `delete` y `commit`. Comprueba también el mapa de identidad: dos `session.get` del mismo id devuelven el mismo objeto (`is`).

Ejercicio 3.3 — Relaciones

Dificultad: ★★

Modela una tienda: `Cliente` 1:N `Pedido`, `Pedido` 1:N `LineaPedido` (con cascada `delete-orphan`) y `LineaPedido` N:1 `Producto`; y `Producto` N:M `Categoria`. Crea datos de prueba y consulta: total facturado por cliente; productos que nunca se han vendido; categorías con su número de productos; el pedido de mayor importe. Provoca el problema N+1 al listar los pedidos con sus líneas, cuenta las consultas con `echo=True` y resuélvelo con carga ansiosa.

Ejercicio 3.4 — Herencia

Dificultad: ★★

Modela la jerarquía `Vehiculo` → `Coche`, `Moto`, `Furgoneta` con atributos comunes y propios. Impleméntala primero con tabla única y después con tablas unidas. Compara las tablas generadas y el SQL de una consulta polimórfica y de una consulta sólo de `Moto`. Resume en una tabla las ventajas e inconvenientes que has observado.

Ejercicio 3.5 — Migrar un DAO a ORM

Dificultad: ★★

Toma el proyecto del videoclub del Tema 2 y sustituye la capa DAO por un modelo SQLAlchemy y repositorios. Mantén la misma interfaz pública de la capa de servicio, de manera que `main.py` no cambie. Compara el número de líneas de código de ambas versiones.

Ejercicio 3.6 — Transacciones con el ORM

Dificultad: ★★★

Implementa una importación de alumnos desde un CSV en la que cada fila se inserte dentro de un `begin_nested()`: las filas con errores (DNI duplicado, curso inexistente, datos obligatorios vacíos) se registran en un fichero `errores.csv` con el motivo, y el resto se confirman todas juntas al final. Añade una opción `--todo-o-nada` en la que un solo error deshaga toda la importación.

Proyecto: Gestión académica de un instituto

Desarrolla la capa de persistencia y una aplicación de consola para gestionar la información académica de un centro usando SQLAlchemy sobre PostgreSQL (y SQLite para las pruebas).

Modelo: `Persona` (jerarquía con `Profesor` y `Alumno`), `Ciclo`, `Curso`, `Modulo` (pertenece a un curso, lo imparte un profesor), `Matricula` (objeto de asociación alumno–módulo con las notas de cada evaluación), `Falta` (alumno, módulo, fecha, justificada).

Requisitos:

1. Configuración en `config.toml` y creación del esquema con el ORM (opcional: con migraciones de Alembic).
2. Mapeo de todas las relaciones (1:1, 1:N, N:M con objeto de asociación) y de la herencia, justificando la estrategia elegida.
3. Capa de servicios con transacciones: matricular a un alumno en todos los módulos de un curso; registrar las faltas de un día para un grupo; cerrar la evaluación (calcula notas finales y marca módulos aprobados).
4. Consultas con `select()`: boletín de notas de un alumno; alumnos con más de un 15 % de faltas injustificadas en un módulo (pérdida de evaluación continua); media de cada módulo; profesores con su carga horaria.
5. Al menos una consulta con SQL nativo y una llamada a una función almacenada.
6. Ausencia de problemas N+1 en los listados (verificado con `echo=True`).
7. Exportación del boletín de un alumno a JSON (validado con JSON Schema), XML y HTML (Tema 1).
8. Pruebas con `pytest` de la capa de servicios.

TEMA 4

Bases de datos objeto-relacionales

En este tema aprenderás a…

- Conocer las características, ventajas e inconvenientes de las bases de datos que almacenan objetos, en especial las objeto-relacionales.
- Utilizar las extensiones objeto-relacionales del estándar SQL:1999 tal como las implementa PostgreSQL: tipos compuestos, arrays, herencia de tablas, tablas tipadas, enumerados, dominios, JSONB y funciones.
- Establecer y cerrar conexiones y adaptar los tipos del gestor a objetos de Python.
- Gestionar la persistencia de objetos simples y estructurados, consultarlos y modificarlos.
- Gestionar transacciones y probar y documentar las aplicaciones.

Resultado de aprendizaje asociado: **RA4**.

Actualización del currículo. El currículo agrupa en este bloque las bases de datos objeto-relacionales y las bases de datos orientadas a objetos "puras". Estas últimas ocupan hoy un nicho muy pequeño: el estándar ODMG y su lenguaje OQL dejaron de mantenerse en 2001 y gestores clásicos como db4o están descontinuados. Por eso este tema se centra en las **bases de datos objeto-relacionales**, cuyas características están vivas en gestores muy utilizados como PostgreSQL y Oracle. Las bases de datos orientadas a objetos, el lenguaje OQL y un ejemplo práctico con ZODB se han trasladado al Anexo C.

4.1 Bases de datos objeto-relacionales

4.1.1 Características

Una base de datos **objeto-relacional** (BDOR) es una base de datos relacional ampliada con conceptos de la orientación a objetos, para reducir el desfase objeto-relacional **desde el lado del gestor**. Sigue habiendo tablas y SQL, pero además:

- Se pueden definir **tipos de datos de usuario**, incluidos tipos estructurados (con varios atributos) que pueden usarse como tipo de una columna.
- Una columna puede contener **colecciones**: arrays (y en algunos gestores, tablas anidadas).
- Existe la **herencia** entre tipos y entre tablas.
- Los tipos pueden tener **métodos** (funciones asociadas).
- Existen **referencias** entre objetos (`REF`), que hacen de identificadores de objeto.

Estas extensiones se estandarizaron en **SQL:1999** (también llamado SQL3).

4.1.2 Ventajas e inconvenientes

+ Modelado más natural de datos complejos (direcciones, coordenadas, listas de teléfonos) sin crear tablas auxiliares.
+ Reutilización de tipos y comportamiento en el propio gestor.
+ Se conserva todo lo bueno del modelo relacional: SQL, transacciones, madurez, herramientas.
– Cada gestor implementa un subconjunto distinto del estándar con sintaxis propia: poca portabilidad.
– Se rompe la primera forma normal; las consultas pueden volverse más complejas.
– Los conectores y los ORM no siempre soportan bien los tipos estructurados.

4.1.3 Gestores objeto-relacionales

Oracle implementa el modelo de SQL:1999 de forma bastante completa: `CREATE TYPE ... AS OBJECT` con métodos, tablas de objetos, `REF`, `VARRAY` y tablas anidadas.

PostgreSQL se define a sí mismo como "base de datos objeto-relacional". Ofrece tipos compuestos, arrays de cualquier tipo, herencia de tablas, dominios, enumerados, rangos, tipos geométricos, `JSONB` y la posibilidad de definir tipos y operadores nuevos.

IBM Db2 e Informix también incluyen tipos estructurados y herencia.

Usaremos PostgreSQL (consulta el Apéndice A para levantarlo con Docker).

4.1.4 Bases de datos objeto-relacionales y orientadas a objetos

Las BDOR no son la única forma de almacenar objetos. Las bases de datos **orientadas a objetos** (BDOO) van más allá: guardan directamente los objetos del lenguaje de programación, sin tablas ni SQL. La tabla resume las diferencias; el Anexo C estudia las BDOO con más detalle.

	Objeto-relacional	**Orientada a objetos**
Modelo	Tablas ampliadas con tipos de usuario, colecciones y herencia.	Objetos y referencias entre ellos.
Lenguaje	SQL:1999 (con extensiones de cada gestor).	API del lenguaje de programación; OQL (poco implantado).
Desfase objeto-relacional	Se reduce.	Desaparece.
Independencia del lenguaje	Alta: cualquier programa con un conector accede a los datos.	Baja: los datos están ligados a las clases de un lenguaje.
Madurez y mercado	Muy alta (PostgreSQL, Oracle).	Nicho (CAD, telecomunicaciones, sistemas financieros).

4.2 Gestión de objetos con SQL:1999

4.2.1 El estándar

Como referencia, este es el aspecto de las construcciones de SQL:1999 (la sintaxis exacta es la de Oracle, que es la más próxima al estándar):

```
-- Tipo estructurado
CREATE TYPE direccion_t AS OBJECT (
    calle VARCHAR2(80), numero NUMBER, cp CHAR(5), ciudad VARCHAR2(40)
);
-- Tipo con herencia (NOT FINAL permite subtipos) y método
CREATE TYPE persona_t AS OBJECT (
    dni CHAR(9), nombre VARCHAR2(50), direccion direccion_t,
    MEMBER FUNCTION etiqueta RETURN VARCHAR2
) NOT FINAL;
CREATE TYPE profesor_t UNDER persona_t (departamento VARCHAR2(40));
-- Colección
CREATE TYPE telefonos_t AS VARRAY(5) OF VARCHAR2(15);
-- Tabla de objetos: cada fila es un objeto con identificador (OID)
CREATE TABLE personas OF persona_t (PRIMARY KEY (dni));
-- Referencia a un objeto de otra tabla
CREATE TABLE tutoria (grupo VARCHAR2(10), tutor REF persona_t SCOPE IS personas);
SELECT t.grupo, DEREF(t.tutor).nombre FROM tutoria t;
```

4.2.2 Tipos compuestos en PostgreSQL

Un tipo compuesto es un tipo estructurado formado por varios atributos:

```
CREATE TYPE direccion AS (
    calle   VARCHAR(80),
    numero  INTEGER,
    cp      CHAR(5),
    ciudad  VARCHAR(40)
);

CREATE TABLE cliente (
    id         SERIAL PRIMARY KEY,
    nombre     VARCHAR(60) NOT NULL,
    domicilio  direccion,                -- columna de tipo estructurado
    envio      direccion
);

INSERT INTO cliente (nombre, domicilio)
VALUES ('Ana', ROW('Santiago', 12, '47001', 'Valladolid'));

-- Acceso a los atributos: (columna).atributo
SELECT nombre, (domicilio).ciudad FROM cliente
WHERE (domicilio).cp LIKE '47%';

UPDATE cliente SET domicilio.numero = 14 WHERE id = 1;
```

4.2.3 Colecciones: arrays

```sql
CREATE TABLE contacto (
    id         SERIAL PRIMARY KEY,
    nombre     VARCHAR(60),
    telefonos  VARCHAR(15)[],             -- array de cadenas
    etiquetas  TEXT[] DEFAULT '{}',
    direcciones direccion[]               -- array de tipos compuestos
);

INSERT INTO contacto (nombre, telefonos, etiquetas)
VALUES ('Luis', ARRAY['600111222', '983000000'], '{trabajo,amigos}');

SELECT nombre, telefonos[1] FROM contacto;            -- los índices empiezan en 1
SELECT nombre FROM contacto WHERE 'amigos' = ANY(etiquetas);
SELECT nombre FROM contacto WHERE etiquetas @> ARRAY['trabajo'];   -- contiene
SELECT nombre, unnest(telefonos) AS tlf FROM contacto;            -- array -> filas
UPDATE contacto SET etiquetas = array_append(etiquetas, 'familia') WHERE id = 1;
SELECT nombre, cardinality(telefonos) FROM contacto;
```

4.2.4 Herencia de tablas

```sql
CREATE TABLE persona (
    id      SERIAL PRIMARY KEY,
    dni     CHAR(9) UNIQUE,
    nombre  VARCHAR(60)
);
CREATE TABLE profesor (departamento VARCHAR(40)) INHERITS (persona);
CREATE TABLE alumno   (ciclo VARCHAR(10), curso INTEGER) INHERITS (persona);

INSERT INTO profesor (dni, nombre, departamento)
VALUES ('11111111H', 'Marta', 'Informática');

SELECT * FROM persona;          -- incluye las filas de profesor y alumno
SELECT * FROM ONLY persona;     -- sólo las filas de la tabla padre
SELECT p.nombre, p.tableoid::regclass AS tipo FROM persona p;
```

Cuidado. En PostgreSQL, las restricciones `UNIQUE` y `PRIMARY KEY` y las claves ajenas **no se heredan entre tablas**: el DNI podría repetirse entre `profesor` y `alumno`. Por eso la herencia de tablas se usa con prudencia.

4.2.5 Otros tipos: enumerados y dominios

```sql
CREATE TYPE estado_pedido AS ENUM ('pendiente', 'enviado', 'entregado');
CREATE DOMAIN email AS VARCHAR(120) CHECK (VALUE ~ '^[^@]+@[^@]+\.[^@]+$');
CREATE DOMAIN importe AS NUMERIC(10, 2) CHECK (VALUE >= 0);

CREATE TABLE pedido (
    id       SERIAL PRIMARY KEY,
    estado   estado_pedido DEFAULT 'pendiente',
    contacto email,
    total    importe,
    extra    JSONB                        -- ver el apartado siguiente
```

```
);
SELECT id FROM pedido WHERE estado > 'pendiente';   -- los ENUM tienen orden
```

Un **enumerado** restringe los valores posibles y les da un orden; un **dominio** es un tipo existente con restricciones añadidas, reutilizable en muchas tablas.

4.2.6 Documentos dentro de la tabla: JSONB

PostgreSQL permite guardar documentos JSON en una columna. El tipo `JSONB` los almacena en un formato binario que se puede consultar e indexar. Es útil para datos con estructura variable (atributos que cambian de un producto a otro, metadatos, respuestas de servicios web) sin renunciar al resto del modelo relacional, y es un buen puente hacia las bases de datos documentales del Tema 5.

```
CREATE TABLE producto (
    id     SERIAL PRIMARY KEY,
    nombre VARCHAR(80) NOT NULL,
    precio NUMERIC(8, 2),
    ficha  JSONB
);
INSERT INTO producto (nombre, precio, ficha) VALUES
 ('Portátil', 899, '{"ram": 16, "cpu": "i7", "puertos": ["usb-c", "hdmi"]}'),
 ('Camiseta', 15,  '{"talla": "M", "color": "azul"}');

SELECT nombre, ficha->>'cpu' AS cpu FROM producto;        -- ->> devuelve texto
SELECT nombre FROM producto WHERE (ficha->>'ram')::int >= 16;
SELECT nombre FROM producto WHERE ficha @> '{"color": "azul"}';  -- contiene
SELECT nombre FROM producto WHERE ficha ? 'talla';               -- tiene la clave
SELECT nombre, jsonb_array_elements_text(ficha->'puertos') FROM producto;
UPDATE producto SET ficha = jsonb_set(ficha, '{ram}', '32') WHERE id = 1;
UPDATE producto SET ficha = ficha || '{"garantia": 3}' WHERE id = 1;  -- añadir
CREATE INDEX idx_ficha ON producto USING GIN (ficha);     -- acelera @> y ?
```

`->` / `->>`	Valor de una clave (como JSON / como texto).
`#>` / `#>>`	Valor de una ruta: `ficha #>> '{dimensiones,alto}'`.
`@>`	¿Contiene el documento indicado?
`?`, `?|`, `?&`	¿Tiene la clave / alguna de las claves / todas?
`||`	Concatena (fusiona) dos documentos.
`jsonb_set`, `jsonb_build_object`, `jsonb_agg`	Modificar, construir y agregar documentos.

4.2.7 Tablas tipadas

Además de usar un tipo compuesto como tipo de una columna, PostgreSQL permite crear una tabla **a partir de un tipo**: cada fila es entonces un valor de ese tipo, de forma parecida a las *tablas de objetos* de SQL:1999.

```
CREATE TYPE persona_t AS (dni CHAR(9), nombre VARCHAR(60), domicilio direccion);

CREATE TABLE socio OF persona_t (
    PRIMARY KEY (dni),
```

```
    nombre WITH OPTIONS NOT NULL
);
INSERT INTO socio VALUES ('11111111H', 'Ana', ROW('Real', 3, '24001', 'León'));
SELECT s FROM socio s;                      -- la fila completa como un valor
SELECT (s.domicilio).ciudad FROM socio s;
```

Si se añade un atributo al tipo (`ALTER TYPE persona_t ADD ATTRIBUTE email TEXT CASCADE`), la tabla lo incorpora automáticamente.

4.2.8 Métodos: funciones sobre tipos

En PostgreSQL los tipos compuestos no tienen métodos propiamente dichos, pero se consigue el mismo efecto con funciones que reciben el tipo como parámetro:

```
CREATE FUNCTION texto(d direccion) RETURNS TEXT
LANGUAGE sql IMMUTABLE AS $$
    SELECT d.calle || ', ' || d.numero || ' - ' || d.cp || ' ' || d.ciudad;
$$;

SELECT nombre, texto(domicilio) FROM cliente;
-- Notación funcional como atributo: también vale (domicilio).texto
```

4.3 Acceso a las funciones del gestor desde Python

4.3.1 Conectores y conexiones

Para acceder a una base de datos objeto-relacional se usan los mismos conectores que para una relacional: **psycopg** en PostgreSQL (Tema 2) u `oracledb` en Oracle. Las conexiones se establecen y se cierran exactamente igual, preferiblemente con `with`. Lo nuevo es cómo el conector **adapta los tipos** del gestor a objetos de Python y viceversa:

PostgreSQL	Python	¿Automático?
`INTEGER`, `NUMERIC`, `DATE`…	`int`, `Decimal`, `date`…	Sí.
Arrays (`TEXT[]`, `INTEGER[]`…)	`list`	Sí.
`JSONB` / `JSON`	`dict`, `list`	Al leer, sí; al escribir, envolviendo el valor en `Jsonb(…)`.
Dominios	El tipo base	Sí.
`ENUM`	`str` o una `Enum`	Hay que registrar el tipo para obtener una `Enum`.
Tipos compuestos	`tuple` o una clase propia	Hay que registrar el tipo para obtener objetos.

En los ejemplos siguientes, `conninfo` es la cadena de conexión del Tema 2 (por ejemplo, `"host=localhost dbname=instituto user=dam password=dam"`).

4.3.2 Persistencia de objetos simples: arrays, enumerados y JSONB

```python
from enum import Enum
import psycopg
from psycopg.types.enum import EnumInfo, register_enum
from psycopg.types.json import Jsonb

class Estado(Enum):
    PENDIENTE = "pendiente"
    ENVIADO = "enviado"
    ENTREGADO = "entregado"

with psycopg.connect(conninfo) as conn:
    # Arrays <-> listas
    conn.execute(
        "INSERT INTO contacto (nombre, telefonos, etiquetas) VALUES (%s, %s, %s)",
        ("Eva", ["611222333"], ["clase", "dam"]))
    for nombre, tlfs in conn.execute(
            "SELECT nombre, telefonos FROM contacto WHERE %s = ANY(etiquetas)",
            ("dam",)):
        print(nombre, tlfs)                 # tlfs es una list[str]

    # ENUM <-> Enum de Python
    register_enum(EnumInfo.fetch(conn, "estado_pedido"), conn, Estado,
                  mapping={m: m.value for m in Estado})

    # JSONB <-> dict
    conn.execute("INSERT INTO pedido (estado, extra) VALUES (%s, %s)",
                 (Estado.ENVIADO, Jsonb({"urgente": True, "bultos": 3})))
    estado, extra = conn.execute(
        "SELECT estado, extra FROM pedido ORDER BY id DESC LIMIT 1").fetchone()
    print(estado, extra["bultos"])          # Estado.ENVIADO 3
```

4.3.3 Persistencia de objetos estructurados: tipos compuestos

Por defecto, un tipo compuesto llega a Python como una tupla. Para trabajar con objetos, se **registra** el tipo indicando la clase Python que lo representa. La forma más sencilla es usar una `NamedTuple`, porque psycopg necesita recorrer los campos en orden para enviarlos al servidor (con una *dataclass* habría que indicar además cómo convertirla en secuencia con el argumento `make_sequence`). Al registrar el tipo, también quedan registrados los **arrays** de ese tipo.

```python
from typing import NamedTuple
import psycopg
from psycopg.types.composite import CompositeInfo, register_composite

class Direccion(NamedTuple):           # una tupla con nombres de campo
    calle: str
    numero: int
    cp: str
    ciudad: str

with psycopg.connect(conninfo) as conn:
    register_composite(CompositeInfo.fetch(conn, "direccion"), conn, Direccion)

    d = Direccion("Plaza Mayor", 1, "47001", "Valladolid")
    conn.execute("INSERT INTO cliente (nombre, domicilio) VALUES (%s, %s)",
                 ("Luis", d))
```

```
    for nombre, dom in conn.execute("SELECT nombre, domicilio FROM cliente"):
        print(nombre, dom.ciudad if dom else "-")       # dom es Direccion
```

4.3.4 Un repositorio de objetos estructurados

Combinando lo anterior podemos guardar y recuperar objetos complejos **en una sola fila**, sin tablas auxiliares ni `JOIN`. Partimos de esta tabla, que usa un tipo compuesto, un array de tipos compuestos, un array de texto y un documento JSONB:

```
CREATE TABLE empresa (
    cif         CHAR(9) PRIMARY KEY,
    nombre      VARCHAR(80) NOT NULL,
    sede        direccion,
    delegaciones direccion[] DEFAULT '{}',
    telefonos   VARCHAR(15)[] DEFAULT '{}',
    datos       JSONB DEFAULT '{}'
);
```

```
from dataclasses import dataclass, field
from typing import NamedTuple
import psycopg
from psycopg.rows import class_row
from psycopg.types.composite import CompositeInfo, register_composite
from psycopg.types.json import Jsonb

class Direccion(NamedTuple):
    calle: str
    numero: int
    cp: str
    ciudad: str

@dataclass
class Empresa:
    cif: str
    nombre: str
    sede: Direccion | None = None
    delegaciones: list[Direccion] = field(default_factory=list)
    telefonos: list[str] = field(default_factory=list)
    datos: dict = field(default_factory=dict)


class RepositorioEmpresas:
    def __init__(self, conn: psycopg.Connection) -> None:
        self.conn = conn
        register_composite(CompositeInfo.fetch(conn, "direccion"), conn, Direccion)

    def guardar(self, e: Empresa) -> None:
        """Inserta la empresa o, si ya existe, la sustituye."""
        self.conn.execute(
            """INSERT INTO empresa (cif, nombre, sede, delegaciones, telefonos, datos)
               VALUES (%s, %s, %s, %s, %s, %s)
               ON CONFLICT (cif) DO UPDATE SET nombre = EXCLUDED.nombre,
                   sede = EXCLUDED.sede, delegaciones = EXCLUDED.delegaciones,
                   telefonos = EXCLUDED.telefonos, datos = EXCLUDED.datos""",
```

```
            (e.cif, e.nombre, e.sede, e.delegaciones, e.telefonos, Jsonb(e.datos)))

    def buscar(self, cif: str) -> Empresa | None:
        with self.conn.cursor(row_factory=class_row(Empresa)) as cur:
            cur.execute("SELECT * FROM empresa WHERE cif = %s", (cif,))
            return cur.fetchone()

    def en_ciudad(self, ciudad: str) -> list[Empresa]:
        """Empresas con sede o alguna delegación en la ciudad indicada."""
        with self.conn.cursor(row_factory=class_row(Empresa)) as cur:
            cur.execute(
                """SELECT * FROM empresa e
                   WHERE (e.sede).ciudad = %s
                      OR EXISTS (SELECT 1 FROM unnest(e.delegaciones) d
                                 WHERE d.ciudad = %s)
                   ORDER BY nombre""", (ciudad, ciudad))
            return cur.fetchall()

    def anadir_telefono(self, cif: str, telefono: str) -> bool:
        """Modifica una parte del objeto en el propio servidor."""
        cur = self.conn.execute(
            "UPDATE empresa SET telefonos = array_append(telefonos, %s) "
            "WHERE cif = %s AND NOT (%s = ANY(telefonos))",
            (telefono, cif, telefono))
        return cur.rowcount == 1

    def mover_sede(self, cif: str, nueva: Direccion) -> None:
        self.conn.execute("UPDATE empresa SET sede = %s WHERE cif = %s",
                          (nueva, cif))

    def borrar(self, cif: str) -> bool:
        return self.conn.execute("DELETE FROM empresa WHERE cif = %s",
                                 (cif,)).rowcount == 1


with psycopg.connect(conninfo) as conn:
    repo = RepositorioEmpresas(conn)
    repo.guardar(Empresa(
        "B47000001", "Software del Duero",
        sede=Direccion("Acera de Recoletos", 2, "47004", "Valladolid"),
        delegaciones=[Direccion("Gran Vía", 10, "37001", "Salamanca")],
        telefonos=["983000111"], datos={"empleados": 25, "sector": "TIC"}))
    repo.anadir_telefono("B47000001", "983000222")
    e = repo.buscar("B47000001")
    print(e.sede.ciudad, e.delegaciones[0].ciudad, e.telefonos, e.datos["sector"])
    print([x.nombre for x in repo.en_ciudad("Salamanca")])
```

Nota. Fíjate en que `class_row(Empresa)` construye directamente objetos `Empresa` con sus atributos estructurados ya convertidos: `sede` es una `Direccion`, `delegaciones` una lista de `Direccion` y `datos` un diccionario. Es la reducción del desfase objeto-relacional que buscan las BDOR.

4.3.5 Funciones del gestor

Las funciones definidas en el servidor (los "métodos" de los tipos) se usan desde Python dentro de las sentencias SQL, como cualquier otra función:

```
with psycopg.connect(conninfo) as conn:
    for nombre, texto in conn.execute(
            "SELECT nombre, texto(domicilio) FROM cliente"):
        print(nombre, texto)
```

4.3.6 Transacciones

Las transacciones funcionan exactamente igual que en el Tema 2 (`commit`, `rollback`, `conn.transaction()`). Como un objeto estructurado se guarda en una única fila, **modificar un objeto completo es siempre atómico**; las transacciones son necesarias cuando la operación afecta a varios objetos:

```
def trasladar_delegaciones(conn, origen: str, destino: str) -> None:
    """Pasa todas las delegaciones de una empresa a otra (todo o nada)."""
    with conn.transaction():
        fila = conn.execute(
            "SELECT delegaciones FROM empresa WHERE cif = %s FOR UPDATE",
            (origen,)).fetchone()
        if fila is None:
            raise ValueError(f"No existe la empresa {origen}")
        cur = conn.execute(
            "UPDATE empresa SET delegaciones = delegaciones || %s WHERE cif = %s",
            (fila[0], destino))
        if cur.rowcount != 1:
            raise ValueError(f"No existe la empresa {destino}")   # -> rollback
        conn.execute("UPDATE empresa SET delegaciones = '{}' WHERE cif = %s",
                     (origen,))
```

4.3.7 Con SQLAlchemy

El ORM del Tema 3 dispone de tipos específicos de PostgreSQL (`ARRAY`, `JSONB`, `ENUM`, rangos...), que se importan del módulo `sqlalchemy.dialects.postgresql`. Los tipos compuestos de PostgreSQL como tipo de columna no están soportados directamente; en su lugar, `composite()` agrupa varias columnas normales en un objeto.

```
from dataclasses import dataclass
from sqlalchemy import String, select
from sqlalchemy.dialects.postgresql import ARRAY, JSONB
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, composite

class Base(DeclarativeBase):
    pass

@dataclass
class Coordenada:
    lat: float
    lon: float

class Lugar(Base):
```

```
    __tablename__ = "lugar"
    id: Mapped[int] = mapped_column(primary_key=True)
    nombre: Mapped[str] = mapped_column(String(80))
    etiquetas: Mapped[list[str]] = mapped_column(ARRAY(String), default=list)
    info: Mapped[dict] = mapped_column(JSONB, default=dict)
    lat: Mapped[float]
    lon: Mapped[float]
    posicion: Mapped[Coordenada] = composite("lat", "lon")   # objeto <- 2 columnas

# Consultas con los operadores propios de PostgreSQL
stmt = select(Lugar).where(Lugar.etiquetas.contains(["museo"]))  # etiquetas @> ...
stmt = select(Lugar).where(Lugar.info["accesible"].as_boolean())
stmt = select(Lugar).where(Lugar.info.contains({"gratis": True}))   # @>
```

Cuidado. SQLAlchemy no detecta las modificaciones *internas* de una lista o un diccionario (`lugar.etiquetas.append("x")`); hay que reasignar el atributo (`lugar.etiquetas = lugar.etiquetas + ["x"]`) o usar `sqlalchemy.ext.mutable.MutableList` y `MutableDict`.

4.4 Pruebas y documentación

Para probar el código que usa las características objeto-relacionales necesitamos un PostgreSQL real (SQLite no tiene tipos compuestos ni arrays). Una técnica habitual es ejecutar cada prueba dentro de una transacción que se deshace al terminar, de modo que la base de datos queda siempre limpia:

```
# tests/conftest.py
import os
import psycopg
import pytest

@pytest.fixture
def conn():
    with psycopg.connect(os.environ.get("TEST_DB", "dbname=pruebas user=dam")) as c:
        with c.transaction(force_rollback=True):     # nunca se confirma
            c.execute(open("esquema.sql", encoding="utf-8").read())
            yield c

def test_anadir_telefono_no_duplica(conn):
    repo = RepositorioEmpresas(conn)
    repo.guardar(Empresa("B00000000", "Prueba"))
    assert repo.anadir_telefono("B00000000", "900000000")
    assert not repo.anadir_telefono("B00000000", "900000000")
    assert repo.buscar("B00000000").telefonos == ["900000000"]
```

Para documentar el esquema, PostgreSQL permite asociar comentarios a tipos, tablas y columnas, que muestran las herramientas como DBeaver o `psql` (`\d+ empresa`):

```
COMMENT ON TYPE direccion IS 'Dirección postal española';
COMMENT ON COLUMN empresa.delegaciones IS 'Direcciones de las delegaciones';
```

4.5 Ejercicios

Ejercicio 4.1 — Tipos compuestos y arrays

Dificultad: ★

En PostgreSQL, crea un tipo compuesto `coordenada(lat, lon)` y una tabla `ruta` con un nombre, un array de coordenadas y un array de etiquetas. Inserta varias rutas y consulta: rutas con una etiqueta dada; número de puntos de cada ruta; primera y última coordenada de cada ruta. Escribe una función SQL `distancia(coordenada, coordenada)` (fórmula de Haversine) y úsala para calcular la longitud total de cada ruta.

Ejercicio 4.2 — Objetos estructurados desde Python

Dificultad: ★★

Desde Python, registra el tipo `coordenada` y escribe un `RepositorioRutas` con operaciones para guardar, buscar, añadir un punto al final de una ruta (en el servidor, con `array_append`) y listar las rutas que pasan cerca de un punto. Incluye un importador de ficheros GPX (son XML: usa ElementTree, Tema 1) y muestra la longitud de cada ruta calculada por la función del gestor.

Ejercicio 4.3 — Herencia de tablas

Dificultad: ★★

Modela con herencia de tablas de PostgreSQL `empleado` → `tecnico`, `comercial`. Comprueba la diferencia entre consultar con y sin `ONLY`, y demuestra el problema de las restricciones `UNIQUE` no heredadas. Propón y aplica una solución (por ejemplo, un disparador que compruebe el DNI en toda la jerarquía) y compárala con la estrategia de herencia *joined* del ORM (Tema 3).

Ejercicio 4.4 — JSONB

Dificultad: ★★

Crea una tabla `producto` con columnas fijas (código, nombre, precio, categoría) y una columna `ficha JSONB` con atributos que dependen de la categoría. Carga 200 productos generados aleatoriamente desde un fichero JSON (Tema 1) y resuelve con los operadores de JSONB: productos con una característica concreta, productos con un valor dentro de un rango, valores distintos de una clave por categoría y una actualización que añada una clave a todos los productos de una categoría. Mide el tiempo de una consulta con `@>` antes y después de crear un índice GIN (`EXPLAIN ANALYZE`).

Ejercicio 4.5 — Enumerados, dominios y tablas tipadas

Dificultad: ★

Define un dominio `dni` con su comprobación de formato, un enumerado `turno` (mañana, tarde, noche) y un tipo compuesto `persona_t`. Crea una tabla tipada a partir de él, añade después un atributo al tipo con `CASCADE` y comprueba el efecto. Accede a todo desde Python registrando el enumerado como una `Enum`.

Ejercicio 4.6 — Tipos de PostgreSQL con el ORM

Dificultad: ★★★

Repite el ejercicio de JSONB usando SQLAlchemy: mapea la ficha como `JSONB` (con `MutableDict` para que se detecten los cambios internos), las etiquetas como `ARRAY` y unas coordenadas con `composite()`. Escribe las mismas consultas con `select()` y compara el SQL generado (`echo=True`) con el que escribiste a mano.

Proyecto: Catálogo de un museo

Se quiere gestionar el catálogo de obras de un museo, con datos muy estructurados: cada obra tiene autores (con sus datos biográficos), un conjunto variable de técnicas y materiales, dimensiones (alto, ancho, profundidad), un historial de ubicaciones (fecha, sala, motivo) y un historial de restauraciones.

Requisitos:

1. Diseña el esquema en PostgreSQL usando: tipos compuestos (`dimensiones`, `ubicacion`, `restauracion`), arrays (técnicas, historial de ubicaciones y de restauraciones), enumerados (estado de conservación), dominios (código de inventario), herencia de tablas o tablas tipadas (`obra` → `pintura`, `escultura`, `fotografia`) y `JSONB` para metadatos que varían según el tipo de obra.
2. Funciones del gestor: superficie o volumen de una obra; ubicación actual (último elemento del historial); obras de una sala; obras que necesitan restauración (sin restaurar en los últimos 10 años).
3. Aplicación Python con un repositorio que registre los tipos y trabaje con *dataclasses*/`NamedTuple`: alta, consulta y modificación de obras completas.
4. Movimiento de obras entre salas como operación transaccional que comprueba el aforo máximo de obras de la sala de destino.
5. Consultas: obras de un autor, por técnica, por estado, por sala, y búsqueda por metadatos JSONB.
6. Exportación de la ficha completa de una obra a JSON validada con JSON Schema (Tema 1).
7. Pruebas con `pytest` con transacciones que se deshacen y documentación del esquema con `COMMENT ON`.
8. **Informe** breve: compara este diseño con uno relacional clásico (normalizado) para el mismo problema: número de tablas, complejidad de las consultas y código necesario en Python.

TEMA 5

Bases de datos documentales: MongoDB

En este tema aprenderás a…

- Conocer las bases de datos NoSQL y, en particular, las documentales, valorando sus ventajas e inconvenientes.
- Comprender el modelo de datos de MongoDB: documentos BSON, colecciones e identificadores únicos.
- Utilizar el lenguaje de consultas de MongoDB desde `mongosh`: inserción, consulta, actualización, borrado y agregación.
- Diseñar el modelo de datos de una aplicación (documentos embebidos o referencias) y validarlo con JSON Schema.
- Crear índices y analizar su efecto en las consultas.
- Desarrollar aplicaciones Python con `pymongo`: conexión, colecciones, documentos, consultas, transacciones y excepciones.

Resultado de aprendizaje asociado: **RA5**.

Actualización del currículo. El currículo titula este bloque *Bases de datos XML*, pero su resultado de aprendizaje habla de *bases de datos documentales nativas*. Hoy, cuando se habla de bases de datos documentales, se piensa casi siempre en bases de datos de documentos **JSON** como MongoDB, que está entre los gestores más utilizados del mundo, mientras que las bases de datos XML nativas se usan en nichos muy concretos. Por eso este tema trabaja todos los conceptos del currículo (colecciones, documentos, conexiones, consultas, índices, identificadores únicos, transacciones y excepciones) con **MongoDB**. Los contenidos sobre bases de datos XML nativas (BaseX y XQuery) se han trasladado al Anexo D.

5.1 Bases de datos NoSQL

5.1.1 ¿Qué es NoSQL?

Bajo el nombre **NoSQL** (*"not only SQL"*) se agrupan los gestores de bases de datos que no siguen el modelo relacional. Surgieron a finales de los años 2000 en grandes empresas de Internet que necesitaban:

- **Escalar horizontalmente**: repartir los datos entre muchos servidores baratos en lugar de comprar un servidor cada vez más potente.
- **Flexibilidad de esquema**: guardar datos cuya estructura cambia con frecuencia o varía de un registro a otro, sin tener que modificar tablas.

- **Alto rendimiento** en operaciones sencillas sobre enormes volúmenes de datos, aunque sea renunciando a algunas garantías del modelo relacional.
- Un modelo de datos más cercano a los **objetos** de la aplicación (sin desfase objeto-relacional).

5.1.2 Tipos de bases de datos NoSQL

Tipo	Modelo y uso típico	Ejemplos
Clave-valor	Un gran diccionario: a cada clave le corresponde un valor opaco. Cachés, sesiones, colas.	Redis, Valkey, DynamoDB
Documental	Documentos (JSON/BSON, XML) con estructura propia, agrupados en colecciones y consultables por cualquier campo. Aplicaciones web y móviles, catálogos, contenidos.	MongoDB, CouchDB, Firestore
Columnar	Filas con un número variable de columnas agrupadas en familias; escrituras masivas y distribuidas.	Cassandra, HBase
Grafos	Nodos y relaciones con propiedades; consultas que recorren relaciones. Redes sociales, recomendaciones, rutas.	Neo4j, Amazon Neptune

5.1.3 ACID, BASE y el teorema CAP

Las bases de datos relacionales garantizan las propiedades ACID (Tema 2). Muchas bases de datos NoSQL distribuidas relajan esas garantías siguiendo el modelo **BASE** (*Basically Available, Soft state, Eventually consistent*): el sistema siempre responde, aunque durante un breve tiempo distintos servidores puedan devolver datos diferentes hasta que se sincronizan (**consistencia eventual**).

El **teorema CAP** afirma que un sistema distribuido no puede garantizar a la vez **C**onsistencia, disponibilidad (*Availability*) y tolerancia a la **P**artición de la red: cuando la red falla, hay que elegir entre responder siempre (aunque sea con datos desactualizados) o mantener la consistencia (aunque algunas peticiones fallen). MongoDB, por defecto, prioriza la consistencia, y desde la versión 4.0 ofrece transacciones ACID.

5.1.4 Bases de datos documentales

Una base de datos documental almacena **documentos**: estructuras jerárquicas autodescriptivas (cada documento lleva los nombres de sus campos) que pueden contener valores simples, listas y otros documentos anidados. Los documentos se agrupan en **colecciones**, y los de una misma colección no tienen por qué tener los mismos campos.

+ **Correspondencia directa** con los objetos y diccionarios del programa: un pedido con sus líneas se guarda como un único documento.
+ **Esquema flexible**: se pueden añadir campos sin migraciones; útil cuando los datos son heterogéneos o evolucionan rápido.

+ Lecturas muy rápidas cuando los datos que se usan juntos se guardan juntos (sin `JOIN`).
+ Escalabilidad horizontal (*sharding*) y alta disponibilidad (réplicas) integradas.
– La flexibilidad se paga: sin disciplina (o sin validación) aparecen documentos inconsistentes, y la integridad referencial es responsabilidad de la aplicación.
– Información duplicada cuando se embeben datos compartidos (por ejemplo, el nombre del cliente en cada pedido), que hay que mantener sincronizada.
– Las consultas que cruzan muchas colecciones son más complejas y lentas que un `JOIN` relacional.
– No hay un lenguaje de consulta estándar como SQL: cada gestor tiene el suyo.

Gestor	Comentario
MongoDB	El gestor documental más utilizado. Versión *Community* libre (licencia SSPL), versión *Enterprise* comercial y servicio en la nube *Atlas*. Lo usaremos en el curso.
CouchDB	Libre (Apache). Acceso mediante API REST y replicación entre dispositivos.
Couchbase	Documental con caché integrada y lenguaje de consulta SQL++.
Firebase Firestore	Servicio en la nube de Google, muy usado en aplicaciones móviles.
Amazon DocumentDB, Azure Cosmos DB	Servicios en la nube compatibles con la API de MongoDB.
FerretDB	Libre; ofrece la API de MongoDB guardando los datos en PostgreSQL.
PostgreSQL (`JSONB`)	No es documental, pero permite combinar tablas y documentos (Tema 4).

5.2 El modelo de datos de MongoDB

5.2.1 Documentos, colecciones y bases de datos

Modelo relacional	MongoDB
Servidor	Servidor (`mongod`)
Base de datos	Base de datos
Tabla	Colección
Fila	Documento
Columna	Campo
Clave primaria	Campo `_id`
`JOIN`	Documentos embebidos o `$lookup`
SQL	MQL (*MongoDB Query Language*) y *aggregation pipeline*

Un documento de MongoDB tiene este aspecto:

```
{
  _id: ObjectId("6710b2c4e1a2f3d4c5b6a7f8"),
  isbn: "978-84-376-0494-7",
  titulo: "Cien años de soledad",
  autor: { nombre: "Gabriel García Márquez", pais: "Colombia" },   // documento embebido
  anio: 1967,
  precio: NumberDecimal("19.95"),
  generos: ["novela", "realismo mágico"],                          // array
  ejemplares: [                                                    // array de
    ↪ documentos
    { codigo: "E1", estado: "disponible" },
    { codigo: "E2", estado: "prestado", hasta: ISODate("2026-11-02") }
  ],
  alta: ISODate("2026-09-15T10:30:00Z")
}
```

5.2.2 BSON y tipos de datos

MongoDB no guarda texto JSON sino **BSON** (*Binary JSON*): una representación binaria que se recorre más deprisa y que añade tipos que JSON no tiene. Cada documento puede ocupar como máximo 16 MB.

Tipo BSON	Python (pymongo)	Comentario
String	`str`	UTF-8.
Int32 / Int64	`int`	
Double	`float`	
Decimal128	`bson.Decimal128`	Decimales exactos (importes). Se convierte con `Decimal128(Decimal("19.95"))` y `.to_decimal()`.
Boolean	`bool`	
Date	`datetime`	Fecha y hora en UTC, con precisión de milisegundos. No existe un tipo sólo fecha.
ObjectId	`bson.ObjectId`	Identificador de 12 bytes.
Array	`list`	
Object (documento)	`dict`	
Null	`None`	
Binary	`bytes`	

5.2.3 Identificadores únicos: `_id` y ObjectId

Todo documento tiene un campo `_id` único dentro de su colección, que actúa como clave primaria y está indexado automáticamente. Puede ser de cualquier tipo (un ISBN, un DNI, un número), pero si no se indica, el cliente genera un **ObjectId**: 12 bytes formados por la marca de tiempo de creación (4 bytes), un valor aleatorio propio de cada proceso (5 bytes) y un contador (3 bytes). Así cualquier cliente puede generar identificadores únicos sin consultar al servidor, y además los ObjectId se ordenan aproximadamente por fecha de creación.

5.3 Instalación y herramientas

Servidor. La forma más cómoda es Docker (Apéndice A); también hay instaladores para Windows, macOS y Linux en `mongodb.com`. El servidor escucha en el puerto **27017**. Otra opción es crear un clúster gratuito en **MongoDB Atlas**, el servicio en la nube.

`mongosh`. La consola oficial. Es un intérprete de JavaScript conectado a la base de datos; en ella escribiremos las consultas para aprender el lenguaje.

MongoDB Compass. Cliente gráfico: explorar colecciones, editar documentos, construir consultas y agregaciones visualmente y ver los planes de ejecución.

Database Tools. `mongoimport` / `mongoexport` (importar y exportar JSON y CSV) y `mongodump` / `mongorestore` (copias de seguridad).

```
$ docker compose exec mongo mongosh                        # consola dentro del
    ↪ contenedor
$ mongosh "mongodb://localhost:27017/biblioteca"           # consola local
$ mongoimport --db biblioteca --collection libros --jsonArray --file libros.json
$ mongoexport --db biblioteca --collection libros --out libros.json --jsonArray
```

5.4 El lenguaje de consultas de MongoDB

En `mongosh`, la variable `db` representa la base de datos activa y `db.<colección>` cada colección. Las operaciones son métodos que reciben **documentos** como parámetros: el filtro, los cambios, las opciones... Lo que aprendamos aquí se traslada casi literalmente a Python, cambiando la sintaxis de JavaScript por diccionarios.

5.4.1 Bases de datos y colecciones

```
show dbs                                // bases de datos existentes
use biblioteca                          // cambia a (o "crea") una base de datos
show collections
db.createCollection("socios")           // crear una colección explícitamente
db.libros.insertOne({ titulo: "Nada" }) // ...o se crea sola al insertar
db.socios.drop()                        // eliminar una colección
db.dropDatabase()                       // eliminar la base de datos activa
```

> **Nota.** Una base de datos o una colección no existen realmente hasta que se guarda en ellas el primer documento. Crear la colección explícitamente sólo es necesario para darle opciones, como reglas de validación (apartado 5.5.1).

5.4.2 Inserción

```
db.libros.insertOne({
  isbn: "978-84-376-0494-7", titulo: "Cien años de soledad",
  autor: { nombre: "Gabriel García Márquez", pais: "Colombia" },
  anio: 1967, precio: 19.95, generos: ["novela", "realismo mágico"], prestado: false
})
```

```
db.libros.insertMany([
  { isbn: "978-84-206-3311-6", titulo: "La colmena",
    autor: { nombre: "Camilo José Cela", pais: "España" }, anio: 1951,
    precio: 12.50, generos: ["novela"], prestado: true },
  { isbn: "978-84-233-0000-1", titulo: "Nada",
    autor: { nombre: "Carmen Laforet", pais: "España" }, anio: 1945,
    precio: 9.90, generos: ["novela", "posguerra"], prestado: false },
  { isbn: "978-84-9759-000-2", titulo: "Poeta en Nueva York",
    autor: { nombre: "Federico García Lorca", pais: "España" }, anio: 1940,
    precio: 8.50, generos: ["poesía"], prestado: false, valoraciones: [5, 4, 5] }
])
```

5.4.3 Consultas: `find`

```
db.libros.find({ anio: { $lt: 1960 } },{ titulo: 1, _id: 0 }).sort({ anio: 1 }).limit(5)
```

filtro — proyección — orden y límite

- El **filtro** es un documento con las condiciones. `{}` (o nada) selecciona todos los documentos.
- La **proyección** indica qué campos devolver (`1`) u ocultar (`0`). El `_id` se devuelve siempre salvo que se oculte.
- `find` devuelve un **cursor** sobre el que se encadenan `sort`, `skip` y `limit`; `findOne` devuelve un único documento (o `null`).

```
db.libros.find()                                          // todos
db.libros.findOne({ isbn: "978-84-206-3311-6" })          // igualdad
db.libros.find({ prestado: false, anio: 1945 })           // varias condiciones: AND
db.libros.find({ "autor.pais": "España" })                // campo embebido (entre
    ↪ comillas)
db.libros.find({}, { titulo: 1, "autor.nombre": 1, _id: 0 })   // proyección
db.libros.find().sort({ anio: -1, titulo: 1 }).skip(10).limit(10)   // paginación
db.libros.countDocuments({ prestado: true })
db.libros.distinct("autor.pais")
```

Operadores de consulta

Comparación	`$eq`, `$ne`, `$gt`, `$gte`, `$lt`, `$lte`, `$in`, `$nin`
Lógicos	`$and`, `$or`, `$nor`, `$not`
Elementos	`$exists` (el campo existe), `$type` (es de un tipo BSON)
Evaluación	`$regex` (expresión regular), `$expr` (comparar campos entre sí), `$text` (búsqueda de texto)
Arrays	`$all` (contiene todos), `$size` (tamaño exacto), `$elemMatch` (algún elemento cumple varias condiciones)

```
db.libros.find({ anio: { $gte: 1940, $lt: 1960 } })              // rango
db.libros.find({ "autor.pais": { $in: ["España", "México"] } })
db.libros.find({ $or: [{ prestado: true }, { precio: { $lt: 10 } }] })
db.libros.find({ valoraciones: { $exists: true } })
```

```
db.libros.find({ titulo: { $regex: "^la", $options: "i" } })       // empieza por "la"
db.libros.find({ titulo: /^la/i })                                  // ídem, literal JS
db.libros.find({ $expr: { $gt: ["$precio", { $multiply: ["$anio", 0.01] }] } })
```

Consultas sobre arrays

```
db.libros.find({ generos: "novela" })                       // el array CONTIENE "novela
    ↪ "
db.libros.find({ generos: ["novela"] })                     // el array ES exactamente
    ↪ ["novela"]
db.libros.find({ generos: { $all: ["novela", "posguerra"] } })
db.libros.find({ generos: { $size: 2 } })
db.libros.find({ "generos.0": "poesía" })                   // primer elemento
db.libros.find({ valoraciones: { $gte: 5 } })               // ALGÚN elemento >= 5
// Algún ejemplar que esté prestado Y sea el E2 (en el mismo elemento):
db.libros.find({ ejemplares: { $elemMatch: { codigo: "E2", estado: "prestado" } } })
```

5.4.4 Actualización

`updateOne(filtro, cambios)`	Modifica el primer documento que cumple el filtro.
`updateMany(filtro, cambios)`	Modifica todos los que lo cumplen.
`replaceOne(filtro, documento)`	Sustituye el documento completo (salvo el `_id`).
`findOneAndUpdate(...)`	Modifica y devuelve el documento (antes o después del cambio), de forma atómica.
Opción `{ upsert: true }`	Si no existe ningún documento que cumpla el filtro, lo inserta.

Los cambios se expresan con **operadores de actualización**, que modifican sólo los campos indicados:

`$set`, `$unset`	Asignar un valor / eliminar un campo.
`$inc`, `$mul`	Sumar / multiplicar.
`$min`, `$max`	Asignar sólo si el nuevo valor es menor / mayor.
`$rename`	Renombrar un campo.
`$currentDate`	Asignar la fecha actual.
`$push`, `$addToSet`	Añadir a un array (`$addToSet` sólo si no está). Con `$each` se añaden varios.
`$pull`, `$pop`	Quitar de un array los elementos que cumplen una condición / el primero o el último.
`.$`, `.$[]`	Operador posicional: el elemento del array que cumplió el filtro / todos los elementos.

```
db.libros.updateOne({ isbn: "978-84-206-3311-6" },
                    { $set: { prestado: false }, $currentDate: { modificado: true } })
db.libros.updateMany({ anio: { $lt: 1950 } }, { $mul: { precio: 1.10 } })
db.libros.updateMany({}, { $unset: { valoraciones: "" } })
```

```
db.libros.updateOne({ titulo: "Nada" }, { $inc: { visitas: 1 } })        // crea el
    ↪ campo
db.libros.updateOne({ titulo: "Nada" }, { $addToSet: { generos: "clásico" } })
db.libros.updateOne({ titulo: "Nada" },
                    { $push: { valoraciones: { $each: [4, 5] } } })
db.libros.updateOne({ titulo: "Nada" }, { $pull: { generos: "posguerra" } })
// Cambiar el estado del ejemplar E2 (el elemento que cumple el filtro):
db.libros.updateOne({ "ejemplares.codigo": "E2" },
                    { $set: { "ejemplares.$.estado": "disponible" } })
db.libros.updateOne({ isbn: "978-84-000-0000-0" },
                    { $set: { titulo: "Nuevo" } }, { upsert: true })
db.libros.replaceOne({ isbn: "978-84-000-0000-0" },
                     { isbn: "978-84-000-0000-0", titulo: "Sustituido" })
```

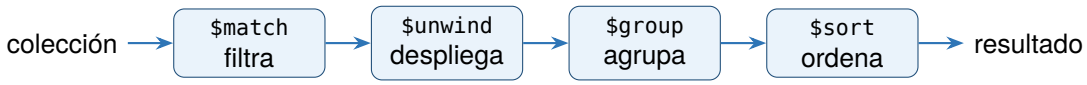
Cuidado. `updateOne({...}, { titulo: "X" })` sin operadores da error en las versiones actuales; en versiones antiguas **sustituía el documento entero**. Usa siempre `$set` para modificar campos y `replaceOne` cuando de verdad quieras sustituirlo.

5.4.5 Borrado

```
db.libros.deleteOne({ isbn: "978-84-000-0000-0" })
db.libros.deleteMany({ prestado: true, anio: { $lt: 1900 } })
db.libros.deleteMany({})              // vacía la colección (pero no la elimina)
```

5.4.6 Agregación

Para consultas más complejas (agrupar, calcular, combinar colecciones, transformar documentos) se usa el **pipeline de agregación**: una lista de **etapas** por las que pasan los documentos, cada una de las cuales transforma el resultado de la anterior.



Etapa	Función	En SQL
`$match`	Filtra documentos (mismo filtro que `find`).	`WHERE` / `HAVING`
`$project`, `$set`	Elige, renombra o calcula campos.	`SELECT`
`$group`	Agrupa por una clave y calcula acumuladores.	`GROUP BY`
`$sort`, `$skip`, `$limit`	Ordena y pagina.	`ORDER BY`, `OFFSET`, `LIMIT`
`$unwind`	Genera un documento por cada elemento de un array.	—
`$lookup`	Combina con documentos de otra colección.	`LEFT JOIN`
`$count`	Cuenta los documentos.	`COUNT(*)`
`$out`, `$merge`	Guarda el resultado en una colección.	`INSERT INTO ... SELECT`

En las etapas, `"$campo"` (con dólar y entre comillas) significa "el valor del campo", y los acumuladores de `$group` son `$sum`, `$avg`, `$min`, `$max`, `$first`, `$last`, `$push` y `$addToSet`.

```
// Número de libros y precio medio por país del autor, de mayor a menor
db.libros.aggregate([
  { $group: { _id: "$autor.pais", libros: { $sum: 1 },
              precio_medio: { $avg: "$precio" } } },
  { $sort: { libros: -1 } }
])

// Número de libros por género (un libro cuenta en cada uno de sus géneros)
db.libros.aggregate([
  { $unwind: "$generos" },
  { $group: { _id: "$generos", total: { $sum: 1 },
              titulos: { $push: "$titulo" } } },
  { $sort: { total: -1, _id: 1 } }
])

// Campos calculados
db.libros.aggregate([
  { $match: { prestado: false } },
  { $project: { _id: 0, titulo: 1, autor: "$autor.nombre",
                antiguedad: { $subtract: [2026, "$anio"] },
                precio_iva: { $round: [{ $multiply: ["$precio", 1.04] }, 2] } } }
])

// Combinar colecciones: cada préstamo con los datos de su libro
db.prestamos.aggregate([
  { $lookup: { from: "libros", localField: "isbn",
               foreignField: "isbn", as: "libro" } },
  { $unwind: "$libro" },
  { $project: { socio: 1, fecha: 1, titulo: "$libro.titulo" } }
])
```

5.5 Modelado de datos

En una base de datos relacional el diseño se basa en normalizar. En MongoDB la pregunta clave es otra: **¿qué datos se leen y se escriben juntos?** Para relacionar información hay dos opciones:

Documentos embebidos. Los datos relacionados se guardan dentro del documento principal (las líneas dentro del pedido). Se leen en una sola operación y se modifican de forma atómica.

Referencias. Se guarda el `_id` del documento relacionado, como una clave ajena (el pedido guarda el `cliente_id`). Hace falta otra consulta o un `$lookup` para obtenerlo.

Embeber cuando...	Referenciar cuando...
La relación es de "contención": las partes no tienen sentido sin el todo (líneas de un pedido, direcciones de un cliente).	Las entidades tienen vida propia y se consultan por separado (clientes y pedidos).
Relación 1:1 o 1:pocos.	Relación 1:muchos con "muchos" grande o que crece sin límite (comentarios de un producto popular).
Los datos se leen casi siempre juntos.	Los datos se comparten entre muchos documentos y cambian a menudo (el precio actual de un producto).

```
// Embebido: el pedido contiene sus líneas y una copia de los datos del cliente
// en ese momento (nombre y dirección de envío)
{ _id: 1001, fecha: ISODate("2026-10-14"),
  cliente: { id: ObjectId("..."), nombre: "Ana García" },
  envio: { calle: "Santiago 12", cp: "47001", ciudad: "Valladolid" },
  lineas: [ { producto: "Teclado", cantidad: 1, precio: 24.90 },
            { producto: "Ratón",   cantidad: 2, precio: 12.50 } ],
  total: 49.90 }

// Referencia N:M: un alumno guarda los _id de sus módulos
{ _id: "11111111H", nombre: "Ana", modulos: ["0486", "0490", "0489"] }
```

5.5.1 Validación del esquema

Que MongoDB no exija un esquema no significa que no deba haberlo. Una colección puede tener reglas de validación escritas con **JSON Schema**, el mismo estándar que usamos en el Tema 1: el servidor rechaza los documentos que no las cumplen.

```
db.createCollection("libros", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["isbn", "titulo", "anio"],
    properties: {
      isbn:    { bsonType: "string", pattern: "^97[89]-" },
      titulo:  { bsonType: "string", minLength: 1 },
      anio:    { bsonType: "int", minimum: 1450 },
      precio:  { bsonType: ["double", "decimal"], minimum: 0 },
      generos: { bsonType: "array", items: { bsonType: "string" } }
    }
  } },
  validationAction: "error"          // "warn" sólo lo registra
})
```

5.6 Índices

Sin índices, MongoDB tiene que examinar todos los documentos de la colección en cada consulta (*collection scan*). Los índices, como en una base de datos relacional, aceleran las búsquedas y ordenaciones a cambio de espacio y de un pequeño coste en cada escritura.

De un campo	`{ isbn: 1 }` (1 ascendente, -1 descendente).
Compuesto	`{ "autor.pais": 1, anio: -1 }`: sirve para consultas por país, o por país y año.
Multiclave	Se crea automáticamente al indexar un campo array: un índice por cada elemento.
Único	`{ unique: true }`: impide valores repetidos (identificador de negocio).
De texto	`{ titulo: "text" }`: búsquedas por palabras con `$text`.
TTL	`{ expireAfterSeconds: 3600 }` sobre un campo fecha: los documentos se borran solos (sesiones, cachés).

```
db.libros.createIndex({ isbn: 1 }, { unique: true })
db.libros.createIndex({ "autor.pais": 1, anio: -1 })
db.libros.createIndex({ titulo: "text" })
db.libros.find({ $text: { $search: "soledad colmena" } })
db.libros.getIndexes()
db.libros.dropIndex("autor.pais_1_anio_-1")
db.libros.find({ "autor.pais": "España" }).explain("executionStats")
// En el plan: IXSCAN = usa un índice; COLLSCAN = recorre toda la colección
```

5.7 MongoDB desde Python: `pymongo`

`pymongo` es el conector oficial de MongoDB para Python. Los documentos se representan como **diccionarios** y los nombres de los métodos son los mismos que en `mongosh`, pero en estilo Python (`insert_one` en lugar de `insertOne`).

```
$ pip install pymongo
```

mongosh	**pymongo**
`db.libros.insertOne({ titulo: "Nada" })`	`db.libros.insert_one({"titulo": "Nada"})`
`{ anio: { $lt: 1960 } }`	`{"anio": {"$lt": 1960}}`
`.sort({ anio: -1 })`	`.sort("anio", -1)` o `.sort([("anio", -1)])`
`true`, `false`, `null`	`True`, `False`, `None`
`ISODate("2026-10-14")`	`datetime(2026, 10, 14)`
`ObjectId("...")`	`bson.ObjectId("...")`

5.7.1 Establecimiento y cierre de conexiones

La clase `MongoClient` representa la conexión con el servidor (en realidad, un *pool* de conexiones). Se crea una vez por aplicación a partir de una **URI de conexión**:

```
mongodb://usuario:contraseña@servidor:27017/basededatos?opciones
mongodb+srv://usuario:contraseña@cluster0.xxxxx.mongodb.net/    (MongoDB Atlas)
```

```
import os
from pymongo import MongoClient
```

```python
from pymongo.errors import ConnectionFailure

URI = os.getenv("MONGO_URI", "mongodb://localhost:27017/")

# MongoClient no conecta hasta la primera operación; "ping" lo comprueba
with MongoClient(URI, serverSelectionTimeoutMS=3000) as cliente:
    try:
        cliente.admin.command("ping")
    except ConnectionFailure as e:
        raise SystemExit(f"No se pudo conectar con MongoDB: {e}")
    print(cliente.list_database_names())
# al salir del with se cierra el cliente (también: cliente.close())
```

5.7.2 Bases de datos y colecciones

```python
from pymongo import MongoClient

cliente = MongoClient("mongodb://localhost:27017/")
bd = cliente["biblioteca"]              # o cliente.biblioteca
libros = bd["libros"]                   # o bd.libros

print(bd.list_collection_names())
if "socios" not in bd.list_collection_names():
    bd.create_collection("socios")      # crear explícitamente
bd.drop_collection("socios")            # o bd.socios.drop()
cliente.drop_database("pruebas")
```

5.7.3 Añadir, modificar y eliminar documentos

Cada operación devuelve un objeto con información sobre su resultado:

```python
from datetime import datetime, timezone

r = libros.insert_one({
    "isbn": "978-84-376-0494-7", "titulo": "Cien años de soledad",
    "autor": {"nombre": "Gabriel García Márquez", "pais": "Colombia"},
    "anio": 1967, "precio": 19.95, "generos": ["novela", "realismo mágico"],
    "prestado": False,
})
print(r.inserted_id)                    # ObjectId generado

r = libros.insert_many([
    {"isbn": "978-84-206-3311-6", "titulo": "La colmena",
     "autor": {"nombre": "Camilo José Cela", "pais": "España"},
     "anio": 1951, "precio": 12.50, "generos": ["novela"], "prestado": True},
    {"isbn": "978-84-233-0000-1", "titulo": "Nada",
     "autor": {"nombre": "Carmen Laforet", "pais": "España"},
     "anio": 1945, "precio": 9.90, "generos": ["novela", "posguerra"], "prestado": False
    ↪ },
])
print(len(r.inserted_ids), "insertados")

r = libros.update_one({"isbn": "978-84-206-3311-6"},
                      {"$set": {"prestado": False,
```

```
                                "modificado": datetime.now(timezone.utc)}})
print(r.matched_count, r.modified_count)          # encontrados / modificados

libros.update_many({"anio": {"$lt": 1950}}, {"$addToSet": {"generos": "clásico"}})
libros.update_one({"isbn": "978-84-000-0000-0"},
                  {"$set": {"titulo": "Nuevo"}}, upsert=True)

r = libros.delete_many({"titulo": "Nuevo"})
print(r.deleted_count, "eliminados")
```

5.7.4 Consultas

`find` devuelve un **cursor**, que se recorre con `for` y va pidiendo los documentos al servidor por lotes; `find_one` devuelve un diccionario o `None`.

```
from pymongo import ASCENDING, DESCENDING

libro = libros.find_one({"isbn": "978-84-376-0494-7"})
if libro is None:
    print("No existe")

for l in libros.find({"anio": {"$gte": 1940, "$lt": 1960}}):
    print(l["titulo"], l["anio"])

cursor = (libros.find({"autor.pais": "España"},
                      {"_id": 0, "titulo": 1, "autor.nombre": 1})   # proyección
          .sort([("anio", DESCENDING), ("titulo", ASCENDING)])
          .skip(0).limit(10))
pagina = list(cursor)

libros.find({"$or": [{"prestado": True}, {"precio": {"$lt": 10}}]})
libros.find({"titulo": {"$regex": "^la", "$options": "i"}})
print(libros.count_documents({"generos": "novela"}))
print(libros.distinct("autor.pais"))

pipeline = [
    {"$unwind": "$generos"},
    {"$group": {"_id": "$generos", "total": {"$sum": 1},
                "precio_medio": {"$avg": "$precio"}}},
    {"$sort": {"total": -1, "_id": 1}},
]
for g in libros.aggregate(pipeline):
    print(f"{g['_id']:<16} {g['total']:>3}  {g['precio_medio']:.2f}")
```

Inyección en MongoDB. Aunque no hay SQL, también existe la inyección: si un filtro se construye con datos que llegan sin comprobar (por ejemplo, un JSON enviado por el usuario), alguien puede enviar `{"$ne": null}` como contraseña y la condición se cumplirá siempre. Comprueba los **tipos** de los datos externos (que una contraseña sea `str`, no `dict`) antes de usarlos en un filtro, y nunca construyas filtros a partir de cadenas.

5.7.5 Documentos y objetos

Igual que con los conectores relacionales, conviene que la aplicación trabaje con objetos del dominio y que la conversión desde y hacia documentos quede encapsulada en un **repositorio**:

```
from dataclasses import dataclass, field, asdict
from bson import ObjectId
from pymongo.collection import Collection

@dataclass
class Autor:
    nombre: str
    pais: str | None = None

@dataclass
class Libro:
    isbn: str
    titulo: str
    autor: Autor
    anio: int
    precio: float
    generos: list[str] = field(default_factory=list)
    prestado: bool = False
    id: ObjectId | None = None

    def a_documento(self) -> dict:
        doc = asdict(self)                # asdict también convierte el Autor
        doc.pop("id")
        if self.id is not None:
            doc["_id"] = self.id
        return doc

    @classmethod
    def desde_documento(cls, doc: dict) -> "Libro":
        doc = dict(doc)
        ident = doc.pop("_id", None)
        autor = Autor(**doc.pop("autor"))
        campos = {k: v for k, v in doc.items() if k in cls.__dataclass_fields__}
        return cls(autor=autor, id=ident, **campos)


class RepositorioLibros:
    def __init__(self, coleccion: Collection) -> None:
        self.col = coleccion
        self.col.create_index("isbn", unique=True)

    def guardar(self, libro: Libro) -> Libro:
        if libro.id is None:
            libro.id = self.col.insert_one(libro.a_documento()).inserted_id
        else:
            self.col.replace_one({"_id": libro.id}, libro.a_documento())
        return libro

    def por_isbn(self, isbn: str) -> Libro | None:
        doc = self.col.find_one({"isbn": isbn})
        return Libro.desde_documento(doc) if doc else None

    def por_genero(self, genero: str) -> list[Libro]:
```

```
        return [Libro.desde_documento(d)
                for d in self.col.find({"generos": genero}).sort("titulo")]

    def prestar(self, isbn: str) -> bool:
        """Operación atómica: sólo presta si no estaba prestado."""
        r = self.col.update_one({"isbn": isbn, "prestado": False},
                                {"$set": {"prestado": True}})
        return r.modified_count == 1

    def borrar(self, isbn: str) -> bool:
        return self.col.delete_one({"isbn": isbn}).deleted_count == 1
```

ODM. Lo que es un ORM para las bases de datos relacionales es un **ODM** (*Object-Document Mapper*) para las documentales: bibliotecas que hacen este mapeo automáticamente a partir de las clases. Para MongoDB en Python destacan **Beanie** (basado en Pydantic, asíncrono) y **MongoEngine**.

5.7.6 Operaciones masivas

Para ejecutar muchas escrituras con una sola petición al servidor se usa `bulk_write`:

```
from pymongo import InsertOne, UpdateOne, DeleteOne
from pymongo.errors import BulkWriteError

operaciones = [
    InsertOne({"isbn": "978-84-000-0001-0", "titulo": "Uno"}),
    UpdateOne({"isbn": "978-84-233-0000-1"}, {"$inc": {"visitas": 1}}),
    DeleteOne({"isbn": "978-84-000-0000-0"}),
]
try:
    r = libros.bulk_write(operaciones, ordered=False)   # sigue aunque alguna falle
    print(r.inserted_count, r.modified_count, r.deleted_count)
except BulkWriteError as e:
    print("Errores:", e.details["writeErrors"])
```

5.7.7 Índices y validación desde Python

```
from pymongo import ASCENDING, DESCENDING, TEXT

libros.create_index([("isbn", ASCENDING)], unique=True)
libros.create_index([("autor.pais", ASCENDING), ("anio", DESCENDING)])
libros.create_index([("titulo", TEXT)])
print(libros.index_information())
print(libros.find({"autor.pais": "España"}).explain()["queryPlanner"]["winningPlan"])

bd.create_collection("socios", validator={"$jsonSchema": {
    "bsonType": "object",
    "required": ["dni", "nombre"],
    "properties": {"dni": {"bsonType": "string", "pattern": "^[0-9]{8}[A-Z]$"},
                   "nombre": {"bsonType": "string"}},
}})
```

5.7.8 Gestión de transacciones

En MongoDB **las operaciones sobre un único documento son siempre atómicas**, incluidas las que modifican varios campos o arrays embebidos. Por eso, un buen modelado (embeber lo que se modifica junto) hace innecesarias muchas transacciones; por ejemplo, el método `prestar` anterior comprueba y modifica en una sola operación.

Cuando una operación afecta a varios documentos o colecciones, MongoDB ofrece **transacciones ACID** (requieren que el servidor funcione como *replica set*, aunque sea de un solo nodo; el Apéndice A lo configura así). Las operaciones de la transacción reciben el parámetro `session`:

```
from datetime import datetime, timezone

def prestar_libro(cliente, isbn: str, socio: str) -> None:
    bd = cliente.biblioteca
    with cliente.start_session() as sesion:
        with sesion.start_transaction():
            r = bd.libros.update_one({"isbn": isbn, "prestado": False},
                                     {"$set": {"prestado": True}}, session=sesion)
            if r.modified_count != 1:
                raise ValueError("El libro no existe o ya está prestado")  # abort
            n = bd.prestamos.count_documents({"socio": socio, "devuelto": False},
                                             session=sesion)
            if n >= 3:
                raise ValueError("El socio ya tiene 3 préstamos")          # abort
            bd.prestamos.insert_one({"isbn": isbn, "socio": socio, "devuelto": False,
                                     "fecha": datetime.now(timezone.utc)},
                                    session=sesion)
        # commit al salir del bloque sin excepciones
```

Para reintentar automáticamente las transacciones que fallan por errores transitorios (conflictos, cambio del servidor primario), la sesión de pymongo ofrece el método `with_transaction`.

5.7.9 Tratamiento de excepciones

Todas las excepciones de pymongo derivan de `pymongo.errors.PyMongoError`:

`ConnectionFailure`	No se puede conectar con el servidor o se ha perdido la conexión.
`  ServerSelectionTimeoutError`	No se encontró un servidor disponible en el tiempo indicado.
`OperationFailure`	El servidor rechaza la operación: sintaxis incorrecta, sin permisos...
`  WriteError` / `DuplicateKeyError`	Error al escribir / clave única repetida.
`BulkWriteError`	Alguna operación de `bulk_write` o `insert_many` falló.
`InvalidDocument`	El documento contiene un valor que no se puede convertir a BSON.

```
from pymongo.errors import (DuplicateKeyError, ServerSelectionTimeoutError,
                            OperationFailure, PyMongoError)
```

```python
try:
    libros.insert_one({"isbn": "978-84-376-0494-7", "titulo": "Repetido"})
except DuplicateKeyError:
    print("Ya existe un libro con ese ISBN")
except ServerSelectionTimeoutError:
    print("No se encuentra el servidor de MongoDB")
except OperationFailure as e:
    print("El servidor rechazó la operación:", e.details.get("errmsg"))
except PyMongoError as e:
    print("Error de MongoDB:", e)
```

5.7.10 Pruebas

Para las pruebas se puede usar una base de datos de pruebas en el mismo servidor, que se elimina al terminar, o la biblioteca `mongomock`, que simula MongoDB en memoria (implementa la mayor parte de las operaciones habituales, pero no todas: por ejemplo, las transacciones).

```python
# tests/conftest.py
import pytest
from pymongo import MongoClient

@pytest.fixture
def coleccion():
    cliente = MongoClient("mongodb://localhost:27017/")
    bd = cliente["biblioteca_test"]
    yield bd["libros"]
    cliente.drop_database("biblioteca_test")
    cliente.close()

# Alternativa sin servidor:  import mongomock; cliente = mongomock.MongoClient()

def test_prestar_dos_veces(coleccion):
    repo = RepositorioLibros(coleccion)
    repo.guardar(Libro("978-84-1", "Prueba", Autor("A"), 2000, 10.0))
    assert repo.prestar("978-84-1") is True
    assert repo.prestar("978-84-1") is False
```

5.8 Ejercicios

Ejercicio 5.1 — Primeros pasos con mongosh

Dificultad: ★

Crea en `mongosh` una base de datos `instituto` con una colección `alumnos` e inserta al menos diez alumnos con nombre, apellidos, fecha de nacimiento, ciclo, curso, dirección (documento embebido), módulos matriculados (array) y notas (array de documentos `{modulo, nota}`). Escribe consultas para: alumnos de un ciclo; alumnos nacidos después de una fecha; alumnos matriculados en un módulo; alumnos con alguna nota inferior a 5 (`$elemMatch`); alumnos sin dirección; listado ordenado por apellidos mostrando sólo nombre y apellidos.

Ejercicio 5.2 — Actualizaciones

Dificultad: ★

Sobre la colección del ejercicio anterior, escribe en `mongosh`: cambiar de curso a todos los alumnos de 1.º; añadir un módulo a un alumno sin repetirlo; quitar un módulo a todos los alumnos; subir un punto la nota de un módulo concreto de un alumno (operador posicional); añadir un campo `fecha_modificacion` a todos los documentos modificados; y un *upsert* que cree un alumno si no existe.

Ejercicio 5.3 — Agregación

Dificultad: ★★

Importa con `mongoimport` un conjunto de datos abiertos en JSON (por ejemplo, de `https://datosabiertos.jcyl.es`: establecimientos turísticos, museos, centros educativos...). Con el *pipeline* de agregación obtén: número de registros por provincia; los diez municipios con más registros; un campo calculado; un listado agrupado que incluya en cada grupo la lista de nombres (`$push`). Escribe al lado de cada consulta su equivalente en SQL.

Ejercicio 5.4 — Modelado

Dificultad: ★★

Diseña el modelo de documentos de una red social sencilla: usuarios, publicaciones, comentarios, "me gusta" y seguidores. Justifica para cada relación si la embebes o la referencias, teniendo en cuenta qué consultas serán las más frecuentes (muro de un usuario, publicación con sus comentarios, número de seguidores). Escribe el `$jsonSchema` de validación de cada colección y crea los índices necesarios.

Ejercicio 5.5 — CRUD con pymongo

Dificultad: ★★

Implementa en Python un programa de consola para gestionar la colección `alumnos` del primer ejercicio, con una *dataclass* `Alumno` y un `RepositorioAlumnos` que haga la conversión entre objetos y documentos. Operaciones: alta, baja, modificación, búsqueda por DNI, por apellidos (expresión regular, sin distinguir mayúsculas) y por ciclo, y poner una nota. Trata las excepciones (servidor no disponible, DNI duplicado) y escribe pruebas con `pytest`.

Ejercicio 5.6 — Índices

Dificultad: ★★

Genera desde Python, con `insert_many`, 200 000 documentos de ventas aleatorias (fecha, tienda, producto, categoría, unidades, importe). Mide con `explain()` (`executionStats`) y con `time.perf_counter` el tiempo de tres consultas típicas antes y después de crear los índices adecuados, comprobando el paso de `COLLSCAN` a `IXSCAN`. Comprueba también cuánto más tarda la inserción con los índices creados.

Ejercicio 5.7 — Migración de relacional a documental

Dificultad: ★★★

Escribe un programa que lea la base de datos SQLite del videoclub del Tema 2 (clientes, películas, copias y alquileres) y la migre a MongoDB con un modelo documental

razonado: por ejemplo, las copias embebidas en la película y los alquileres en su propia colección con referencias. Comprueba después con agregaciones que se obtienen los mismos resultados que con las consultas SQL originales (ranking de películas más alquiladas, alquileres vencidos).

Proyecto: Repositorio de recetas

Desarrolla una aplicación para gestionar un repositorio de recetas de cocina sobre MongoDB.
Cada receta tiene: título, autor, categoría, tiempo, dificultad, raciones, lista de ingredientes (nombre, cantidad, unidad), pasos ordenados, etiquetas, fecha de publicación y valoraciones de usuarios (usuario, puntuación, comentario, fecha). Los usuarios se guardan en su propia colección y pueden marcar recetas como favoritas.
Requisitos:

1. **Modelado** razonado (qué se embebe y qué se referencia) y reglas de validación `$jsonSchema` en cada colección.
2. Script de **inicialización** que cree (o elimine y vuelva a crear) las colecciones, sus índices y la validación, e importe recetas de ejemplo desde una carpeta de ficheros JSON, informando de las que no superen la validación.
3. Modelo con *dataclasses* o Pydantic y una capa de repositorios que encapsule la conversión entre objetos y documentos.
4. **Operaciones**: alta, modificación parcial (añadir una valoración, cambiar el tiempo, añadir una etiqueta sin repetir) y baja de recetas; alta de usuarios; marcar y desmarcar favoritos.
5. **Consultas**: por categoría, por ingrediente, por tiempo máximo, recetas que se pueden preparar con una lista de ingredientes disponibles, búsqueda de texto completo y paginación de resultados.
6. **Agregaciones**: recetas mejor valoradas (media y número de valoraciones), ingredientes más usados, número de recetas por categoría y dificultad, y las recetas favoritas de un usuario con sus datos (`$lookup`).
7. Al menos una operación **transaccional** que afecte a varias colecciones (por ejemplo, eliminar un usuario junto con sus valoraciones y favoritos).
8. **Exportación** de una receta a HTML y de la lista de la compra de varias recetas (sumando cantidades del mismo ingrediente y unidad) a CSV (Tema 1).
9. Tratamiento de excepciones, configuración de la conexión fuera del código y pruebas con `pytest`.

Ampliación: compara el diseño con el de una base de datos relacional para el mismo problema, o implementa una versión con PostgreSQL y `JSONB` (Tema 4).

TEMA 6

Programación de componentes de acceso a datos

En este tema aprenderás a…

- Explicar qué es un componente, sus características y las ventajas e inconvenientes de la programación orientada a componentes.
- Definir propiedades simples, indexadas, compartidas (*bound*) y restringidas (*constrained*).
- Programar componentes que generan eventos y permiten asociarles acciones.
- Usar la introspección y la reflexión de Python para construir editores de propiedades y cargar componentes dinámicamente.
- Hacer persistente el estado de un componente.
- Empaquetar, documentar, probar e integrar componentes de acceso a datos para ficheros, conectores, ORM, bases de datos de objetos y documentales.

Resultado de aprendizaje asociado: **RA6**.

Actualización del currículo. Los contenidos de este bloque en el currículo (editores de propiedades, propiedades compartidas y restringidas, introspección, persistencia del componente…) están redactados pensando en el modelo de componentes **JavaBeans**. Los conceptos siguen siendo válidos, pero en Python se implementan con otras herramientas: propiedades y descriptores, el patrón observador, el módulo `inspect` y los paquetes instalables con `pyproject.toml`. Este tema presenta cada concepto, indica su equivalente en JavaBeans y muestra cómo se hace en Python.

6.1 Concepto de componente

Un **componente** es una unidad de software **reutilizable**, **autocontenida** y con una **interfaz bien definida**, que puede combinarse con otros componentes para construir aplicaciones sin necesidad de conocer ni modificar su código interno.

Sus características principales son:

Encapsulación. Oculta su implementación; sólo se usa a través de su interfaz.

Interfaz definida. Expone **propiedades** (estado configurable), **métodos** (operaciones) y **eventos** (avisos de que algo ha ocurrido).

Reutilizable e independiente del contexto. No depende de la aplicación concreta en la que se use.

Configurable. Su comportamiento se adapta modificando sus propiedades, no su código.

Introspectable. Una herramienta puede descubrir en tiempo de ejecución qué propiedades, métodos y eventos tiene.

Persistente. Su configuración puede guardarse y restaurarse.

Desplegable. Se distribuye como una unidad (un `.jar` en Java, un paquete *wheel* en Python).

Un **componente de acceso a datos** es un componente que encapsula el almacenamiento y la recuperación de la información: la aplicación le pide "guarda este alumno" o "dame los libros prestados" sin saber si los datos están en un CSV, en PostgreSQL, en ZODB o en MongoDB.

6.2 Programación orientada a componentes

La programación orientada a componentes construye las aplicaciones **ensamblando** componentes existentes, del mismo modo que un equipo electrónico se monta con piezas estándar.

+ **Reutilización**: un componente probado se usa en muchas aplicaciones.
+ **Menor tiempo y coste de desarrollo** y mayor calidad, al partir de piezas probadas.
+ **Mantenimiento** más sencillo: se puede sustituir un componente por otra versión o implementación sin tocar el resto.
+ Permite el **desarrollo en paralelo** por equipos distintos y el uso de componentes de terceros.
– Diseñar componentes realmente reutilizables es más costoso que escribir código para un caso concreto.
– Dependencia de terceros: versiones, licencias, componentes abandonados.
– Posibles incompatibilidades entre versiones y sobrecarga por la generalidad del componente.
– Puede ser difícil adaptar un componente a necesidades que su diseño no previó.

6.2.1 De JavaBeans a Python

Concepto	JavaBeans	Python
Propiedad	Pareja `getX()` / `setX()`	`@property` o descriptor
Propiedad indexada	`getX(int)` / `setX(int, v)`	Objeto con `__getitem__` / `__setitem__`
Propiedad compartida	`PropertyChangeSupport`	Patrón observador (este tema)
Propiedad restringida	`VetoableChangeSupport`	Observador con veto mediante excepción
Eventos	Interfaces `XxxListener`	*Callbacks*, señales (Qt, `blinker`)
Introspección	`Introspector`, `BeanInfo`	`inspect`, `dataclasses.fields`, anotaciones
Editor de propiedades	`PropertyEditor`	Conversores texto ↔ valor según el tipo
Persistencia	`Serializable`, `XMLEncoder`	JSON a partir de la introspección, `pickle`
Empaquetado	Fichero `.jar` con manifiesto	Paquete *wheel* con `pyproject.toml`

6.3 Propiedades y atributos

Un **atributo** es un dato interno del objeto. Una **propiedad** es un elemento de la interfaz pública del componente que se lee y escribe como si fuera un atributo, pero cuyo acceso pasa por código del componente (validación, conversión, avisos...). En Python, gracias a `@property`, la diferencia es invisible para quien usa el componente.

6.3.1 Propiedades simples

```python
class ConfiguracionConexion:
    """Componente con propiedades simples validadas."""

    def __init__(self, servidor: str = "localhost", puerto: int = 5432) -> None:
        self.servidor = servidor
        self.puerto = puerto

    @property
    def puerto(self) -> int:
        return self._puerto

    @puerto.setter
    def puerto(self, valor: int) -> None:
        if not 1 <= valor <= 65535:
            raise ValueError("Puerto fuera de rango")
        self._puerto = valor

    @property
    def url(self) -> str:                # propiedad de sólo lectura (calculada)
        return f"{self.servidor}:{self.puerto}"
```

6.3.2 Descriptores: propiedades reutilizables

Si muchas propiedades necesitan la misma lógica, escribir un `@property` para cada una es repetitivo. Un **descriptor** es una clase que define cómo se lee y escribe un atributo, y se reutiliza en tantas propiedades como se quiera. (De hecho, `property` y los atributos de SQLAlchemy son descriptores.)

```
class Rango:
    """Descriptor: propiedad numérica con límites."""

    def __init__(self, minimo: float, maximo: float) -> None:
        self.minimo, self.maximo = minimo, maximo

    def __set_name__(self, propietario, nombre: str) -> None:
        self.nombre = nombre                # nombre de la propiedad
        self.privado = "_" + nombre

    def __get__(self, obj, tipo=None):
        if obj is None:                     # acceso desde la clase
            return self
        return getattr(obj, self.privado)

    def __set__(self, obj, valor) -> None:
        if not self.minimo <= valor <= self.maximo:
            raise ValueError(f"{self.nombre} debe estar entre "
                             f"{self.minimo} y {self.maximo}")
        setattr(obj, self.privado, valor)


class ConfiguracionPool:
    tamano = Rango(1, 100)
    timeout = Rango(0.1, 60)

    def __init__(self, tamano: int = 5, timeout: float = 30) -> None:
        self.tamano = tamano
        self.timeout = timeout
```

6.3.3 Propiedades indexadas

Una propiedad indexada representa una colección de valores a la que se accede por posición. En Python se implementa devolviendo un objeto que se comporta como una secuencia y que controla el acceso a sus elementos:

```
from collections.abc import MutableSequence

class ListaValidada(MutableSequence):
    """Secuencia que valida cada elemento al asignarlo."""

    def __init__(self, validador, valores=()) -> None:
        self._validar = validador
        self._datos: list = []
        for v in valores:
            self.append(v)

    def __getitem__(self, i):            return self._datos[i]
    def __len__(self) -> int:            return len(self._datos)
```

```
    def __delitem__(self, i) -> None:    del self._datos[i]

    def __setitem__(self, i, valor) -> None:
        self._validar(valor)
        self._datos[i] = valor

    def insert(self, i, valor) -> None:
        self._validar(valor)
        self._datos.insert(i, valor)

    def __repr__(self) -> str:
        return repr(self._datos)


def validar_servidor(s: str) -> None:
    if not s or " " in s:
        raise ValueError(f"Nombre de servidor no válido: {s!r}")

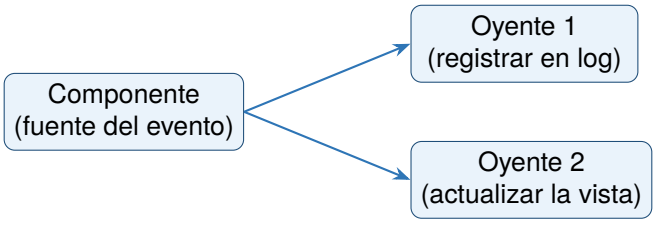
class ClusterBD:
    def __init__(self) -> None:
        self._servidores = ListaValidada(validar_servidor)

    @property
    def servidores(self) -> ListaValidada:      # propiedad indexada
        return self._servidores

c = ClusterBD()
c.servidores.append("bd1.instituto.local")
c.servidores[0] = "bd-principal"                 # validado
print(c.servidores[0], len(c.servidores))
```

6.4 Eventos y asociación de acciones a eventos

Un **evento** es un aviso que emite un componente cuando ocurre algo relevante (se ha guardado un registro, ha fallado la conexión, ha cambiado una propiedad). Otros objetos, los **oyentes** (*listeners*), se registran en el componente para que se ejecute una acción cuando se produzca el evento. Es el **patrón observador**, y permite que el componente avise a la aplicación sin conocerla.



Una implementación sencilla y reutilizable de un evento en Python:

```
from collections.abc import Callable
from typing import Any

class Evento:
    """Lista de oyentes a los que se avisa al emitir el evento."""
```

```
    def __init__(self) -> None:
        self._oyentes: list[Callable[..., Any]] = []

    def conectar(self, oyente: Callable[..., Any]) -> Callable[..., Any]:
        self._oyentes.append(oyente)
        return oyente                    # permite usarlo como decorador

    def desconectar(self, oyente: Callable[..., Any]) -> None:
        self._oyentes.remove(oyente)

    def emitir(self, *args, **kwargs) -> None:
        for oyente in list(self._oyentes):
            oyente(*args, **kwargs)
```

```
class RepositorioLibros:
    def __init__(self) -> None:
        self.libro_guardado = Evento()    # eventos públicos del componente
        self.libro_borrado = Evento()
        self._libros: dict[str, dict] = {}

    def guardar(self, libro: dict) -> None:
        self._libros[libro["isbn"]] = libro
        self.libro_guardado.emitir(libro)

    def borrar(self, isbn: str) -> None:
        libro = self._libros.pop(isbn)
        self.libro_borrado.emitir(libro)

# Asociación de acciones a eventos (en la aplicación, no en el componente)
repo = RepositorioLibros()
repo.libro_guardado.conectar(lambda l: print("Guardado:", l["titulo"]))

@repo.libro_borrado.conectar
def auditar(libro: dict) -> None:
    with open("auditoria.log", "a", encoding="utf-8") as f:
        f.write(f"BORRADO {libro['isbn']}\n")

repo.guardar({"isbn": "1", "titulo": "Nada"})
```

> **Nota.** Las bibliotecas de interfaz gráfica tienen su propio sistema de eventos (señales y *slots* en Qt/PySide, `bind` en Tkinter). Para componentes no visuales existe la biblioteca `blinker`, que implementa señales con nombre; también la usan Flask y otros *frameworks*.

6.5 Propiedades compartidas y restringidas

Propiedad compartida (*bound property*). Cuando cambia su valor, el componente **notifica** el cambio a los oyentes registrados, indicando el nombre de la propiedad, el valor anterior y el nuevo. Permite, por ejemplo, que varios objetos compartan y sincronicen un mismo dato.

Propiedad restringida (*constrained property*). Antes de cambiar, el componente **consulta** a los oyentes, y cualquiera de ellos puede **vetar** el cambio lanzando una excepción. Si se veta, la propiedad conserva su valor.

El siguiente descriptor implementa ambas cosas y lo reutilizaremos en los componentes de acceso a datos:

```
# componentes/propiedades.py
from dataclasses import dataclass
from typing import Any
from collections.abc import Callable

@dataclass(frozen=True)
class CambioPropiedad:
    fuente: Any
    nombre: str
    anterior: Any
    nuevo: Any

class CambioVetado(Exception):
    """Un oyente ha rechazado el cambio de una propiedad restringida."""

class SoportePropiedades:
    """Clase base (mixin) que gestiona los oyentes de cambios."""

    def __init__(self) -> None:
        self._oyentes_cambio: list[Callable[[CambioPropiedad], None]] = []
        self._oyentes_veto: list[Callable[[CambioPropiedad], None]] = []

    def al_cambiar(self, oyente: Callable[[CambioPropiedad], None]) -> None:
        self._oyentes_cambio.append(oyente)

    def al_intentar_cambiar(self, oyente: Callable[[CambioPropiedad], None]) -> None:
        self._oyentes_veto.append(oyente)


class Propiedad:
    """Descriptor de propiedad simple, compartida y/o restringida."""

    def __init__(self, defecto: Any = None, *, compartida: bool = True,
                 restringida: bool = False, tipo: type | None = None) -> None:
        self.defecto = defecto
        self.compartida = compartida
        self.restringida = restringida
        self.tipo = tipo

    def __set_name__(self, propietario, nombre: str) -> None:
        self.nombre = nombre
        self.privado = "_" + nombre

    def __get__(self, obj, tipo=None):
        if obj is None:
            return self
        return getattr(obj, self.privado, self.defecto)

    def __set__(self, obj: SoportePropiedades, valor: Any) -> None:
        if self.tipo is not None and not isinstance(valor, self.tipo):
            raise TypeError(f"{self.nombre} debe ser de tipo {self.tipo.__name__}")
        anterior = self.__get__(obj)
        if anterior == valor:
            return
        cambio = CambioPropiedad(obj, self.nombre, anterior, valor)
```

```
        if self.restringida:
            for oyente in obj._oyentes_veto:
                oyente(cambio)                  # puede lanzar CambioVetado
        setattr(obj, self.privado, valor)
        if self.compartida:
            for oyente in obj._oyentes_cambio:
                oyente(cambio)
```

```
from componentes.propiedades import (SoportePropiedades, Propiedad,
                                     CambioPropiedad, CambioVetado)

class FuenteDatos(SoportePropiedades):
    ruta = Propiedad("datos.db", restringida=True, tipo=str)
    solo_lectura = Propiedad(False, tipo=bool)

fuente = FuenteDatos()
fuente.al_cambiar(lambda c: print(f"{c.nombre}: {c.anterior!r} -> {c.nuevo!r}"))

def no_permitir_temporales(c: CambioPropiedad) -> None:
    if c.nombre == "ruta" and c.nuevo.startswith("/tmp"):
        raise CambioVetado("No se permiten bases de datos en /tmp")

fuente.al_intentar_cambiar(no_permitir_temporales)
fuente.ruta = "instituto.db"            # ruta: 'datos.db' -> 'instituto.db'
try:
    fuente.ruta = "/tmp/x.db"
except CambioVetado as e:
    print("Vetado:", e, "| valor actual:", fuente.ruta)
```

6.6 Introspección y reflexión

Introspección es la capacidad de un programa de examinar en tiempo de ejecución la estructura de los objetos: su tipo, sus atributos, sus métodos y sus firmas. **Reflexión** va un paso más allá: permite además *actuar* a partir de esa información (leer o modificar atributos por su nombre, invocar métodos, crear objetos de clases cuyo nombre sólo se conoce al ejecutar).

Las herramientas de desarrollo de componentes usan la introspección para mostrar las propiedades de un componente y permitir editarlas sin conocerlo de antemano. Python es especialmente rico en este aspecto:

`type(obj)`, `isinstance(obj, C)`	Tipo de un objeto.
`dir(obj)`, `vars(obj)`	Nombres de sus atributos; diccionario de atributos de instancia.
`getattr`, `setattr`, `hasattr`, `delattr`	Acceso a atributos por nombre (reflexión).
`callable(x)`	¿Se puede invocar?
`inspect.getmembers(obj, pred)`	Miembros que cumplen un predicado (métodos, funciones, propiedades...).
`inspect.signature(f)`	Parámetros de una función o método.
`inspect.getdoc(obj)`	Documentación.
`typing.get_type_hints(C)`	Anotaciones de tipo resueltas.
`dataclasses.fields(C)`	Campos de una *dataclass*.
`importlib.import_module(n)`	Importar un módulo cuyo nombre se conoce en ejecución.

```python
import inspect

def describir_componente(clase: type) -> None:
    """Muestra propiedades, métodos públicos y eventos de un componente."""
    print(f"Componente {clase.__name__}: {inspect.getdoc(clase) or ''}")
    for nombre, valor in vars(clase).items():
        if isinstance(valor, (Propiedad, property)):
            extra = ""
            if isinstance(valor, Propiedad):
                extra = (" [compartida]" * valor.compartida
                         + " [restringida]" * valor.restringida)
            print(f"  propiedad {nombre}{extra}")
    for nombre, metodo in inspect.getmembers(clase, inspect.isfunction):
        if not nombre.startswith("_"):
            print(f"  método    {nombre}{inspect.signature(metodo)}")
    obj = clase()
    for nombre, valor in vars(obj).items():
        if isinstance(valor, Evento):
            print(f"  evento    {nombre}")
```

6.6.1 Carga dinámica de componentes

Con la reflexión, la aplicación puede elegir el componente a usar a partir de la configuración, sin importarlo explícitamente en el código:

```python
import importlib

def crear_componente(ruta_clase: str, **propiedades):
    """crear_componente("almacenes.sqlite:AlmacenSQLite", ruta="x.db")"""
    nombre_modulo, nombre_clase = ruta_clase.split(":")
    modulo = importlib.import_module(nombre_modulo)
    clase = getattr(modulo, nombre_clase)
    componente = clase()
    for nombre, valor in propiedades.items():
        if not hasattr(clase, nombre):
            raise AttributeError(f"{nombre_clase} no tiene la propiedad {nombre}")
        setattr(componente, nombre, valor)
    return componente
```

6.7 Editores de propiedades

En JavaBeans, un **editor de propiedades** es una clase que sabe mostrar y modificar el valor de una propiedad de un tipo determinado (convertirlo a texto y desde texto, o mostrar un control gráfico), y que usan las herramientas de diseño para construir la "hoja de propiedades" de un componente.

En Python podemos construir un editor de propiedades genérico combinando introspección y conversores por tipo:

```
from pathlib import Path

# Un "editor" por tipo: convierte texto -> valor
EDITORES = {
    bool: lambda t: t.strip().lower() in ("s", "si", "sí", "true", "1"),
    int: int,
    float: float,
    str: str,
    Path: Path,
}

def editar_propiedades(componente) -> None:
    """Hoja de propiedades de consola para cualquier componente."""
    clase = type(componente)
    for nombre, desc in vars(clase).items():
        if not isinstance(desc, Propiedad):
            continue
        actual = getattr(componente, nombre)
        texto = input(f"{nombre} [{actual}]: ")
        if not texto:
            continue
        editor = EDITORES.get(desc.tipo or type(actual), str)
        try:
            setattr(componente, nombre, editor(texto))
        except (ValueError, TypeError, CambioVetado) as e:
            print(f"  Valor no aceptado: {e}")
```

> **Nota.** La misma idea, con una biblioteca gráfica (Tkinter, PySide), permite generar automáticamente un formulario de configuración para cualquier componente: un campo de texto para `str`, una casilla para `bool`, un selector numérico para `int`...

6.8 Persistencia del componente

La **persistencia del componente** es la capacidad de guardar su estado (sus propiedades configuradas) y restaurarlo más tarde, por ejemplo al reiniciar la aplicación. No hay que confundirla con los datos que el componente gestiona: aquí se guarda su *configuración*.

Con la introspección se pueden guardar las propiedades de cualquier componente en JSON:

```python
import json
from pathlib import Path

def guardar_estado(componente, fichero: Path) -> None:
    clase = type(componente)
    estado = {
        "clase": f"{clase.__module__}:{clase.__name__}",
        "propiedades": {n: getattr(componente, n) for n, d in vars(clase).items()
                        if isinstance(d, Propiedad)},
    }
    fichero.write_text(json.dumps(estado, indent=2, default=str), encoding="utf-8")

def cargar_estado(fichero: Path):
    estado = json.loads(fichero.read_text(encoding="utf-8"))
    return crear_componente(estado["clase"], **estado["propiedades"])
```

Si se usa `pickle`, los métodos especiales `__getstate__` y `__setstate__` permiten decidir qué se guarda; por ejemplo, para excluir una conexión abierta, que no se puede serializar:

```python
class AlmacenSQLite(SoportePropiedades):
    NO_PERSISTENTES = ("_conexion", "_oyentes_cambio", "_oyentes_veto",
                       "guardado", "borrado", "error")

    def __getstate__(self) -> dict:
        # ni la conexión ni los oyentes (funciones) se pueden serializar
        return {k: v for k, v in self.__dict__.items()
                if k not in self.NO_PERSISTENTES}

    def __setstate__(self, estado: dict) -> None:
        self.__init__()                     # recrea conexión (cerrada) y eventos
        self.__dict__.update(estado)        # restaura las propiedades
```

6.9 Componentes de acceso a datos

Reunimos todo lo anterior en un componente de acceso a datos. Primero se define la **interfaz** que deben cumplir todas las implementaciones. En Python se puede expresar con una clase abstracta (`abc`) o con un `Protocol` (tipado estructural):

```python
# componentes/interfaz.py
from abc import ABC, abstractmethod
from dataclasses import dataclass

@dataclass
class Alumno:
    dni: str
    nombre: str
    apellidos: str
    email: str | None = None

class AlmacenAlumnos(ABC):
    """Interfaz común de todos los componentes de acceso a alumnos."""

    @abstractmethod
    def abrir(self) -> None: ...
    @abstractmethod
```

```
    def cerrar(self) -> None: ...
    @abstractmethod
    def guardar(self, alumno: Alumno) -> None: ...
    @abstractmethod
    def buscar(self, dni: str) -> Alumno | None: ...
    @abstractmethod
    def listar(self) -> list[Alumno]: ...
    @abstractmethod
    def borrar(self, dni: str) -> bool: ...

    # Todos los componentes se pueden usar con "with"
    def __enter__(self):
        self.abrir()
        return self

    def __exit__(self, *exc) -> None:
        self.cerrar()
```

Y una implementación concreta con SQLite (Tema 2), con propiedades, eventos y validación:

```
# componentes/sqlite.py
import sqlite3
from componentes.interfaz import AlmacenAlumnos, Alumno
from componentes.propiedades import SoportePropiedades, Propiedad
from componentes.eventos import Evento

class ErrorAlmacen(Exception):
    """Error de cualquier componente de almacenamiento."""

class AlmacenSQLite(SoportePropiedades, AlmacenAlumnos):
    """Componente de acceso a alumnos almacenados en SQLite."""

    ruta = Propiedad("alumnos.db", restringida=True, tipo=str)
    solo_lectura = Propiedad(False, tipo=bool)

    def __init__(self) -> None:
        SoportePropiedades.__init__(self)
        self._conexion: sqlite3.Connection | None = None
        self.guardado = Evento()            # eventos del componente
        self.borrado = Evento()
        self.error = Evento()
        # no se puede cambiar la ruta con la base de datos abierta
        self.al_intentar_cambiar(self._vetar_si_abierta)

    def _vetar_si_abierta(self, cambio) -> None:
        if cambio.nombre == "ruta" and self._conexion is not None:
            from componentes.propiedades import CambioVetado
            raise CambioVetado("Cierra el almacén antes de cambiar la ruta")

    def abrir(self) -> None:
        try:
            self._conexion = sqlite3.connect(self.ruta)
            self._conexion.row_factory = sqlite3.Row
            self._conexion.execute(
                "CREATE TABLE IF NOT EXISTS alumno (dni TEXT PRIMARY KEY, "
                "nombre TEXT NOT NULL, apellidos TEXT NOT NULL, email TEXT)")
        except sqlite3.Error as e:
```

```
            self.error.emitir(e)
            raise ErrorAlmacen(f"No se pudo abrir {self.ruta}: {e}") from e

    def cerrar(self) -> None:
        if self._conexion is not None:
            self._conexion.close()
            self._conexion = None

    def _con(self) -> sqlite3.Connection:
        if self._conexion is None:
            raise ErrorAlmacen("El almacén no está abierto")
        return self._conexion

    def guardar(self, alumno: Alumno) -> None:
        if self.solo_lectura:
            raise ErrorAlmacen("Almacén en modo sólo lectura")
        with self._con() as c:              # transacción
            c.execute("INSERT INTO alumno VALUES (:dni, :nombre, :apellidos, :email) "
                      "ON CONFLICT(dni) DO UPDATE SET nombre=excluded.nombre, "
                      "apellidos=excluded.apellidos, email=excluded.email",
                      vars(alumno))
        self.guardado.emitir(alumno)

    def buscar(self, dni: str) -> Alumno | None:
        f = self._con().execute("SELECT * FROM alumno WHERE dni = ?",
                                (dni,)).fetchone()
        return Alumno(**dict(f)) if f else None

    def listar(self) -> list[Alumno]:
        return [Alumno(**dict(f)) for f in
                self._con().execute("SELECT * FROM alumno ORDER BY apellidos")]

    def borrar(self, dni: str) -> bool:
        if self.solo_lectura:
            raise ErrorAlmacen("Almacén en modo sólo lectura")
        with self._con() as c:
            borrado = c.execute("DELETE FROM alumno WHERE dni = ?",
                                (dni,)).rowcount == 1
        if borrado:
            self.borrado.emitir(dni)
        return borrado
```

La aplicación sólo depende de la interfaz; la implementación concreta se elige en la configuración:

```
# aplicacion.py
from componentes.interfaz import Alumno
from componentes.cargador import crear_componente   # función del apartado anterior

almacen = crear_componente("componentes.sqlite:AlmacenSQLite", ruta="instituto.db")
almacen.guardado.conectar(lambda a: print("Guardado", a.dni))

with almacen:
    almacen.guardar(Alumno("11111111H", "Ana", "García"))
    for a in almacen.listar():
        print(a)
```

Siguiendo el mismo esquema se programan componentes para el resto de tecnologías del módulo: `AlmacenJSON`, `AlmacenCSV` y `AlmacenXML` (Tema 1), `AlmacenPostgreSQL` (Tema 2), `AlmacenSQLAlchemy` (Tema 3), `AlmacenObjetoRelacional` (Tema 4) y `AlmacenMongo` (Tema 5). Una única batería de pruebas, parametrizada, comprueba que todos cumplen la interfaz:

```python
# tests/test_almacenes.py
import pytest
from componentes.interfaz import Alumno
from componentes.sqlite import AlmacenSQLite
from componentes.csv_ import AlmacenCSV    # otro componente con la misma interfaz

@pytest.fixture(params=[AlmacenSQLite, AlmacenCSV])
def almacen(request, tmp_path):
    a = request.param()
    a.ruta = str(tmp_path / "datos")
    with a:
        yield a

def test_guardar_y_buscar(almacen):
    almacen.guardar(Alumno("11111111H", "Ana", "García"))
    assert almacen.buscar("11111111H").nombre == "Ana"

def test_borrar_inexistente(almacen):
    assert almacen.borrar("00000000T") is False

def test_evento_guardado(almacen):
    recibidos = []
    almacen.guardado.conectar(recibidos.append)
    almacen.guardar(Alumno("22222222J", "Luis", "Pérez"))
    assert [a.dni for a in recibidos] == ["22222222J"]
```

6.10 Herramientas para el desarrollo de componentes no visuales

Entornos de desarrollo	PyCharm, Visual Studio Code (extensión de Python): depuración, refactorización, ejecución de pruebas, introspección de los componentes en el autocompletado.
Gestión de proyectos y dependencias	`venv` + `pip`, `uv`, `poetry`, `hatch`.
Construcción de paquetes	`build` con un *backend* (`setuptools`, `hatchling`).
Calidad	`ruff` (estilo y errores), `mypy` (comprobación de tipos estática).
Pruebas	`pytest`, `pytest-cov` (cobertura).
Documentación	*Docstrings* + `pdoc` o `Sphinx` para generar la documentación HTML.
Distribución	PyPI o un repositorio privado; instalación directa desde Git.

6.11 Empaquetado de componentes

Para que un componente se pueda reutilizar en otros proyectos hay que **empaquetarlo**: convertirlo en un paquete instalable con `pip`, con su nombre, versión, dependencias y documentación. Es el equivalente al `.jar` de Java.

6.11.1 Estructura del proyecto

```
componentes-ad/
├── pyproject.toml
├── README.md
├── LICENSE
├── src/
│   └── componentes_ad/
│       ├── __init__.py
│       ├── interfaz.py
│       ├── propiedades.py
│       ├── eventos.py
│       ├── sqlite.py
│       └── py.typed          (indica que el paquete incluye anotaciones de tipo)
└── tests/
    └── test_almacenes.py
```

6.11.2 El fichero `pyproject.toml`

```toml
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "componentes-ad"
version = "0.1.0"
description = "Componentes de acceso a datos para el módulo de AD"
readme = "README.md"
requires-python = ">=3.12"
authors = [{ name = "2º DAM" }]
dependencies = []                       # el componente SQLite no necesita nada

[project.optional-dependencies]
postgres = ["psycopg[binary]>=3.2"]
orm = ["sqlalchemy>=2.0"]
mongo = ["pymongo>=4"]
xml = ["lxml>=5"]
json = ["jsonschema>=4", "pydantic>=2"]
dev = ["pytest", "ruff", "mypy", "pdoc"]

# Puntos de entrada: permiten descubrir los almacenes instalados (plugins)
[project.entry-points."componentes_ad.almacenes"]
sqlite = "componentes_ad.sqlite:AlmacenSQLite"
```

6.11.3 Construir, instalar y distribuir

```
$ pip install build
$ python -m build                    # genera dist/componentes_ad-0.1.0-py3-none-any.
    ↪ whl
$ pip install dist/componentes_ad-0.1.0-py3-none-any.whl
$ pip install -e ".[dev]"            # instalación editable para desarrollar
$ pip install "componentes-ad[postgres,orm]"   # con dependencias opcionales
$ pip install git+https://github.com/usuario/componentes-ad.git
```

Gracias a los puntos de entrada, una aplicación puede descubrir qué componentes de almacenamiento están instalados, incluso los que publiquen terceros en otros paquetes:

```python
from importlib.metadata import entry_points

for ep in entry_points(group="componentes_ad.almacenes"):
    print(ep.name, "->", ep.value)
    clase = ep.load()                    # importa la clase
```

6.11.4 Documentación

```
$ pdoc componentes_ad -o docs/       # genera la documentación HTML desde los
    ↪ docstrings
```

6.12 Ejercicios

Ejercicio 6.1 — Propiedades y descriptores

Dificultad: ★

Escribe un descriptor `Texto(min_longitud, max_longitud, patron=None)` que valide cadenas con una expresión regular opcional, y úsalo para definir un componente `ConfiguracionCorreo` (servidor, puerto con el descriptor `Rango`, usuario, remitente con patrón de email). Comprueba que se rechazan los valores no válidos.

Ejercicio 6.2 — Propiedades compartidas

Dificultad: ★★

Usando `SoportePropiedades` y `Propiedad`, crea un componente `Termostato` con las propiedades `temperatura` (compartida) y `consigna` (restringida: entre 15 y 28 grados, vetada fuera de ese rango por un oyente). Registra dos oyentes que muestren los cambios en consola y en un fichero de registro. Escribe pruebas con `pytest`.

Ejercicio 6.3 — Introspección

Dificultad: ★★

Escribe una función `documentar(clase) -> str` que genere, mediante introspección, un fichero Markdown con la documentación de un componente: descripción, propiedades (con su tipo, valor por defecto y si son compartidas o restringidas), métodos públicos (con su firma y *docstring*) y eventos. Aplícala a `AlmacenSQLite`.

Ejercicio 6.4 — Editor de propiedades gráfico

Dificultad: ★★★

Con Tkinter, crea una ventana que, a partir de cualquier componente que use `Propiedad`, construya automáticamente un formulario con un control adecuado al tipo de cada propiedad (`Entry`, `Checkbutton`, `Spinbox`). Al pulsar "Aplicar" se asignan los valores, mostrando en la ventana los errores y los vetos. Añade botones para guardar y cargar el estado del componente en JSON.

Ejercicio 6.5 — Empaquetado

Dificultad: ★★

Empaqueta el componente `AlmacenSQLite` con su interfaz, propiedades y eventos en un paquete `componentes-ad`. Genera el *wheel*, instálalo en un entorno virtual nuevo y crea un pequeño programa que lo use sin copiar ningún fichero del proyecto original. Genera su documentación con `pdoc`.

Proyecto: Biblioteca de componentes de acceso a datos

Este es el proyecto final del módulo. Consiste en desarrollar, empaquetar e integrar una biblioteca de componentes de acceso a datos reutilizando el trabajo de los temas anteriores.

Requisitos:

1. Elige un dominio (puede ser el de uno de tus proyectos anteriores) y define su modelo con *dataclasses* y una interfaz abstracta `Almacen` con las operaciones de alta, baja, modificación, búsqueda por clave, listado con filtros y operaciones transaccionales.
2. Programa **al menos un componente por cada tecnología del módulo**:
   - ficheros (JSON validado con JSON Schema y, al menos, otro formato);
   - conector de base de datos relacional (SQLite y/o PostgreSQL con su conector);
   - mapeo objeto-relacional (SQLAlchemy);
   - base de datos objeto-relacional (PostgreSQL con tipos compuestos, arrays o JSONB);
   - base de datos documental (MongoDB).
3. Todos los componentes tienen propiedades configurables (con al menos una compartida y una restringida), emiten eventos (`guardado`, `borrado`, `error`) y se pueden usar con `with`.
4. Una función de fábrica crea el componente a partir de un fichero de configuración usando reflexión o puntos de entrada, y el estado de configuración del componente es persistente.
5. Una herramienta de migración que copie todos los datos de un componente a otro (por ejemplo, de CSV a PostgreSQL o de SQLite a MongoDB) usando sólo la interfaz común.
6. Batería de pruebas parametrizada con `pytest` que se ejecuta contra todos los componentes.

7. Empaquetado con `pyproject.toml`, dependencias opcionales por tecnología y documentación generada con `pdoc`.
8. Integración: una aplicación (de consola, gráfica o web) que use los componentes y permita cambiar de almacenamiento sólo modificando la configuración.

Entrega: repositorio Git con el código, el paquete construido, la documentación y una memoria breve que justifique el diseño y compare las tecnologías empleadas.

APÉNDICE A

Preparación del entorno de trabajo

A.1 Python y el editor

1. Instala **Python 3.12 o superior** desde `https://www.python.org` (en Windows, marca la opción *Add python.exe to PATH*). En Linux suele venir instalado; comprueba la versión con `python3 --version`.
2. Instala un editor: **Visual Studio Code** con la extensión *Python* de Microsoft, o **PyCharm Community**.
3. Para cada proyecto, crea un entorno virtual (`python -m venv .venv`) y selecciónalo como intérprete en el editor.

Paquetes que usaremos a lo largo del curso:

```
$ pip install pytest jsonschema pydantic lxml xsdata requests beautifulsoup4
$ pip install "psycopg[binary]" sqlalchemy pymongo
$ pip install ZODB BaseXClient xmltodict          # sólo para los anexos
```

A.2 Servidores de bases de datos con Docker

Para no instalar cada gestor en el equipo, los levantaremos en contenedores con **Docker** (Docker Desktop en Windows y macOS; Docker Engine en Linux). Guarda este fichero como `compose.yaml` en una carpeta del curso:

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: dam
      POSTGRES_PASSWORD: dam
      POSTGRES_DB: instituto
    ports: ["5432:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]

  mysql:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: instituto
      MYSQL_USER: dam
      MYSQL_PASSWORD: dam
    ports: ["3306:3306"]
    volumes: [mysqldata:/var/lib/mysql]

  basex:                              # sólo para el Anexo D
    image: basex/basexhttp:latest
    ports: ["1984:1984", "8080:8080"]
```

```
    volumes: [basexdata:/srv/basex/data]

  mongo:
    image: mongo:8
    command: ["--replSet", "rs0", "--bind_ip_all"]
    ports: ["27017:27017"]
    volumes: [mongodata:/data/db]

volumes:
  pgdata:
  mysqldata:
  basexdata:
  mongodata:
```

```
$ docker compose up -d              # arranca todos los servidores
$ docker compose up -d postgres     # o sólo los que se necesiten
$ docker compose ps                 # comprueba que están en marcha
$ docker compose stop               # los detiene (los datos se conservan)
$ docker compose down -v            # los elimina, !`incluidos los datos!
```

A.2.1 PostgreSQL

Conexión: servidor `localhost`, puerto `5432`, base de datos `instituto`, usuario y contraseña `dam`. Para abrir una consola SQL:

```
$ docker compose exec postgres psql -U dam -d instituto
```

También puedes usar un cliente gráfico como **DBeaver** o **pgAdmin**.

A.2.2 MySQL

Conexión: servidor `localhost`, puerto `3306`, base de datos `instituto`, usuario y contraseña `dam` (el usuario `root` tiene la contraseña `root`). Para abrir una consola SQL:

```
$ docker compose exec mysql mysql -u dam -p instituto
```

Para conectarse mediante ODBC (Tema 2) hay que instalar además en el equipo el controlador **MySQL Connector/ODBC**, que se descarga de `https://dev.mysql.com/downloads/connector/odbc/`. En Linux también hace falta el gestor de controladores unixODBC (paquete `unixodbc` en Debian y Ubuntu).

A.2.3 BaseX (Anexo D)

Antes de conectarte por primera vez hay que asignar una contraseña al usuario `admin`:

```
$ docker compose exec basex basex -c "ALTER PASSWORD admin dam"
```

Conexión desde Python: servidor `localhost`, puerto `1984`, usuario `admin`, contraseña `dam`. En `http://localhost:8080` está disponible la interfaz web (DBA). Para practicar XQuery es muy cómodo instalar también la **BaseX GUI** desde `https://basex.org`.

Nota. Si la imagen de Docker cambia su forma de configurar la contraseña, consulta la documentación de BaseX (*Docker* y *User Management*). La alternativa es descargar BaseX (necesita Java 17+) y ejecutar `basexserver` directamente.

A.2.4 MongoDB

La primera vez hay que inicializar el *replica set* (necesario para las transacciones):

```
$ docker compose exec mongo mongosh --eval \
    "rs.initiate({_id: 'rs0', members: [{_id: 0, host: 'localhost:27017'}]})"
```

Conexión desde Python: `MongoClient("mongodb://localhost:27017/")`. Como cliente gráfico se recomienda **MongoDB Compass**.

Cuidado. Esta configuración no tiene autenticación en MongoDB y usa contraseñas triviales en el resto: es adecuada sólo para el equipo de desarrollo, nunca para un servidor accesible desde Internet.

APÉNDICE B

Procesamiento avanzado de XML

Este anexo amplía la sección de XML del Tema 1 con contenidos que en versiones anteriores del módulo tenían más peso y que hoy se consideran complementarios. Los ejemplos usan el documento `biblioteca.xml` del Tema 1.

B.1 Los modelos DOM y SAX en la práctica

B.1.1 DOM con `minidom`

Es la API estándar del W3C, idéntica en concepto a la de Java o JavaScript (métodos como `getElementsByTagName`, `createElement` o `appendChild`).

```
from xml.dom import minidom

doc = minidom.parse("biblioteca.xml")
raiz = doc.documentElement
print("Raíz:", raiz.tagName, raiz.getAttribute("nombre"))

for libro in doc.getElementsByTagName("libro"):
    isbn = libro.getAttribute("isbn")
    titulo = libro.getElementsByTagName("titulo")[0].firstChild.data
    print(isbn, titulo)

# Añadir un libro nuevo
nuevo = doc.createElement("libro")
nuevo.setAttribute("isbn", "978-84-9759-000-1")
nuevo.setAttribute("prestado", "no")
titulo = doc.createElement("titulo")
titulo.appendChild(doc.createTextNode("Nada"))
nuevo.appendChild(titulo)
raiz.appendChild(nuevo)

with open("biblioteca_dom.xml", "w", encoding="utf-8") as f:
    doc.writexml(f, encoding="utf-8")
```

B.1.2 SAX

Con SAX escribimos una clase manejadora (*handler*) cuyos métodos se llaman a medida que el parser encuentra cada parte del documento.

```
import xml.sax

class ManejadorLibros(xml.sax.ContentHandler):
    def __init__(self) -> None:
        super().__init__()
        self.libros: list[dict] = []
        self.actual: dict | None = None
```

```
        self.texto = ""

    def startElement(self, nombre, atributos):
        if nombre == "libro":
            self.actual = {"isbn": atributos.get("isbn")}
        self.texto = ""

    def characters(self, contenido):
        self.texto += contenido           # puede llegar en varios trozos

    def endElement(self, nombre):
        if self.actual is not None and nombre in ("titulo", "autor", "anio"):
            self.actual[nombre] = self.texto.strip()
        elif nombre == "libro":
            self.libros.append(self.actual)
            self.actual = None

manejador = ManejadorLibros()
xml.sax.parse("biblioteca.xml", manejador)
for libro in manejador.libros:
    print(libro)
```

B.1.3 Procesamiento incremental con `iterparse`

`ElementTree.iterparse` combina las dos ideas: recorre el documento de forma secuencial, como SAX, pero entrega elementos completos de ElementTree. Si se eliminan los elementos ya procesados, la memoria se mantiene constante aunque el fichero ocupe varios gigabytes.

```
import xml.etree.ElementTree as ET

total = 0.0
n = 0
for evento, elem in ET.iterparse("biblioteca_enorme.xml", events=("end",)):
    if elem.tag == "libro":
        total += float(elem.findtext("precio", "0"))
        n += 1
        elem.clear()                      # libera la memoria del elemento
print(f"{n} libros, precio medio {total / n:.2f}")
```

B.2 XPath completo con `lxml`

ElementTree sólo admite un subconjunto de XPath. `lxml` implementa XPath 1.0 completo: ejes, funciones, comparaciones numéricas y expresiones que devuelven números, cadenas o booleanos.

```
from lxml import etree

arbol = etree.parse("biblioteca.xml")
titulos = arbol.xpath("//libro[anio > 1960]/titulo/text()")
total = arbol.xpath("sum(//precio)")                  # número (float)
isbns = arbol.xpath("//libro/@isbn")                  # lista de cadenas
caro = arbol.xpath("//libro[not(precio < //libro/precio)]/titulo/text()")
n = arbol.xpath("count(//libro[@prestado='si'])")
```

```
autores = arbol.xpath("//libro[starts-with(autor, 'Ca')]/autor/text()")
anio = 1950
recientes = arbol.xpath("//libro[anio > $a]/titulo/text()", a=anio)   # variables
```

Ejes	`child::`, `descendant::`, `parent::`, `ancestor::`, `following-sibling::`, `preceding-sibling::`, `attribute::` (`@`)
Funciones de nodos	`count()`, `position()`, `last()`, `name()`, `text()`
Funciones de cadenas	`contains()`, `starts-with()`, `concat()`, `substring()`, `normalize-space()`, `string-length()`
Funciones numéricas	`sum()`, `round()`, `floor()`, `ceiling()`, `number()`
Operadores	`and`, `or`, `not()`, `=`, `!=`, `<`, `>`, `|` (unión)

B.3 Vinculación de datos ampliada

B.3.1 Generación de clases a partir de un XSD

En el Tema 1 escribimos a mano las clases vinculadas. Si se dispone de un esquema, xsdata las genera automáticamente, con todos los tipos y restricciones:

```
$ pip install "xsdata[cli]"
$ xsdata generate biblioteca.xsd --package modelo
```

```
from pathlib import Path
from xsdata.formats.dataclass.parsers import XmlParser
from modelo import Biblioteca                # clase generada

bib = XmlParser().from_path(Path("biblioteca.xml"), Biblioteca)
```

B.3.2 Otras herramientas

- **pydantic-xml**: extensión de Pydantic para XML; útil si ya se usa Pydantic para JSON.
- **xmltodict**: no vincula a clases propias, sino a diccionarios. Muy cómodo para conversiones rápidas.

Con `xmltodict` la conversión genérica a diccionario es inmediata (los atributos aparecen con el prefijo `@`):

```
import json, xmltodict

with open("biblioteca.xml", encoding="utf-8") as f:
    datos = xmltodict.parse(f.read())
print(json.dumps(datos, indent=2, ensure_ascii=False))
xml_de_nuevo = xmltodict.unparse(datos, pretty=True)
```

B.4 Transformaciones XSLT

XSLT es un lenguaje (también escrito en XML) para transformar un documento XML en otro documento: XML con otra estructura, HTML o texto plano. Es la herramienta estándar de conversión de XML. `lxml` incluye un procesador XSLT 1.0.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!-- biblioteca_html.xsl -->
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:output method="html" encoding="UTF-8" indent="yes"/>
  <xsl:template match="/biblioteca">
    <html>
      <head><title><xsl:value-of select="@nombre"/></title></head>
      <body>
        <h1><xsl:value-of select="@nombre"/></h1>
        <table border="1">
          <tr><th>Título</th><th>Autor</th><th>Año</th></tr>
          <xsl:for-each select="libro">
            <xsl:sort select="anio" data-type="number"/>
            <tr>
              <td><xsl:value-of select="titulo"/></td>
              <td><xsl:value-of select="autor"/></td>
              <td><xsl:value-of select="anio"/></td>
            </tr>
          </xsl:for-each>
        </table>
      </body>
    </html>
  </xsl:template>
</xsl:stylesheet>
```

```python
from lxml import etree

xml = etree.parse("biblioteca.xml")
transformar = etree.XSLT(etree.parse("biblioteca_html.xsl"))
resultado = transformar(xml)
resultado.write_output("biblioteca.html")   # respeta xsl:output
```

> **Nota.** Para convertir XML a otros formatos también son útiles: `pandas` (`read_xml`/`to_xml`, `to_excel`), `openpyxl` para hojas de cálculo y, para generar PDF, la combinación XML → HTML (XSLT) → PDF con `weasyprint`.

B.5 Ejercicios

Ejercicio B.1 — DOM vs SAX

Dificultad: ★★

Genera un fichero XML con 200 000 libros aleatorios. Después, escribe dos programas que calculen el precio medio de los libros publicados en cada década: uno con ElementTree (`parse`) y otro con SAX. Mide el tiempo y la memoria máxima de cada uno (`time.perf_counter` y `tracemalloc`) y explica los resultados en un breve informe. Como

ampliación, prueba `ElementTree.iterparse`, que combina ambos enfoques.

Ejercicio B.2 — Validación y XPath

Dificultad: ★★

Escribe el esquema XSD de `biblioteca.xml` (el ISBN con un patrón, el año entre 1450 y el año actual, el atributo `prestado` con valores `si`/`no`). Con `lxml`: valida varios documentos, mostrando los errores con su número de línea, y resuelve con XPath las consultas: títulos de los libros prestados; autor del libro más caro; número de libros por autor; precio medio.

Ejercicio B.3 — XSLT

Dificultad: ★★★

Escribe dos hojas XSLT para `biblioteca.xml`: una que genere una página HTML con los libros agrupados por estado (prestados / disponibles) y otra que genere un fichero CSV (`xsl:output method="text"`). Aplica ambas desde Python con `lxml`, pasando como parámetro de la transformación el año mínimo de los libros a incluir.

APÉNDICE C

Bases de datos orientadas a objetos

Este anexo recoge los contenidos del currículo sobre bases de datos orientadas a objetos que, por su escasa implantación actual, se han separado del Tema 4: el modelo de las BDOO, el estándar ODMG, el lenguaje de consultas OQL y un ejemplo práctico con **ZODB**, una base de datos de objetos nativa de Python que sigue mantenida.

C.1 El modelo orientado a objetos

C.1.1 Características

Una base de datos **orientada a objetos** (BDOO) almacena directamente los objetos del lenguaje de programación, sin traducirlos a tablas. Desaparece el desfase objeto-relacional: el modelo de la base de datos *es* el modelo de clases de la aplicación. Sus características principales son:

- **Identidad de objeto** (OID): cada objeto tiene un identificador único e inmutable, independiente de sus valores.
- Soporte de **objetos complejos**: atributos que son otros objetos o colecciones.
- **Encapsulación**, **herencia** y **polimorfismo**.
- **Navegación**: las relaciones se recorren siguiendo referencias, sin `JOIN`.
- **Persistencia transparente** (u ortogonal): cualquier objeto alcanzable desde un objeto persistente se vuelve persistente (*persistencia por alcance*).
- Transacciones, concurrencia y recuperación, como en cualquier SGBD.

C.1.2 El estándar ODMG

El *Object Data Management Group* publicó el estándar ODMG (última versión 3.0, 2001), que definía:

- Un **modelo de objetos** común.
- **ODL** (*Object Definition Language*): lenguaje para definir clases, atributos y relaciones.
- **OQL** (*Object Query Language*): lenguaje de consulta declarativo, basado en SQL.
- Enlaces (*bindings*) con C++, Smalltalk y Java.

C.1.3 Ventajas e inconvenientes

+ Sin desfase objeto-relacional ni código de mapeo.
+ Muy eficientes al navegar por grafos de objetos complejos (CAD, simulaciones, sistemas de telecomunicaciones).
+ Se almacenan comportamiento y tipos complejos de forma natural.

- Falta de un estándar efectivo: cada gestor tiene su API. OQL nunca llegó a implantarse por completo.
- Fuerte acoplamiento con un lenguaje de programación concreto: es difícil compartir los datos con otras aplicaciones.
- Consultas *ad hoc* e informes más difíciles que con SQL; menos herramientas.
- Cuota de mercado muy pequeña y menos profesionales que las conozcan.

C.1.4 Gestores de bases de datos orientadas a objetos

ZODB	Base de datos de objetos para Python. Usada por el servidor de aplicaciones Zope y el gestor de contenidos Plone. Libre y mantenida.
ObjectDB	BDOO para Java que usa la API estándar JPA (la misma que Hibernate).
GemStone/S	BDOO comercial para Smalltalk, en uso en sistemas financieros.
Versant / Actian NoSQL, ObjectStore	BDOO comerciales clásicas para C++ y Java.
db4o	Muy popular en docencia; descontinuada desde 2014.

C.2 Tipos de datos: básicos y estructurados

En el modelo ODMG se distinguen:

Tipos básicos (literales atómicos). `long`, `short`, `float`, `double`, `boolean`, `char`, `string`, `enum`, `date`, `time`, `timestamp`.

Tipos estructurados. `struct` (registro con campos con nombre, sin identidad propia).

Colecciones. `set` (sin orden, sin repetidos), `bag` (sin orden, con repetidos), `list` (con orden), `array` y `dictionary`.

Objetos. Instancias de clases definidas por el usuario, con identidad.

Un ejemplo de definición en ODL:

```
class Departamento (extent departamentos key nombre) {
    attribute string nombre;
    relationship set<Profesor> profesores inverse Profesor::departamento;
};
class Profesor (extent profesores key dni) {
    attribute string dni;
    attribute string nombre;
    attribute struct Direccion { string calle; string ciudad; } direccion;
    attribute set<string> telefonos;
    relationship Departamento departamento inverse Departamento::profesores;
    float sueldo_neto();
};
```

El `extent` es la colección de todos los objetos de la clase (el equivalente a una tabla, y el punto de partida de las consultas).

C.3 El lenguaje de consultas OQL

OQL toma la sintaxis de `SELECT` de SQL y la extiende para trabajar con objetos. Sus elementos principales son:

C.3.1 Sintaxis básica y expresiones de camino

```
select p.nombre
from p in profesores
where p.departamento.nombre = "Informática"
```

- En `from` se declaran variables que recorren colecciones (`p in profesores`, también `profesores p` o `profesores as p`).
- Las **expresiones de camino** (`p.departamento.nombre`) navegan por las relaciones sin necesidad de `JOIN`.
- Se pueden invocar métodos: `p.sueldo_neto()`.

C.3.2 Resultados estructurados

```
select struct(nombre: p.nombre, ciudad: p.direccion.ciudad)
from p in profesores
where p.sueldo_neto() > 2000
order by p.nombre
```

El resultado puede ser un conjunto de literales, de objetos o de estructuras; `select distinct` devuelve un `set` y `select` un `bag`.

C.3.3 Recorrido de colecciones anidadas

```
select d.nombre, p.nombre
from d in departamentos, p in d.profesores
where "983000000" in p.telefonos
```

C.3.4 Operadores

Aritméticos y de comparación	`+ - * / mod`, `= != < <= > >=`, `like`
Lógicos	`and`, `or`, `not`
Pertenencia	`x in coleccion`
Cuantificadores	`for all x in c: condición`, `exists x in c: condición`
Agregados	`count`, `sum`, `avg`, `min`, `max`
Conjuntos	`union`, `intersect`, `except`
Conversión	`element(c)` (único elemento), `flatten(c)`, `listtoset(l)`
Agrupación	`group by … partition`, `having`

```
// Departamentos en los que todos los profesores cobran más de 1800
select d.nombre from d in departamentos
where for all p in d.profesores: p.sueldo_neto() > 1800

// Número de profesores por departamento
select d.nombre, count(d.profesores) from d in departamentos

// Una consulta OQL puede ser una simple expresión
count(profesores)
element(select p from p in profesores where p.dni = "11111111H")
```

Herederos de OQL. Aunque OQL apenas se implementó tal cual, su idea (consultar objetos y navegar por sus relaciones con una sintaxis parecida a SQL) sobrevive en **HQL/JPQL** de Hibernate y JPA, y en los lenguajes de consulta de los ORM como el `select()` de SQLAlchemy que vimos en el Tema 3.

C.4 Una base de datos de objetos en Python: ZODB

C.4.1 Instalación y conceptos

```
$ pip install ZODB
```

- Los objetos se guardan en un **almacenamiento** (*storage*); el más sencillo es `FileStorage`, un fichero local (`.fs`).
- La base de datos tiene un **objeto raíz**, un diccionario a partir del cual se alcanzan todos los demás (*persistencia por alcance*).
- Para que ZODB detecte los cambios de un objeto, su clase debe heredar de `persistent.Persistent`.
- Todo cambio ocurre dentro de una **transacción**, que se confirma con `transaction.commit()` y se deshace con `transaction.abort()`.

C.4.2 API de la base de datos: conexión y objetos simples

```
import ZODB, ZODB.FileStorage
import transaction
from persistent import Persistent

class Alumno(Persistent):
    def __init__(self, dni: str, nombre: str, edad: int) -> None:
        self.dni = dni
        self.nombre = nombre
        self.edad = edad

    def __repr__(self) -> str:
        return f"Alumno({self.dni}, {self.nombre}, {self.edad})"

# Apertura: almacenamiento -> base de datos -> conexión -> raíz
storage = ZODB.FileStorage.FileStorage("instituto.fs")
```

```
db = ZODB.DB(storage)
conexion = db.open()
raiz = conexion.root()

# Guardar: basta con hacerlo alcanzable desde la raíz y confirmar
raiz["ana"] = Alumno("11111111H", "Ana", 20)
transaction.commit()

# Modificar: se cambia el objeto y se confirma
raiz["ana"].edad += 1
transaction.commit()

# Cierre
conexion.close()
db.close()
```

Nota. `ZODB.DB` admite también un nombre de fichero directamente (`ZODB.DB("instituto.fs")`) y `None` para una base de datos en memoria. Y `db.transaction()` abre una conexión con una transacción en un bloque `with`:

```
with db.transaction() as conn:
    conn.root()["luis"] = Alumno("22222222J", "Luis", 19)
# commit automático al salir (abort si hay excepción)
```

C.4.3 Objetos estructurados y colecciones

Los objetos pueden contener otros objetos y colecciones. Hay que tener en cuenta un detalle: ZODB detecta cuándo se *asigna* un atributo de un objeto `Persistent`, pero **no** cuándo se modifica por dentro una lista o un diccionario normales. Por eso se usan las colecciones persistentes:

`PersistentList`	Lista que avisa de sus cambios.
`PersistentMapping`	Diccionario que avisa de sus cambios. Adecuado para pocos elementos.
`BTrees.OOBTree`	Diccionario ordenado que se guarda en nodos separados: escala a millones de elementos y sólo carga en memoria lo que se usa. Variantes según el tipo de clave y valor (`IOBTree`: clave entera, `OOBTree`: claves y valores objeto).

```
from dataclasses import dataclass
import ZODB, transaction
from persistent import Persistent
from persistent.list import PersistentList
from BTrees.OOBTree import OOBTree

@dataclass(frozen=True)
class Direccion:                    # valor inmutable: se guarda dentro del objeto
    calle: str
    ciudad: str

class Modulo(Persistent):
```

```
    def __init__(self, codigo: str, nombre: str, horas: int) -> None:
        self.codigo, self.nombre, self.horas = codigo, nombre, horas

class Alumno(Persistent):
    def __init__(self, dni: str, nombre: str, direccion: Direccion) -> None:
        self.dni = dni
        self.nombre = nombre
        self.direccion = direccion
        self.modulos = PersistentList()     # referencias a otros objetos
        self.telefonos: list[str] = []      # lista normal (ver aviso)

    def matricular(self, m: Modulo) -> None:
        self.modulos.append(m)              # se detecta: PersistentList

    def anadir_telefono(self, t: str) -> None:
        self.telefonos = self.telefonos + [t]   # reasignar para que se detecte
        # alternativa: self.telefonos.append(t); self._p_changed = True

db = ZODB.DB("instituto.fs")
with db.transaction() as conn:
    raiz = conn.root()
    raiz.setdefault("alumnos", OOBTree())       # "extent" de alumnos por DNI
    raiz.setdefault("modulos", OOBTree())
    ad = Modulo("0486", "Acceso a Datos", 126)
    raiz["modulos"][ad.codigo] = ad
    ana = Alumno("11111111H", "Ana", Direccion("Santiago 12", "Valladolid"))
    ana.matricular(ad)                          # referencia, no copia
    ana.anadir_telefono("600111222")
    raiz["alumnos"][ana.dni] = ana
```

Cambios no detectados. Si haces `alumno.telefonos.append(...)` sobre una `list` normal y confirmas, el cambio **no se guarda**: el objeto no sabe que ha cambiado. Usa colecciones persistentes, reasigna el atributo o marca el objeto con `self._p_changed = True`.

C.4.4 Consultas

ZODB no tiene un lenguaje de consultas: las consultas se hacen **navegando** y con expresiones de Python, que cumplen el papel de OQL. Los BTrees permiten búsquedas por rango de claves de forma eficiente.

```
with db.transaction() as conn:
    raiz = conn.root()
    alumnos = raiz["alumnos"]

    # Acceso por clave (como un índice)
    ana = alumnos["11111111H"]

    # select a.nombre from a in alumnos where a.direccion.ciudad = "Valladolid"
    nombres = [a.nombre for a in alumnos.values()
               if a.direccion.ciudad == "Valladolid"]

    # Navegación: módulos de cada alumno
    for a in alumnos.values():
        print(a.nombre, [m.nombre for m in a.modulos])
```

```
    # Rango de claves: DNIs entre dos valores (eficiente en un BTree)
    for dni, a in alumnos.items(min="10000000A", max="30000000Z"):
        print(dni, a.nombre)

    # Cuantificador universal: alumnos con todos sus módulos de más de 100 h
    largos = [a for a in alumnos.values()
              if all(m.horas > 100 for m in a.modulos)]
```

Para consultas frecuentes sobre otro atributo, se mantiene un **índice** propio: otro BTree que asocie, por ejemplo, ciudad → conjunto de alumnos (`BTrees.OOBTree.OOTreeSet`), y que se actualiza al dar de alta o modificar un alumno.

C.4.5 Borrado

Para borrar un objeto basta con eliminar todas las referencias que llegan a él; el espacio se recupera al **empaquetar** la base de datos (`db.pack()`), que además elimina las versiones antiguas de los objetos.

```
with db.transaction() as conn:
    del conn.root()["alumnos"]["11111111H"]
db.pack()
```

C.4.6 Transacciones

```
import transaction
from ZODB.POSException import ConflictError

conexion = db.open()
raiz = conexion.root()
try:
    raiz["modulos"]["0486"].horas = 130
    if raiz["modulos"]["0486"].horas > 200:
        raise ValueError("Horas no válidas")
    transaction.commit()
except ValueError:
    transaction.abort()                # descarta los cambios en memoria
except ConflictError:
    transaction.abort()                # otro proceso modificó el mismo objeto
    print("Conflicto: vuelve a intentarlo")

sp = transaction.savepoint()           # punto intermedio
raiz["temporal"] = 1
sp.rollback()                          # deshace sólo desde el savepoint
conexion.close()
```

ZODB usa **control de concurrencia optimista**: cada conexión trabaja con una instantánea consistente de la base de datos y, si al confirmar otra transacción ha modificado los mismos objetos, se lanza `ConflictError`. Para acceso desde varios procesos o máquinas se usa el servidor ZEO o el almacenamiento RelStorage (que guarda los objetos en PostgreSQL).

C.4.7 Pruebas

Para probar el código que usa ZODB, se crea una base de datos en memoria en cada prueba:

```
import pytest, ZODB

@pytest.fixture
def raiz():
    db = ZODB.DB(None)                  # en memoria
    conn = db.open()
    yield conn.root()
    conn.close()
    db.close()
```

C.5 Ejercicios

Ejercicio C.1 — OQL

Dificultad: ★

Dado el esquema ODL de `Departamento` y `Profesor` de este anexo, ampliado con una clase `Modulo` (nombre, horas, relación con el profesor que lo imparte), escribe en OQL: los módulos de más de 100 horas con el nombre de su profesor; los departamentos con más de cinco profesores; los profesores que imparten algún módulo de más de 150 horas (`exists`); el total de horas impartidas por cada departamento. Traduce después cada consulta a SQL relacional y compara.

Ejercicio C.2 — Primeros pasos con ZODB

Dificultad: ★★

Guarda en ZODB una colección de libros (`OOBTree` por ISBN) con sus autores (objetos compartidos: varios libros referencian al mismo autor). Comprueba que, al modificar el nombre de un autor, el cambio se ve desde todos sus libros, y que si añades un elemento a una `list` normal sin marcar `_p_changed`, el cambio se pierde tras reabrir la base de datos.

Ejercicio C.3 — Concurrencia en ZODB

Dificultad: ★★★

Abre dos conexiones a la misma base de datos ZODB (con `transaction.TransactionManager()` distintos), modifica el mismo objeto desde las dos y confirma ambas. Captura el `ConflictError` e implementa una función `con_reintentos(funcion, intentos=3)` que vuelva a ejecutar la transacción. Compara este control optimista con el bloqueo pesimista (`SELECT ... FOR UPDATE`) del Tema 2.

APÉNDICE D

Bases de datos XML nativas

Este anexo recoge los contenidos del currículo sobre bases de datos XML nativas, que se han separado del Tema 5 para dedicar ese tema a las bases de datos documentales JSON (MongoDB). Se trabaja con el gestor libre **BaseX** y los lenguajes XPath, XQuery y XQuery Update. Muchos conceptos (colecciones, documentos, índices, identificadores únicos) son comunes a ambos tipos de bases de datos documentales.

D.1 Bases de datos XML nativas

D.1.1 Almacenar XML: estrategias

Cuando una aplicación maneja muchos documentos XML, hay varias formas de guardarlos:

En ficheros. Sencillo, pero sin consultas eficientes, sin transacciones ni control de concurrencia (Tema 1).

En una base de datos relacional con soporte XML (*XML-enabled*). Hay dos variantes:

- **Descomposición** (*shredding*): el documento se trocea y sus datos se reparten en tablas. Se pierde la estructura del documento (orden, comentarios, mezcla de texto y elementos) y reconstruirlo es costoso.
- **Columna de tipo XML**: el documento completo se guarda en una columna (tipo `XML` de PostgreSQL, `XMLType` de Oracle, `xml` de SQL Server) y el gestor ofrece funciones XPath para consultarlo.

En una base de datos XML nativa (NXD, *Native XML Database*). La unidad lógica de almacenamiento es el **documento XML**; el gestor lo guarda conservando su modelo de árbol (normalmente en estructuras internas optimizadas, no como texto) y se consulta con lenguajes del W3C: **XPath** y **XQuery**.

D.1.2 Características de una base de datos XML nativa

- Define un modelo lógico para el documento XML (elementos, atributos, texto, orden) y almacena y recupera documentos según ese modelo.
- Organiza los documentos en **colecciones**, que hacen el papel de las tablas o de las carpetas de un sistema de ficheros. Los documentos de una colección no tienen por qué seguir el mismo esquema.
- Ofrece **índices** sobre la estructura y el contenido (textos, atributos, texto completo).
- Proporciona lenguajes de consulta y actualización estándar (XPath, XQuery, XQuery Update, XQuery Full Text).
- Ofrece API de acceso para los lenguajes de programación: REST, protocolos propios cliente-servidor y, históricamente, la API **XML:DB**.

D.1.3 Ventajas e inconvenientes

+ Los documentos se guardan y recuperan tal cual, sin pérdida de estructura ni código de conversión.
+ Ideales para información semiestructurada, con estructura variable o con mucho texto (contenido mixto).
+ Lenguajes de consulta potentes y estandarizados.
+ Validación con esquemas (XSD, DTD) y transformación (XSLT) integradas.
– Peor rendimiento que una base de datos relacional para datos muy estructurados y operaciones de agregación.
– XML es un formato verboso; ocupa más espacio.
– Menor madurez en transacciones, concurrencia y herramientas que los gestores relacionales.
– Comunidad y mercado reducidos.

D.1.4 Gestores comerciales y libres

Gestor	Licencia	Comentario
BaseX	Libre (BSD)	Ligero, rápido, muy buen soporte de XQuery 3.1 y XQuery Update. Servidor y cliente gráfico. Lo usaremos en el curso.
eXist-db	Libre (LGPL)	Muy popular; incluye un servidor web para construir aplicaciones completas con XQuery. API REST y XML:DB.
MarkLogic	Comercial	Base de datos multimodelo (XML, JSON, RDF) de uso empresarial.
Oracle XML DB	Comercial	Soporte XML integrado en Oracle Database (`XMLType`).
IBM Db2 pureXML	Comercial	Almacenamiento XML nativo dentro de Db2.
Sedna, Apache Xindice	Libres	Históricos; ya no se mantienen.

D.2 Instalación y configuración de BaseX

BaseX está escrito en Java. Se puede instalar descargando el paquete de `basex.org` (requiere Java 17 o superior) o con Docker (Apéndice A). Incluye varias herramientas:

BaseX GUI Interfaz gráfica para crear bases de datos, ejecutar consultas y visualizar los documentos. Muy útil para aprender XQuery.

basexserver El servidor. Por defecto escucha en el puerto **1984**.

basexclient Cliente de terminal.

basexhttp Servidor HTTP con API REST y aplicaciones web (RESTXQ), en el puerto 8080.

```
$ basexserver -S                    # arranca el servidor en segundo plano
$ basex -c "PASSWORD dam"           # (en local) establece la contraseña de admin
$ basexclient -U admin -P dam       # se conecta como cliente
> CREATE DB biblioteca
> LIST
> exit
$ basexserver stop
```

> **Nota.** Desde BaseX 10, el usuario `admin` no tiene contraseña por defecto y no se puede conectar al servidor hasta que se le asigna una. Las opciones de configuración (puerto, carpeta de datos, tamaño de caché...) se guardan en el fichero `.basex` de la carpeta de instalación.

Para conectarnos desde Python instalamos el cliente oficial:

```
$ pip install BaseXClient
```

D.3 Establecimiento y cierre de conexiones

El cliente se comunica con el servidor por TCP. La clase `Session` representa una conexión autenticada:

```python
from BaseXClient import BaseXClient

session = BaseXClient.Session("localhost", 1984, "admin", "dam")
try:
    print(session.execute("INFO"))      # información del servidor
finally:
    session.close()                     # cerrar siempre la sesión
```

El cliente no implementa `with`, así que conviene envolverlo en un gestor de contexto propio. Lo reutilizaremos en el resto de ejemplos:

```python
# basex_util.py
import os
from contextlib import contextmanager
from BaseXClient import BaseXClient

@contextmanager
def conectar(host: str = "localhost", port: int = 1984):
    session = BaseXClient.Session(
        host, port,
        os.getenv("BASEX_USER", "admin"),
        os.getenv("BASEX_PASSWORD", "dam"))
    try:
        yield session
    finally:
        session.close()
```

D.4 Colecciones y documentos

En BaseX, una **base de datos** es una colección de documentos (y opcionalmente de ficheros binarios). Dentro de ella, los documentos se identifican por una **ruta** (`libros/2026/nada.xml`), de modo que las carpetas actúan como subcolecciones.

D.4.1 Creación y borrado de colecciones

Las operaciones de administración se hacen con **comandos** mediante `session.execute()` o, desde XQuery, con las funciones del módulo `db:`.

`CREATE DB nombre [entrada]`	Crea una base de datos (vacía o a partir de un fichero, carpeta o URL) y la abre.
`OPEN nombre` / `CLOSE`	Abre / cierra la base de datos sobre la que actúan los comandos.
`LIST [nombre]`	Lista las bases de datos o los documentos de una.
`DROP DB nombre`	Borra una base de datos.
`INFO DB`	Información de la base de datos abierta.
`OPTIMIZE`	Reorganiza estructuras e índices.

```python
from basex_util import conectar

with conectar() as s:
    s.execute("CREATE DB biblioteca")              # vacía
    # s.execute("CREATE DB biblioteca /ruta/a/carpeta/xml")   # con documentos
    print(s.execute("LIST"))
    s.execute("DROP DB biblioteca")
```

Las mismas operaciones en XQuery: `db:create("biblioteca")`, `db:drop("biblioteca")`, `db:list()`, `db:exists("biblioteca")`.

D.4.2 Añadir, modificar y eliminar documentos

`session.add(ruta, xml)`	Añade un documento a la base de datos abierta (comando `ADD TO`).
`session.replace(ruta, xml)`	Sustituye el documento de esa ruta, o lo añade si no existe (comando `PUT`, llamado `REPLACE` en versiones anteriores a la 10).
`DELETE ruta`	Elimina el documento o todos los de una carpeta.
`RENAME ruta nueva`	Cambia la ruta de un documento.
`session.store(ruta, bytes)`	Guarda un fichero binario (imagen, PDF…).

```python
from pathlib import Path
from basex_util import conectar

with conectar() as s:
    s.execute("CREATE DB biblioteca")
    # Añadir desde una cadena
```

```
    s.add("libros/nada.xml",
          '<libro isbn="978-84-233-0000-1"><titulo>Nada</titulo>'
          '<autor>Carmen Laforet</autor><anio>1945</anio></libro>')
    # Añadir todos los XML de una carpeta
    for f in Path("datos").glob("*.xml"):
        s.add(f"libros/{f.name}", f.read_text(encoding="utf-8"))
    # Modificar = sustituir el documento completo
    s.replace("libros/nada.xml",
              '<libro isbn="978-84-233-0000-1"><titulo>Nada</titulo>'
              '<autor>Carmen Laforet</autor><anio>1945</anio>'
              '<premio>Nadal</premio></libro>')
    # Eliminar
    s.execute("DELETE libros/obsoleto.xml")
    print(s.execute("LIST biblioteca"))
```

Para modificar *partes* de un documento sin reemplazarlo entero se usa XQuery Update (apartado D.5.3).

D.5 Lenguajes de consulta: XPath y XQuery

D.5.1 XPath

XPath (que vimos en el Tema 1 y, con más detalle, en el Anexo B) selecciona nodos de un documento mediante expresiones de camino. En BaseX, `collection("biblioteca")` devuelve todos los documentos de la base de datos y `doc("biblioteca/libros/nada.xml")` uno concreto.

```
collection("biblioteca")//libro/titulo
collection("biblioteca")//libro[anio < 1950]/titulo/text()
collection("biblioteca")//libro[@prestado = "si"]
count(collection("biblioteca")//libro)
collection("biblioteca")//libro[contains(autor, "Cela")]
```

D.5.2 XQuery

XQuery es a XML lo que SQL es a las bases de datos relacionales. Es un lenguaje funcional que incluye XPath y añade las expresiones **FLWOR**:

`for` recorre una secuencia.

`let` asigna un valor a una variable.

`where` filtra.

`order by` ordena.

`return` construye el resultado (puede crear XML nuevo).

```
(: Libros anteriores a 1960 ordenados por año, como una lista HTML :)
<ul>{
  for $l in collection("biblioteca")//libro
  let $anio := xs:integer($l/anio)
  where $anio < 1960
  order by $anio descending
  return <li>{ $l/titulo/text() } ({ $anio })</li>
```

```
}</ul>
```

```
(: Número de libros y precio medio por autor :)
for $l in collection("biblioteca")//libro
group by $autor := $l/autor
order by count($l) descending
return <autor nombre="{ $autor }" libros="{ count($l) }"
              media="{ round(avg($l/precio), 2) }"/>
```

```
(: Combinación (join) de dos colecciones :)
for $p in collection("prestamos")//prestamo
for $l in collection("biblioteca")//libro[@isbn = $p/@isbn]
return <prestamo socio="{ $p/socio }">{ $l/titulo }</prestamo>
```

```
(: Funciones definidas por el usuario :)
declare function local:precio-iva($precio as xs:decimal) as xs:decimal {
  round($precio * 1.04, 2)
};
for $l in collection("biblioteca")//libro
return <precio isbn="{ $l/@isbn }">{ local:precio-iva($l/precio) }</precio>
```

XQuery 3.1 también trabaja con mapas, arrays y JSON (`json-doc`, `parse-json`, serialización `method=json`).

D.5.3 Actualización: XQuery Update

```
(: Insertar un nodo :)
insert node <premio>Nadal</premio>
  into collection("biblioteca")//libro[titulo = "Nada"]

insert node <libro isbn="1"><titulo>Nuevo</titulo></libro>
  as last into doc("biblioteca/catalogo.xml")/biblioteca

(: Modificar el valor de un nodo o de un atributo :)
replace value of node collection("biblioteca")//libro[@isbn="1"]/titulo
  with "Título corregido"

for $p in collection("biblioteca")//libro/precio
return replace value of node $p with round($p * 1.1, 2)

(: Sustituir un nodo completo :)
replace node collection("biblioteca")//libro[@isbn="1"]/titulo
  with <titulo idioma="es">Otro título</titulo>

(: Renombrar y borrar :)
rename node collection("biblioteca")//libro[1]/anio as "publicacion"
delete node collection("biblioteca")//libro[@isbn="1"]
delete node collection("biblioteca")//libro/@prestado
```

Las expresiones de actualización no se aplican una a una: se acumulan en una **lista de actualizaciones pendientes** que se aplica **de forma atómica** al terminar la consulta.

D.6 Realización de consultas desde Python

Las consultas se crean con `session.query(texto)`, que devuelve un objeto `Query`:

`q.bind("$var", valor, tipo)`	Asigna un valor a una variable externa de la consulta (equivalente a los parámetros de SQL).
`q.execute()`	Ejecuta la consulta y devuelve el resultado completo como cadena.
`q.iter()`	Ejecuta la consulta y devuelve un iterador de pares `(tipo, valor)`, un elemento del resultado cada vez.
`q.info()`	Información de la ejecución (tiempos, plan, índices usados).
`q.close()`	Libera la consulta en el servidor.

```python
import xml.etree.ElementTree as ET
from basex_util import conectar

CONSULTA = """
declare variable $anio external;
for $l in collection("biblioteca")//libro
where xs:integer($l/anio) < $anio
order by $l/titulo
return $l
"""

with conectar() as s:
    q = s.query(CONSULTA)
    try:
        q.bind("$anio", "1960", "xs:integer")    # !`nunca concatenar valores!
        for tipo, item in q.iter():
            libro = ET.fromstring(item)           # cada item es un <libro>
            print(libro.get("isbn"), libro.findtext("titulo"))
    finally:
        q.close()

    # Consulta que devuelve un único valor
    q = s.query('count(collection("biblioteca")//libro)')
    total = int(q.execute())
    q.close()
```

Inyección XQuery. Igual que con SQL, no construyas consultas concatenando datos introducidos por el usuario: usa variables externas (`declare variable $x external;`) y `bind`.

Para transformar los resultados en objetos se puede usar ElementTree, como arriba, o la vinculación con `xsdata` del Tema 1 (`XmlParser().from_string(item, Libro)`).

D.6.1 Actualizaciones desde Python

```python
def prestar(s, isbn: str) -> None:
    q = s.query("""
```

```
        declare variable $isbn external;
        let $l := collection("biblioteca")//libro[@isbn = $isbn]
        return if (empty($l)) then error(xs:QName("NOEXISTE"), "No existe " || $isbn)
               else replace value of node $l/@prestado with "si"
    """)
    try:
        q.bind("$isbn", isbn)
        q.execute()
    finally:
        q.close()

with conectar() as s:
    try:
        prestar(s, "978-84-206-3311-6")
    except OSError as e:                    # el cliente lanza IOError/OSError
        print("Error:", e)
```

D.7 Indexación

Sin índices, cada consulta recorre todos los nodos de la base de datos. BaseX mantiene varios índices que el optimizador usa automáticamente cuando una consulta puede aprovecharlos:

Estructurales (siempre)	Nombres de elementos y atributos y *resumen de caminos*: aceleran expresiones como `//libro/titulo`.
`TEXT`	Valores de los nodos de texto: `//libro[autor = "Cela"]`. Activo por defecto.
`ATTRIBUTE`	Valores de los atributos: `//libro[@isbn = "…"]`. Activo por defecto.
`TOKEN`	Palabras dentro de atributos (como `class="a b"`).
`FULLTEXT`	Búsqueda de texto completo con `contains text`. Hay que crearlo.

```
with conectar() as s:
    s.execute("OPEN biblioteca")
    s.execute("CREATE INDEX FULLTEXT")
    s.execute("SET UPDINDEX true")   # mantener índices al actualizar (por defecto
                                     # se invalidan y se reconstruyen con OPTIMIZE)
    q = s.query('collection("biblioteca")//libro[titulo contains text "soledad"]')
    print(q.execute())
    print(q.info())                  # muestra si se ha usado el índice
    q.close()
```

D.8 Identificadores únicos

Una base de datos XML necesita identificar documentos y nodos de forma única:

- Cada **documento** se identifica por su **ruta** dentro de la base de datos (`biblioteca/libros/nada.xml`).
- Cada **nodo** tiene en BaseX un identificador interno estable, el `id`, que no cambia aunque se modifique el documento. La función `db:node-id` obtiene el identificador de un nodo y

`db:get-id` recupera el nodo a partir de su identificador (en BaseX 10 o superior). También existe el valor `pre`, la posición en el documento, que sí cambia con las actualizaciones.

- A nivel de datos, lo habitual es que cada entidad tenga un atributo con un identificador de negocio (el ISBN) o generado (`random:uuid()`), declarado en el esquema con el tipo `xs:ID` para poder usar la función `id()`.

```
let $l := collection("biblioteca")//libro[titulo = "Nada"]
return db:node-id($l)            (: p. ej. 42 :)

db:get-id("biblioteca", 42)/titulo

insert node <libro id="{ random:uuid() }"><titulo>Nuevo</titulo></libro>
  into doc("biblioteca/catalogo.xml")/biblioteca
```

D.9 Gestión de transacciones

En BaseX **cada comando y cada consulta es una transacción**: se ejecuta de forma atómica y aislada. Las actualizaciones de XQuery Update se acumulan y se aplican todas a la vez al final; si se produce un error, no se aplica ninguna. Para que varias operaciones sean atómicas, se agrupan en una sola consulta XQuery:

```
(: Préstamo: se registran ambos cambios o ninguno :)
declare variable $isbn external;
declare variable $socio external;
let $l := collection("biblioteca")//libro[@isbn = $isbn]
return (
  if ($l/@prestado = "si") then error(xs:QName("PRESTADO"), "Ya prestado") else (),
  replace value of node $l/@prestado with "si",
  insert node <prestamo isbn="{ $isbn }" socio="{ $socio }"
                        fecha="{ current-date() }"/>
    into doc("prestamos/prestamos.xml")/prestamos
)
```

La concurrencia se controla con bloqueos de lectura y escritura sobre las bases de datos implicadas en cada transacción. No hay transacciones de varias sentencias controladas desde el cliente (`commit`/`rollback`) como en SQL.

D.10 Tratamiento de excepciones

- El cliente de Python lanza `IOError` (alias de `OSError`) tanto si falla la conexión como si el servidor devuelve un error en un comando o consulta. El mensaje incluye el código de error de XQuery.
- Los errores de XQuery tienen códigos estándar: `XPST0003` (error de sintaxis), `XPTY0004` (tipos incompatibles), `FODC0002` (documento no encontrado)…, además de los propios de BaseX para sus módulos.
- En XQuery se pueden lanzar errores con `error()` y capturarlos con `try {…} catch * {…}`.

```
try:
    with conectar() as s:
```

```
        q = s.query("collection('noexiste')//libro")
        print(q.execute())
except ConnectionRefusedError:
    print("El servidor BaseX no está arrancado")
except OSError as e:
    mensaje = str(e)
    if "Access denied" in mensaje or "Access Denied" in mensaje:
        print("Usuario o contraseña incorrectos")
    else:
        print("Error de BaseX:", mensaje)
```

D.11 Ejercicios

Ejercicio D.1 — Primeros pasos con BaseX

Dificultad: ★

Instala BaseX y, desde la interfaz gráfica, crea una base de datos a partir de la carpeta con varios documentos XML de libros. Escribe y prueba consultas XPath y XQuery para: títulos de todos los libros; libros de un autor; libros ordenados por precio; precio medio por autor; un listado HTML de los libros prestados. Guarda las consultas en ficheros `.xq`.

Ejercicio D.2 — Administración desde Python

Dificultad: ★★

Escribe un programa de línea de comandos `basexadmin.py` con subórdenes: `crear <bd> [carpeta]`, `borrar <bd>`, `listar [bd]`, `anadir <bd> <fichero> [ruta]`, `sustituir <bd> <fichero> <ruta>`, `eliminar <bd> <ruta>` y `exportar <bd> <carpeta>` (guarda cada documento en un fichero). Gestiona todas las excepciones con mensajes claros.

Ejercicio D.3 — XQuery con parámetros

Dificultad: ★★

Usando la base de datos de `https://www.w3schools.com/xml/books.xml` (o una similar), escribe funciones Python que ejecuten consultas parametrizadas con `bind` y devuelvan objetos (*dataclasses*): buscar por categoría; libros entre dos precios; autores con más de un libro. Comprueba con `q.info()` qué consultas usan índices.

Ejercicio D.4 — XQuery Update

Dificultad: ★★

Implementa en Python, mediante XQuery Update, las operaciones: añadir un libro a un catálogo, cambiar su precio, añadir un elemento `<valoracion>` a todos los libros de un autor, renombrar el elemento `year` a `anio` en todos los libros y eliminar los libros sin precio. Demuestra que una consulta con varias actualizaciones que termina con `error()` no aplica ninguna.

APÉNDICE E

Pruebas y documentación del código

Casi todos los resultados de aprendizaje del módulo terminan con el mismo criterio: *"se han probado y documentado las aplicaciones desarrolladas"*. Este anexo explica cómo hacer ambas cosas en Python de forma sistemática: escribir **pruebas automáticas** con `pytest` y **documentar** módulos, funciones y programas.

E.1 Por qué escribir pruebas automáticas

Probar un programa a mano (ejecutarlo, introducir datos y mirar si el resultado parece correcto) es lento, poco fiable y hay que repetirlo cada vez que se cambia algo. Una **prueba automática** es un pequeño fragmento de código que ejecuta una parte del programa con unos datos concretos y **comprueba** que el resultado es el esperado. Una vez escritas, las pruebas se ejecutan en segundos tantas veces como haga falta, y avisan inmediatamente si un cambio ha estropeado algo que antes funcionaba (una *regresión*).

Las pruebas que veremos son **pruebas unitarias**: cada una comprueba una unidad pequeña (una función, un método) de forma aislada. Además sirven de documentación: muestran con ejemplos cómo se usa cada función y qué debe devolver.

E.2 De `assert` a las pruebas

En el Tema 0 vimos que `assert condición` lanza un `AssertionError` si la condición no se cumple. Una prueba no es más que una función que llama al código y comprueba el resultado con `assert`:

```
def test_media_basica():
    assert media([7, 8, 9]) == 8.0
```

Podríamos llamar a mano a todas estas funciones, pero un **marco de pruebas** se encarga de encontrarlas, ejecutarlas una a una (aunque alguna falle), mostrar un informe claro de las que fallan y por qué, y ofrecer herramientas para los casos habituales. En Python hay dos: `unittest`, incluido en la biblioteca estándar y de estilo similar a JUnit de Java, y `pytest`, que es el más utilizado por su sencillez. Usaremos `pytest`.

```
$ pip install pytest
```

E.3 Un ejemplo completo

Partimos de un módulo `notas.py` con dos funciones:

```
# notas.py
def media(notas: list[float]) -> float:
    """Calcula la media de una lista de notas, redondeada a dos decimales."""
    if not notas:
```

```
        raise ValueError("La lista de notas está vacía")
    if any(not 0 <= n <= 10 for n in notas):
        raise ValueError("Las notas deben estar entre 0 y 10")
    return round(sum(notas) / len(notas), 2)


def calificacion(nota: float) -> str:
    """Devuelve la calificación textual de una nota (Suspenso ... Sobresaliente)."""
    if not 0 <= nota <= 10:
        raise ValueError(f"Nota fuera de rango: {nota}")
    if nota < 5:
        return "Suspenso"
    if nota < 7:
        return "Aprobado"
    if nota < 9:
        return "Notable"
    return "Sobresaliente"
```

Las pruebas se guardan en una carpeta `tests`, en ficheros cuyo nombre empieza por `test_`:

```
proyecto/
├── pyproject.toml
├── notas.py
└── tests/
    ├── conftest.py
    └── test_notas.py
```

```
# tests/test_notas.py
import pytest
from notas import media, calificacion


def test_media_basica():
    assert media([7, 8, 9]) == 8.0


def test_media_redondea():
    assert media([7, 8, 8]) == 7.67


def test_media_lista_vacia():
    with pytest.raises(ValueError, match="vacía"):
        media([])
```

E.3.1 Reglas que sigue `pytest`

- Busca las pruebas en los ficheros `test_*.py` (o `*_test.py`) y, dentro de ellos, ejecuta las funciones cuyo nombre empieza por `test_`.
- Cada prueba es independiente: si una falla, las demás se siguen ejecutando.
- Una prueba **pasa** si termina sin lanzar ninguna excepción y **falla** si un `assert` no se cumple o se produce una excepción no esperada.

- Los nombres de las pruebas deben describir qué se comprueba: cuando una falle, el nombre será lo primero que veas.

E.3.2 Ejecutar las pruebas

Para que las pruebas puedan importar los módulos del proyecto, se indica en `pyproject.toml` dónde están el código y las pruebas:

```
# pyproject.toml
[tool.pytest.ini_options]
pythonpath = ["."]
testpaths = ["tests"]
```

```
$ pytest              # ejecuta todas las pruebas
$ pytest -v           # muestra cada prueba
$ pytest tests/test_notas.py::test_media_basica    # sólo una prueba
$ pytest -k media     # sólo las pruebas cuyo nombre contiene "media"
$ pytest -x           # se detiene en el primer fallo
```

Cuando todo va bien, `pytest` muestra un punto por cada prueba superada. Si una prueba falla, muestra exactamente qué línea ha fallado y con qué valores. Por ejemplo, si en `test_media_redondea` hubiéramos escrito `7.66` en lugar de `7.67`:

```
$ pytest
tests/test_notas.py .F.........                                          [100%]

================================== FAILURES ===================================
_____________________________ test_media_redondea _____________________________

    def test_media_redondea():
>       assert media([7, 8, 8]) == 7.66
E       assert 7.67 == 7.66
E        +  where 7.67 = media([7, 8, 8])

tests/test_notas.py:11: AssertionError
============================ short test summary info ============================
FAILED tests/test_notas.py::test_media_redondea - assert 7.67 == 7.66
========================= 1 failed, 11 passed in 0.04s =========================
```

E.4 Herramientas de `pytest`

E.4.1 Comprobar que se lanza una excepción

Probar los casos de error es tan importante como probar los correctos. `pytest.raises` comprueba que el bloque lanza la excepción indicada (y, opcionalmente, que su mensaje contiene un texto); si no la lanza, la prueba falla:

```
def test_media_lista_vacia():
    with pytest.raises(ValueError, match="vacía"):
        media([])
```

E.4.2 Pruebas parametrizadas

Para probar la misma función con muchos datos distintos no hace falta escribir una prueba por cada caso. Con `@pytest.mark.parametrize` se escribe una sola y `pytest` la ejecuta una vez por cada juego de datos, informando de cada uno por separado:

```
@pytest.mark.parametrize("nota, esperada", [
    (0, "Suspenso"),
    (4.99, "Suspenso"),
    (5, "Aprobado"),
    (7, "Notable"),
    (9, "Sobresaliente"),
    (10, "Sobresaliente"),
])
def test_calificacion(nota, esperada):
    assert calificacion(nota) == esperada


@pytest.mark.parametrize("nota", [-1, 10.5])
def test_calificacion_fuera_de_rango(nota):
    with pytest.raises(ValueError):
        calificacion(nota)
```

```
$ pytest -v
...
tests/test_notas.py::test_calificacion[0-Suspenso] PASSED                [ 41%]
tests/test_notas.py::test_calificacion[4.99-Suspenso] PASSED             [ 50%]
tests/test_notas.py::test_calificacion[5-Aprobado] PASSED                [ 58%]
...
tests/test_notas.py::test_calificacion_fuera_de_rango[10.5] PASSED       [100%]
```

> **Qué casos probar.** Además del caso "normal", conviene probar los **valores límite** (justo antes y justo después de cada frontera: 4.99 y 5, 0 y 10), los **casos vacíos** (lista vacía, cadena vacía, fichero sin datos) y los **datos no válidos**, que deben producir la excepción adecuada.

E.4.3 Números decimales

Por cómo se representan los números reales en el ordenador, `0.1 + 0.2 == 0.3` es `False`. Para comparar resultados con decimales se usa `pytest.approx`, que admite un pequeño margen de error:

```
def test_suma_decimales():
    assert 0.1 + 0.2 == pytest.approx(0.3)
```

E.4.4 *Fixtures*: preparar los datos de las pruebas

Muchas pruebas necesitan preparar algo antes de empezar: un fichero con datos, una base de datos en memoria, una conexión... Una **fixture** es una función decorada con `@pytest.fixture` que prepara ese recurso; cualquier prueba que tenga un **parámetro con el mismo nombre** lo recibe automáticamente. Las *fixtures* que se usan en varios ficheros de pruebas se guardan en

`tests/conftest.py`, que `pytest` carga solo.

`pytest` incluye algunas *fixtures* muy útiles. La más usada en este módulo es `tmp_path`: un directorio temporal nuevo y vacío para cada prueba, en el que se pueden crear ficheros sin ensuciar el proyecto.

```python
# tests/conftest.py
import json
import pytest


@pytest.fixture
def fichero_notas(tmp_path):
    """Crea un fichero JSON de notas de prueba y devuelve su ruta."""
    ruta = tmp_path / "notas.json"
    ruta.write_text(json.dumps({"Ana": [7, 8], "Luis": [4, 5]}), encoding="utf-8")
    return ruta
```

```python
# tests/test_fichero.py
import json
from notas import media


def test_medias_desde_fichero(fichero_notas):    # recibe la ruta creada
    datos = json.loads(fichero_notas.read_text(encoding="utf-8"))
    medias = {nombre: media(n) for nombre, n in datos.items()}
    assert medias == {"Ana": 7.5, "Luis": 4.5}
```

Si la *fixture* tiene que liberar el recurso al terminar la prueba (cerrar una conexión, borrar datos), se usa `yield` en lugar de `return`: lo que va después del `yield` se ejecuta al acabar la prueba, como en un gestor de contexto. En los temas 2 a 5 hay ejemplos con bases de datos.

Fixture	**Qué proporciona**
`tmp_path`	Un directorio temporal (`Path`) distinto para cada prueba.
`capsys`	Lo que el código ha escrito por pantalla: `capsys.readouterr().out`.
`monkeypatch`	Permite sustituir temporalmente variables de entorno, atributos o funciones (por ejemplo, simular `input()`).

E.4.5 Cobertura

La **cobertura** indica qué porcentaje de las líneas del código se ejecuta durante las pruebas, y sobre todo **qué líneas no se prueban nunca**. Se mide con el complemento `pytest-cov`:

```
$ pip install pytest-cov
$ pytest --cov=notas --cov-report=term-missing
Name       Stmts   Miss  Cover   Missing
----------------------------------------
notas.py      16      1    94%   7
----------------------------------------
TOTAL         16      1    94%
```

La línea 7 de `notas.py` (el `raise` de `media` para notas fuera de rango) no la ejecuta ninguna prueba: falta ese caso. Una cobertura alta no garantiza que el código sea correcto, pero una baja

indica con seguridad que hay partes sin probar.

E.5 Documentar el código

La documentación tiene dos destinatarios: quien **usa** el código (necesita saber qué hace cada función, qué recibe y qué devuelve) y quien lo **mantiene** (necesita entender por qué está escrito así). Python ofrece herramientas distintas para cada caso.

E.5.1 Comentarios y *docstrings*

- Los **comentarios** (`# ...`) explican el *porqué* de un fragmento concreto de código para quien lo lee. No deben repetir lo que el código ya dice (`i += 1  # suma 1 a i` no aporta nada).
- Las ***docstrings*** son cadenas entre triples comillas situadas en la **primera línea** de un módulo, clase o función. Describen *qué* hace y *cómo* se usa. A diferencia de los comentarios, forman parte del programa: las muestran los editores al pasar el ratón, la función `help()` y las herramientas de documentación.

Las convenciones para escribirlas están en la PEP 257 (`https://peps.python.org/pep-0257/`): una primera línea con un resumen breve terminado en punto, una línea en blanco y, si hace falta, una descripción más detallada. Para describir parámetros, valores devueltos y excepciones se usa un formato estructurado; el más extendido es el **estilo Google**:

```
def media(notas: list[float]) -> float:
    """Calcula la media aritmética de una lista de notas.

    Args:
        notas: Notas entre 0 y 10. No puede estar vacía.

    Returns:
        La media, redondeada a dos decimales.

    Raises:
        ValueError: Si la lista está vacía o alguna nota está fuera de rango.
    """
```

Las anotaciones de tipo complementan a la *docstring*: el tipo de cada parámetro ya está en la firma, así que la documentación se centra en su significado y sus restricciones.

E.5.2 Documentar un *script*

Un *script* (un programa que se ejecuta directamente, como las utilidades de línea de comandos de los ejercicios) debería incluir:

1. Una ***docstring* de módulo** al principio del fichero, con qué hace el programa, cómo se usa (con ejemplos de llamada), qué ficheros o datos necesita y qué produce.
2. *Docstrings* en todas las funciones y clases públicas.
3. Las importaciones agrupadas al principio (primero la biblioteca estándar, luego las de terceros y por último los módulos propios) y las constantes en mayúsculas.

4. Una función `main()` y el bloque `if __name__ == "__main__":`, para que el fichero se pueda importar (por ejemplo, desde las pruebas) sin ejecutar el programa.
5. Ayuda en la línea de comandos con `argparse`, que se puede generar a partir de la propia *docstring* del módulo.

```
"""Calcula las medias de las notas de los alumnos a partir de un fichero JSON.

El fichero de entrada debe contener un objeto cuyas claves son los nombres
de los alumnos y cuyos valores son listas de notas entre 0 y 10:

    {"Ana": [7, 8, 9], "Luis": [4, 5]}

Uso:
    python medias.py notas.json
    python medias.py notas.json --minimo 5      # sólo los aprobados

Autor: 2.º DAM - IES Ejemplo
"""
import argparse
import json
import sys
from pathlib import Path

from notas import calificacion, media

NOTA_APROBADO = 5.0


def cargar_notas(ruta: Path) -> dict[str, list[float]]:
    """Lee el fichero JSON de notas y devuelve un diccionario alumno -> notas."""
    with open(ruta, encoding="utf-8") as f:
        return json.load(f)


def main(argv: list[str] | None = None) -> int:
    """Punto de entrada del programa. Devuelve el código de salida."""
    parser = argparse.ArgumentParser(description=__doc__,
                                     formatter_class=argparse.
    ↪ RawDescriptionHelpFormatter)
    parser.add_argument("fichero", type=Path, help="fichero JSON con las notas")
    parser.add_argument("--minimo", type=float, default=0.0,
                        help="muestra sólo los alumnos con media >= MINIMO")
    args = parser.parse_args(argv)

    try:
        notas = cargar_notas(args.fichero)
    except (OSError, json.JSONDecodeError) as e:
        print(f"No se pudo leer {args.fichero}: {e}", file=sys.stderr)
        return 1

    for alumno, lista in sorted(notas.items()):
        m = media(lista)
        if m >= args.minimo:
            print(f"{alumno:<10} {m:5.2f}  {calificacion(m)}")
    return 0
```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

E.5.3 Consultar y generar la documentación

- En el intérprete, `help(media)` o `help(notas)` muestran las *docstrings*. Desde la terminal, `python -m pydoc notas` hace lo mismo.
- `pdoc` genera a partir de las *docstrings* y las anotaciones de tipo una documentación HTML navegable:

```
$ pip install pdoc
$ pdoc notas.py -o docs/
```

- Todo proyecto debería tener un fichero `README.md` en la carpeta raíz que explique qué hace, cómo se instala (dependencias, `requirements.txt`), cómo se ejecuta y cómo se lanzan las pruebas.

E.5.4 *Doctests*: ejemplos que también son pruebas

Los ejemplos de uso escritos en una *docstring* con el formato del intérprete (`>>>`) se pueden ejecutar como pruebas con el módulo `doctest` o con `pytest --doctest-modules`. Así se garantiza que los ejemplos de la documentación siguen siendo correctos:

```
def media(notas: list[float]) -> float:
    """Calcula la media aritmética de una lista de notas.

    >>> media([5, 6])
    5.5
    >>> media([7, 8, 9])
    8.0
    """
```

```
$ pytest --doctest-modules
```

E.6 Guía de estilo

La PEP 8 (`https://peps.python.org/pep-0008/`) es la guía de estilo oficial de Python: sangría de cuatro espacios, nombres de funciones y variables en `minusculas_con_guiones`, clases en `CamelCase`, constantes en `MAYUSCULAS`, líneas de longitud moderada, etc. Herramientas como `ruff` comprueban el estilo y detectan errores habituales automáticamente (`ruff check .`) e incluso dan formato al código (`ruff format .`).

E.7 Ejercicios

Ejercicio E.1 — Pruebas de funciones

Dificultad: ★

Escribe pruebas con `pytest` para la función `validar_dni` del Tema 0: varios DNI correctos (parametrizados), DNI con la letra equivocada, con menos dígitos, con letras en lugar de números y en minúsculas. Comprueba con `pytest-cov` que se ejecutan todas las líneas de la función.

Ejercicio E.2 — Documentar un programa

Dificultad: ★★

Toma uno de los programas de línea de comandos que hayas hecho en el curso y documéntalo siguiendo este anexo: *docstring* de módulo con ejemplos de uso, *docstrings* en estilo Google en todas las funciones, `README.md` y ayuda de `argparse` generada a partir de la *docstring*. Genera la documentación HTML con `pdoc` y comprueba el estilo con `ruff`.

Ejercicio E.3 — Pruebas con ficheros

Dificultad: ★★

Escribe una *fixture* que cree en `tmp_path` un fichero CSV de alumnos de prueba y úsala en varias pruebas de una función `importar_csv(ruta) -> list[Alumno]`: fichero correcto, fichero vacío, fila con un campo de más, nota no numérica y fichero inexistente.

APÉNDICE F

Bibliografía y recursos

Este anexo reúne la documentación oficial, las especificaciones y los libros en los que se apoyan los apuntes, organizados por temas. Además, a lo largo del documento, cada vez que se menciona una biblioteca o un módulo de Python, su nombre aparece en este color y enlaza directamente con su documentación.

> **Sobre las versiones.** La documentación en línea corresponde normalmente a la última versión publicada. Si algo no funciona como se describe, comprueba qué versión tienes instalada (`pip show nombre`) y consulta la documentación de esa versión.

F.1 Documentación general

- Documentación oficial de Python (en español): `https://docs.python.org/es/3/`
- Tutorial oficial de Python (en español): `https://docs.python.org/es/3/tutorial/`
- Índice de la biblioteca estándar: `https://docs.python.org/es/3/library/`
- PEP 8, guía de estilo para el código Python: `https://peps.python.org/pep-0008/`
- Guía de empaquetado de Python (*Python Packaging User Guide*): `https://packaging.python.org/en/latest/`
- Documentación de Docker: `https://docs.docker.com/`

F.2 Tema 1: Manejo de ficheros

- Ficheros y directorios: `https://docs.python.org/es/3/library/pathlib.html`, `https://docs.python.org/es/3/library/shutil.html`
- Entrada y salida, flujos: `https://docs.python.org/es/3/library/io.html`
- Datos binarios: `https://docs.python.org/es/3/library/struct.html`
- CSV: `https://docs.python.org/es/3/library/csv.html`
- JSON: `https://docs.python.org/es/3/library/json.html`
- ElementTree: `https://docs.python.org/es/3/library/xml.etree.elementtree.html`
- JSON Schema, web oficial del estándar: `https://json-schema.org`
- Guía *Understanding JSON Schema*: `https://json-schema.org/understanding-json-schema`
- jsonschema: `https://python-jsonschema.readthedocs.io/`
- Pydantic: `https://docs.pydantic.dev/`
- lxml: `https://lxml.de/`
- xsdata: `https://xsdata.readthedocs.io/`

- Requests: `https://requests.readthedocs.io/`
- Beautiful Soup: `https://www.crummy.com/software/BeautifulSoup/bs4/doc/`
- pytest: `https://docs.pytest.org/`

F.3 Tema 2: Manejo de conectores

- PEP 249, especificación DB-API 2.0: `https://peps.python.org/pep-0249/`
- sqlite3: `https://docs.python.org/es/3/library/sqlite3.html`
- Documentación de SQLite: `https://www.sqlite.org/docs.html`
- psycopg 3: `https://www.psycopg.org/psycopg3/docs/`
- Documentación de PostgreSQL: `https://www.postgresql.org/docs/current/`
- MySQL Connector/Python: `https://dev.mysql.com/doc/connector-python/en/`
- MySQL Connector/ODBC: `https://dev.mysql.com/doc/connector-odbc/en/`
- pyodbc: `https://github.com/mkleehammer/pyodbc/wiki`
- unixODBC: `https://www.unixodbc.org/`
- PL/pgSQL: `https://www.postgresql.org/docs/current/plpgsql.html`

F.4 Tema 3: Herramientas ORM

- SQLAlchemy 2.0: `https://docs.sqlalchemy.org/en/20/`
- Inicio rápido del ORM de SQLAlchemy: `https://docs.sqlalchemy.org/en/20/orm/quickstart.html`
- Tutorial unificado de SQLAlchemy 2.0: `https://docs.sqlalchemy.org/en/20/tutorial/`
- Alembic (migraciones): `https://alembic.sqlalchemy.org/`

F.5 Tema 4: Bases de datos objeto-relacionales

- Tipos compuestos en PostgreSQL: `https://www.postgresql.org/docs/current/rowtypes.html`
- Arrays: `https://www.postgresql.org/docs/current/arrays.html`
- Herencia de tablas: `https://www.postgresql.org/docs/current/ddl-inherit.html`
- Tipos JSON: `https://www.postgresql.org/docs/current/datatype-json.html`
- Adaptación de tipos de PostgreSQL en psycopg: `https://www.psycopg.org/psycopg3/docs/basic/pgtypes.html`
- Tipos específicos de PostgreSQL en SQLAlchemy: `https://docs.sqlalchemy.org/en/20/dialects/postgresql.html`
- ISO/IEC 9075:1999, *Database languages — SQL* (SQL:1999). Organización Internacional de Normalización, 1999.

F.6 Tema 5: Bases de datos documentales (MongoDB)

- Manual de MongoDB: `https://www.mongodb.com/docs/manual/`
- MongoDB Shell (`mongosh`): `https://www.mongodb.com/docs/mongodb-shell/`
- MongoDB Compass: `https://www.mongodb.com/docs/compass/`
- PyMongo: `https://www.mongodb.com/docs/languages/python/pymongo-driver/current/`
- Especificación BSON: `https://bsonspec.org/`

F.7 Tema 6: Componentes de acceso a datos

- Descriptores (guía práctica): `https://docs.python.org/es/3/howto/descriptor.html`
- Introspección: `https://docs.python.org/es/3/library/inspect.html`
- Clases abstractas: `https://docs.python.org/es/3/library/abc.html`
- Especificación de `pyproject.toml`: `https://packaging.python.org/en/latest/specifications/pyproject-toml/`
- pdoc: `https://pdoc.dev/`

F.8 Anexos B, C y D

- XSLT y XPath con lxml: `https://lxml.de/xpathxslt.html`
- ZODB: `https://zodb.org/`
- BaseX: `https://docs.basex.org/`
- R. G. G. Cattell y D. K. Barry (eds.), *The Object Data Standard: ODMG 3.0*. Morgan Kaufmann, 2000.

F.9 Anexo E: pruebas y documentación

- pytest: `https://docs.pytest.org/`
- pytest-cov: `https://pytest-cov.readthedocs.io/`
- doctest: `https://docs.python.org/es/3/library/doctest.html`
- PEP 257, convenciones para las *docstrings*: `https://peps.python.org/pep-0257/`
- Guía de estilo de Google para Python (formato de las *docstrings*): `https://google.github.io/styleguide/pyguide.html`
- pdoc: `https://pdoc.dev/`
- Ruff: `https://docs.astral.sh/ruff/`

F.10 Especificaciones y estándares

- RFC 8259, *The JavaScript Object Notation (JSON) Data Interchange Format*: `https://www.rfc-editor.org/rfc/rfc8259`
- JSON Schema (versión 2020-12): `https://json-schema.org/draft/2020-12`
- JSON Lines: `https://jsonlines.org/`
- W3C, *Extensible Markup Language (XML) 1.0*: `https://www.w3.org/TR/xml/`
- W3C, *XML Schema Definition Language (XSD) 1.1*: `https://www.w3.org/TR/xmlschema11-1/`
- W3C, *XML Path Language (XPath) 1.0*: `https://www.w3.org/TR/xpath-10/`
- W3C, *XSL Transformations (XSLT) 1.0*: `https://www.w3.org/TR/xslt-10/`
- W3C, *XQuery 3.1*: `https://www.w3.org/TR/xquery-31/`
- W3C, *XQuery Update Facility 1.0*: `https://www.w3.org/TR/xquery-update-10/`
- RFC 9309, *Robots Exclusion Protocol* (`robots.txt`): `https://www.rfc-editor.org/rfc/rfc9309`

F.11 Libros

- L. Ramalho, *Fluent Python*, 2.ª ed. O'Reilly Media, 2022.
- M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- S. Bradshaw, E. Brazil y K. Chodorow, *MongoDB: The Definitive Guide*, 3.ª ed. O'Reilly Media, 2019.
- R. Mitchell, *Web Scraping with Python*, 3.ª ed. O'Reilly Media, 2024.
- B. Okken, *Python Testing with pytest*, 2.ª ed. The Pragmatic Bookshelf, 2022.

F.12 Datos abiertos y sitios de práctica

- Portal de datos abiertos de la Junta de Castilla y León: `https://datosabiertos.jcyl.es/`
- Catálogo nacional de datos abiertos: `https://datos.gob.es/`
- Sitios para practicar *web scraping*: `https://books.toscrape.com/` y `https://quotes.toscrape.com/`
- Proyecto Gutenberg (textos de dominio público): `https://www.gutenberg.org/`